```
!pip install transformers
!pip install torch

Requirement already satisfied: transformers in
/usr/local/lib/python3.11/dist-packages (4.55.1)
Requirement already satisfied: filelock in
/usr/local/lib/python3.11/dist-packages (from transformers) (3.18.0)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in
/usr/local/lib/python3.11/dist-packages (from transformers) (0.34.4)
Requirement already satisfied: numpy>=1.17 in
/usr/local/lib/python3.11/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.11/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in
/usr/local/lib/python3.11/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in
/usr/local/lib/python3.11/dist-packages (from transformers)
(2024.11.6)
Requirement already satisfied: requests in
/usr/local/lib/python3.11/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: tokenizers<0.22,>=0.21 in
/usr/local/lib/python3.11/dist-packages (from transformers) (0.21.4)
Requirement already satisfied: safetensors>=0.4.3 in
/usr/local/lib/python3.11/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in
/usr/local/lib/python3.11/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in
/usr/local/lib/python3.11/dist-packages (from huggingface-
hub<1.0,>=0.34.0->transformers) (2025.3.0)
Requirement already satisfied: typing-extensions>=3.7.4.3 in
/usr/local/lib/python3.11/dist-packages (from huggingface-
hub<1.0,>=0.34.0->transformers) (4.14.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in
/usr/local/lib/python3.11/dist-packages (from huggingface-
hub<1.0,>=0.34.0->transformers) (1.1.7)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.11/dist-packages (from requests->transformers)
(3.4.3)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.11/dist-packages (from requests->transformers)
(3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.11/dist-packages (from requests->transformers)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests->transformers)
(2025.8.3)
Requirement already satisfied: torch in
/usr/local/lib/python3.11/dist-packages (2.6.0+cu124)
Requirement already satisfied: filelock in
```

```
/usr/local/lib/python3.11/dist-packages (from torch) (3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.11/dist-packages (from torch) (4.14.1)
Requirement already satisfied: networkx in
/usr/local/lib/python3.11/dist-packages (from torch) (3.5)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.11/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in
/usr/local/lib/python3.11/dist-packages (from torch) (2025.3.0)
Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch)
  Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch)
  Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch)
  Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch)
  Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-curand-cu12==10.3.5.147 (from torch)
  Downloading nvidia_curand_cu12-10.3.5.147-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch)
  Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Collecting nvidia-cusparse-cu12==12.3.1.170 (from torch)
  Downloading nvidia_cusparse_cu12-12.3.1.170-py3-none-
manylinux2014_x86_64.whl.metadata (1.6 kB)
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in
/usr/local/lib/python3.11/dist-packages (from torch) (0.6.2)
Collecting nvidia-nccl-cu12==2.21.5 (from torch)
  Downloading nvidia_nccl_cu12-2.21.5-py3-none-
manylinux2014_x86_64.whl.metadata (1.8 kB)
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in
/usr/local/lib/python3.11/dist-packages (from torch) (12.4.127)
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch)
  Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-
manylinux2014_x86_64.whl.metadata (1.5 kB)
Requirement already satisfied: triton==3.2.0 in
/usr/local/lib/python3.11/dist-packages (from torch) (3.2.0)
Requirement already satisfied: sympy==1.13.1 in
```

```
/usr/local/lib/python3.11/dist-packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from sympy==1.13.1->torch)
(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.11/dist-packages (from jinja2->torch) (3.0.2)
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-
manylinux2014_x86_64.whl (363.4 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 363.4/363.4 MB 3.9 MB/s eta
0:00:00
anylinux2014_x86_64.whl (13.8 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.8/13.8 MB 68.5 MB/s eta
0:00:00
anylinux2014_x86_64.whl (24.6 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.6/24.6 MB 24.3 MB/s eta
0:00:00
e_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 883.7/883.7 kB 33.5 MB/s eta
0:00:00
anylinux2014_x86_64.whl (664.8 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 1.9 MB/s eta
0:00:00
anylinux2014_x86_64.whl (211.5 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 211.5/211.5 MB 2.4 MB/s eta
0:00:00
anylinux2014_x86_64.whl (56.3 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.3/56.3 MB 8.1 MB/s eta
0:00:00
anylinux2014_x86_64.whl (127.9 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 127.9/127.9 MB 5.4 MB/s eta
0:00:00
anylinux2014_x86_64.whl (207.5 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 207.5/207.5 MB 1.8 MB/s eta
0:00:00
anylinux2014_x86_64.whl (188.7 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 188.7/188.7 MB 5.9 MB/s eta
0:00:00
anylinux2014_x86_64.whl (21.1 MB)
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21.1/21.1 MB 33.2 MB/s eta
0:00:00
e-cu12, nvidia-cuda-nvrtc-cu12, nvidia-cuda-cupti-cu12, nvidia-cublas-
cu12, nvidia-cusparse-cu12, nvidia-cudnn-cu12, nvidia-cusolver-cu12
  Attempting uninstall: nvidia-nvjitlink-cu12
    Found existing installation: nvidia-nvjitlink-cu12 12.5.82
    Uninstalling nvidia-nvjitlink-cu12-12.5.82:
      Successfully uninstalled nvidia-nvjitlink-cu12-12.5.82
  Attempting uninstall: nvidia-nccl-cu12
    Found existing installation: nvidia-nccl-cu12 2.23.4
    Uninstalling nvidia-nccl-cu12-2.23.4:
```

```
      Successfully uninstalled nvidia-nccl-cu12-2.23.4
  Attempting uninstall: nvidia-curand-cu12
    Found existing installation: nvidia-curand-cu12 10.3.6.82
    Uninstalling nvidia-curand-cu12-10.3.6.82:
      Successfully uninstalled nvidia-curand-cu12-10.3.6.82
  Attempting uninstall: nvidia-cufft-cu12
    Found existing installation: nvidia-cufft-cu12 11.2.3.61
    Uninstalling nvidia-cufft-cu12-11.2.3.61:
      Successfully uninstalled nvidia-cufft-cu12-11.2.3.61
  Attempting uninstall: nvidia-cuda-runtime-cu12
    Found existing installation: nvidia-cuda-runtime-cu12 12.5.82
    Uninstalling nvidia-cuda-runtime-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-runtime-cu12-12.5.82
  Attempting uninstall: nvidia-cuda-nvrtc-cu12
    Found existing installation: nvidia-cuda-nvrtc-cu12 12.5.82
    Uninstalling nvidia-cuda-nvrtc-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.5.82
  Attempting uninstall: nvidia-cuda-cupti-cu12
    Found existing installation: nvidia-cuda-cupti-cu12 12.5.82
    Uninstalling nvidia-cuda-cupti-cu12-12.5.82:
      Successfully uninstalled nvidia-cuda-cupti-cu12-12.5.82
  Attempting uninstall: nvidia-cublas-cu12
    Found existing installation: nvidia-cublas-cu12 12.5.3.2
    Uninstalling nvidia-cublas-cu12-12.5.3.2:
      Successfully uninstalled nvidia-cublas-cu12-12.5.3.2
  Attempting uninstall: nvidia-cusparse-cu12
    Found existing installation: nvidia-cusparse-cu12 12.5.1.3
    Uninstalling nvidia-cusparse-cu12-12.5.1.3:
      Successfully uninstalled nvidia-cusparse-cu12-12.5.1.3
  Attempting uninstall: nvidia-cudnn-cu12
    Found existing installation: nvidia-cudnn-cu12 9.3.0.75
    Uninstalling nvidia-cudnn-cu12-9.3.0.75:
      Successfully uninstalled nvidia-cudnn-cu12-9.3.0.75
  Attempting uninstall: nvidia-cusolver-cu12
    Found existing installation: nvidia-cusolver-cu12 11.6.3.83
    Uninstalling nvidia-cusolver-cu12-11.6.3.83:
      Successfully uninstalled nvidia-cusolver-cu12-11.6.3.83
Successfully installed nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-
cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-
cu12-12.4.127 nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3
nvidia-curand-cu12-10.3.5.147 nvidia-cusolver-cu12-11.6.1.9 nvidia-
cusparse-cu12-12.3.1.170 nvidia-nccl-cu12-2.21.5 nvidia-nvjitlink-
cu12-12.4.127
```

```
import torch
from transformers import pipeline, AutoTokenizer,
AutoModelForSequenceClassification
import logging

logging.basicConfig(level=logging.INFO)
```

```
MODEL_NAME = "distilbert-base-uncased-finetuned-sst-2-english"

try:
    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
    model =
AutoModelForSequenceClassification.from_pretrained(MODEL_NAME)
    logging.info("Model loaded successfully.")
except Exception as e:
    logging.error(f"Error loading model: {e}")
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```


```
{"model_id":"be75845d8caa40f48b2896899a1e4111","version_major":2,"vers
ion_minor":0}

{"model_id":"c943be5378ca4752907130e57269a525","version_major":2,"vers
ion_minor":0}

{"model_id":"e1502f2dc19f4079a4ade9001d0c5cd3","version_major":2,"vers
ion_minor":0}

{"model_id":"001cd2355577426b938a1bc43adf9c0c","version_major":2,"vers
ion_minor":0}
```


```
device = 0 if torch.cuda.is_available() else -1

nlp = pipeline("sentiment-analysis", model=model, tokenizer=tokenizer,
device=device)
```

```
Device set to use cpu
```

```
texts = [
    "I love learning Python!",
    "This assignment is tough but interesting.",
    "I'm feeling anxious about my exam.",
    "this workshop is very challenging."
]

results = nlp(texts)

for i, res in enumerate(results):
    print(f"Text: {texts[i]}")
    print(f"→ Sentiment: {res['label']} | Score: {res['score']:.4f}")
```

```
Text: I love learning Python!
→ Sentiment: POSITIVE | Score: 0.9996
Text: This assignment is tough but interesting.
→ Sentiment: POSITIVE | Score: 0.9726
Text: I'm feeling anxious about my exam.
→ Sentiment: NEGATIVE | Score: 0.9980
Text: this workshop is very challenging.
→ Sentiment: POSITIVE | Score: 0.9989
```

que.2.2. Prompt Optimization Framework Develop a framework that automatically adapts prompts for: - Summarization - Text classification - Creative text generation The framework should detect the task type and adjust prompt style, length, and structure accordingly

```
# Step 1: Task Detection
def detect_task(input_text):
    input_text = input_text.lower()
    if "summarize" in input_text:
        return "summarization"
    elif "classify" in input_text or "label" in input_text:
        return "classification"
    elif any(word in input_text for word in ["story", "write",
"generate", "imagine"]):
        return "creative_generation"
    else:
        return "unknown"

# Step 2: Prompt Templates
PROMPT_TEMPLATES = {
    "summarization": lambda text: f"Summarize this clearly and
concisely:\n{text}",
    "classification": lambda text: f"Classify the sentiment or topic
of this text:\n{text}",
    "creative_generation": lambda text: f"Write a creative
continuation of the following:\n{text}"
}

# Step 3: Prompt Adapter
def adapt_prompt(task_type, input_text, style="default",
length="medium"):
    base_prompt = PROMPT_TEMPLATES.get(task_type, lambda x: x)
(input_text)
    if style == "formal":
        base_prompt = "Please respond in a formal tone.\n" +
base_prompt
    elif style == "playful":
        base_prompt = "Add a touch of humor and creativity.\n" +
base_prompt
    if length == "short":
        base_prompt += "\nLimit your response to 2-3 sentences."
```

```
    elif length == "long":
        base_prompt += "\nExpand with detailed reasoning or
narrative."
    return base_prompt

# Step 4: Run it
input_text = "Write a story about a robot who learns to dream."
task = detect_task(input_text)
prompt = adapt_prompt(task, input_text, style="playful",
length="long")

print("Detected Task:", task)
print("Final Prompt:\n", prompt)
```

```
Detected Task: creative_generation
Final Prompt:
 Add a touch of humor and creativity.
Write a creative continuation of the following:
Write a story about a robot who learns to dream.
Expand with detailed reasoning or narrative.
```

q3. Automated Data Exploration with pandas Create a reusable function or class that: - Generates statistical summaries for each column - Detects and reports missing values, duplicates, and outliers - Recommends preprocessing steps - Outputs a well-structured report

```
import pandas as pd
import numpy as np

class DataExplorer:
    """
    A reusable class for automated data exploration using pandas.
    Provides statistical summaries, detects data issues, and
recommends preprocessing steps.
    """

    def __init__(self, dataframe):
        self.df = dataframe.copy()
        self.report = {}

    def generate_summary(self):
        """Generates statistical summary for each column."""
        self.report['summary'] =
self.df.describe(include='all').transpose()

    def detect_missing_values(self):
        """Detects missing values and calculates percentage."""
        missing_count = self.df.isnull().sum()
        missing_percent = (missing_count / len(self.df)) * 100
        self.report['missing_values'] = pd.DataFrame({
            'Missing Count': missing_count,
```

```
            'Missing %': missing_percent.round(2)
        })

    def detect_duplicates(self):
        """Detects duplicate rows in the dataset."""
        duplicate_count = self.df.duplicated().sum()
        self.report['duplicates'] = f"{duplicate_count} duplicate rows
found."

    def detect_outliers(self):
        """Detects outliers in numeric columns using IQR method."""
        outlier_summary = {}
        numeric_cols =
self.df.select_dtypes(include=np.number).columns
        for col in numeric_cols:
            Q1 = self.df[col].quantile(0.25)
            Q3 = self.df[col].quantile(0.75)
            IQR = Q3 - Q1
            lower_bound = Q1 - 1.5 * IQR
            upper_bound = Q3 + 1.5 * IQR
            outliers = self.df[(self.df[col] < lower_bound) |
(self.df[col] > upper_bound)]
            outlier_summary[col] = {
                'Outlier Count': outliers.shape[0],
                'Outlier Indices': outliers.index.tolist()
            }
        self.report['outliers'] = outlier_summary

    def recommend_preprocessing(self):
        """Suggests preprocessing steps based on data issues."""
        recommendations = []
        if self.df.isnull().sum().any():
            recommendations.append("□ Handle missing values:
imputation or removal.")
        if self.df.duplicated().any():
            recommendations.append("□ Remove duplicate rows.")
        for col in self.df.select_dtypes(include=np.number).columns:
            skewness = self.df[col].skew()
            if abs(skewness) > 1:
                recommendations.append(f"□ Column '{col}' is highly
skewed (skew={skewness:.2f}). Consider transformation.")
        self.report['recommendations'] = recommendations

    def generate_report(self):
        """Runs all analysis methods and returns a structured
report."""
        self.generate_summary()
        self.detect_missing_values()
        self.detect_duplicates()
        self.detect_outliers()
```

```
        self.recommend_preprocessing()
        return self.report

# Load your dataset
df = pd.read_csv("/content/hospital.csv")

# Create an instance and generate report
explorer = DataExplorer(df)
report = explorer.generate_report()

# View sections of the report
print("Summary:\n", report['summary'])
print("Missing Values:\n", report['missing_values'])
print("Duplicates:\n", report['duplicates'])
print("Outliers:\n", report['outliers'])
print("Recommendations:\n", report['recommendations'])
```

```
Summary:
                       count unique
top  \
Admission_ID           40235  40235  a129b050-2edc-444b-9036-
fc1886798172
Name                   40235  40235                          Tony
Collins
Age                  40235.0    NaN
NaN
Gender                 40235      2
Female
BMI                  40235.0    NaN
NaN
Ethnicity              40235      5
Caucasian
Height               40235.0    NaN
NaN
Weight               40235.0    NaN
NaN
Blood_Type             40235      8
AB+
Medical_Condition      40235      6
Arthritis
Admission_Date         40235   1827                           2020-10-
22
Doctor                 40235  33556                          John
Smith
Hospital               40235  32783                           LLC
Smith
Insurance_Provider     40235      5
Cigna
Billing_Amount       40235.0    NaN
NaN
```

```
Room_Number         40235.0    NaN
NaN
Admission_Type        40235      3
Elective
Discharge_Date        40235   1856                                    2019-09-
10
Medication            40235      5
Lipitor
Test_Results          40235      3
Abnormal
Days_Hospitalised   40235.0    NaN
NaN

                     freq         mean           std      min      25%
\
Admission_ID            1          NaN           NaN      NaN      NaN

Name                    1          NaN           NaN      NaN      NaN

Age                   NaN    51.697353     19.575471     18.0     35.0

Gender              20127          NaN           NaN      NaN      NaN

BMI                   NaN    29.285556       8.29259    14.84    23.72

Ethnicity           31726          NaN           NaN      NaN      NaN

Height                NaN   169.915074     10.674313    137.0    163.0

Weight                NaN    84.565279      25.07438     39.0     67.0

Blood_Type           5101          NaN           NaN      NaN      NaN

Medical_Condition    6821          NaN           NaN      NaN      NaN

Admission_Date         38          NaN           NaN      NaN      NaN

Doctor                 18          NaN           NaN      NaN      NaN

Hospital               31          NaN           NaN      NaN      NaN

Insurance_Provider   8122          NaN           NaN      NaN      NaN

Billing_Amount        NaN  25559.60293  14220.951676 -2008.49  13236.5

Room_Number           NaN   300.946017     115.33754    101.0    202.0

Admission_Type      13563          NaN           NaN      NaN      NaN

Discharge_Date         38          NaN           NaN      NaN      NaN
```

Medication	8156	NaN	NaN	NaN	NaN
Test_Results	13487	NaN	NaN	NaN	NaN
Days_Hospitalised	NaN	15.524692	8.657143	1.0	8.0

	50%	75%	max
Admission_ID	NaN	NaN	NaN
Name	NaN	NaN	NaN
Age	52.0	69.0	85.0
Gender	NaN	NaN	NaN
BMI	27.76	33.06	67.81
Ethnicity	NaN	NaN	NaN
Height	170.0	178.0	196.0
Weight	81.0	98.0	186.0
Blood_Type	NaN	NaN	NaN
Medical_Condition	NaN	NaN	NaN
Admission_Date	NaN	NaN	NaN
Doctor	NaN	NaN	NaN
Hospital	NaN	NaN	NaN
Insurance_Provider	NaN	NaN	NaN
Billing_Amount	25544.68	37836.79	52764.28
Room_Number	302.0	401.0	500.0
Admission_Type	NaN	NaN	NaN
Discharge_Date	NaN	NaN	NaN
Medication	NaN	NaN	NaN
Test_Results	NaN	NaN	NaN
Days_Hospitalised	16.0	23.0	30.0

Missing Values:

	Missing Count	Missing %
Admission_ID	0	0.0
Name	0	0.0
Age	0	0.0
Gender	0	0.0
BMI	0	0.0
Ethnicity	0	0.0
Height	0	0.0
Weight	0	0.0
Blood_Type	0	0.0
Medical_Condition	0	0.0
Admission_Date	0	0.0
Doctor	0	0.0
Hospital	0	0.0
Insurance_Provider	0	0.0
Billing_Amount	0	0.0
Room_Number	0	0.0
Admission_Type	0	0.0
Discharge_Date	0	0.0
Medication	0	0.0

```
Test_Results                    0        0.0
Days_Hospitalised               0        0.0
Duplicates:
 0 duplicate rows found.
Outliers:
 {'Age': {'Outlier Count': 0, 'Outlier Indices': []}, 'BMI': {'Outlier
Count': 1480, 'Outlier Indices': [2, 6, 21, 47, 87, 91, 99, 137, 161,
166, 187, 247, 260, 321, 330, 347, 410, 414, 419, 435, 449, 457, 513,
521, 546, 629, 669, 683, 687, 741, 746, 791, 802, 843, 881, 905, 920,
1007, 1011, 1039, 1066, 1085, 1119, 1123, 1157, 1159, 1167, 1179,
1191, 1197, 1202, 1216, 1217, 1225, 1241, 1298, 1321, 1328, 1387,
1391, 1448, 1457, 1458, 1505, 1507, 1575, 1600, 1646, 1660, 1681,
1689, 1748, 1801, 1820, 1841, 1858, 1997, 2000, 2001, 2010, 2114,
2141, 2212, 2276, 2310, 2316, 2350, 2352, 2443, 2526, 2545, 2564,
2568, 2569, 2608, 2619, 2672, 2685, 2697, 2769, 2829, 2840, 2848,
2888, 2947, 3011, 3034, 3045, 3075, 3079, 3168, 3186, 3215, 3216,
3246, 3269, 3294, 3302, 3307, 3326, 3331, 3335, 3360, 3390, 3396,
3406, 3415, 3429, 3438, 3442, 3451, 3470, 3512, 3550, 3606, 3678,
3710, 3741, 3747, 3760, 3766, 3767, 3791, 3794, 3810, 3820, 3825,
3829, 3850, 3892, 4016, 4062, 4081, 4147, 4189, 4292, 4312, 4345,
4356, 4389, 4457, 4552, 4561, 4580, 4635, 4675, 4770, 4781, 4787,
4823, 4826, 4827, 4832, 4837, 4886, 4897, 4927, 4942, 4966, 4977,
5000, 5003, 5016, 5055, 5080, 5106, 5118, 5124, 5136, 5137, 5182,
5193, 5209, 5221, 5233, 5292, 5335, 5360, 5391, 5393, 5394, 5406,
5415, 5459, 5476, 5555, 5569, 5576, 5599, 5632, 5655, 5672, 5684,
5695, 5699, 5711, 5725, 5760, 5856, 5939, 6070, 6127, 6201, 6209,
6222, 6261, 6276, 6362, 6374, 6403, 6405, 6418, 6514, 6520, 6551,
6616, 6650, 6664, 6681, 6712, 6730, 6752, 6776, 6817, 6837, 6850,
6861, 6865, 6870, 6884, 6893, 6918, 6924, 6957, 7022, 7050, 7074,
7086, 7105, 7119, 7167, 7174, 7230, 7283, 7298, 7397, 7401, 7411,
7430, 7481, 7482, 7514, 7531, 7615, 7620, 7659, 7680, 7696, 7789,
7804, 7839, 7847, 7905, 7951, 7963, 7976, 8023, 8044, 8132, 8140,
8259, 8294, 8300, 8323, 8357, 8370, 8394, 8453, 8504, 8540, 8576,
8588, 8601, 8610, 8642, 8643, 8649, 8678, 8695, 8746, 8757, 8758,
8812, 8851, 8858, 8863, 8872, 8898, 8924, 8943, 8947, 8948, 8973,
8980, 9001, 9004, 9034, 9053, 9093, 9148, 9160, 9209, 9242, 9267,
9275, 9302, 9309, 9328, 9465, 9534, 9544, 9555, 9563, 9616, 9630,
9651, 9700, 9762, 9793, 9814, 9835, 9881, 9886, 9888, 9895, 9959,
9966, 9977, 10000, 10032, 10106, 10115, 10117, 10124, 10128, 10140,
10155, 10180, 10198, 10202, 10241, 10250, 10253, 10257, 10293, 10297,
10301, 10312, 10335, 10383, 10386, 10450, 10462, 10520, 10539, 10567,
10629, 10635, 10661, 10668, 10669, 10687, 10721, 10734, 10763, 10819,
10831, 10843, 10875, 10884, 10915, 10933, 11029, 11031, 11050, 11078,
11098, 11114, 11117, 11165, 11172, 11191, 11213, 11248, 11326, 11457,
11467, 11470, 11485, 11488, 11498, 11511, 11558, 11583, 11610, 11622,
11695, 11704, 11732, 11786, 11914, 11988, 11999, 12002, 12008, 12132,
12146, 12158, 12245, 12292, 12295, 12324, 12343, 12358, 12393, 12400,
12407, 12461, 12523, 12545, 12557, 12581, 12608, 12611, 12616, 12642,
12651, 12656, 12739, 12742, 12757, 12769, 12786, 12797, 12858, 12879,
```

12918, 12954, 12960, 12983, 12995, 13059, 13083, 13092, 13093, 13100,
13119, 13130, 13136, 13143, 13207, 13224, 13254, 13259, 13305, 13348,
13372, 13407, 13450, 13476, 13496, 13523, 13548, 13605, 13651, 13694,
13735, 13770, 13790, 13807, 13856, 13996, 14015, 14064, 14070, 14085,
14115, 14144, 14235, 14244, 14250, 14275, 14297, 14305, 14344, 14353,
14387, 14417, 14426, 14466, 14478, 14515, 14528, 14564, 14580, 14627,
14635, 14723, 14731, 14734, 14760, 14780, 14801, 14818, 14823, 14833,
14859, 14866, 14940, 15000, 15008, 15042, 15061, 15083, 15209, 15211,
15215, 15218, 15251, 15279, 15284, 15291, 15301, 15313, 15320, 15333,
15335, 15340, 15345, 15391, 15481, 15489, 15493, 15499, 15515, 15516,
15549, 15569, 15582, 15590, 15612, 15629, 15752, 15816, 15819, 15906,
15923, 15932, 15933, 15944, 15995, 16020, 16044, 16052, 16100, 16160,
16183, 16206, 16208, 16214, 16219, 16227, 16230, 16251, 16252, 16269,
16282, 16286, 16335, 16372, 16375, 16382, 16388, 16391, 16432, 16434,
16445, 16529, 16534, 16540, 16564, 16582, 16619, 16640, 16643, 16645,
16655, 16678, 16737, 16803, 16830, 16847, 16857, 16866, 16871, 16901,
16906, 16965, 16967, 16972, 16999, 17053, 17059, 17065, 17073, 17128,
17201, 17204, 17285, 17296, 17303, 17382, 17420, 17432, 17455, 17517,
17531, 17561, 17604, 17682, 17713, 17753, 17758, 17806, 17820, 17866,
17950, 17977, 18039, 18055, 18077, 18126, 18145, 18150, 18154, 18207,
18215, 18357, 18395, 18414, 18424, 18490, 18500, 18517, 18523, 18562,
18563, 18616, 18687, 18785, 18812, 18824, 18849, 18914, 18944, 18958,
18969, 18980, 19022, 19034, 19068, 19076, 19101, 19128, 19147, 19169,
19180, 19181, 19203, 19269, 19287, 19338, 19386, 19404, 19424, 19425,
19434, 19524, 19623, 19627, 19696, 19704, 19732, 19765, 19822, 19877,
19949, 19954, 19963, 19975, 20015, 20048, 20072, 20135, 20154, 20169,
20174, 20179, 20248, 20249, 20298, 20299, 20323, 20349, 20372, 20383,
20447, 20462, 20510, 20520, 20579, 20607, 20684, 20694, 20724, 20727,
20759, 20773, 20786, 20809, 20816, 20820, 20836, 20904, 20906, 20947,
20975, 21056, 21095, 21121, 21147, 21182, 21216, 21238, 21269, 21348,
21358, 21387, 21426, 21431, 21447, 21497, 21515, 21519, 21541, 21542,
21642, 21729, 21778, 21844, 21889, 21906, 21933, 21936, 21939, 21960,
21969, 21970, 22089, 22160, 22164, 22186, 22225, 22230, 22238, 22265,
22269, 22298, 22309, 22324, 22329, 22330, 22383, 22403, 22409, 22435,
22499, 22511, 22558, 22567, 22597, 22598, 22640, 22675, 22676, 22721,
22734, 22783, 22786, 22802, 22862, 22908, 22933, 22959, 22965, 22968,
22972, 22985, 22988, 23001, 23011, 23026, 23043, 23074, 23125, 23128,
23140, 23214, 23235, 23263, 23266, 23393, 23437, 23446, 23490, 23492,
23503, 23516, 23521, 23563, 23600, 23619, 23635, 23655, 23659, 23705,
23730, 23760, 23764, 23793, 23823, 23856, 23859, 23865, 23915, 23941,
24048, 24056, 24074, 24110, 24111, 24120, 24130, 24149, 24357, 24367,
24399, 24420, 24427, 24441, 24456, 24458, 24583, 24615, 24657, 24658,
24668, 24677, 24719, 24735, 24740, 24743, 24748, 24774, 24796, 24798,
24834, 24857, 24873, 24901, 24926, 24940, 24954, 24965, 24970, 25028,
25042, 25097, 25105, 25148, 25225, 25239, 25251, 25276, 25290, 25319,
25331, 25379, 25562, 25575, 25615, 25620, 25653, 25756, 25778, 25797,
25801, 25806, 25812, 25839, 25855, 25906, 25908, 25946, 25955, 26056,
26138, 26140, 26152, 26165, 26196, 26203, 26206, 26217, 26242, 26256,
26271, 26294, 26307, 26314, 26334, 26412, 26445, 26466, 26490, 26524,

26526, 26530, 26543, 26586, 26643, 26656, 26703, 26706, 26730, 26750,
26832, 26857, 26910, 26971, 26980, 26981, 26989, 27032, 27084, 27095,
27116, 27121, 27149, 27185, 27224, 27238, 27243, 27249, 27264, 27268,
27272, 27307, 27340, 27346, 27449, 27489, 27497, 27536, 27577, 27578,
27666, 27709, 27745, 27747, 27759, 27786, 27827, 27832, 27842, 27883,
27911, 27912, 27999, 28000, 28011, 28086, 28134, 28135, 28140, 28146,
28172, 28186, 28217, 28233, 28254, 28266, 28279, 28341, 28392, 28442,
28526, 28560, 28611, 28628, 28834, 28879, 28903, 28912, 29021, 29037,
29042, 29048, 29065, 29123, 29144, 29212, 29214, 29229, 29230, 29256,
29327, 29353, 29356, 29362, 29364, 29394, 29395, 29462, 29497, 29512,
29517, 29534, 29589, 29593, 29595, 29630, 29633, 29651, 29675, 29686,
29705, 29739, 29750, 29803, 29840, 29863, 29905, 29918, 29944, 29950,
30001, 30037, 30076, 30094, 30105, 30117, 30126, 30130, 30210, 30261,
30262, 30311, 30319, 30322, 30324, 30328, 30469, 30528, 30556, 30576,
30577, 30603, 30612, 30631, 30670, 30682, 30702, 30704, 30741, 30749,
30753, 30777, 30785, 30847, 30879, 31006, 31023, 31024, 31028, 31052,
31062, 31109, 31118, 31152, 31196, 31234, 31266, 31299, 31314, 31319,
31325, 31363, 31403, 31405, 31433, 31448, 31508, 31526, 31615, 31619,
31660, 31689, 31694, 31760, 31773, 31800, 31817, 31838, 31856, 31973,
31980, 31997, 32015, 32017, 32040, 32074, 32077, 32083, 32111, 32128,
32135, 32176, 32202, 32244, 32265, 32322, 32348, 32362, 32378, 32416,
32444, 32480, 32482, 32526, 32564, 32565, 32571, 32579, 32597, 32647,
32687, 32712, 32781, 32783, 32791, 32810, 32812, 32866, 32894, 32895,
32915, 32946, 32957, 33086, 33090, 33132, 33191, 33194, 33227, 33247,
33260, 33265, 33302, 33303, 33322, 33342, 33362, 33398, 33423, 33439,
33455, 33515, 33529, 33572, 33631, 33635, 33667, 33712, 33726, 33730,
33736, 33749, 33807, 33933, 33935, 33936, 33987, 34037, 34050, 34115,
34243, 34248, 34259, 34264, 34274, 34295, 34334, 34372, 34450, 34452,
34468, 34506, 34526, 34551, 34561, 34593, 34597, 34609, 34613, 34614,
34681, 34722, 34800, 34801, 34837, 34866, 34877, 34882, 34903, 34908,
34911, 34912, 34925, 35099, 35131, 35136, 35147, 35150, 35170, 35224,
35228, 35268, 35295, 35323, 35341, 35342, 35344, 35350, 35369, 35370,
35398, 35418, 35458, 35467, 35475, 35481, 35483, 35510, 35515, 35525,
35526, 35614, 35618, 35647, 35664, 35671, 35673, 35740, 35746, 35753,
35817, 35826, 35827, 35845, 35851, 35853, 35856, 35862, 35863, 35865,
35887, 35908, 35915, 35924, 35961, 35984, 35986, 36012, 36032, 36100,
36116, 36272, 36294, 36421, 36463, 36494, 36506, 36606, 36612, 36622,
36645, 36651, 36666, 36668, 36687, 36712, 36735, 36830, 36875, 36896,
36916, 36938, 36953, 36959, 36973, 36998, 37006, 37039, 37073, 37092,
37103, 37111, 37154, 37171, 37179, 37215, 37220, 37226, 37270, 37285,
37322, 37332, 37413, 37421, 37428, 37435, 37504, 37551, 37561, 37574,
37586, 37610, 37631, 37653, 37661, 37689, 37697, 37699, 37706, 37709,
37773, 37826, 37841, 37843, 37849, 37861, 37914, 37939, 37961, 37996,
38044, 38045, 38052, 38065, 38066, 38108, 38127, 38155, 38196, 38206,
38266, 38279, 38293, 38299, 38343, 38344, 38449, 38513, 38549, 38551,
38660, 38706, 38713, 38740, 38753, 38801, 38803, 38886, 38888, 38913,
38932, 38949, 38986, 39037, 39042, 39048, 39069, 39096, 39107, 39134,
39137, 39153, 39162, 39182, 39223, 39236, 39253, 39294, 39322, 39329,
39332, 39333, 39345, 39347, 39356, 39361, 39367, 39368, 39422, 39454,

39502, 39529, 39542, 39587, 39629, 39640, 39655, 39666, 39677, 39701, 39719, 39727, 39750, 39781, 39791, 39817, 39838, 39874, 39889, 39892, 39895, 39908, 39937, 39967, 39976, 40015, 40032, 40084, 40092, 40106, 40107, 40129, 40141, 40225]}, 'Height': {'Outlier Count': 227, 'Outlier Indices': [6, 60, 442, 449, 457, 535, 687, 741, 791, 920, 1027, 1179, 1634, 1665, 1967, 1997, 2316, 2537, 2837, 3079, 3183, 3254, 3637, 3664, 3787, 3932, 3955, 4005, 4111, 4376, 4754, 4832, 4933, 4977, 5177, 5221, 5354, 5535, 5580, 5757, 5867, 6074, 6127, 6242, 6403, 6500, 6691, 6712, 6865, 7031, 7114, 7298, 7364, 7401, 7564, 7976, 8259, 8394, 8841, 8943, 8985, 9394, 9746, 9762, 9835, 10124, 10180, 10202, 10286, 10312, 10539, 10571, 10680, 10831, 10898, 11371, 11470, 11491, 11979, 11988, 12103, 12321, 12341, 12960, 12995, 13057, 13207, 13254, 13371, 13735, 13755, 13770, 13856, 14250, 14317, 14352, 14743, 14938, 15024, 15083, 15345, 15424, 15738, 15816, 15912, 16289, 16358, 16434, 16584, 16788, 16950, 17201, 18076, 18126, 18245, 18633, 18816, 19022, 19124, 19490, 19514, 19715, 19733, 19761, 19823, 19907, 20148, 20162, 20694, 20724, 20816, 20906, 21081, 21157, 21183, 21238, 21275, 21431, 21520, 21693, 21768, 21906, 22034, 22058, 22176, 22606, 22933, 22953, 22968, 23635, 23892, 24658, 24719, 24834, 25042, 25230, 25239, 25251, 26138, 26203, 26217, 26271, 26507, 26660, 26910, 27340, 27449, 27638, 27692, 27827, 28186, 28628, 28651, 28681, 29024, 29192, 29320, 29353, 29357, 29498, 29534, 29905, 30117, 30144, 30463, 30670, 30749, 31299, 31433, 31572, 31826, 32077, 32259, 32482, 32749, 33247, 33303, 33355, 33548, 33650, 34234, 34264, 34613, 34668, 34693, 35150, 35344, 35356, 35605, 35915, 36421, 37570, 37608, 38079, 38266, 38613, 38647, 38706, 38801, 38907, 39223, 39587, 39599, 39811, 39913, 39937, 40159]}, 'Weight': {'Outlier Count': 1116, 'Outlier Indices': [2, 21, 47, 87, 89, 91, 137, 247, 255, 260, 321, 330, 414, 419, 435, 513, 521, 576, 669, 683, 714, 802, 843, 905, 1007, 1011, 1029, 1039, 1066, 1085, 1119, 1123, 1157, 1167, 1191, 1202, 1216, 1217, 1225, 1241, 1321, 1387, 1391, 1448, 1458, 1505, 1507, 1575, 1646, 1651, 1660, 1704, 1748, 1820, 1841, 1858, 2000, 2001, 2049, 2051, 2114, 2141, 2212, 2276, 2310, 2350, 2352, 2443, 2545, 2568, 2569, 2608, 2619, 2697, 2769, 2840, 2848, 2874, 2947, 3011, 3033, 3034, 3075, 3168, 3216, 3269, 3302, 3307, 3360, 3396, 3406, 3415, 3429, 3438, 3442, 3470, 3480, 3512, 3528, 3606, 3710, 3722, 3741, 3747, 3766, 3810, 3841, 4016, 4081, 4093, 4098, 4108, 4147, 4292, 4419, 4598, 4652, 4675, 4781, 4787, 4826, 4837, 4886, 4897, 4927, 4966, 5000, 5016, 5055, 5071, 5088, 5124, 5136, 5209, 5212, 5221, 5233, 5292, 5393, 5394, 5406, 5415, 5457, 5459, 5555, 5576, 5599, 5655, 5672, 5711, 5723, 5725, 5747, 5856, 5937, 5939, 6070, 6201, 6249, 6261, 6276, 6353, 6362, 6374, 6392, 6418, 6514, 6520, 6551, 6616, 6752, 6776, 6804, 6833, 6861, 6870, 6884, 6893, 6906, 6921, 6924, 6957, 7012, 7016, 7050, 7105, 7119, 7182, 7230, 7261, 7283, 7318, 7371, 7388, 7407, 7430, 7514, 7531, 7585, 7615, 7659, 7680, 7696, 7789, 7804, 7836, 7846, 7847, 7886, 7905, 7951, 8023, 8132, 8140, 8257, 8294, 8300, 8357, 8504, 8610, 8642, 8649, 8678, 8695, 8746, 8798, 8851, 8863, 8872, 8898, 8947, 8948, 8980, 9001, 9053, 9091, 9093, 9148, 9209, 9221, 9267, 9275, 9309, 9472, 9531, 9546, 9555, 9563,

9588, 9616, 9630, 9651, 9685, 9700, 9706, 9793, 9814, 9826, 9843,
9881, 9886, 9959, 9966, 9977, 10032, 10106, 10117, 10140, 10155,
10159, 10198, 10253, 10257, 10293, 10297, 10368, 10383, 10407, 10462,
10509, 10549, 10629, 10635, 10661, 10668, 10669, 10687, 10721, 10884,
10894, 10915, 10933, 11050, 11117, 11172, 11183, 11213, 11248, 11326,
11334, 11387, 11457, 11467, 11488, 11548, 11558, 11583, 11622, 11704,
11732, 11783, 11786, 11805, 11834, 11914, 11956, 11999, 12008, 12132,
12146, 12158, 12324, 12393, 12400, 12407, 12461, 12485, 12510, 12523,
12545, 12557, 12608, 12611, 12616, 12629, 12642, 12687, 12742, 12786,
12797, 12858, 12918, 12943, 12944, 12954, 12983, 13013, 13059, 13092,
13119, 13130, 13136, 13143, 13224, 13348, 13363, 13407, 13450, 13496,
13548, 13605, 13630, 13651, 13694, 13732, 13807, 13831, 13918, 13996,
14070, 14085, 14144, 14174, 14201, 14235, 14244, 14305, 14344, 14387,
14429, 14466, 14506, 14512, 14515, 14528, 14564, 14602, 14627, 14635,
14645, 14723, 14731, 14734, 14760, 14766, 14794, 14797, 14818, 14833,
14859, 15055, 15061, 15209, 15215, 15218, 15251, 15284, 15320, 15333,
15404, 15458, 15499, 15515, 15516, 15569, 15582, 15627, 15629, 15632,
15678, 15752, 15819, 15906, 15923, 15929, 15932, 15933, 15936, 15944,
15995, 16010, 16020, 16044, 16047, 16052, 16160, 16183, 16208, 16214,
16220, 16227, 16251, 16252, 16269, 16286, 16356, 16382, 16388, 16432,
16529, 16534, 16540, 16564, 16619, 16630, 16641, 16643, 16657, 16678,
16737, 16762, 16791, 16830, 16857, 16871, 16901, 16965, 16972, 16999,
17034, 17053, 17059, 17073, 17078, 17097, 17128, 17201, 17285, 17431,
17432, 17442, 17517, 17542, 17561, 17713, 17731, 17753, 17758, 17766,
17784, 17820, 17866, 18033, 18039, 18055, 18077, 18126, 18150, 18215,
18242, 18283, 18379, 18395, 18413, 18517, 18523, 18562, 18563, 18568,
18626, 18651, 18687, 18812, 18825, 18845, 18951, 18958, 18969, 18980,
19034, 19075, 19076, 19101, 19147, 19169, 19180, 19203, 19287, 19297,
19338, 19364, 19386, 19404, 19424, 19425, 19434, 19578, 19623, 19696,
19704, 19794, 19796, 19822, 19954, 19963, 19975, 20048, 20072, 20154,
20169, 20241, 20248, 20249, 20298, 20299, 20342, 20349, 20372, 20383,
20438, 20579, 20607, 20653, 20684, 20734, 20786, 20820, 20904, 20927,
20947, 21138, 21147, 21182, 21216, 21267, 21269, 21348, 21358, 21369,
21387, 21426, 21447, 21519, 21542, 21642, 21729, 21778, 21880, 21939,
21960, 21970, 21996, 22078, 22104, 22160, 22225, 22230, 22238, 22265,
22269, 22309, 22324, 22328, 22383, 22403, 22409, 22435, 22452, 22470,
22504, 22558, 22567, 22597, 22640, 22675, 22676, 22721, 22734, 22783,
22786, 22802, 22817, 22846, 22862, 22897, 22933, 22959, 22972, 22985,
23026, 23074, 23125, 23140, 23225, 23235, 23263, 23266, 23327, 23393,
23431, 23490, 23503, 23563, 23655, 23659, 23730, 23760, 23793, 23865,
23915, 23941, 24029, 24074, 24110, 24111, 24130, 24149, 24357, 24367,
24399, 24441, 24547, 24583, 24626, 24629, 24657, 24668, 24735, 24748,
24774, 24798, 24814, 24825, 24873, 24926, 24940, 24954, 24965, 25028,
25096, 25148, 25276, 25331, 25379, 25575, 25590, 25615, 25620, 25756,
25797, 25801, 25806, 25812, 25839, 25855, 25906, 25908, 25946, 26056,
26152, 26165, 26196, 26206, 26256, 26294, 26298, 26307, 26314, 26348,
26412, 26466, 26490, 26494, 26524, 26526, 26530, 26586, 26703, 26706,
26712, 26832, 26862, 26893, 26980, 26981, 26989, 27032, 27035, 27084,
27087, 27095, 27116, 27121, 27185, 27205, 27217, 27238, 27249, 27268,

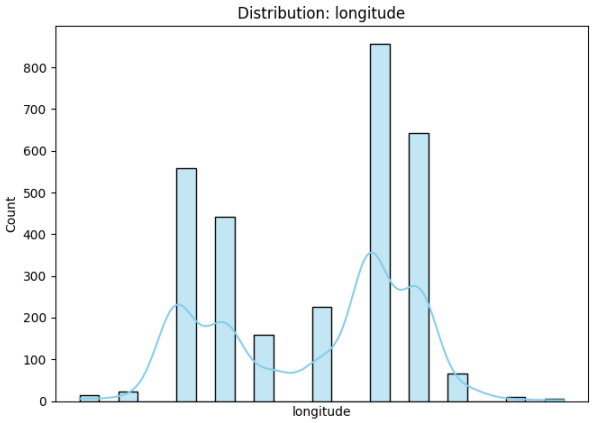
```
27272, 27277, 27303, 27307, 27329, 27346, 27369, 27489, 27547, 27577,
27578, 27637, 27666, 27745, 27747, 27763, 27786, 27832, 27883, 27911,
27912, 28000, 28086, 28135, 28140, 28146, 28172, 28217, 28233, 28254,
28279, 28359, 28384, 28392, 28424, 28433, 28442, 28464, 28560, 28767,
28834, 28903, 28912, 29021, 29042, 29048, 29123, 29144, 29198, 29212,
29214, 29229, 29230, 29256, 29275, 29327, 29356, 29362, 29394, 29395,
29462, 29497, 29512, 29517, 29589, 29593, 29630, 29651, 29686, 29705,
29739, 29750, 29803, 29840, 29863, 29905, 29918, 29944, 30001, 30076,
30126, 30130, 30149, 30210, 30261, 30311, 30320, 30322, 30324, 30576,
30577, 30603, 30612, 30631, 30704, 30753, 30772, 30777, 30785, 30847,
30895, 30920, 30930, 30964, 31006, 31024, 31052, 31109, 31118, 31203,
31214, 31234, 31266, 31314, 31319, 31363, 31394, 31405, 31448, 31508,
31593, 31615, 31689, 31760, 31793, 31800, 31817, 31884, 31943, 31960,
31966, 31973, 31980, 32015, 32017, 32040, 32074, 32076, 32111, 32176,
32265, 32283, 32348, 32362, 32378, 32435, 32480, 32526, 32571, 32579,
32647, 32687, 32712, 32717, 32783, 32812, 32894, 32931, 32935, 32945,
32946, 32957, 33053, 33086, 33194, 33239, 33260, 33390, 33423, 33455,
33515, 33525, 33631, 33667, 33712, 33730, 33749, 33800, 33896, 33933,
33936, 33946, 34050, 34108, 34164, 34203, 34229, 34243, 34259, 34334,
34372, 34452, 34468, 34473, 34526, 34546, 34551, 34577, 34584, 34597,
34609, 34614, 34681, 34800, 34801, 34866, 34877, 34882, 34884, 34908,
34912, 34917, 35119, 35136, 35147, 35268, 35295, 35342, 35350, 35369,
35418, 35432, 35458, 35467, 35475, 35481, 35483, 35510, 35526, 35564,
35598, 35618, 35643, 35664, 35671, 35673, 35740, 35746, 35817, 35839,
35845, 35851, 35853, 35856, 35863, 35908, 35984, 35986, 36012, 36032,
36100, 36128, 36208, 36220, 36272, 36281, 36294, 36421, 36463, 36494,
36606, 36622, 36666, 36701, 36712, 36735, 36875, 36916, 36938, 36953,
36959, 36998, 37006, 37039, 37051, 37073, 37111, 37140, 37154, 37164,
37179, 37215, 37352, 37421, 37435, 37441, 37504, 37530, 37561, 37592,
37611, 37631, 37689, 37692, 37699, 37706, 37826, 37849, 37861, 37914,
37939, 37961, 37996, 38052, 38065, 38066, 38108, 38113, 38127, 38196,
38206, 38279, 38293, 38344, 38449, 38454, 38513, 38551, 38579, 38605,
38643, 38660, 38740, 38753, 38803, 38810, 38886, 38903, 38913, 38932,
38949, 38986, 38994, 39037, 39069, 39096, 39137, 39182, 39253, 39294,
39322, 39332, 39333, 39347, 39368, 39490, 39529, 39542, 39550, 39581,
39612, 39640, 39655, 39666, 39677, 39687, 39701, 39727, 39750, 39781,
39791, 39838, 39874, 39889, 39892, 39895, 39908, 39976, 40015, 40084,
40092, 40106, 40107, 40135, 40141, 40171, 40225]}, 'Billing_Amount':
{'Outlier Count': 0, 'Outlier Indices': []}, 'Room_Number': {'Outlier
Count': 0, 'Outlier Indices': []}, 'Days_Hospitalised': {'Outlier
Count': 0, 'Outlier Indices': []}}
Recommendations:
 ["□ Column 'BMI' is highly skewed (skew=1.41). Consider
transformation.", "□ Column 'Weight' is highly skewed (skew=1.06).
Consider transformation."]
```

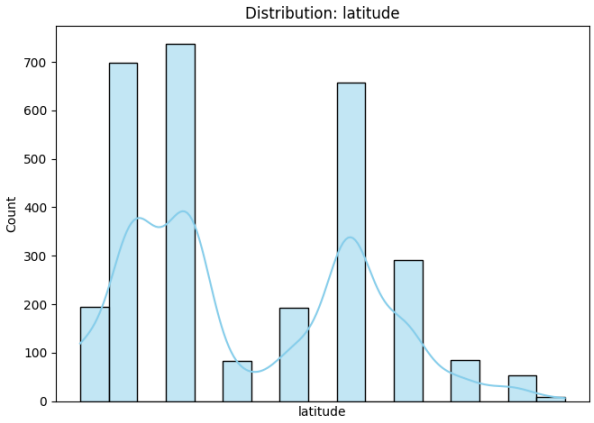
1. Multi-Panel Visualization Dashboard Build a dashboard using matplotlib and seaborn that: - Supports different data types - Automatically selects suitable chart types -

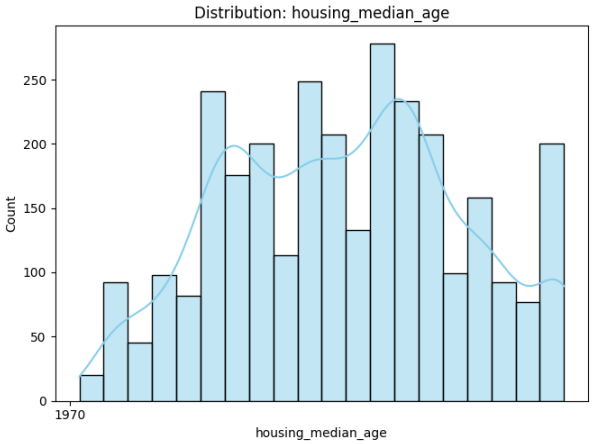
Displays multiple panels for comparison - Includes legends, titles, and annotations for clar

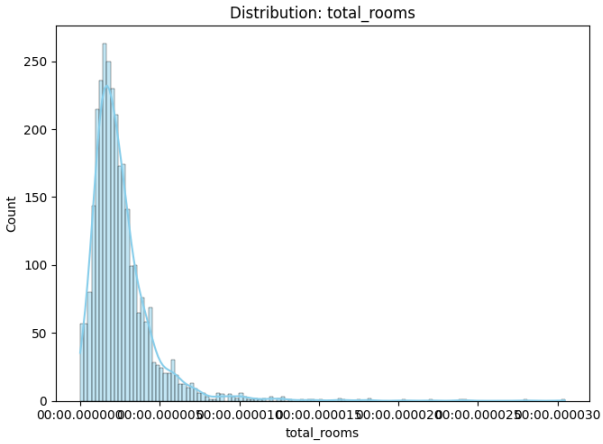
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 📥 Step 1: Load CSV
df = pd.read_csv("/content/sample_data/california_housing_test.csv")
# Replace with any file path
df.columns = df.columns.str.strip().str.replace(' ', '_')  # Clean
column names

# 🧹 Step 2: Basic Cleaning
threshold = len(df) * 0.5
df = df.dropna(thresh=threshold, axis=1)

for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        df[col] = df[col].fillna(df[col].median())
    else:
        df[col] = df[col].fillna("Unknown")

# 🔍 Step 3: Detect Column Types
numeric_cols = df.select_dtypes(include='number').columns.tolist()
categorical_cols = df.select_dtypes(include='object').columns.tolist()
datetime_cols = []

for col in df.columns:
    try:
        df[col] = pd.to_datetime(df[col])
        datetime_cols.append(col)
    except:
        continue

# 📐 Step 4: Setup Plot Grid Dynamically
total_plots = min(10, len(numeric_cols) + len(categorical_cols) +
len(datetime_cols))
rows = (total_plots + 1) // 2
fig, axes = plt.subplots(rows, 2, figsize=(14, 5 * rows))
axes = axes.flatten()
plot_idx = 0

# 📊 Step 5: Plot Numeric Distributions
for col in numeric_cols:
    if plot_idx >= len(axes):
        break
    sns.histplot(df[col], kde=True, ax=axes[plot_idx],
color='skyblue')
    axes[plot_idx].set_title(f"Distribution: {col}")
```

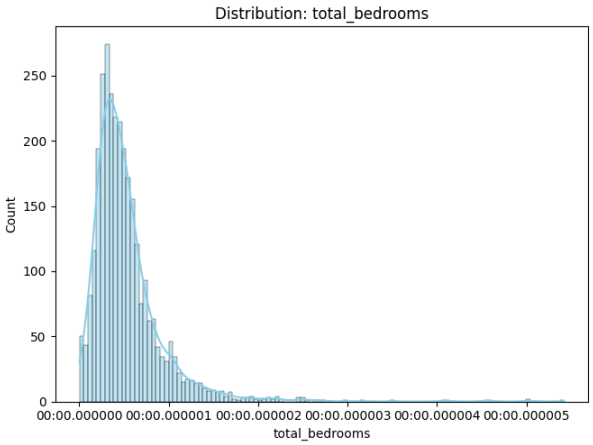
```
    plot_idx += 1

# 🔹 Step 6: Plot Categorical Counts
for col in categorical_cols:
    if plot_idx >= len(axes):
        break
    sns.countplot(x=col, data=df, ax=axes[plot_idx], palette='Set2')
    axes[plot_idx].set_title(f"Countplot: {col}")
    axes[plot_idx].tick_params(axis='x', rotation=45)
    plot_idx += 1

# 🔹 Step 7: Plot Time Series Trends
for date_col in datetime_cols:
    for num_col in numeric_cols:
        if plot_idx >= len(axes):
            break
        df_sorted = df.sort_values(by=date_col)
        sns.lineplot(x=date_col, y=num_col, data=df_sorted,
ax=axes[plot_idx])
        axes[plot_idx].set_title(f"{num_col} over {date_col}")
        plot_idx += 1
    if plot_idx >= len(axes):
        break

# 🔹 Final Touch
plt.tight_layout()
plt.show()
```

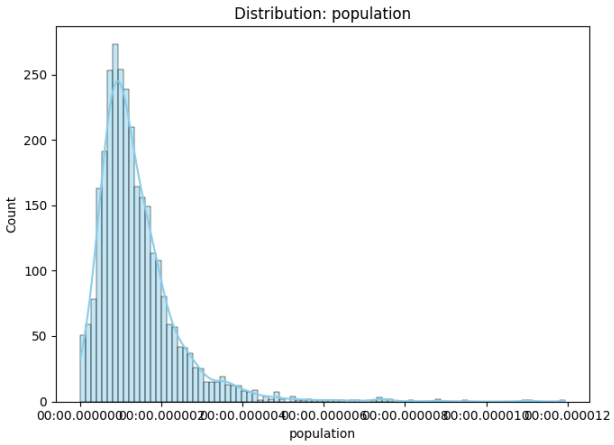

Distribution: longitude
Count
longitude

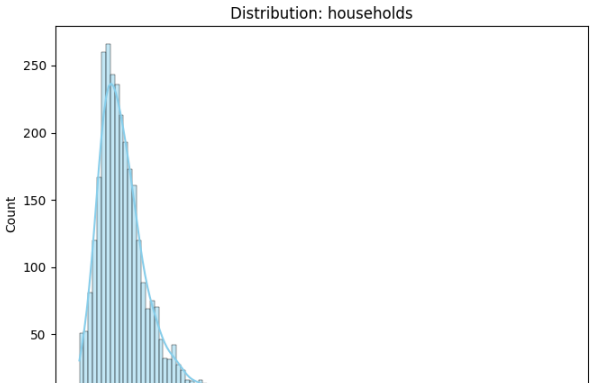

Distribution: latitude
Count
latitude

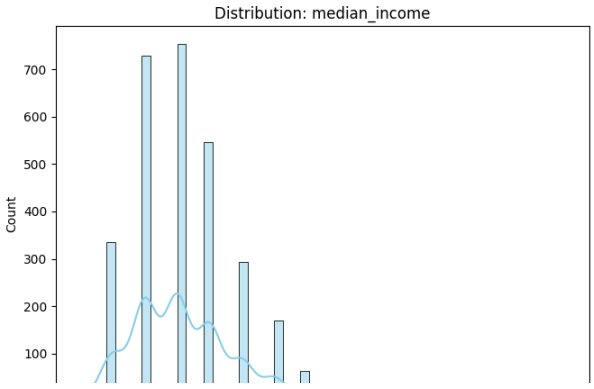

Distribution: housing_median_age
Count
1970
housing_median_age


Distribution: total_rooms
Count
total_rooms


Distribution: total_bedrooms
Count
total_bedrooms


Distribution: population
Count
population


Distribution: households
Count

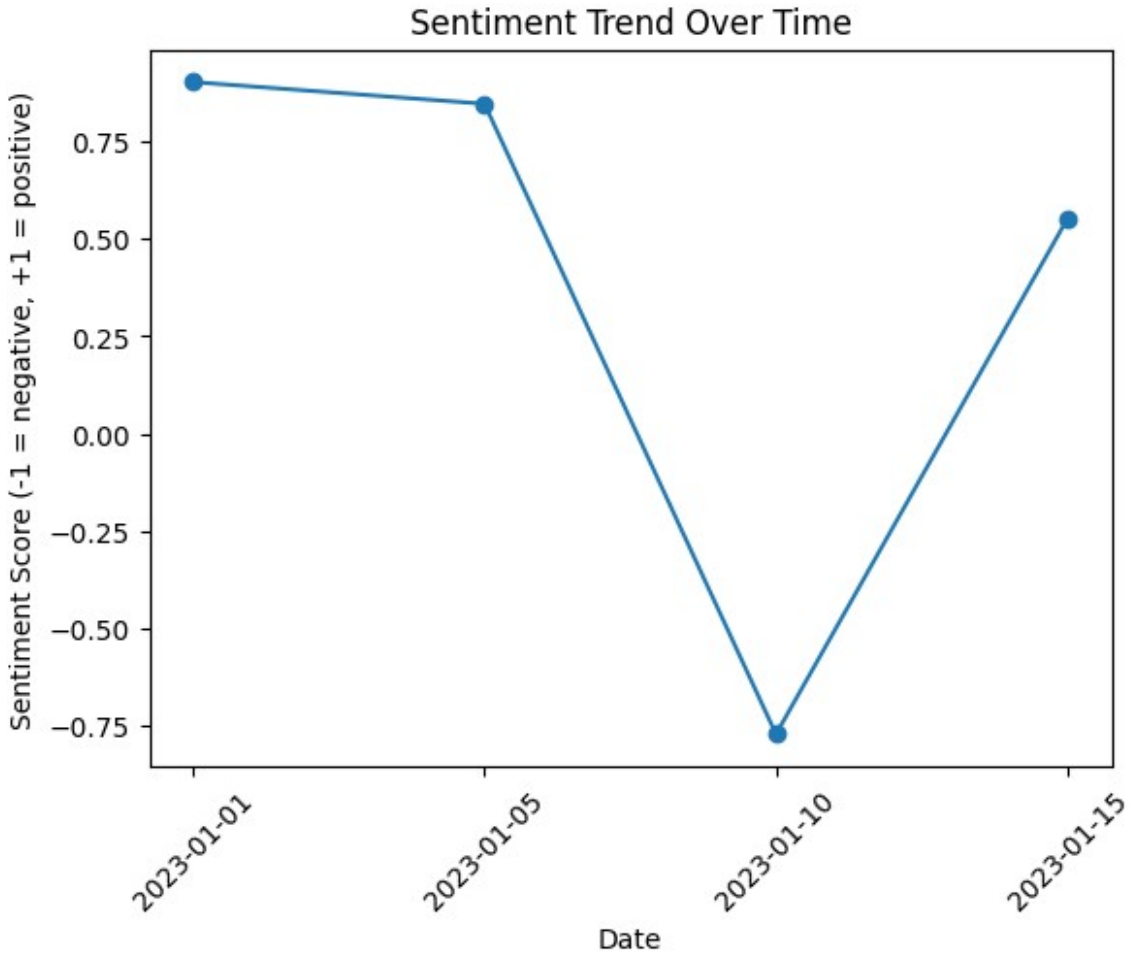

Distribution: median_income
Count

1. NLTK-Based Text Analysis System Develop a script that: - Preprocesses a corpus (tokenization, stopword removal, lemmatization) - Analyzes sentiment trends over time - Detects frequently occurring topics or keywords - Calculates basic readability metrics

```
# 🔹 Install NLTK resources (only needed once in Colab/Notebook)
import nltk
nltk.download("punkt")
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("vader_lexicon")

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.sentiment import SentimentIntensityAnalyzer
from collections import Counter
import matplotlib.pyplot as plt

# -----------------------
# 1. Sample Corpus
# -----------------------
corpus = [
    {"date": "2023-01-01", "text": "I love learning AI. It is amazing
and powerful!"},
    {"date": "2023-01-05", "text": "Data science is difficult, but
rewarding when you succeed."},
    {"date": "2023-01-10", "text": "Sometimes coding can be
frustrating. Bugs are so annoying!"},
    {"date": "2023-01-15", "text": "Streamlit makes building web apps
very simple and fun."},
]

# -----------------------
# 2. Preprocessing
# -----------------------
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    tokens = word_tokenize(text.lower())                    # tokenize
    tokens = [w for w in tokens if w.isalpha()]             # remove
punctuation/numbers
    tokens = [w for w in tokens if w not in stop_words]     # remove
stopwords
    tokens = [lemmatizer.lemmatize(w) for w in tokens]      # lemmatize
    return tokens

preprocessed_corpus = [preprocess(doc["text"]) for doc in corpus]

print("🔹 Preprocessed Corpus:", preprocessed_corpus)
```

```
# ----------------------
# 3. Sentiment Analysis
# ----------------------
sia = SentimentIntensityAnalyzer()

sentiments = []
for doc in corpus:
    score = sia.polarity_scores(doc["text"])
    sentiments.append({"date": doc["date"], "sentiment":
score["compound"]})

print("\n Sentiment Scores Over Time:", sentiments)

# Plot sentiment trend
dates = [s["date"] for s in sentiments]
scores = [s["sentiment"] for s in sentiments]

plt.plot(dates, scores, marker="o")
plt.title("Sentiment Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Sentiment Score (-1 = negative, +1 = positive)")
plt.xticks(rotation=45)
plt.show()

# ----------------------
# 4. Keyword Frequency
# ----------------------
all_tokens = [word for doc in preprocessed_corpus for word in doc]
freq_dist = Counter(all_tokens)
print("\n Most Common Keywords:", freq_dist.most_common(5))

# ----------------------
# 5. Readability Metrics
# ----------------------
def readability_metrics(text):
    sentences = sent_tokenize(text)
    words = word_tokenize(text)
    avg_sentence_length = len(words) / len(sentences)
    avg_word_length = sum(len(w) for w in words if w.isalpha()) /
len(words)
    return {
        "avg_sentence_length": avg_sentence_length,
        "avg_word_length": avg_word_length
    }

metrics = [readability_metrics(doc["text"]) for doc in corpus]
print("\n Readability Metrics:", metrics)

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package vader_lexicon to /root/nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!

 Preprocessed Corpus: [['love', 'learning', 'ai', 'amazing',
'powerful'], ['data', 'science', 'difficult', 'rewarding', 'succeed'],
['sometimes', 'coding', 'frustrating', 'bug', 'annoying'],
['streamlit', 'make', 'building', 'web', 'apps', 'simple', 'fun']]

 Sentiment Scores Over Time: [{'date': '2023-01-01', 'sentiment':
0.902}, {'date': '2023-01-05', 'sentiment': 0.8462}, {'date': '2023-
01-10', 'sentiment': -0.7706}, {'date': '2023-01-15', 'sentiment':
0.552}]
```




```
 Most Common Keywords: [('love', 1), ('learning', 1), ('ai', 1),
('amazing', 1), ('powerful', 1)]
```

```
 Readability Metrics: [{'avg_sentence_length': 5.5,
'avg_word_length': 3.3636363636363638}, {'avg_sentence_length': 11.0,
'avg_word_length': 4.363636363636363}, {'avg_sentence_length': 5.5,
'avg_word_length': 4.363636363636363}, {'avg_sentence_length': 10.0,
'avg_word_length': 4.5}]
```

1. pandas Data Transformation Pipeline Implement a configurable pipeline that: - Handles missing values with various strategies - Detects and removes outliers - Performs feature scaling (normalization, standardization) - Logs each transformation step

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import logging

# Setup logging
logging.basicConfig(filename='transformation_log.txt',
level=logging.INFO, format='%(asctime)s - %(message)s')

# Sample configuration dictionary
config = {
    'missing_value_strategy': 'mean',       # Options: 'mean',
'median', 'drop'
    'outlier_detection': True,              # Enable or disable
outlier removal
    'scaling': 'standard'                   # Options: 'minmax',
'standard'
}

def handle_missing_values(df, strategy):
    logging.info(f"Handling missing values using strategy:
{strategy}")
    if strategy == 'mean':
        return df.fillna(df.mean(numeric_only=True))
    elif strategy == 'median':
        return df.fillna(df.median(numeric_only=True))
    elif strategy == 'drop':
        return df.dropna()
    else:
        logging.warning("Invalid missing value strategy. No changes
applied.")
        return df

def remove_outliers(df):
    logging.info("Removing outliers using IQR method")
    numeric_cols = df.select_dtypes(include=np.number).columns
    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
```

```
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
    return df

def scale_features(df, method):
    logging.info(f"Scaling features using method: {method}")
    numeric_cols = df.select_dtypes(include=np.number).columns
    scaler = MinMaxScaler() if method == 'minmax' else
StandardScaler()
    df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
    return df

def transform_pipeline(df, config):
    logging.info("Starting data transformation pipeline")

    # Step 1: Handle missing values
    df = handle_missing_values(df, config['missing_value_strategy'])

    # Step 2: Remove outliers
    if config['outlier_detection']:
        df = remove_outliers(df)

    # Step 3: Scale features
    df = scale_features(df, config['scaling'])

    logging.info("Data transformation pipeline completed")
    return df

# Example usage with dummy data
if __name__ == "__main__":

    df = pd.read_csv("/content/hospital.csv")  # Your actual dataset
    transformed_df = transform_pipeline(df, config)
    print(transformed_df.head())
    print("Original shape:", df.shape)
df = handle_missing_values(df, config['missing_value_strategy'])
print("After missing value handling:", df.shape)
if config['outlier_detection']:
    df = remove_outliers(df)
    print("After outlier removal:", df.shape)
df = scale_features(df, config['scaling'])
print("After scaling:", df.shape)
```

```
                          Admission_ID             Name       Age
Gender  \
```

```
0  4ba0c14a-62d9-43ae-9f9f-6ba24017f808     Tara Thomas -1.362540
Male
1  e00b340c-279f-4898-9fbc-054552ad6095    Laura Rivera  1.294951
Female
3  ed5088d6-d6a1-45f9-b994-db5a178cc659    Angela Yates  0.528367
Female
4  be20cd2f-42b8-4c83-8871-6adf6a1254d7     Cathy Small -0.033794
Female
5  46770815-f994-42f6-a2f4-e9f637f2c6e9  Kimberly Stone  0.835001
Male

        BMI          Ethnicity    Height    Weight Blood_Type  \
0  0.233979  African American  0.971443  0.722383         B+
1  0.065170         Caucasian  0.777325  0.425858         A-
3 -0.199877         Caucasian  0.292030 -0.068351        AB+
4 -0.977662         Caucasian -1.163857 -1.353292         O-
5 -1.553507          Hispanic  0.292030 -1.353292         B-

  Medical_Condition  ...            Doctor                    Hospital
\
0            Cancer  ...       Susan Bauer                    PLC Ford

1      Hypertension  ...      Amanda Clark                   Henry PLC

3            Asthma  ...  Kimberly Hammond  French Wells, and Schmidt

4            Asthma  ...       Wendy Glenn    Brown, and Jones Weaver

5      Hypertension  ...      Robert Young   Leonard and Olson Lynch,


  Insurance_Provider Billing_Amount  Room_Number  Admission_Type  \
0         Blue Cross      -1.095503     0.876877        Elective
1              Aetna      -0.718423    -0.709682          Urgent
3              Cigna       1.146935    -1.359911          Urgent
4         Blue Cross       0.087121     0.868208        Elective
5         Blue Cross      -1.107228     0.478070       Emergency

  Discharge_Date  Medication  Test_Results Days_Hospitalised
0     2019-07-21     Lipitor        Normal         -0.638353
1     2023-06-19  Penicillin        Normal          0.747788
3     2024-01-06     Aspirin  Inconclusive         -0.176306
4     2024-01-19   Ibuprofen        Normal          1.325346
5     2020-06-08  Penicillin  Inconclusive          0.978811

[5 rows x 21 columns]
Original shape: (40235, 21)
After missing value handling: (40235, 21)
After outlier removal: (38151, 21)
After scaling: (38151, 21)
```

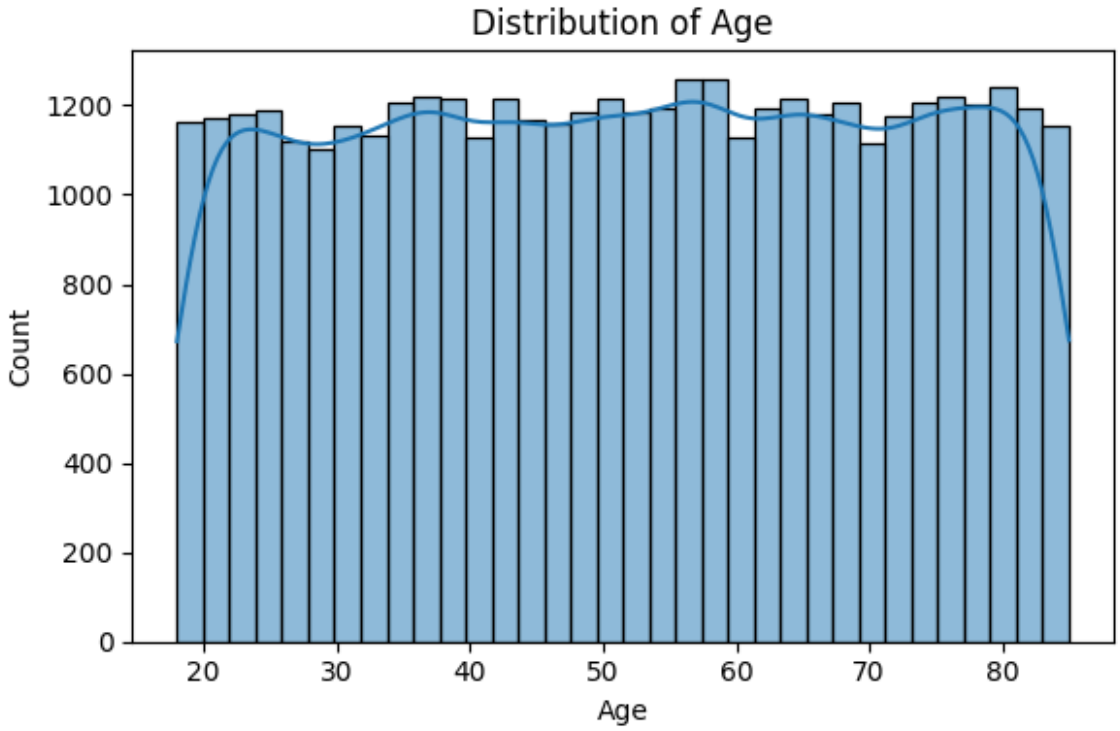
1. Visualization Automation Tool Create a script that: - Accepts any dataset as input - Generates a complete exploratory data analysis (EDA) report - Automatically selects visualization types - Includes statistical summaries alongside plot

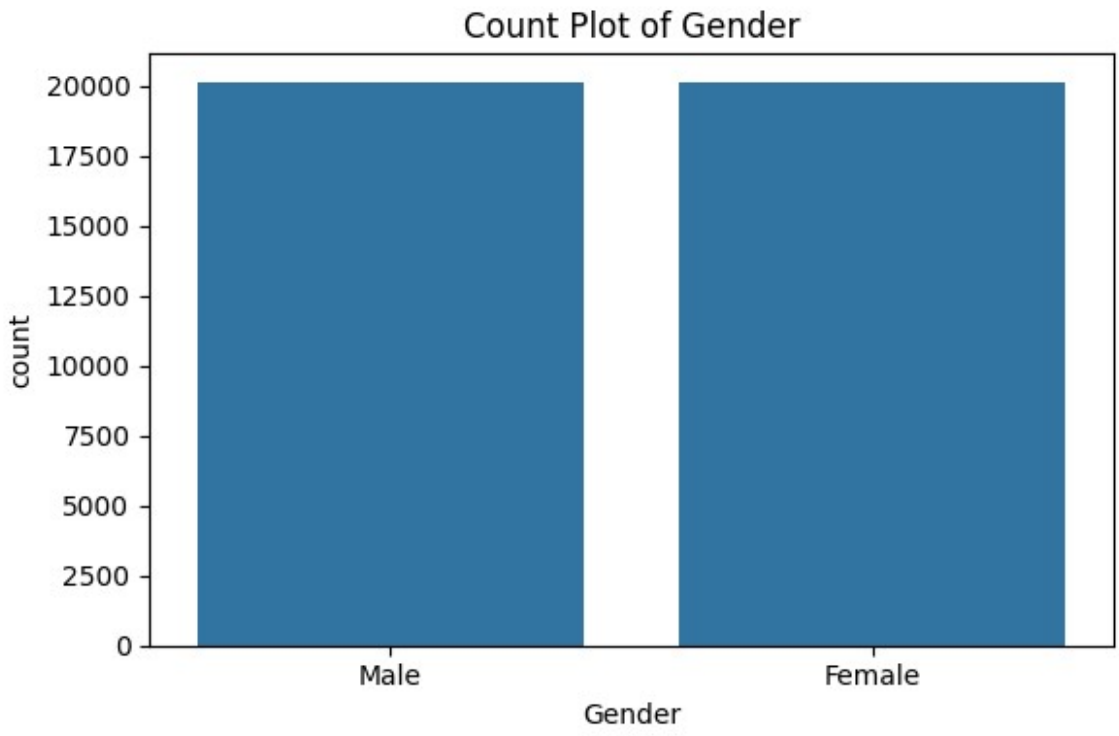
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import os
import numpy as np
from IPython.display import display, Image

#  Optional: Show plots inline in notebooks
%matplotlib inline

#  Create output folder
def create_output_folder(folder_name="eda_report"):
    if not os.path.exists(folder_name):
        os.makedirs(folder_name)
    return folder_name

#  Save summary statistics
def save_summary_stats(df, folder):
    df.info(buf=open(f"{folder}/info.txt", "w"))
    df.describe().to_csv(f"{folder}/describe.csv")
    df.isnull().sum().to_csv(f"{folder}/null_counts.csv")

#  Auto-generate visualizations
def generate_visualizations(df, folder):
    for col in df.columns:
        plt.figure(figsize=(6, 4))
        if pd.api.types.is_numeric_dtype(df[col]):
            sns.histplot(df[col].dropna(), kde=True)
            plt.title(f"Distribution of {col}")
        elif isinstance(df[col].dtype, pd.CategoricalDtype) or
df[col].nunique() < 20:
            sns.countplot(x=col, data=df)
            plt.title(f"Count Plot of {col}")
        else:
            continue
        plt.tight_layout()
        plt.savefig(f"{folder}/{col}_plot.png")
        plt.show()
        plt.close()

#  Correlation heatmap
def plot_correlation_heatmap(df, folder):
    numeric_df = df.select_dtypes(include=np.number)
    if numeric_df.shape[1] >= 2:
        plt.figure(figsize=(10, 8))
        sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
        plt.title("Correlation Heatmap")
```

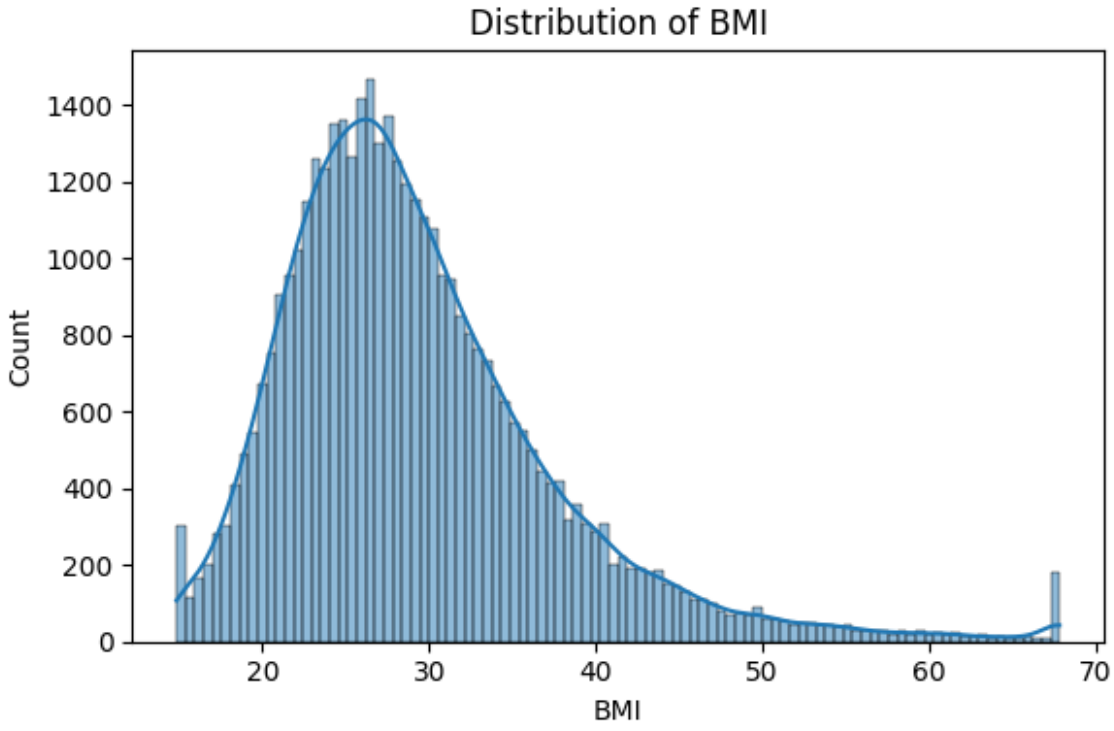
```
        plt.tight_layout()
        plt.savefig(f"{folder}/correlation_heatmap.png")
        plt.show()
        plt.close()

#  Boxplots for numeric vs categorical
def plot_boxplots(df, folder):
    numeric_cols = df.select_dtypes(include=np.number).columns
    cat_cols = df.select_dtypes(exclude=np.number).columns
    for num in numeric_cols:
        for cat in cat_cols:
            if df[cat].nunique() < 10:
                plt.figure(figsize=(6, 4))
                sns.boxplot(x=cat, y=num, data=df)
                plt.title(f"{num} by {cat}")
                plt.tight_layout()
                plt.savefig(f"{folder}/boxplot_{num}_by_{cat}.png")
                plt.show()
                plt.close()

#  Pairplot for numeric features
def plot_pairplot(df, folder):
    numeric_df = df.select_dtypes(include=np.number)
    if numeric_df.shape[1] >= 2:
        sampled = numeric_df.iloc[:, :5]  # limit to 5 features
        sns.pairplot(sampled.dropna())
        plt.savefig(f"{folder}/pairplot.png")
        plt.show()
        plt.close()

#  Preview all saved plots
def preview_all_plots(folder="eda_report"):
    for file in sorted(os.listdir(folder)):
        if file.endswith(".png"):
            print(f" {file}")
            display(Image(filename=os.path.join(folder, file)))

#  Main function
def run_eda_pipeline(file_path):
    print(f" Loading dataset from: {file_path}")
    df = pd.read_csv(file_path)
    folder = create_output_folder()
    print(" Saving summary statistics...")
    save_summary_stats(df, folder)
    print(" Generating visualizations...")
    generate_visualizations(df, folder)
    print(" Creating correlation heatmap...")
    plot_correlation_heatmap(df, folder)
    print(" Creating boxplots...")
    plot_boxplots(df, folder)
```

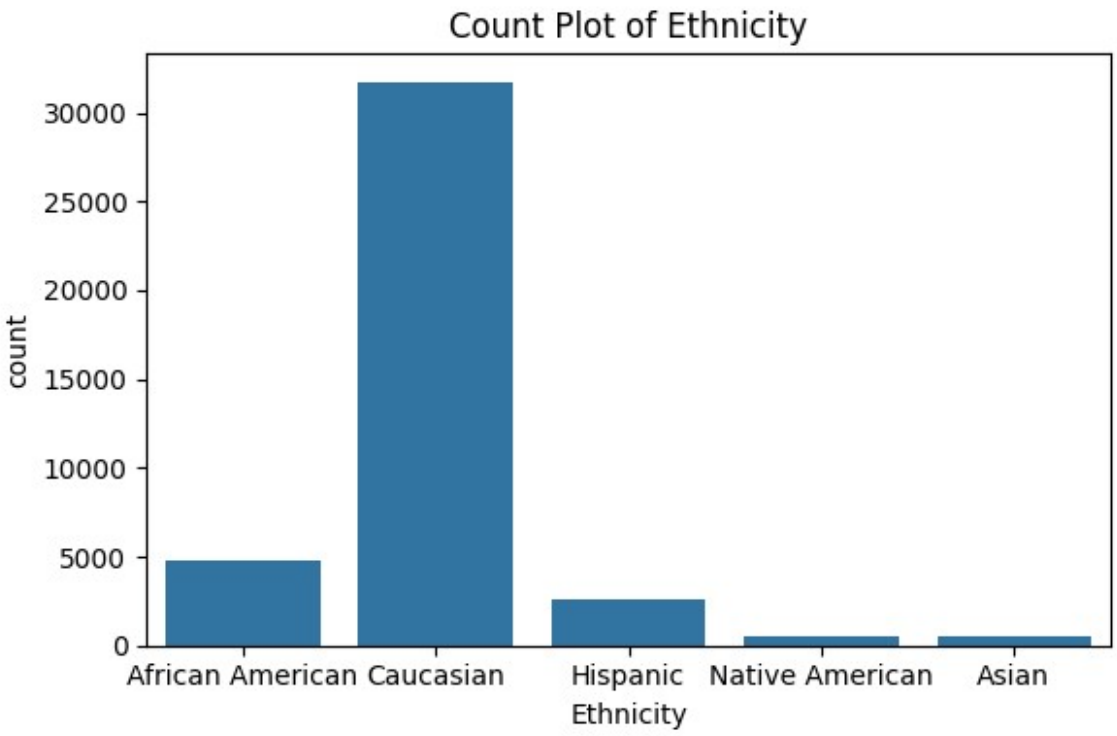
```
    print(" Creating pairplot...")
    plot_pairplot(df, folder)
    print(f" EDA report saved in '{folder}' folder.")
    preview_all_plots(folder)

#  Example usage
run_eda_pipeline("/content/hospital.csv")
```

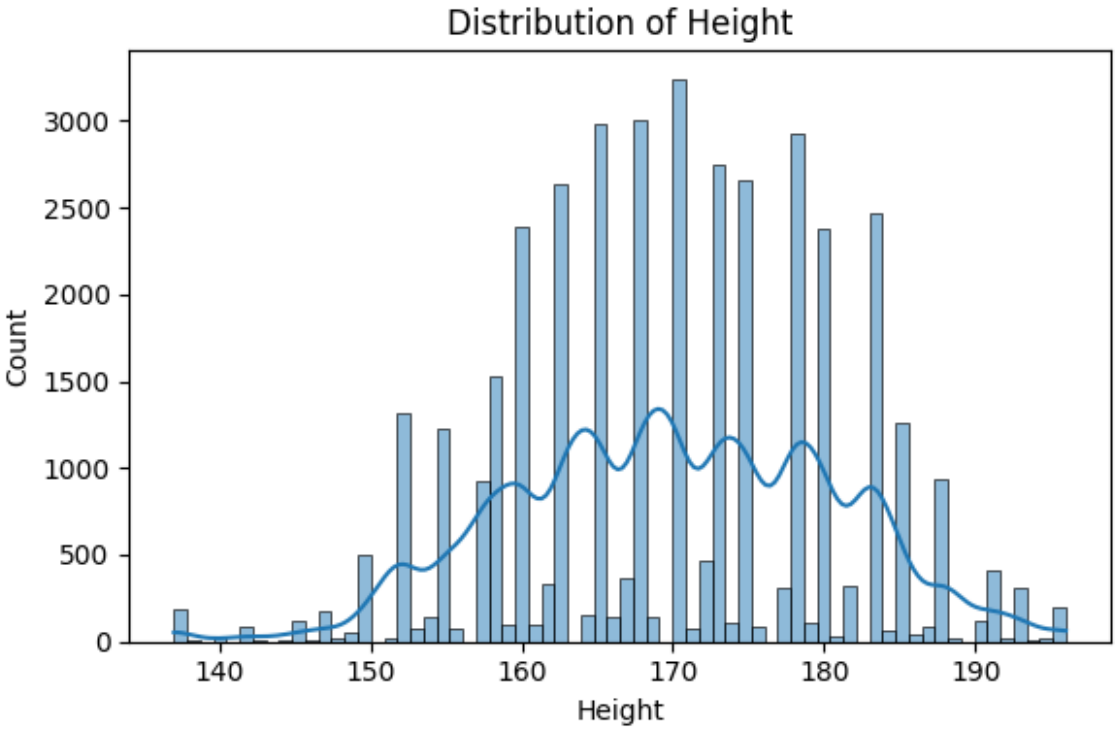
```
 Loading dataset from: /content/hospital.csv
 Saving summary statistics...
 Generating visualizations...

<Figure size 600x400 with 0 Axes>

<Figure size 600x400 with 0 Axes>
```

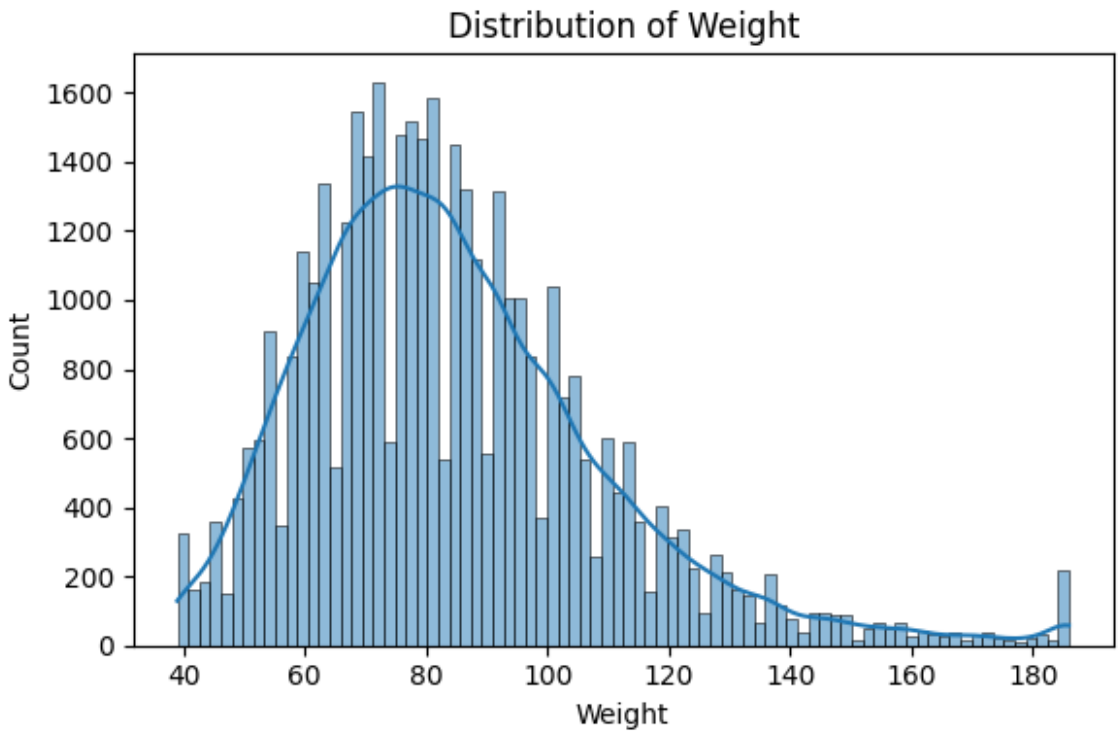


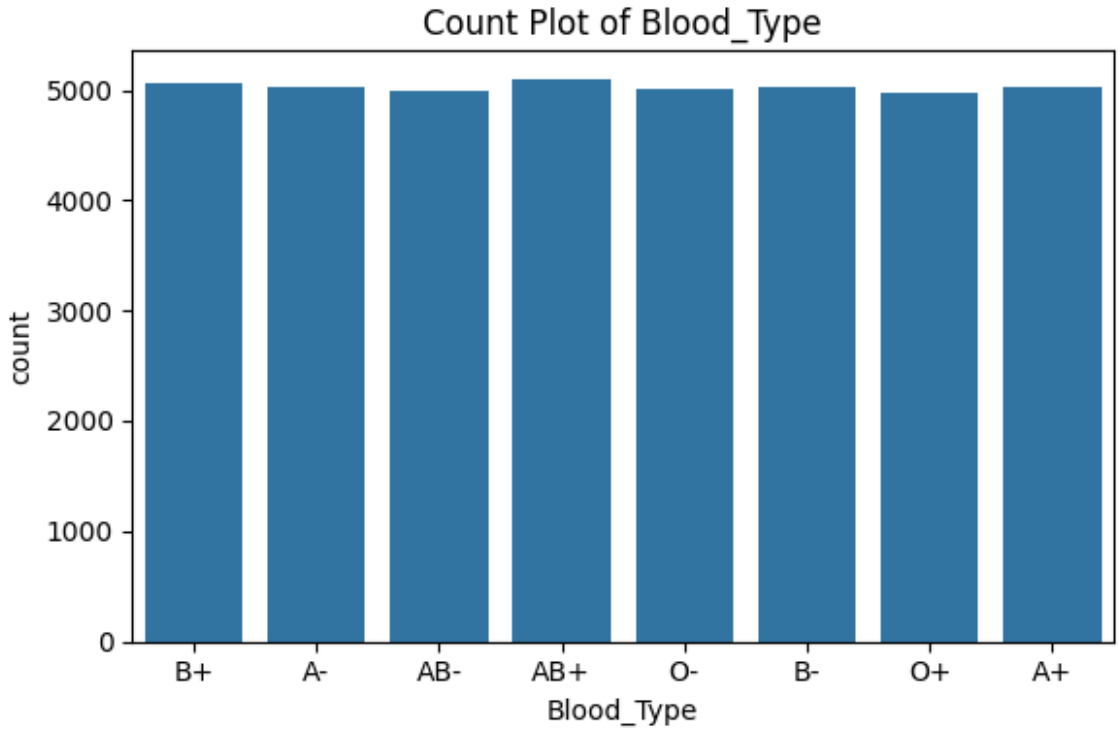

Count Plot of Gender
20000
17500
15000
12500
10000
7500
5000
2500
0
count
Male
Female
Gender

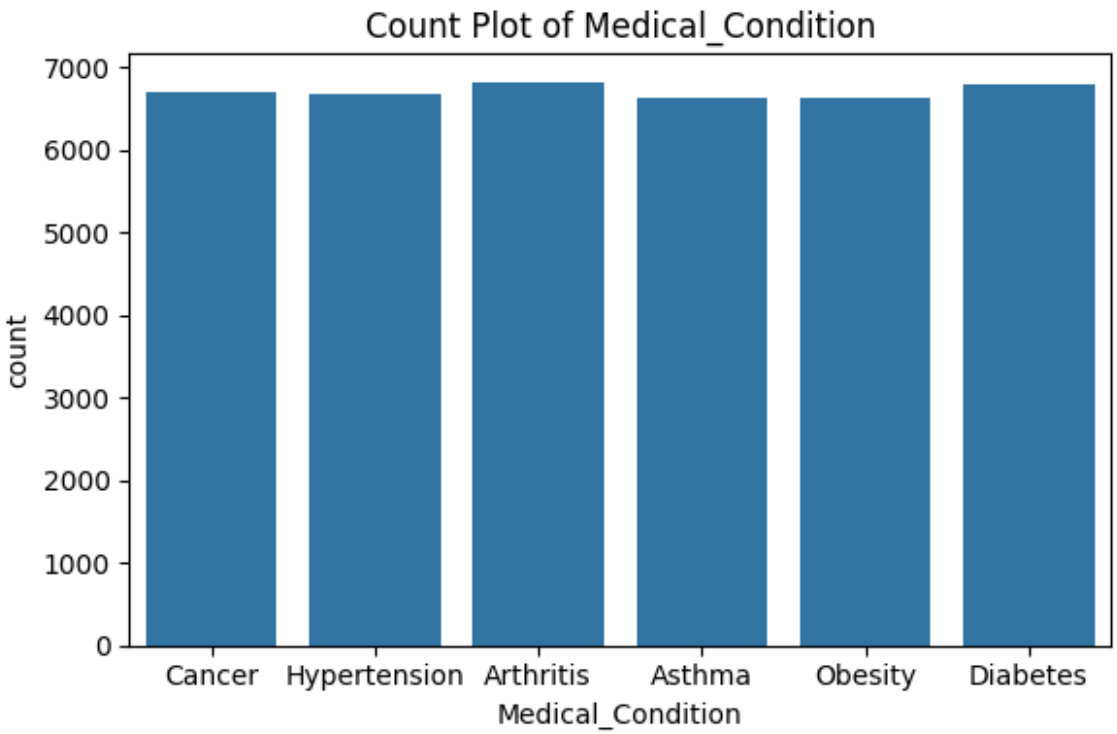

Distribution of BMI
1400
1200
1000
800
600
400
200
0
Count
20
30
40
50
60
70
BMI


Count Plot of Ethnicity
30000
25000
20000
15000
10000
5000
0
count
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

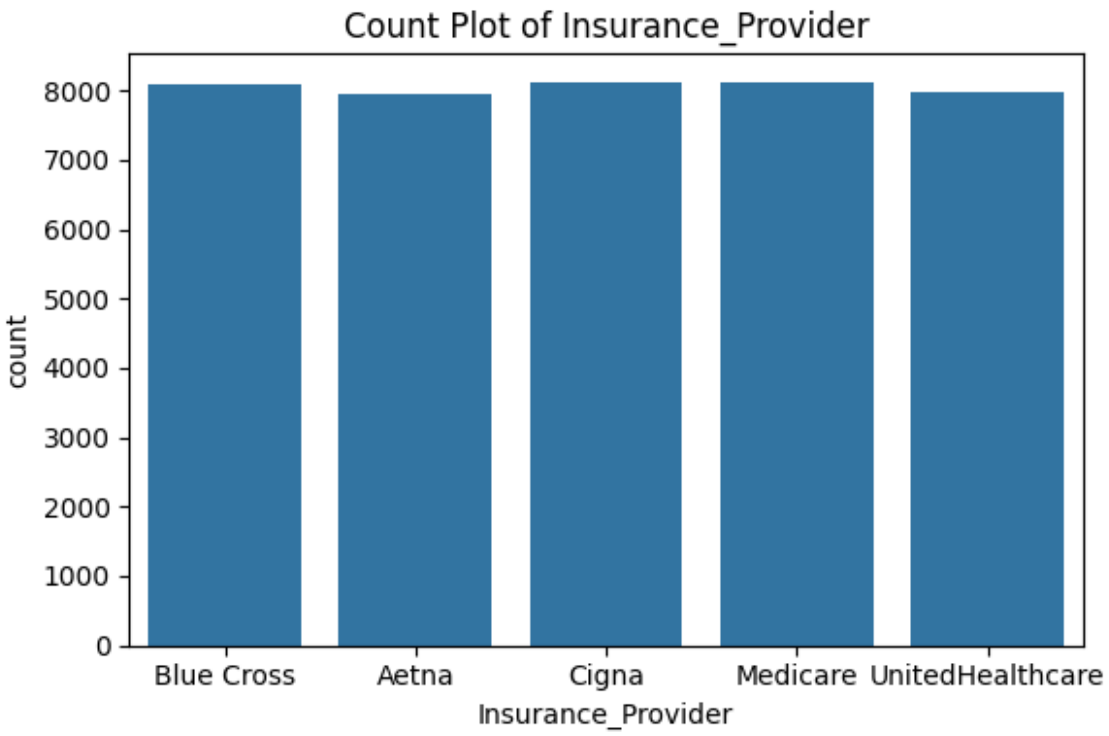

Distribution of Height
3000
2500
2000
1500
1000
500
0
Count
140
150
160
170
180
190
Height

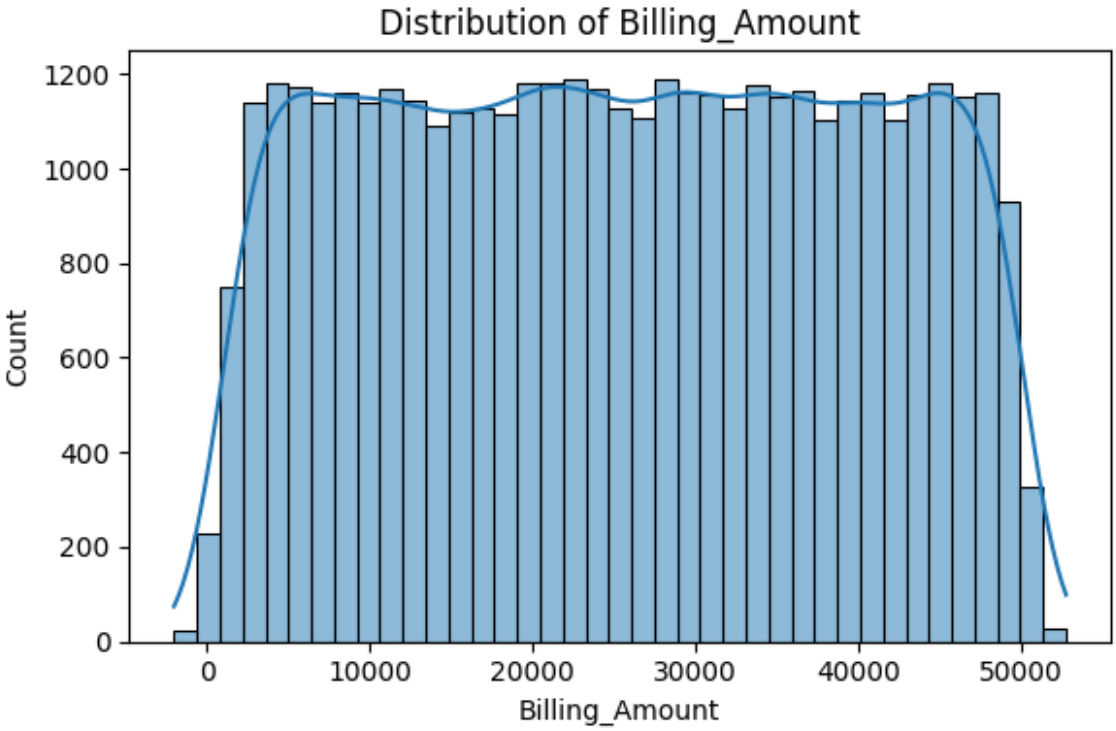

Distribution of Weight
Count
Weight
0
200
400
600
800
1000
1200
1400
1600
40
60
80
100
120
140
160
180

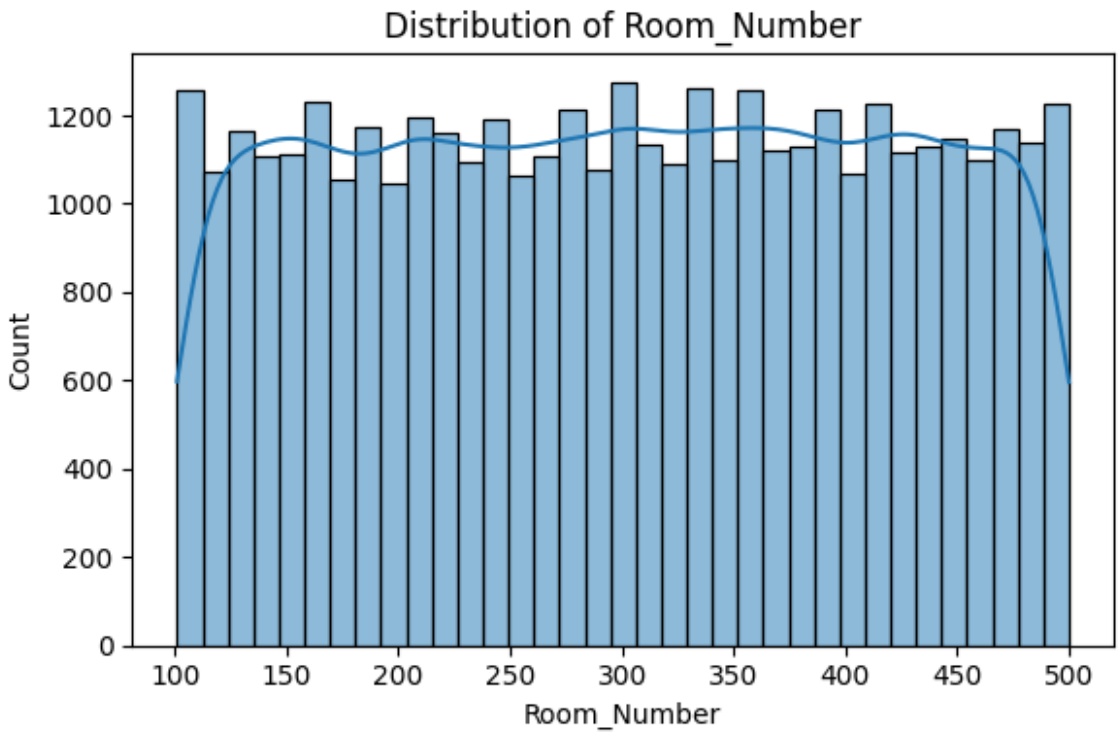

Count Plot of Blood_Type
count
Blood_Type
0
1000
2000
3000
4000
5000
B+
A-
AB-
AB+
O-
B-
O+
A+

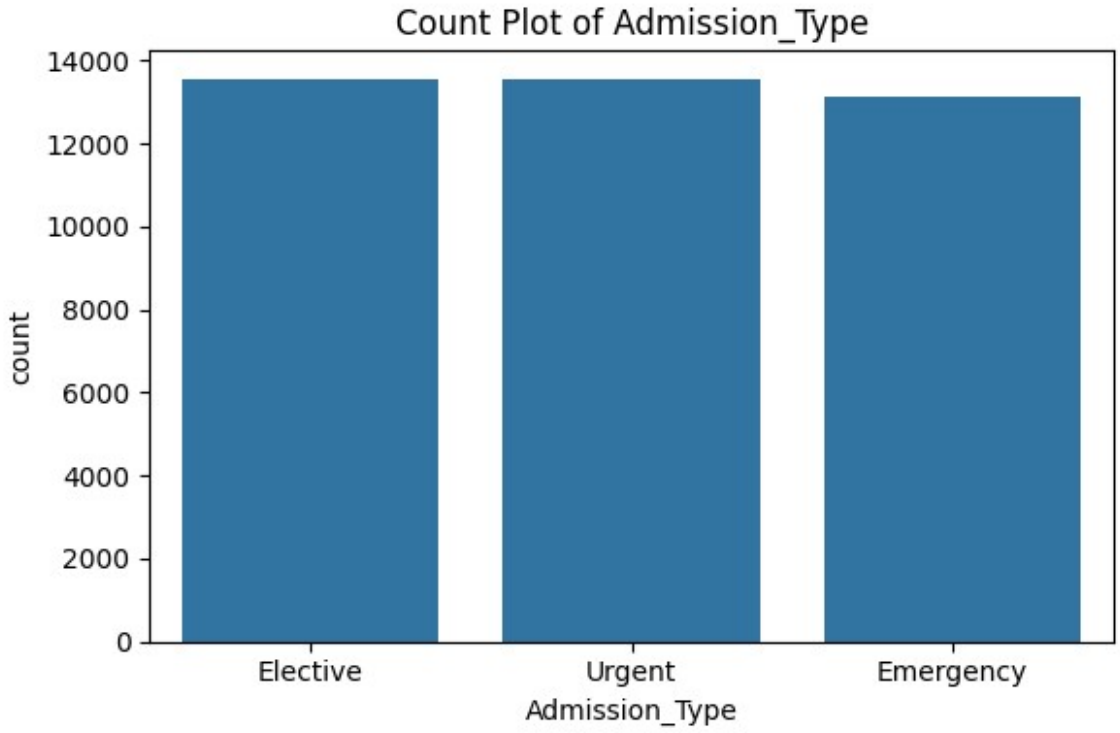



```
<Figure size 600x400 with 0 Axes>
<Figure size 600x400 with 0 Axes>
<Figure size 600x400 with 0 Axes>
```

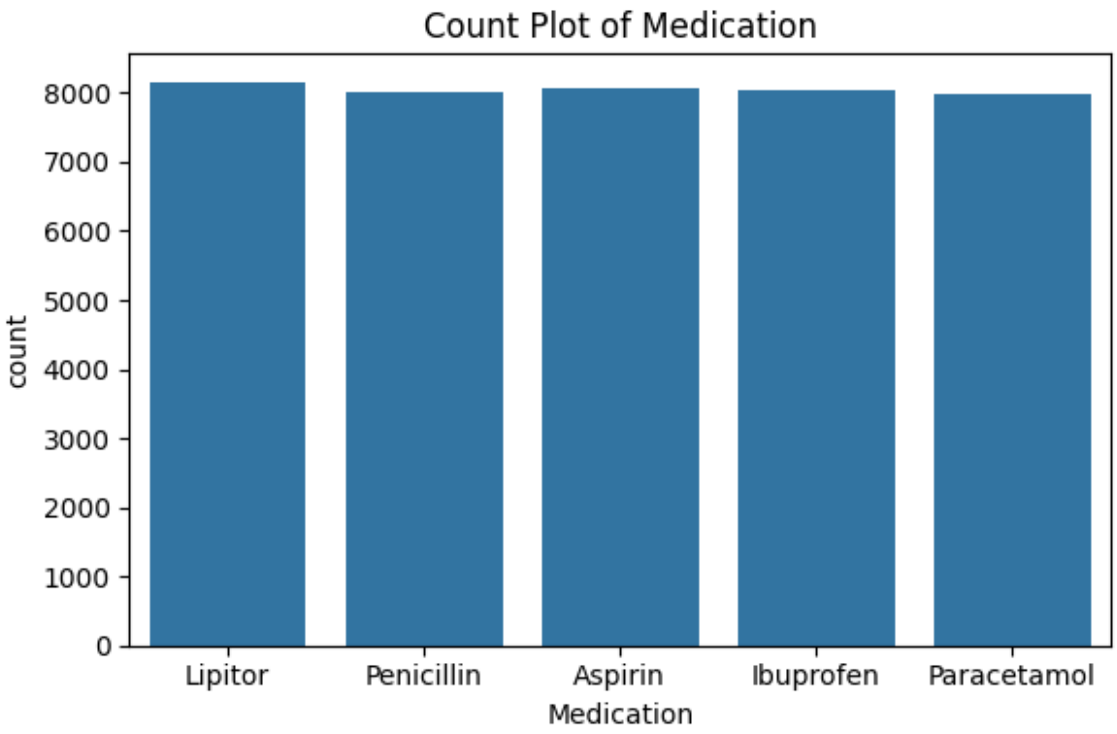

Count Plot of Insurance_Provider
count
8000
7000
6000
5000
4000
3000
2000
1000
0
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider

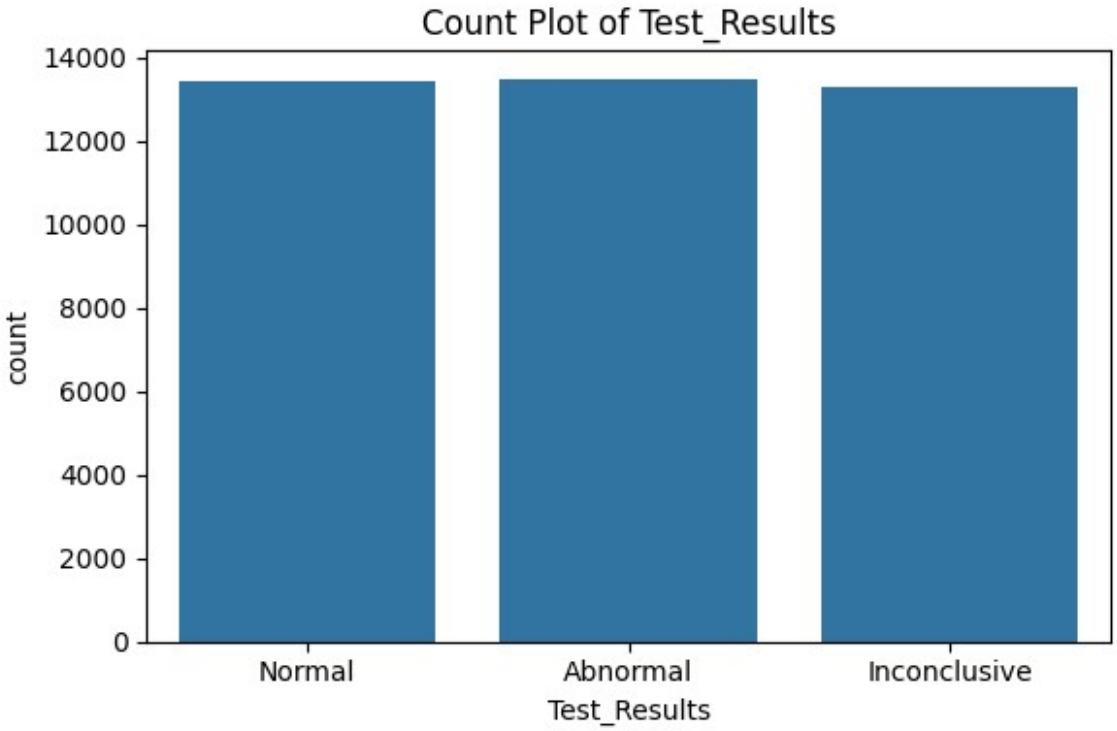

Distribution of Billing_Amount
Count
1200
1000
800
600
400
200
0
0
10000
20000
30000
40000
50000
Billing_Amount

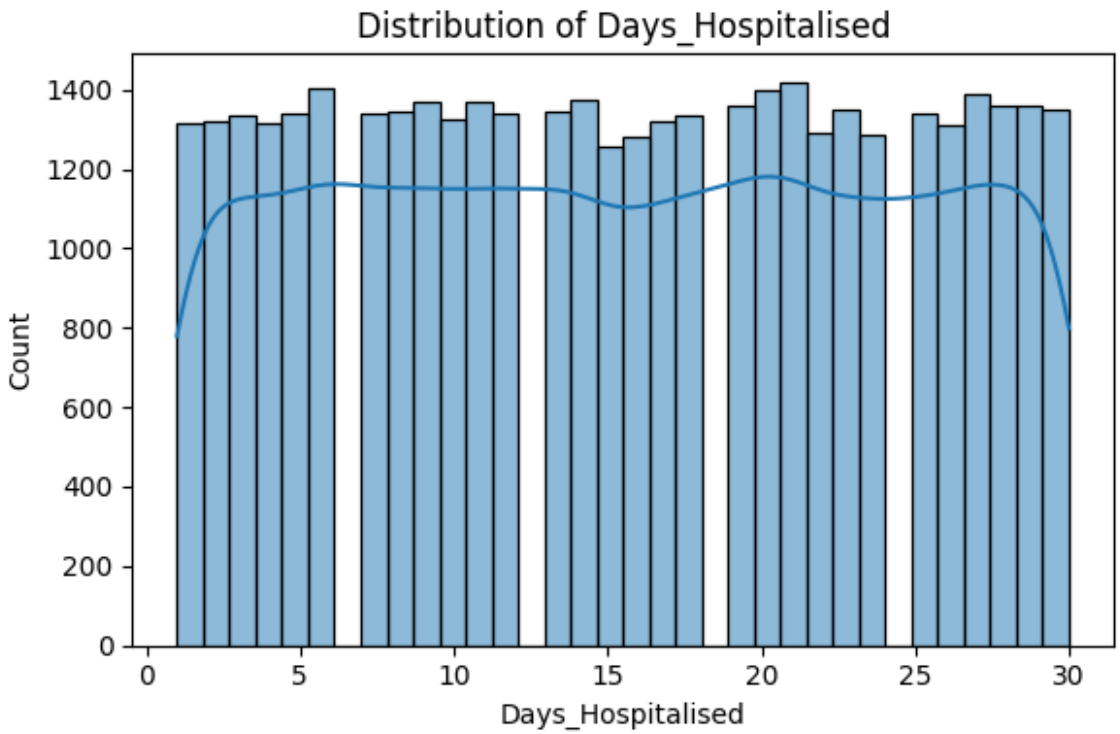




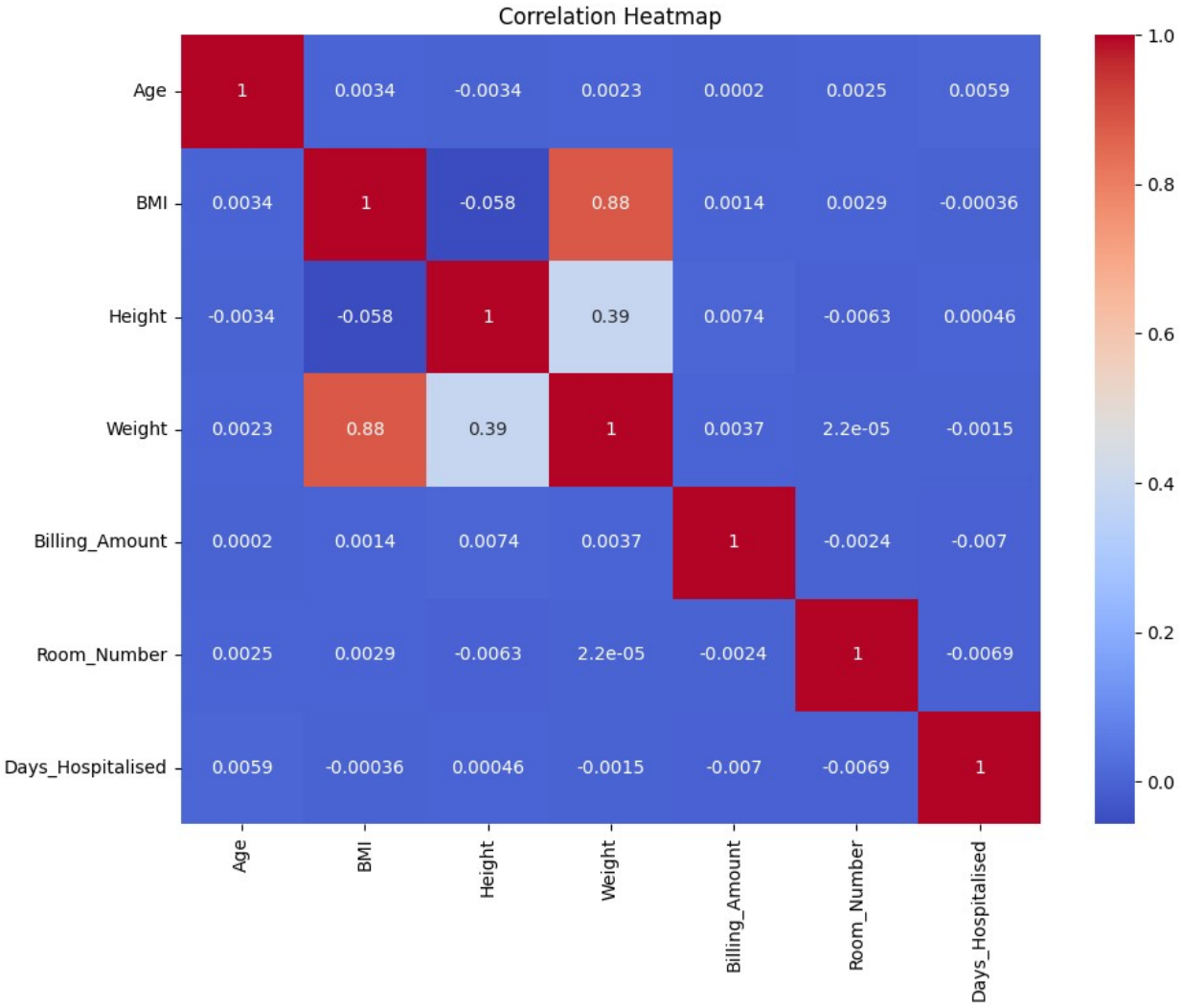

```
<Figure size 600x400 with 0 Axes>
```


Count Plot of Medication
count
8000
7000
6000
5000
4000
3000
2000
1000
0
Lipitor
Penicillin
Aspirin
Ibuprofen
Paracetamol
Medication

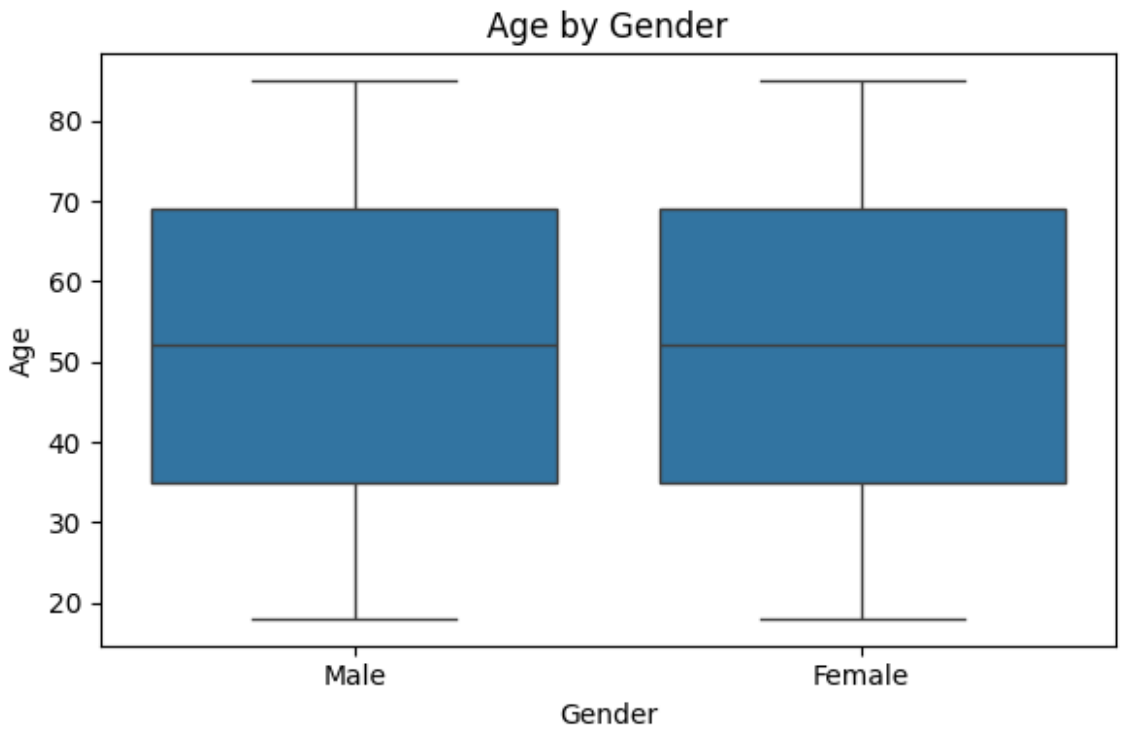

Count Plot of Test_Results
count
14000
12000
10000
8000
6000
4000
2000
0
Normal
Abnormal
Inconclusive
Test_Results

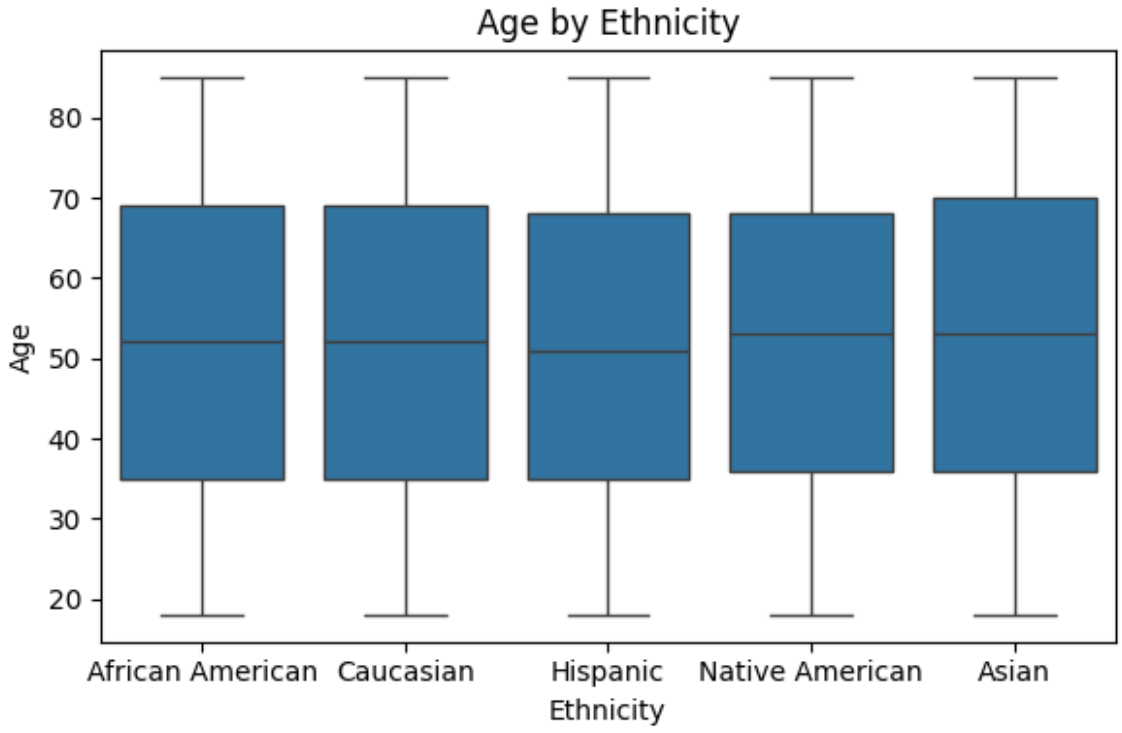


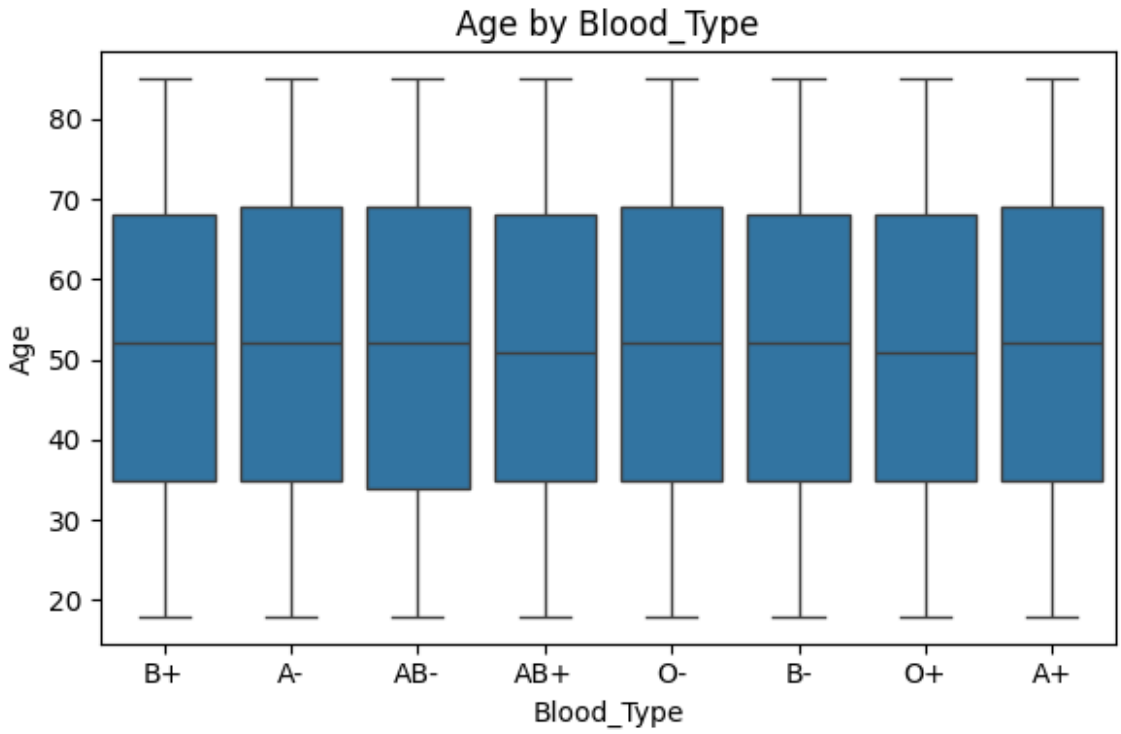

```
Creating correlation heatmap...
```

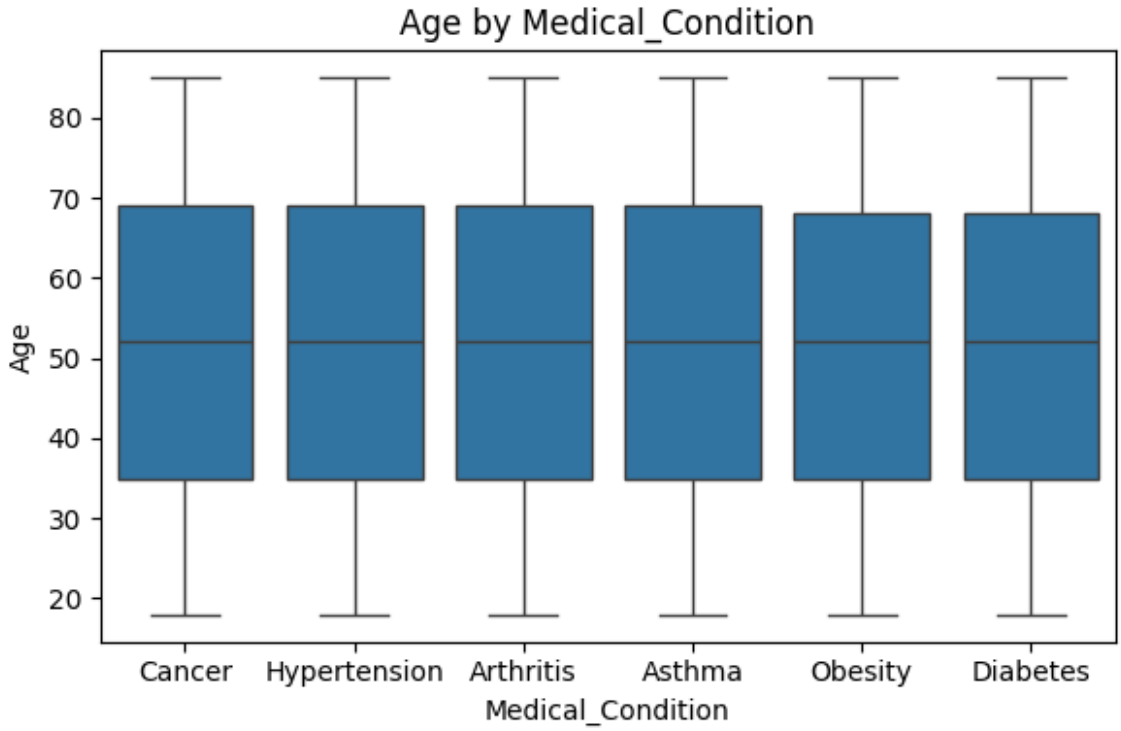


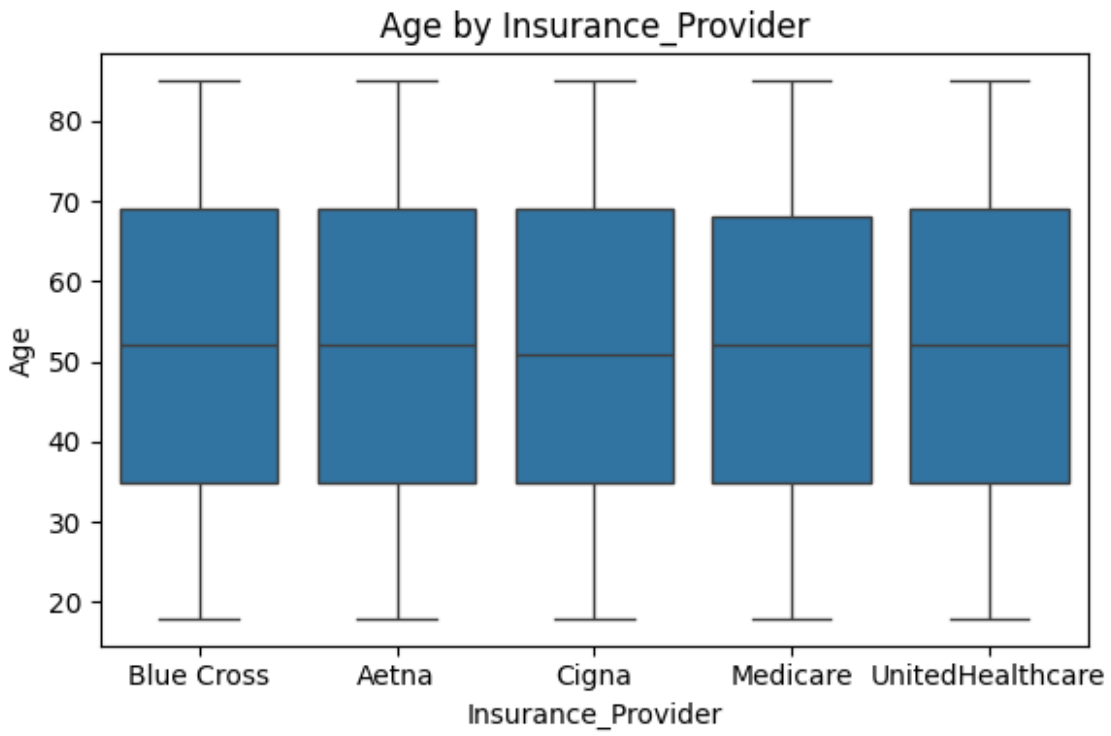

```
Creating boxplots...
```

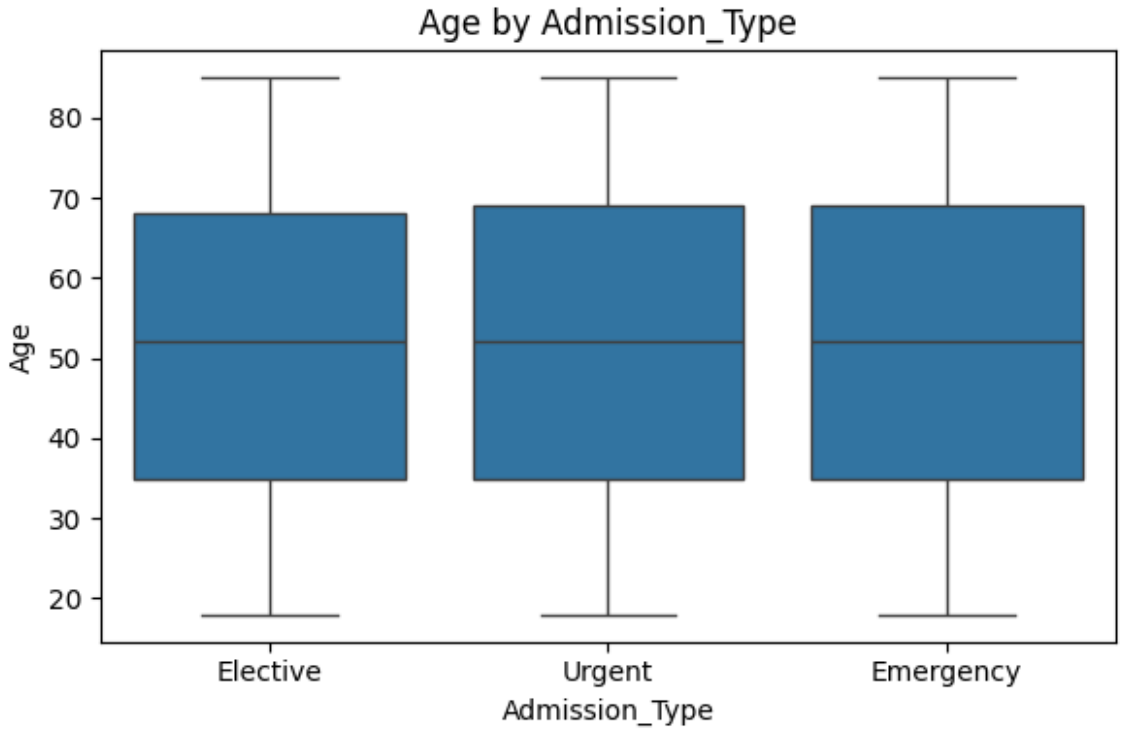

Age by Gender
Age
80
70
60
50
40
30
20
Male
Female
Gender

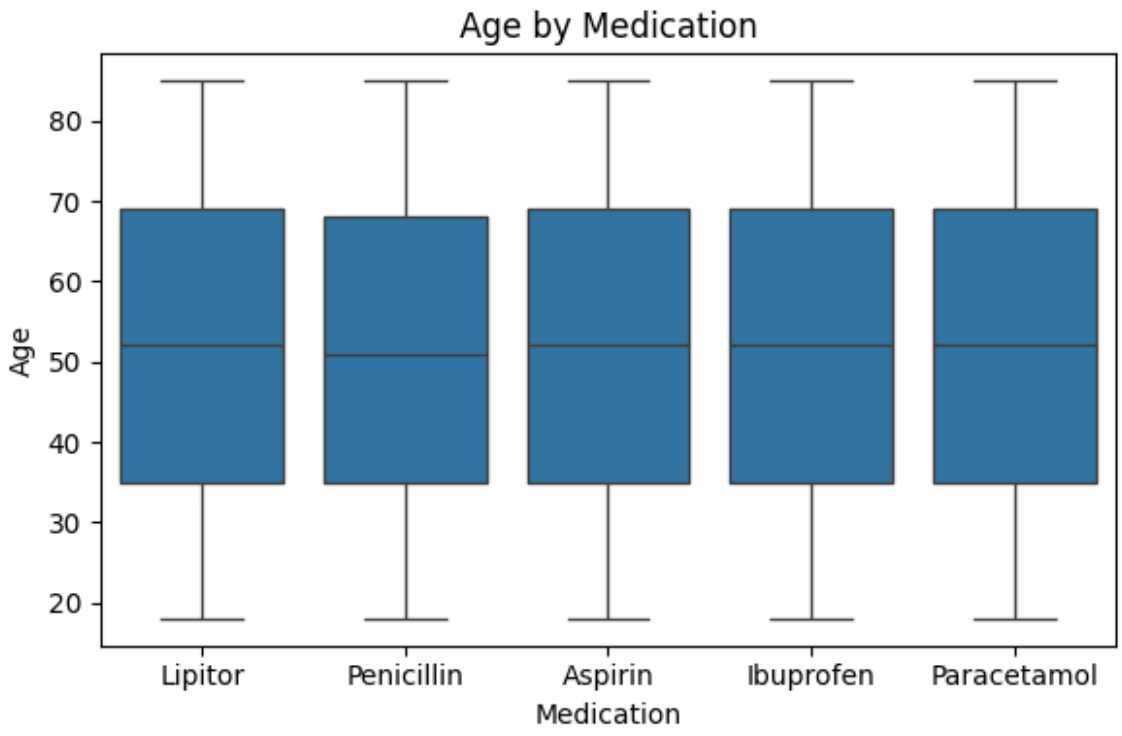

Age by Ethnicity
Age
80
70
60
50
40
30
20
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

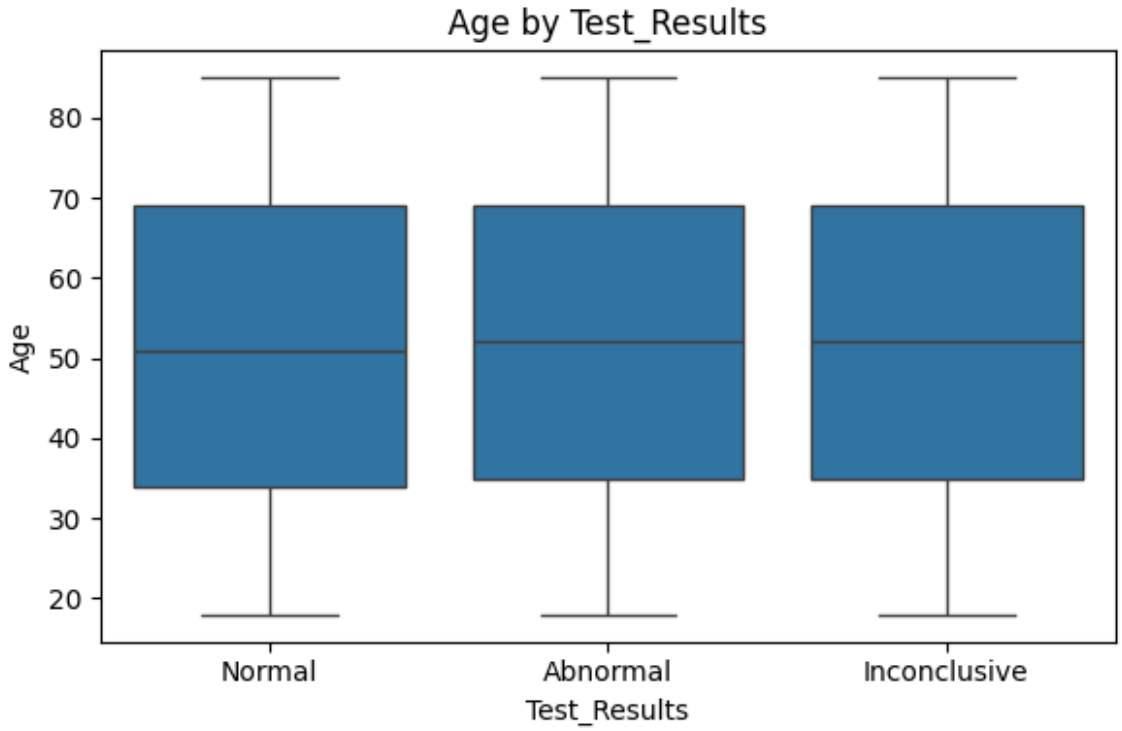

Age by Blood_Type
Age
80
70
60
50
40
30
20
B+
A-
AB-
AB+
O-
B-
O+
A+
Blood_Type

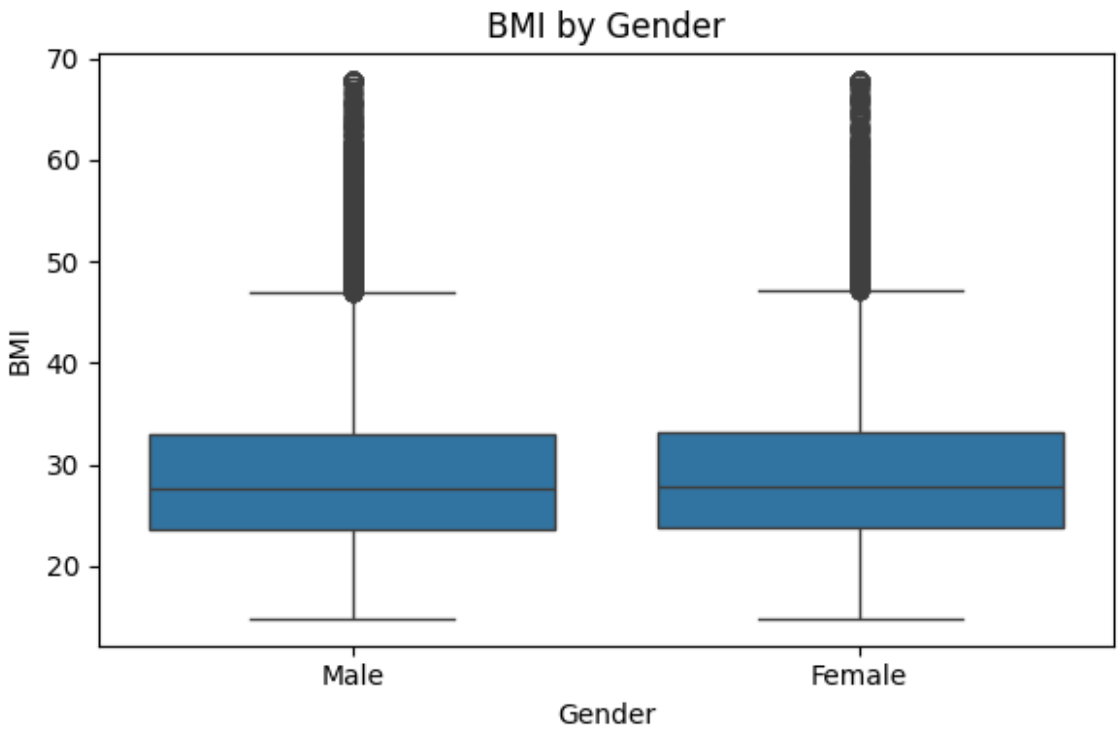

Age by Medical_Condition
Age
80
70
60
50
40
30
20
Cancer
Hypertension
Arthritis
Asthma
Obesity
Diabetes
Medical_Condition

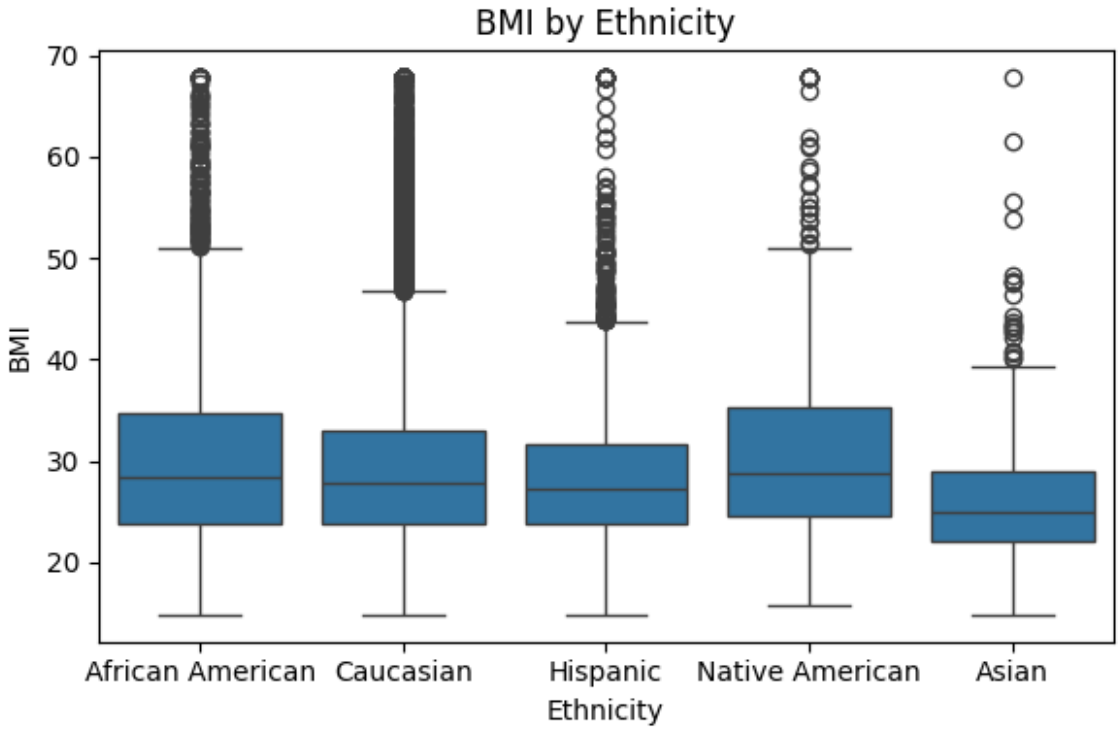

Age by Insurance_Provider
Age
80
70
60
50
40
30
20
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider

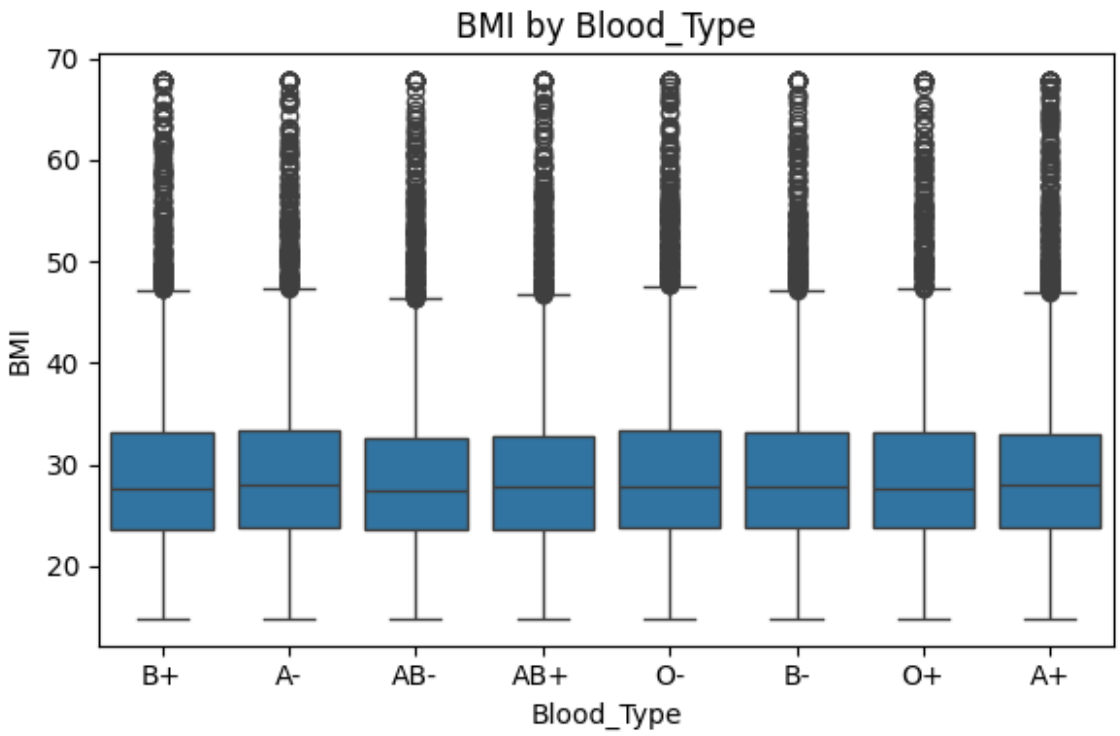

Age by Admission_Type
Age
80
70
60
50
40
30
20
Elective
Urgent
Emergency
Admission_Type

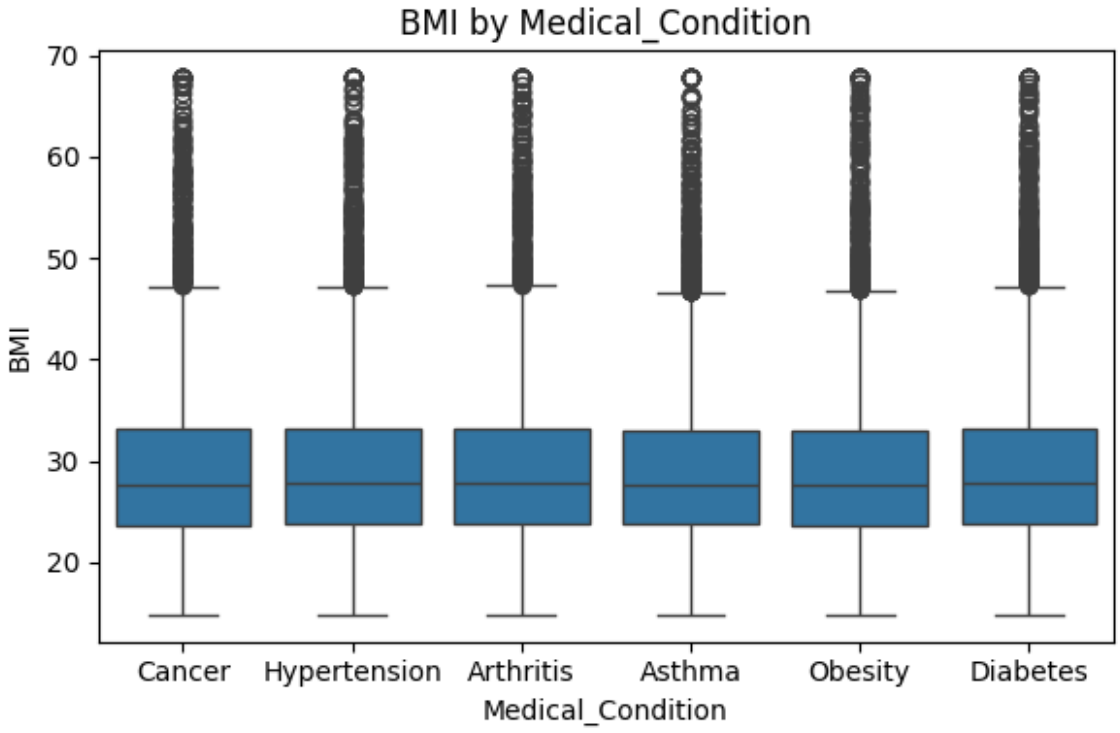

Age by Medication
Age
80
70
60
50
40
30
20
Lipitor
Penicillin
Aspirin
Ibuprofen
Paracetamol
Medication

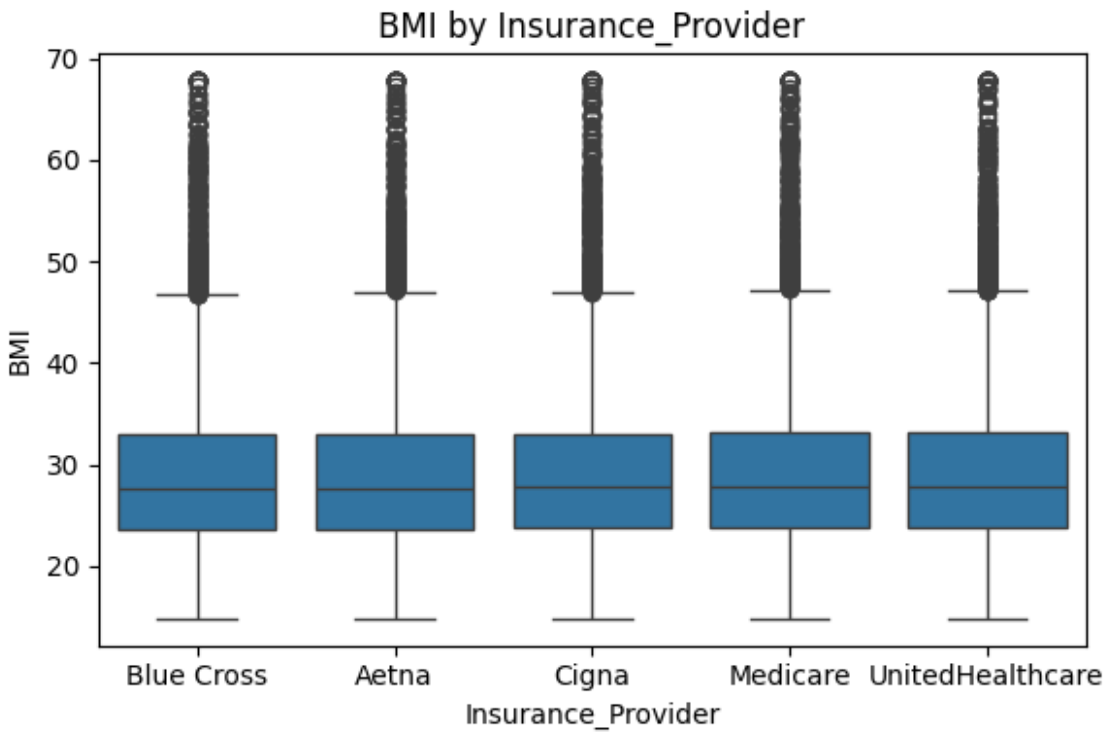

Age by Test_Results
Age
80
70
60
50
40
30
20
Normal
Abnormal
Inconclusive
Test_Results

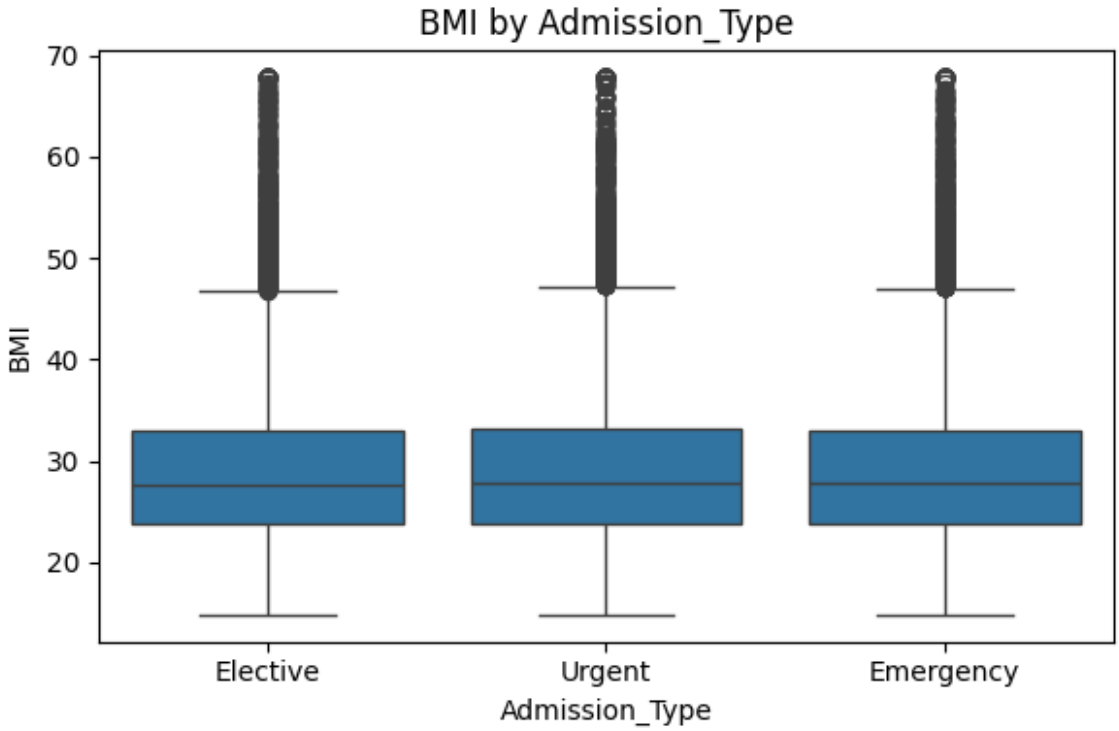

BMI by Gender
70
60
50
40
30
20
BMI
Male
Female
Gender

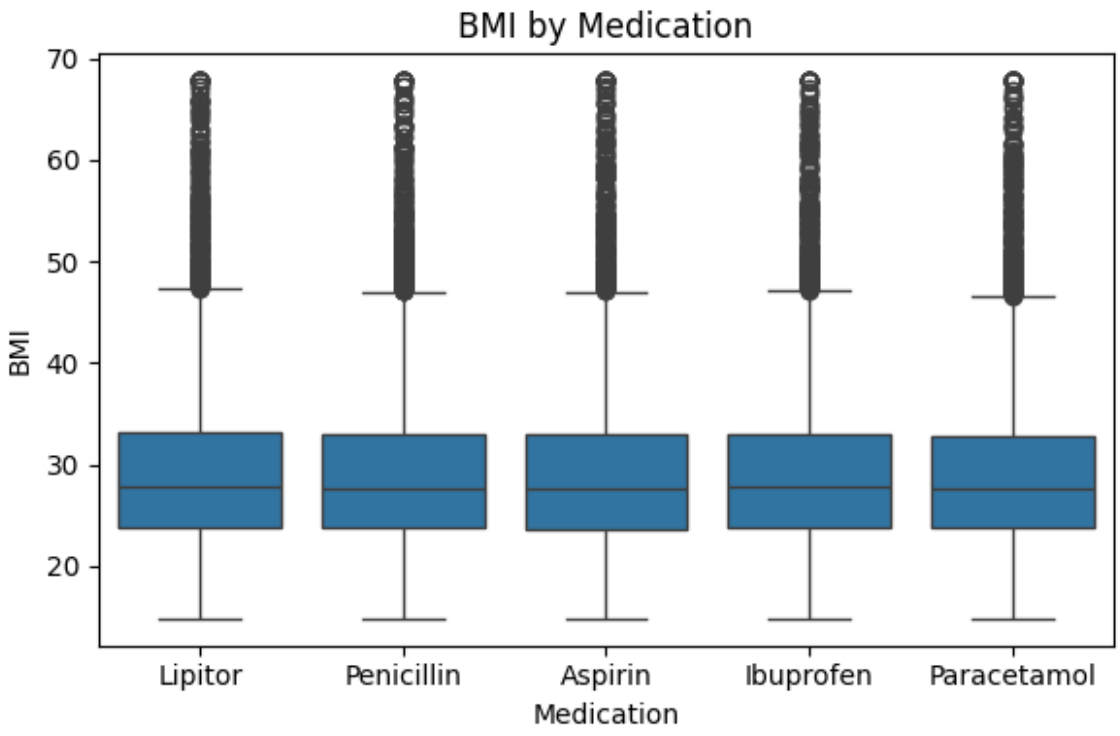

BMI by Ethnicity
70
60
50
40
30
20
BMI
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

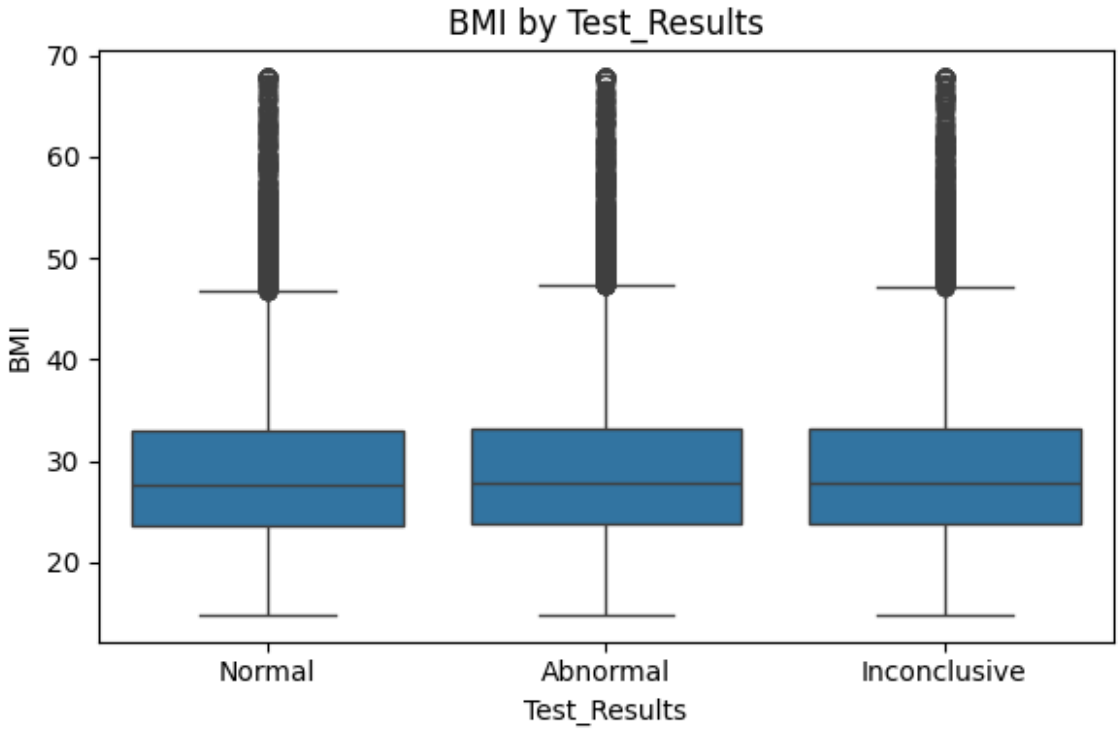

BMI by Blood_Type
BMI
70
60
50
40
30
20
B+
A-
AB-
AB+
O-
B-
O+
A+
Blood_Type

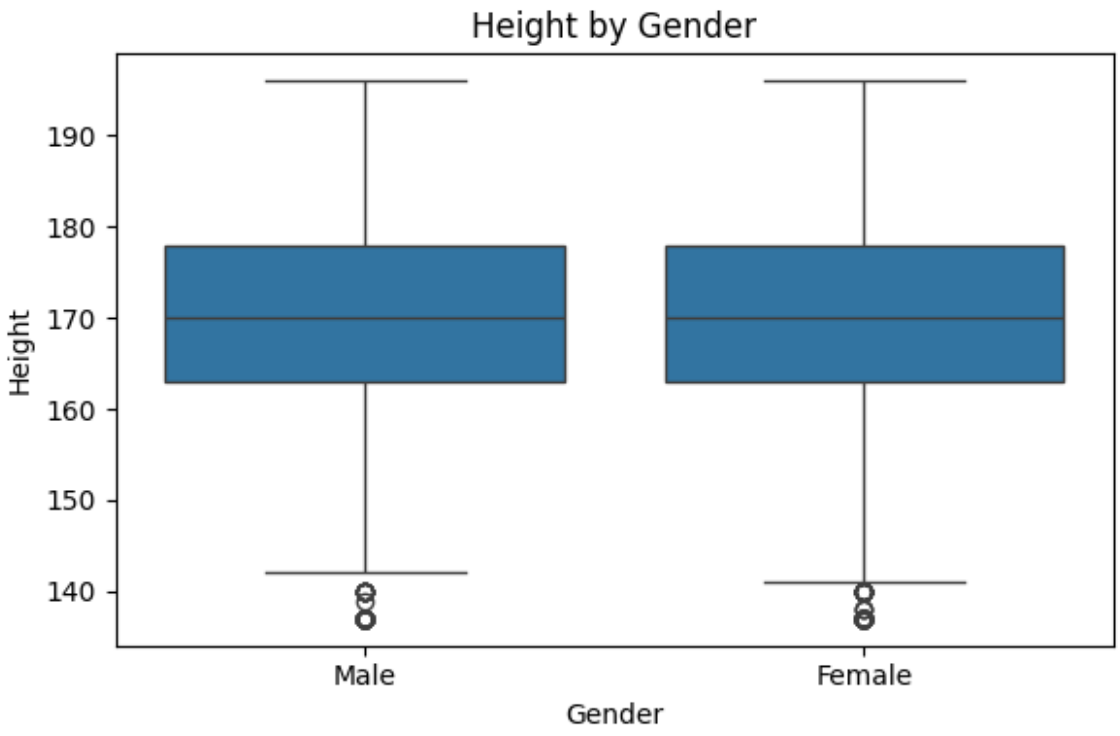

BMI by Medical_Condition
BMI
70
60
50
40
30
20
Cancer
Hypertension
Arthritis
Asthma
Obesity
Diabetes
Medical_Condition

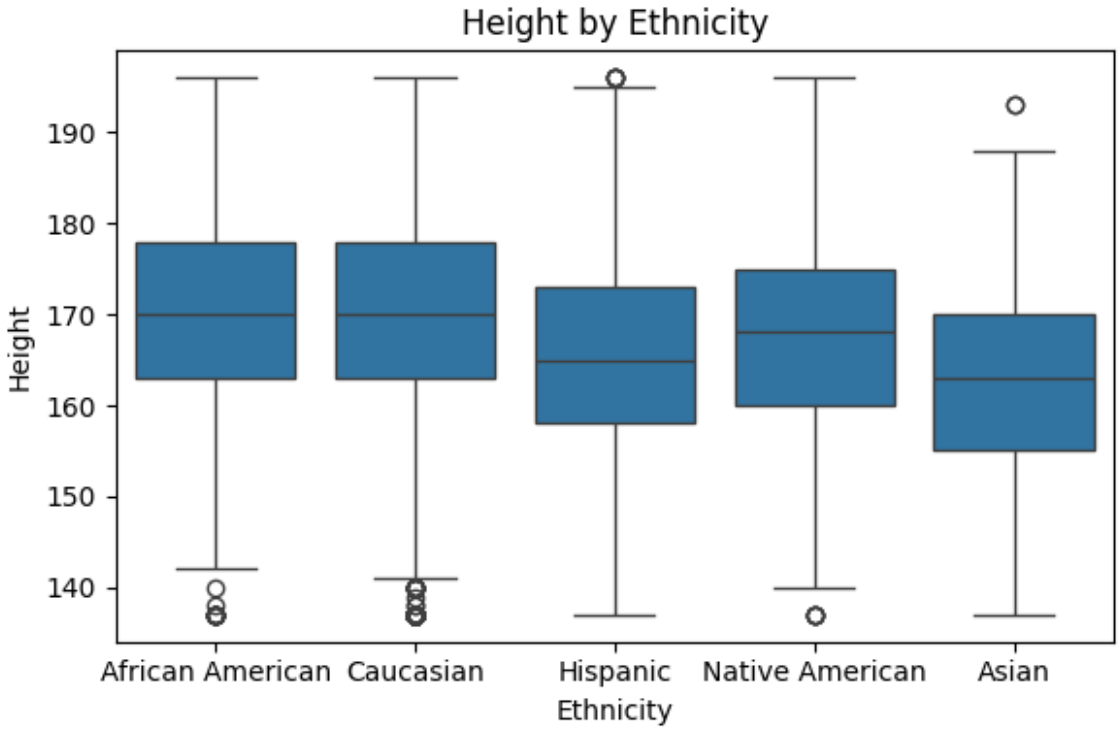

BMI by Insurance_Provider
BMI
70
60
50
40
30
20
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider

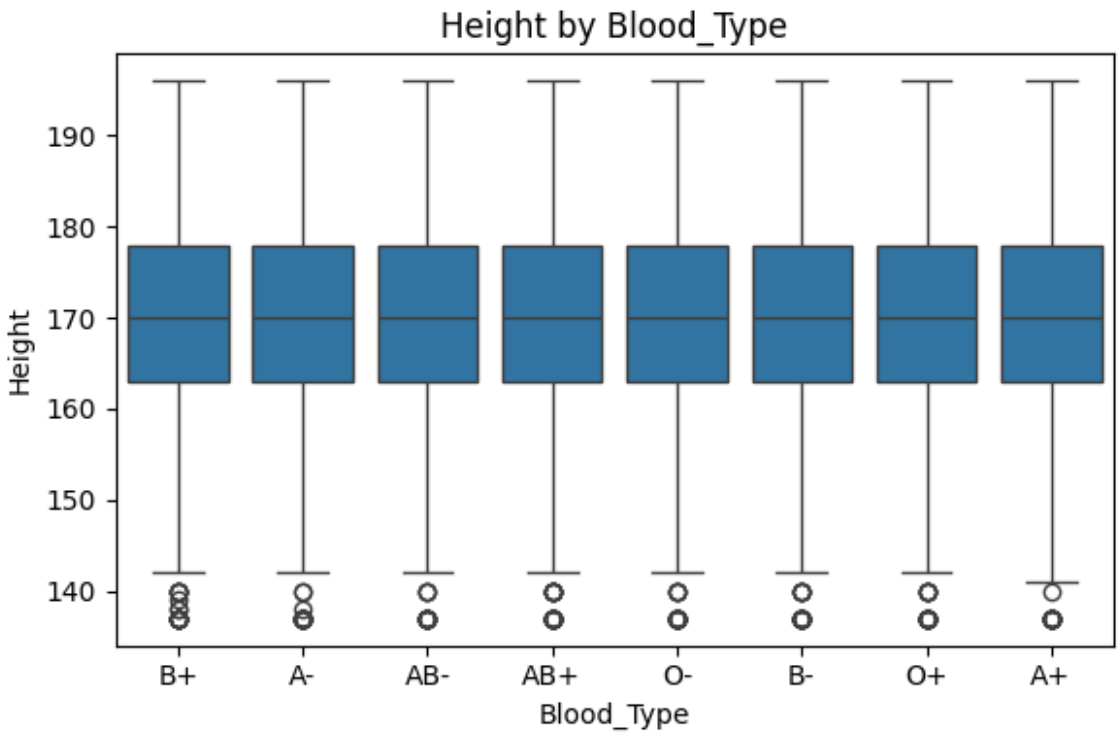

BMI by Admission_Type
BMI
70
60
50
40
30
20
Elective
Urgent
Emergency
Admission_Type

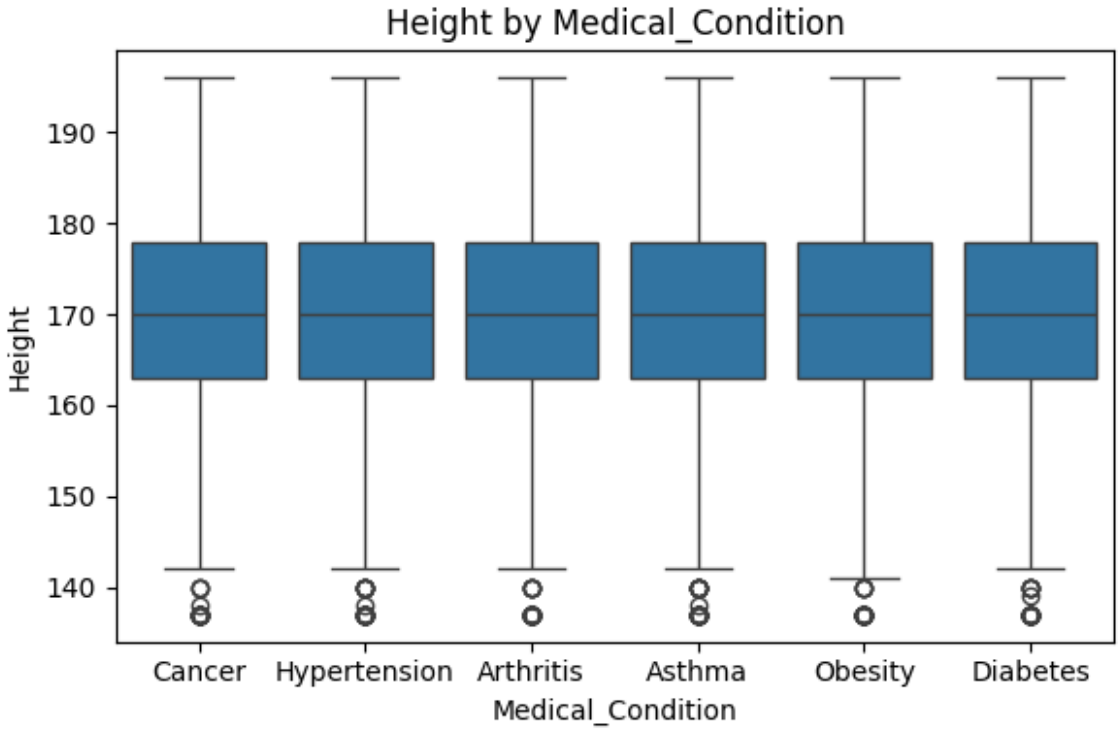

BMI by Medication
BMI
70
60
50
40
30
20
Lipitor
Penicillin
Aspirin
Ibuprofen
Paracetamol
Medication

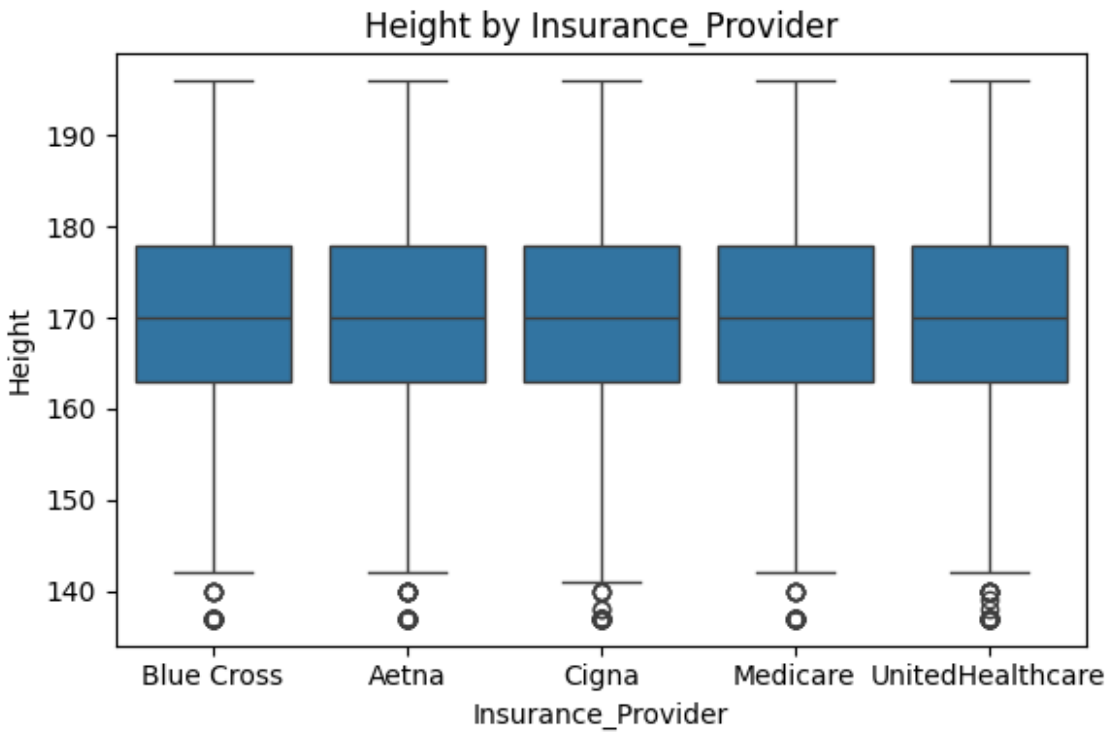

BMI by Test_Results
BMI
70
60
50
40
30
20
Normal
Abnormal
Inconclusive
Test_Results

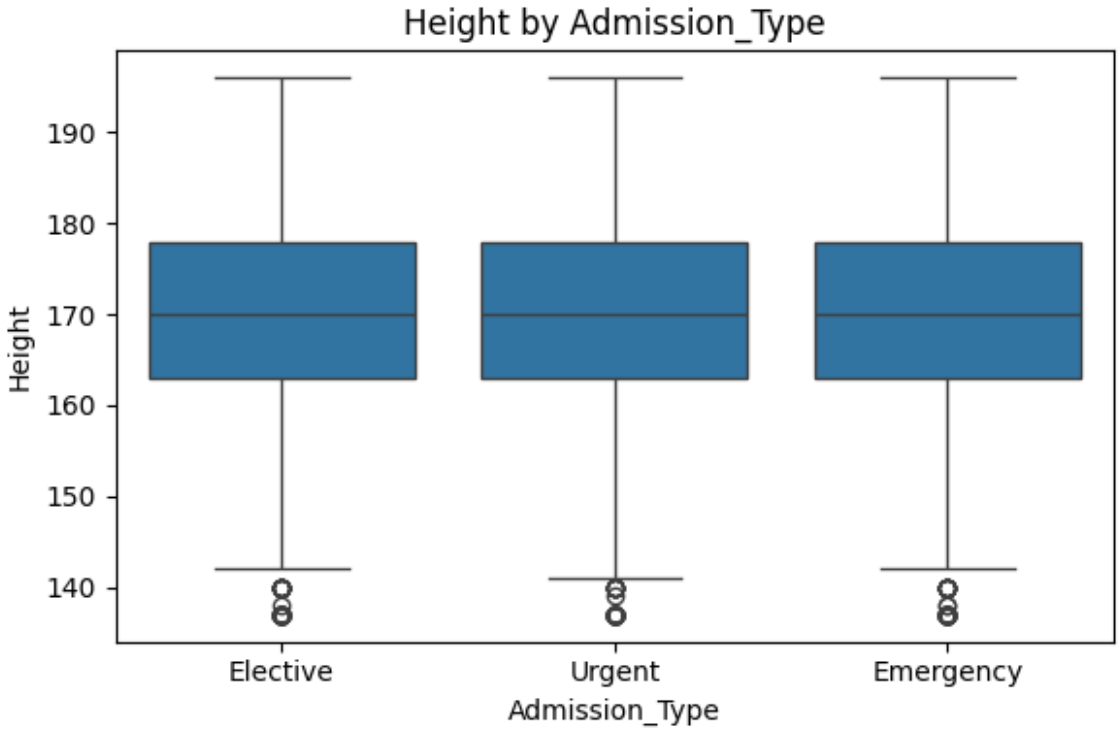

Height by Gender
Height
190
180
170
160
150
140
Male
Female
Gender

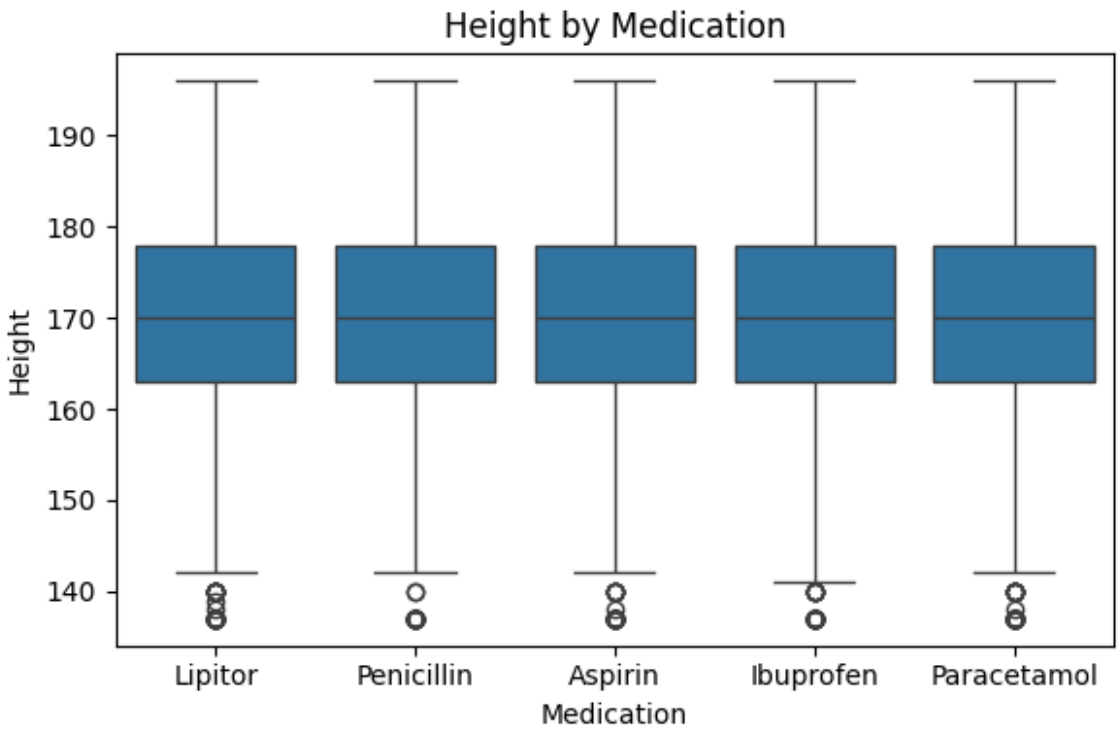

Height by Ethnicity
Height
190
180
170
160
150
140
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

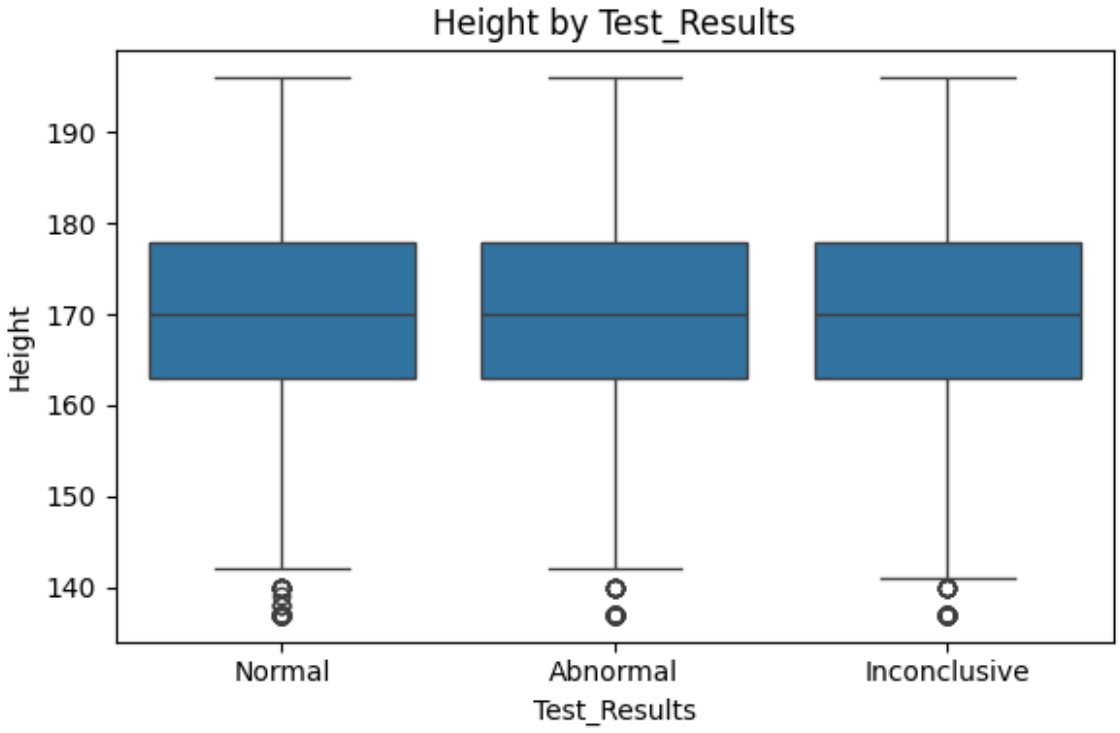

Height by Blood_Type
Height
190
180
170
160
150
140
B+
A-
AB-
AB+
O-
B-
O+
A+
Blood_Type

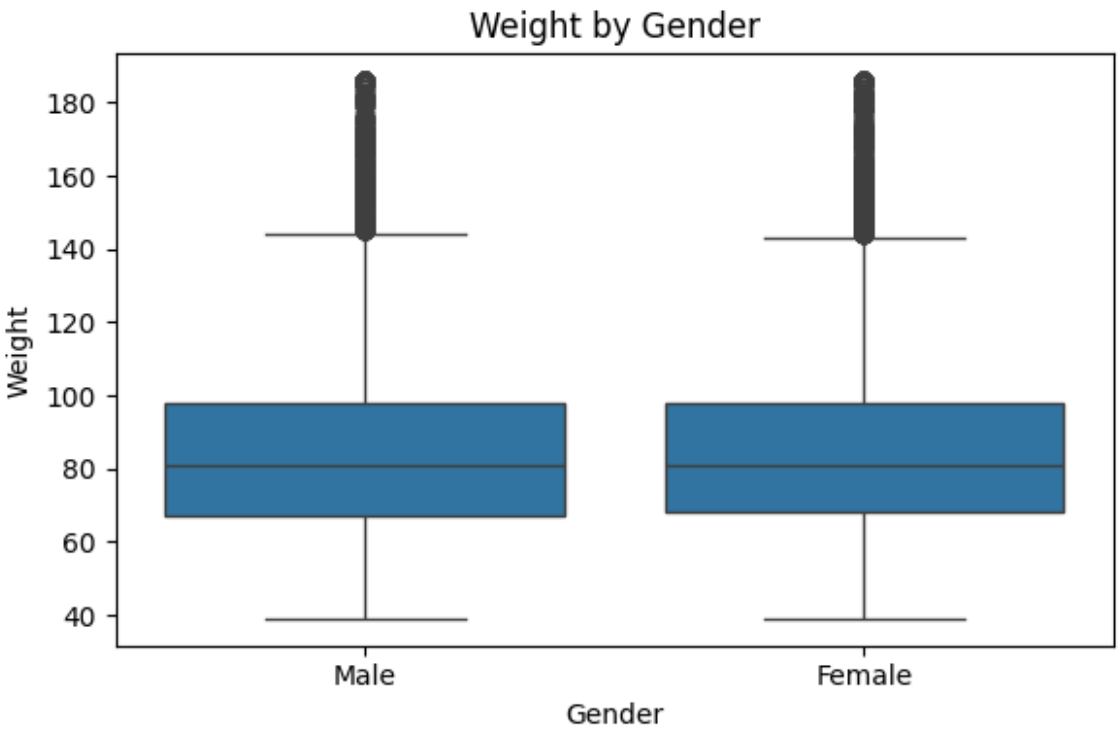

Height by Medical_Condition
Height
190
180
170
160
150
140
Cancer
Hypertension
Arthritis
Asthma
Obesity
Diabetes
Medical_Condition

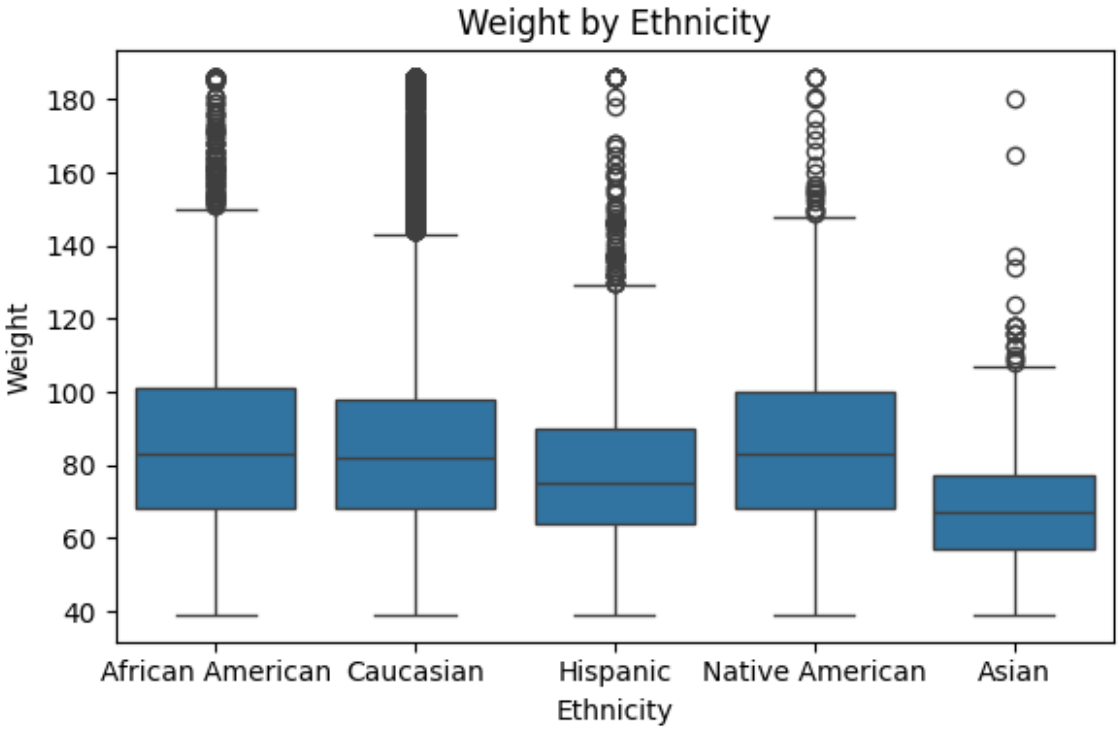

Height by Insurance_Provider
Height
190
180
170
160
150
140
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider

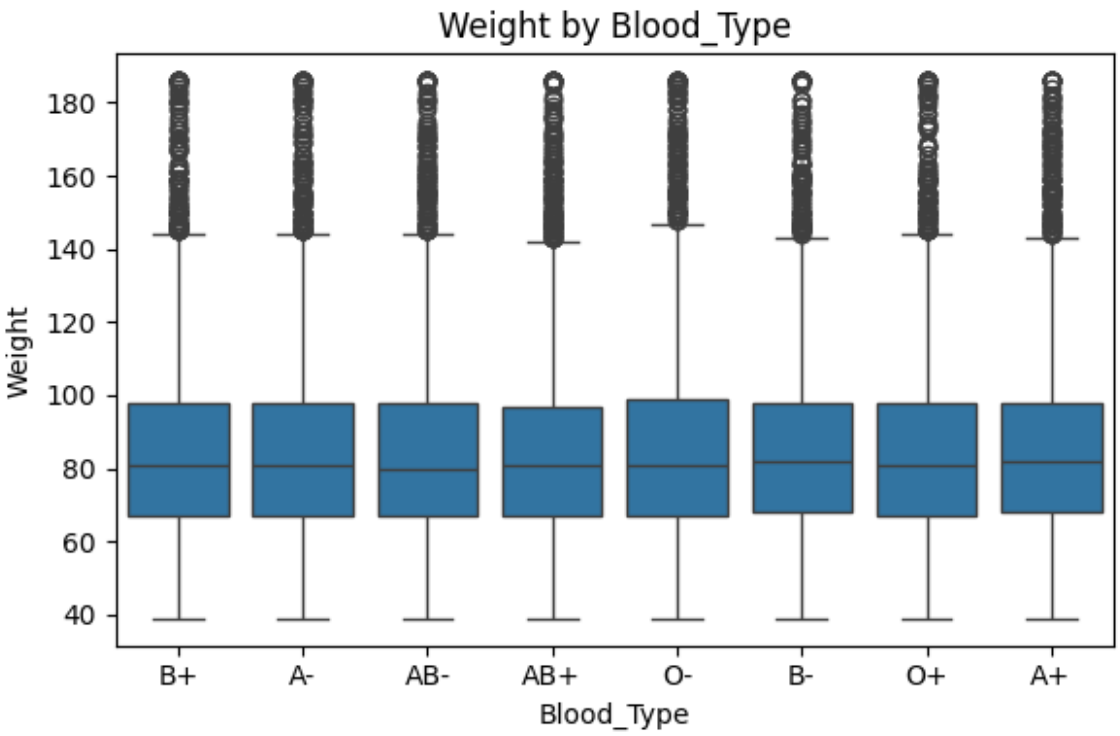

Height by Admission_Type
Height
190
180
170
160
150
140
Elective
Urgent
Emergency
Admission_Type

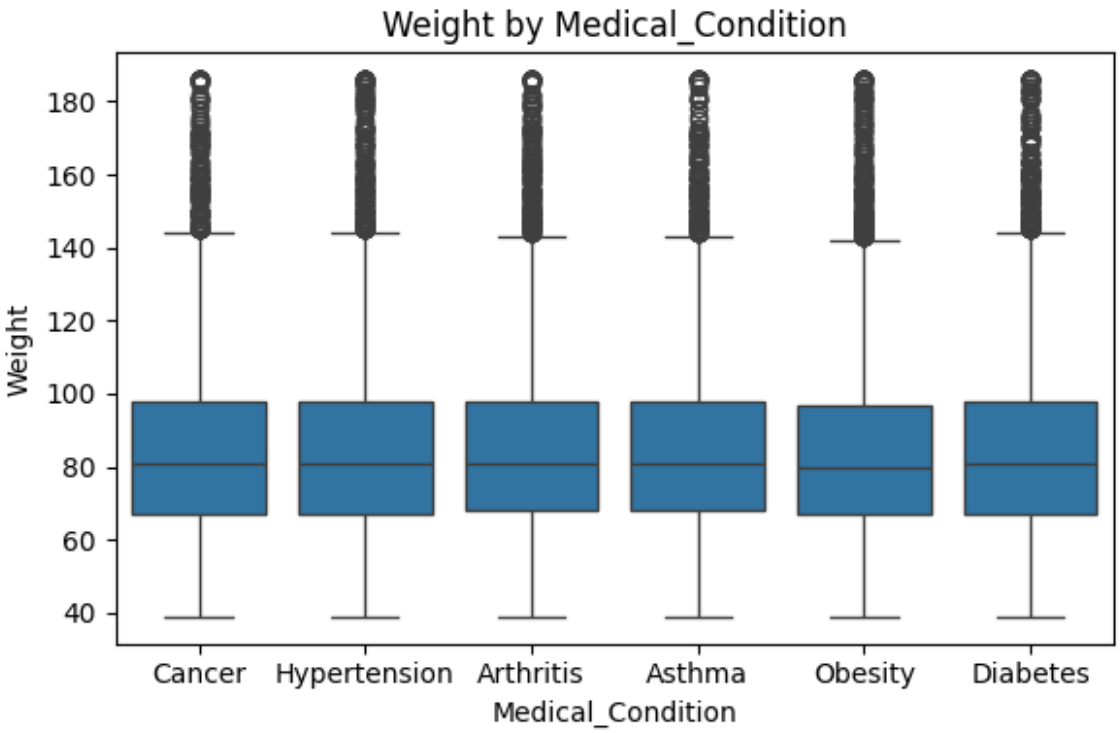

Height by Medication
Height
190
180
170
160
150
140
Lipitor
Penicillin
Aspirin
Ibuprofen
Paracetamol
Medication

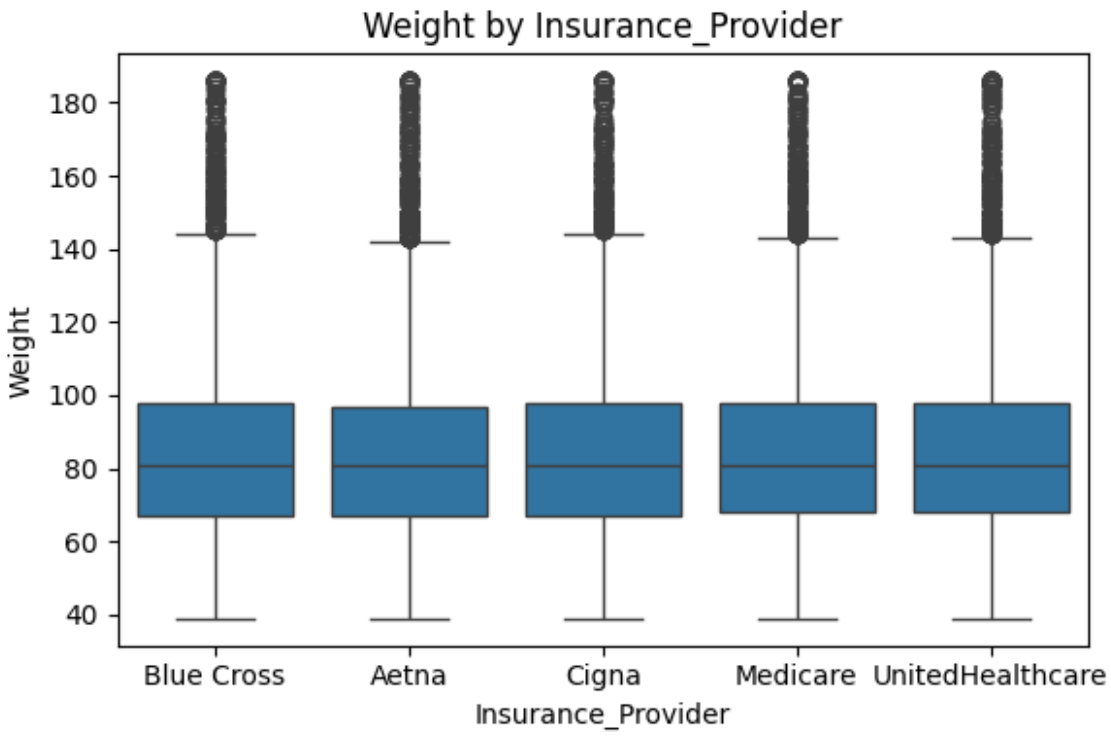

Height by Test_Results
Height
190
180
170
160
150
140
Normal
Abnormal
Inconclusive
Test_Results

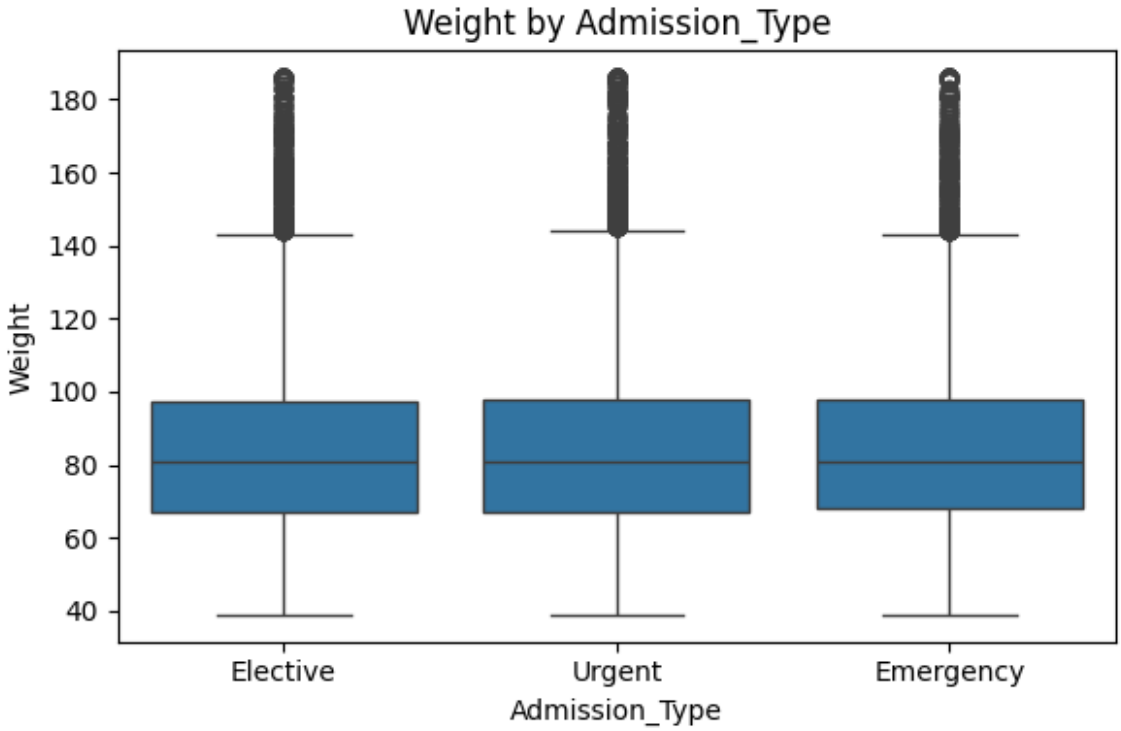

Weight by Gender
Weight
180
160
140
120
100
80
60
40
Male
Female
Gender

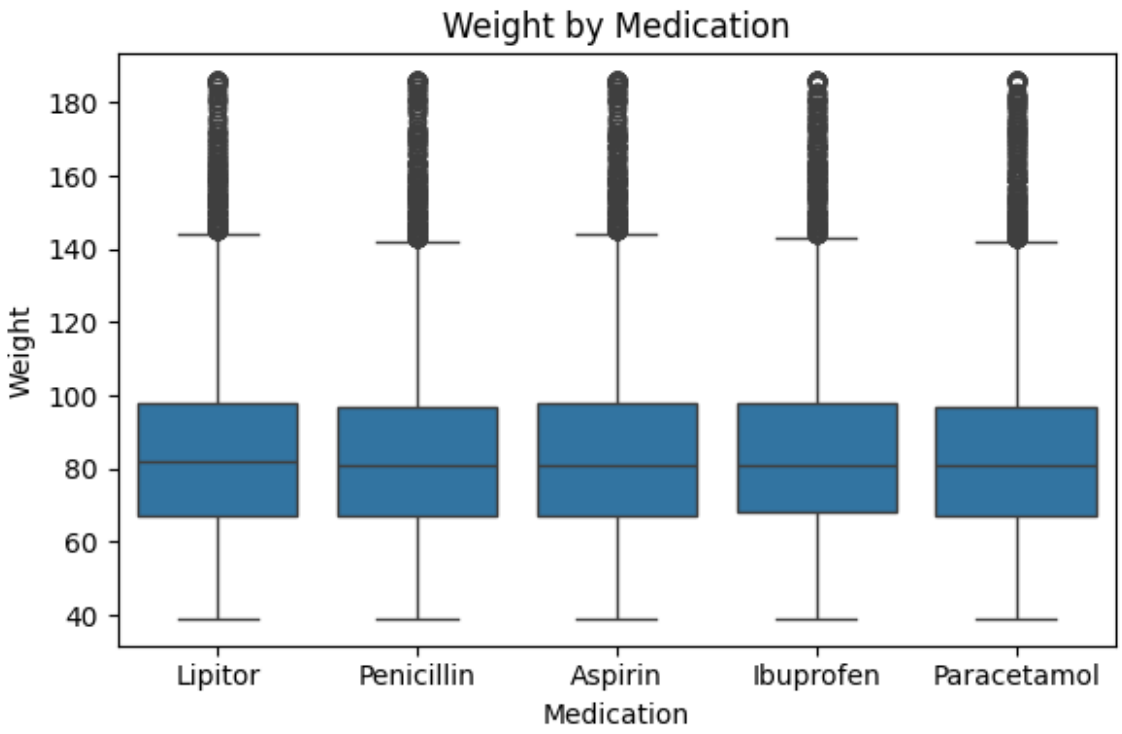

Weight by Ethnicity
Weight
180
160
140
120
100
80
60
40
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

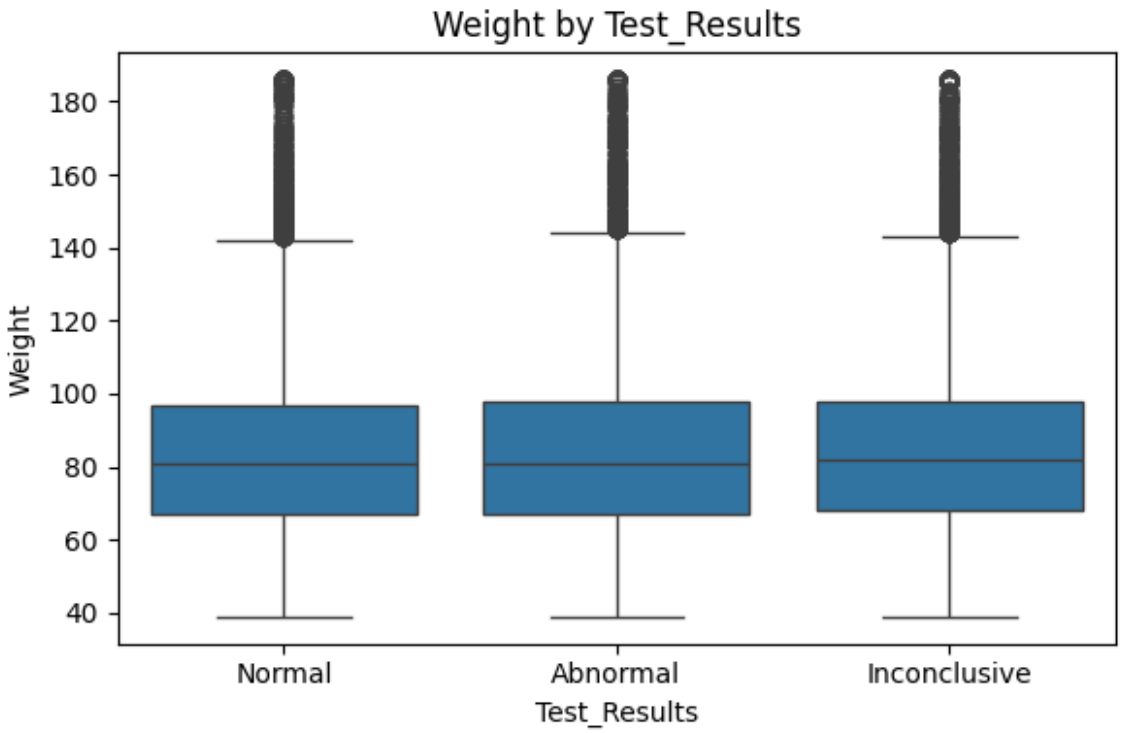

Weight by Blood_Type
Weight
180
160
140
120
100
80
60
40
B+
A-
AB-
AB+
O-
B-
O+
A+
Blood_Type

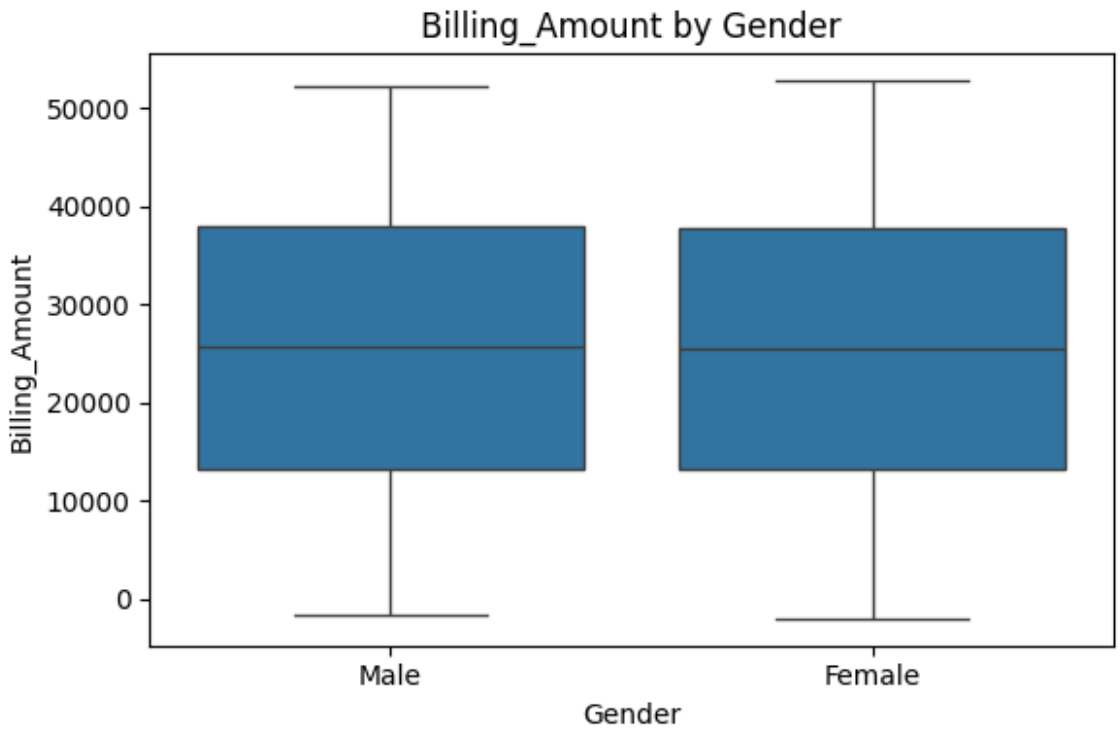

Weight by Medical_Condition
Weight
180
160
140
120
100
80
60
40
Cancer
Hypertension
Arthritis
Asthma
Obesity
Diabetes
Medical_Condition

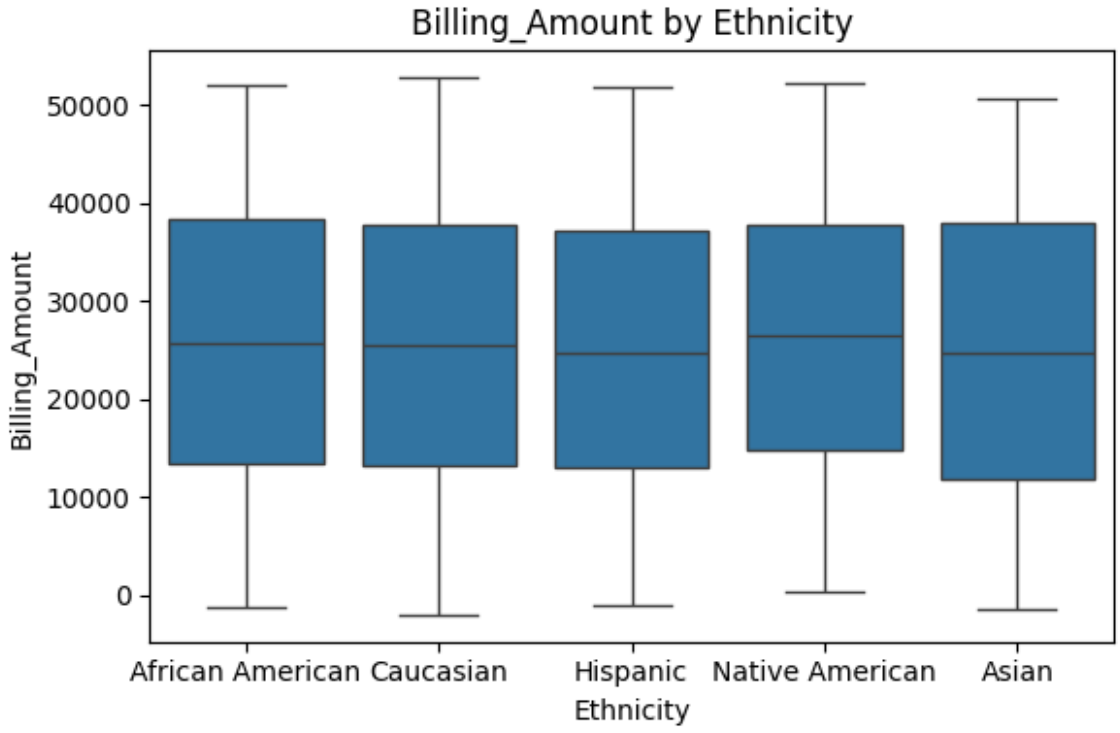

Weight by Insurance_Provider
Weight
180
160
140
120
100
80
60
40
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider

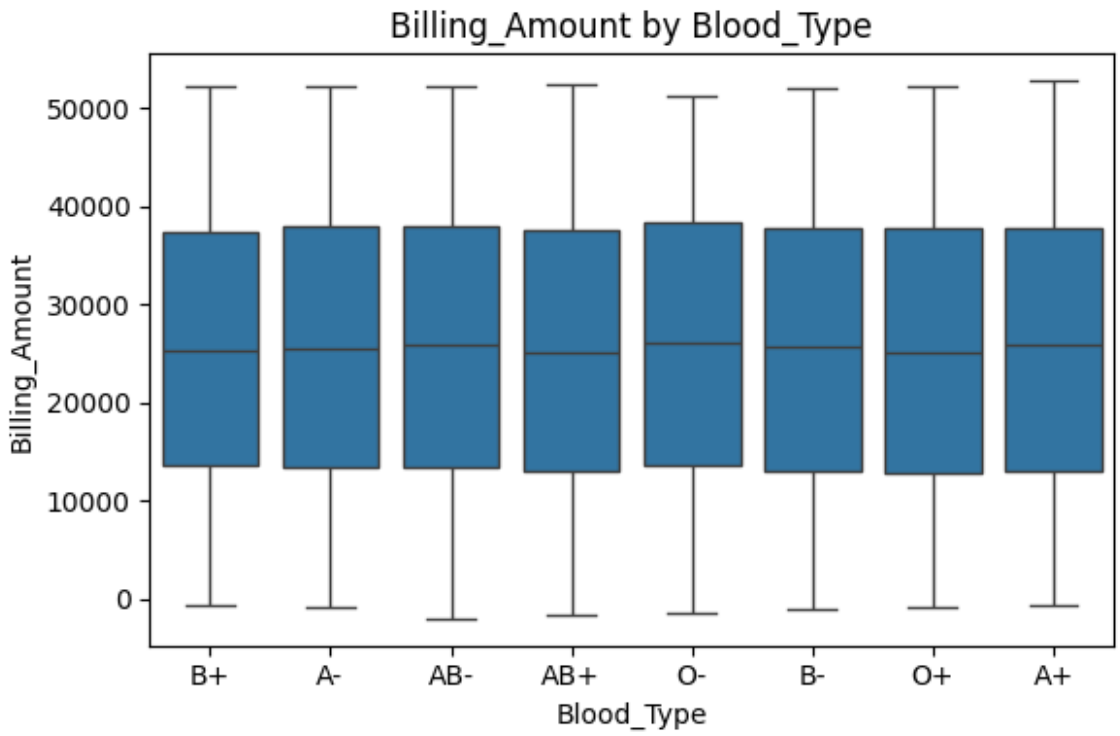

Weight by Admission_Type
Weight
180
160
140
120
100
80
60
40
Elective
Urgent
Emergency
Admission_Type

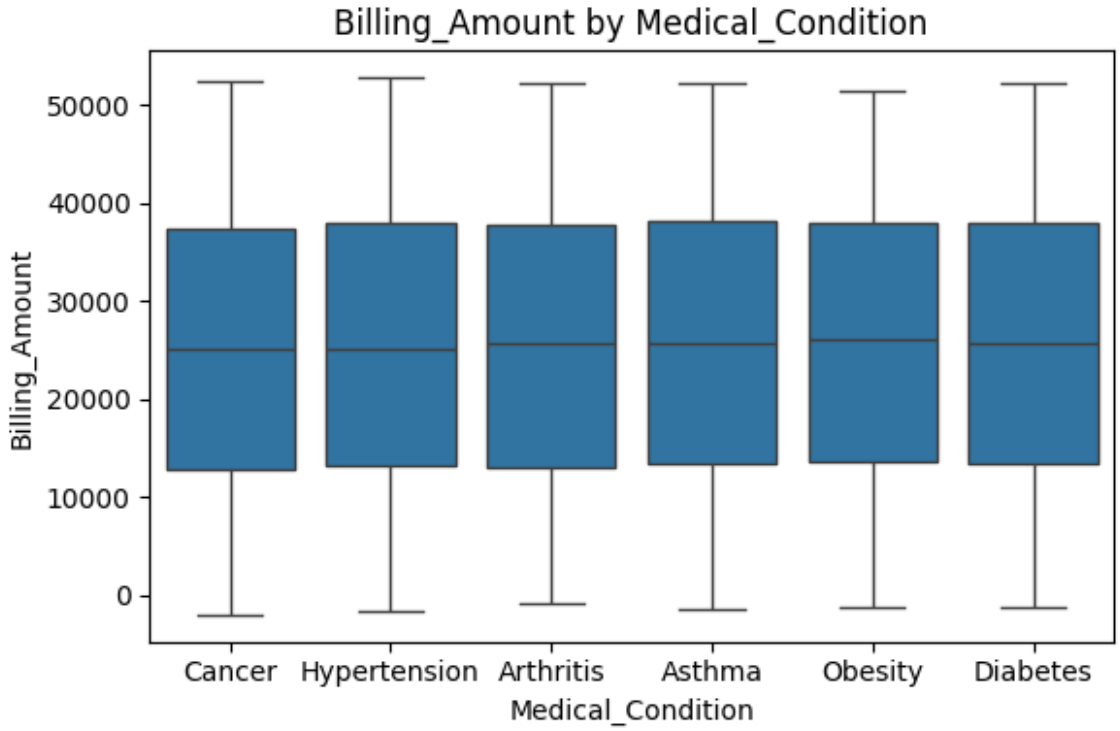

Weight by Medication
Weight
180
160
140
120
100
80
60
40
Lipitor
Penicillin
Aspirin
Ibuprofen
Paracetamol
Medication

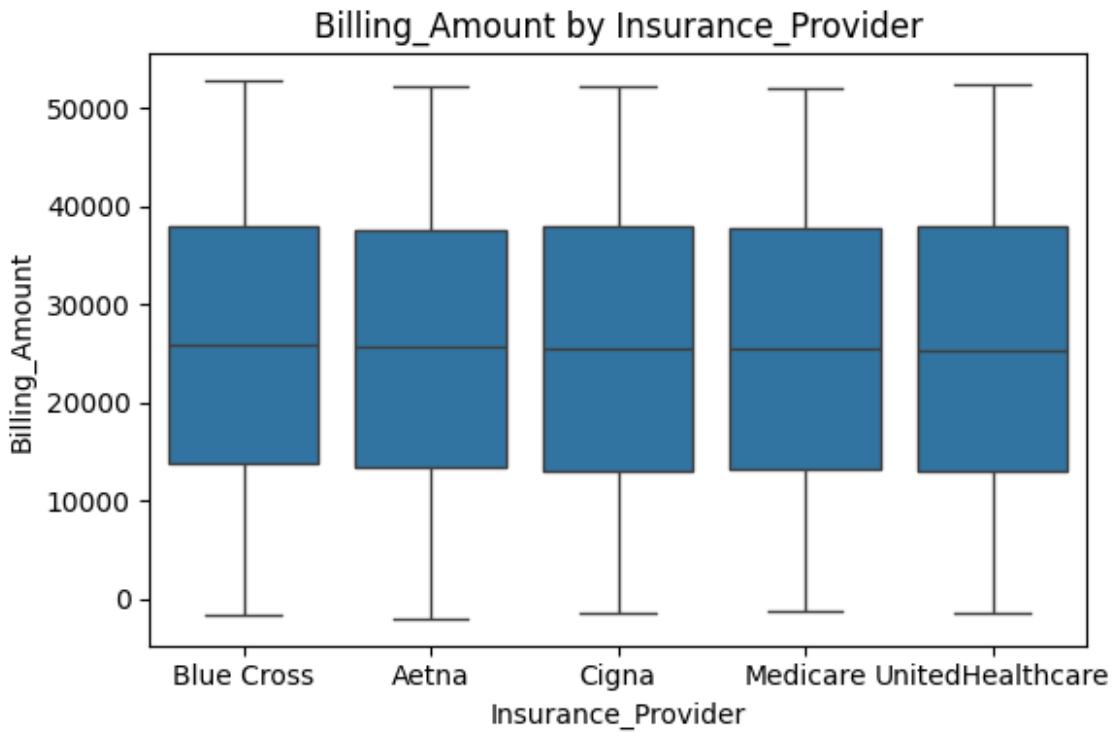

Weight by Test_Results
Weight
180
160
140
120
100
80
60
40
Normal
Abnormal
Inconclusive
Test_Results

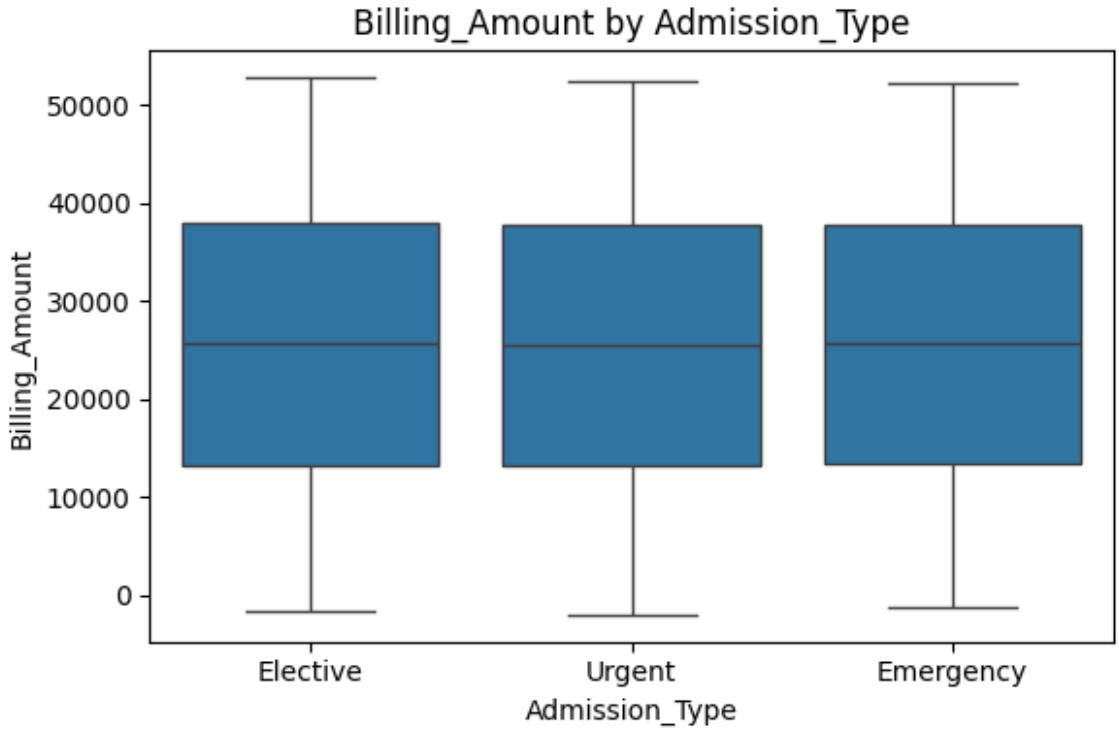

Billing_Amount by Gender
Billing_Amount
50000
40000
30000
20000
10000
0
Male
Female
Gender

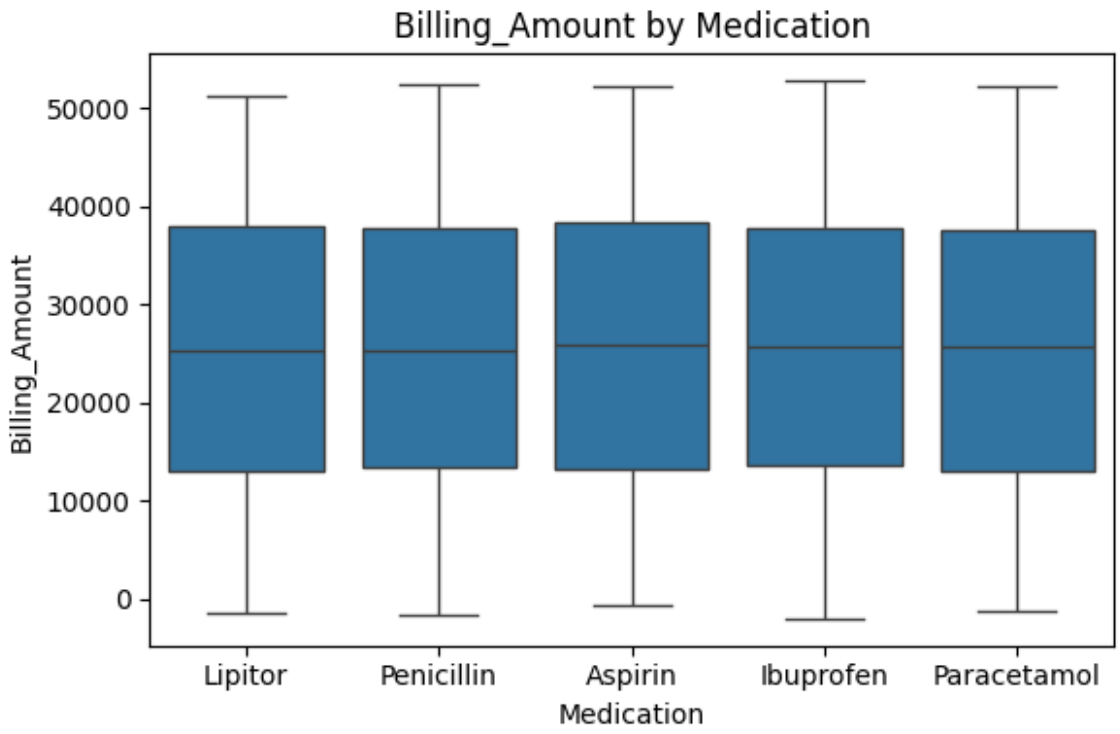

Billing_Amount by Ethnicity
Billing_Amount
50000
40000
30000
20000
10000
0
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

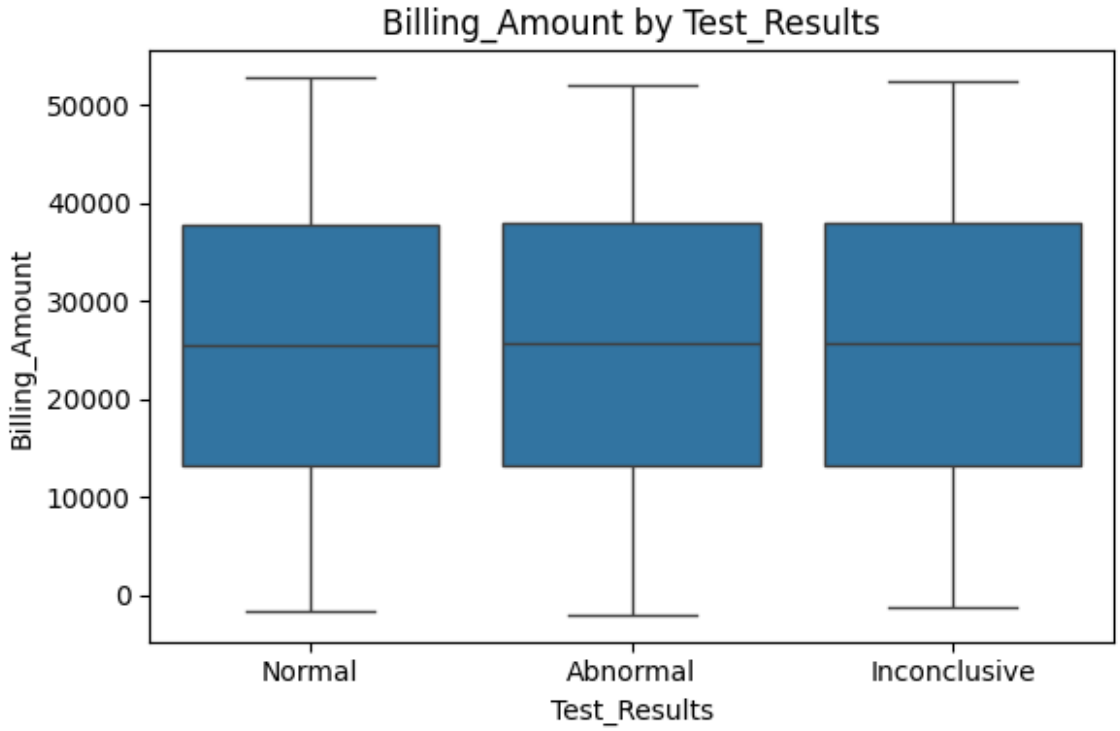

Billing_Amount by Blood_Type
Billing_Amount
50000
40000
30000
20000
10000
0
B+
A-
AB-
AB+
O-
B-
O+
A+
Blood_Type

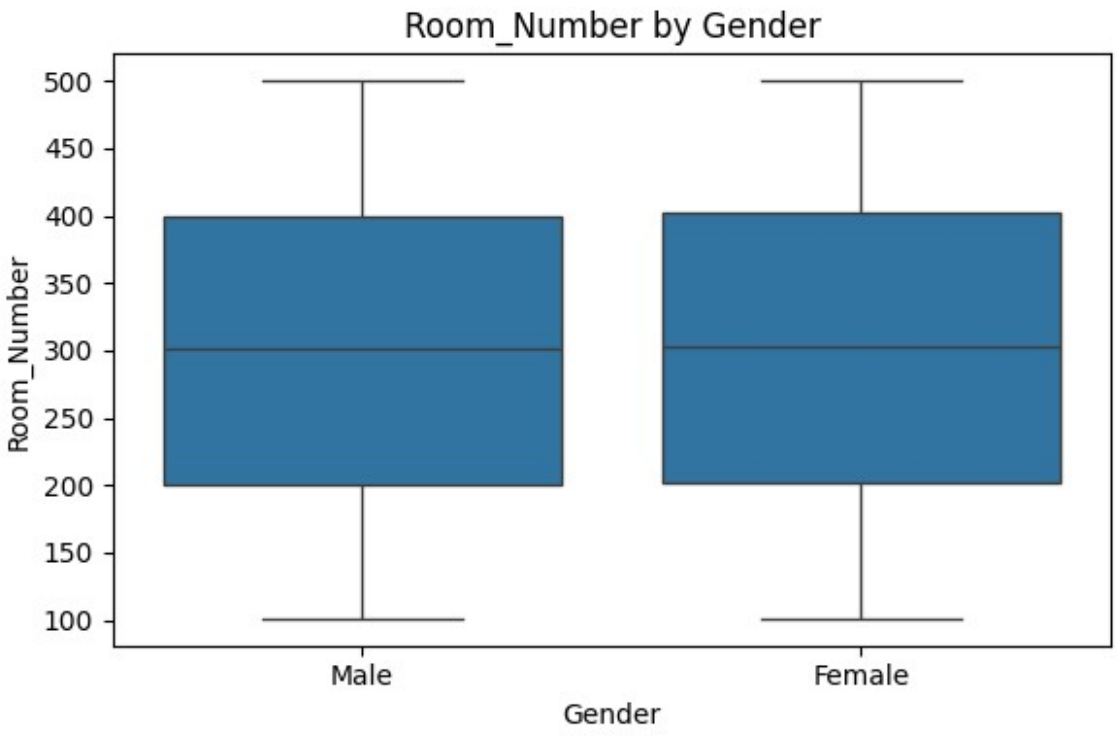

Billing_Amount by Medical_Condition
Billing_Amount
50000
40000
30000
20000
10000
0
Cancer
Hypertension
Arthritis
Asthma
Obesity
Diabetes
Medical_Condition

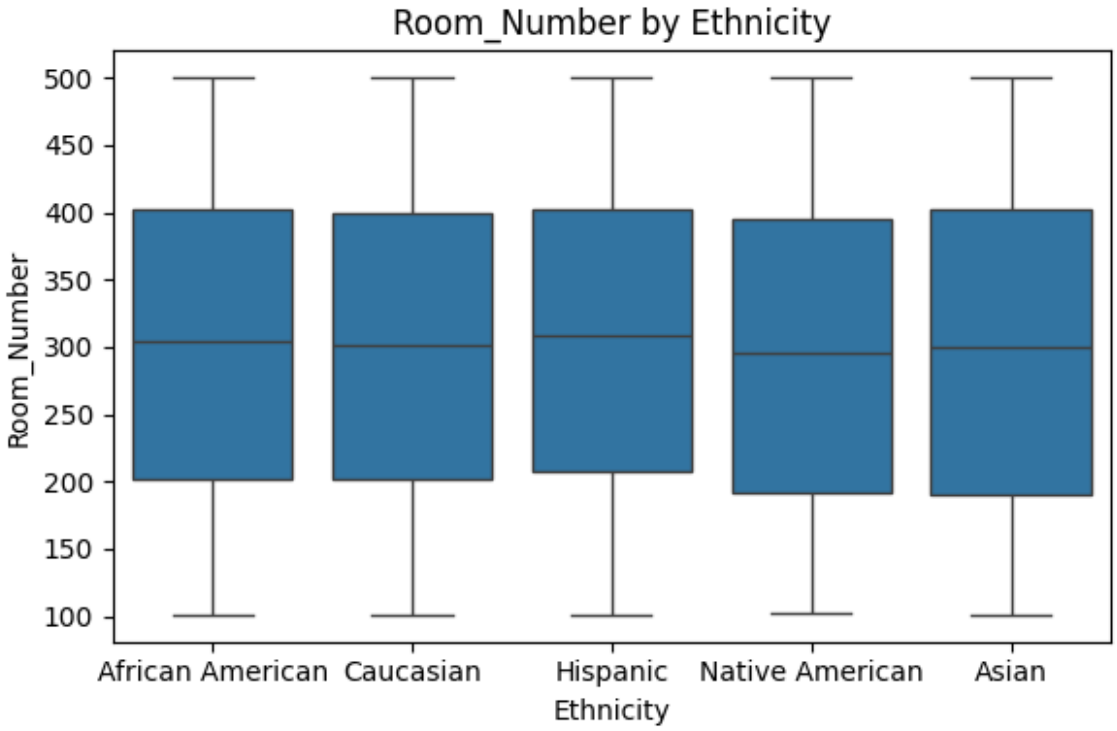

Billing_Amount by Insurance_Provider
Billing_Amount
50000
40000
30000
20000
10000
0
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider

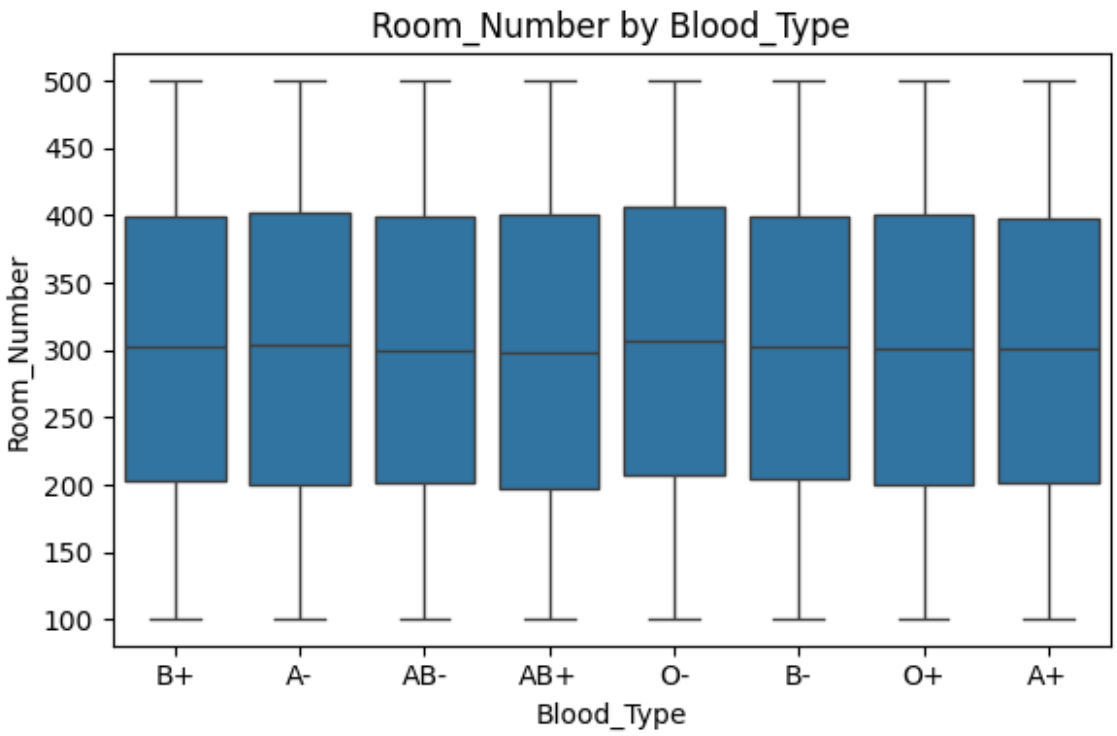

Billing_Amount by Admission_Type
Billing_Amount
50000
40000
30000
20000
10000
0
Elective
Urgent
Emergency
Admission_Type

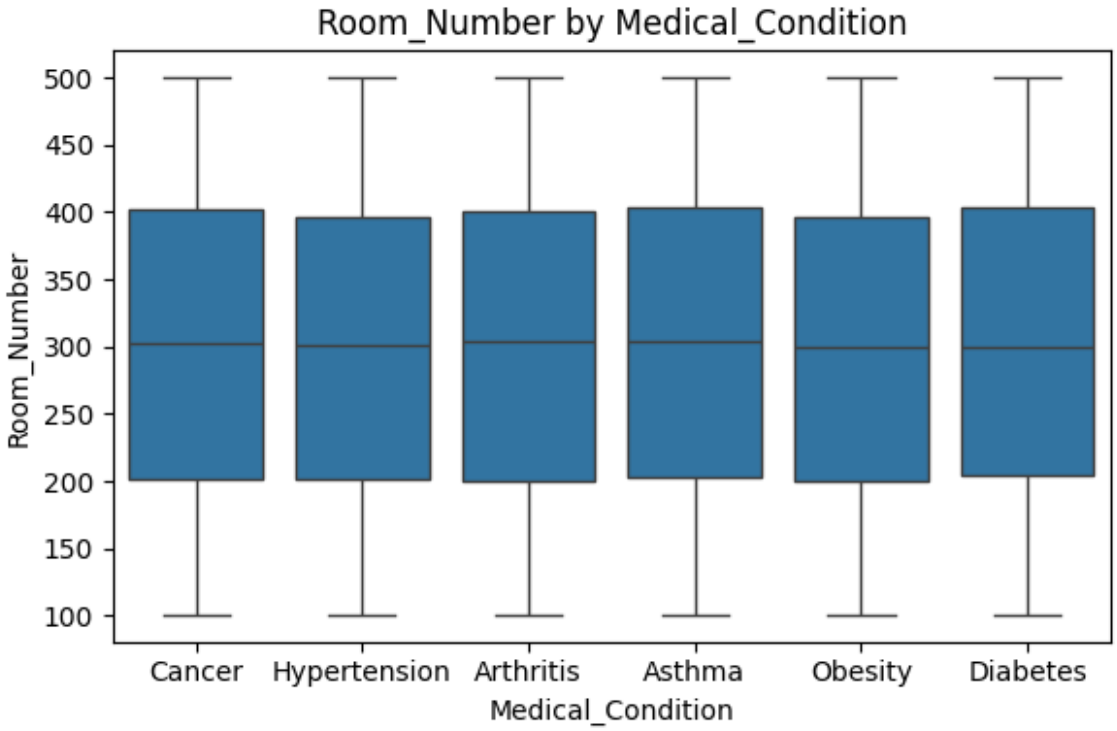

Billing_Amount by Medication
Billing_Amount
50000
40000
30000
20000
10000
0
Lipitor
Penicillin
Aspirin
Ibuprofen
Paracetamol
Medication

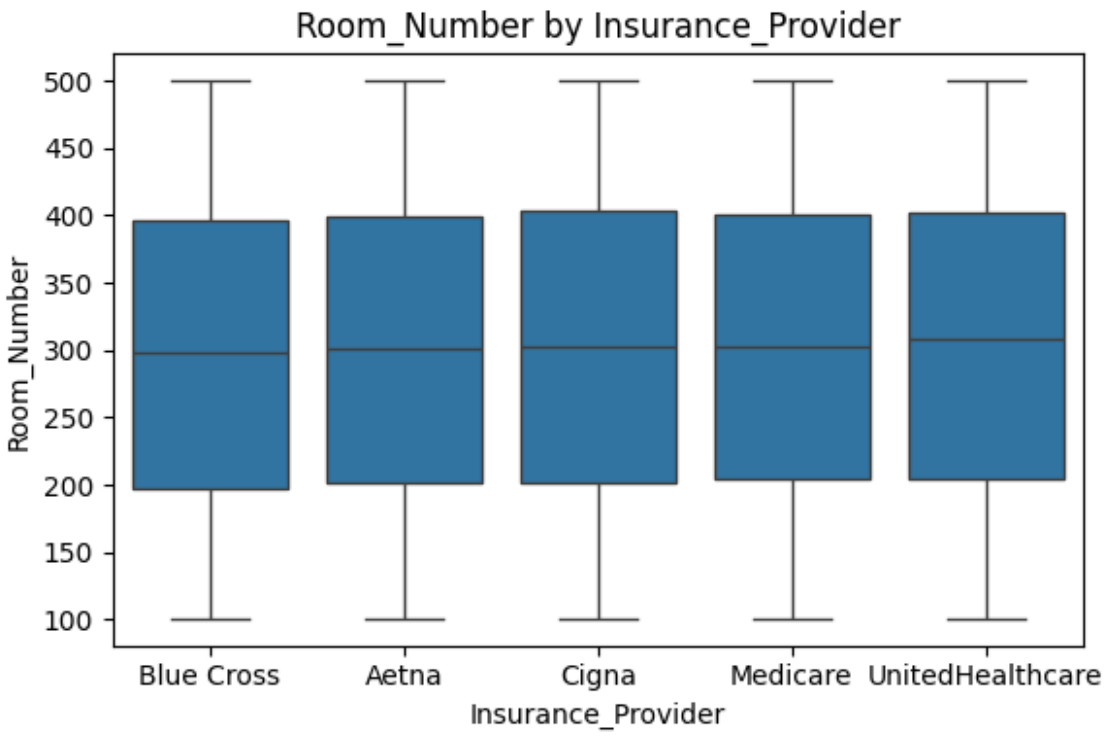

Billing_Amount by Test_Results
Billing_Amount
50000
40000
30000
20000
10000
0
Normal
Abnormal
Inconclusive
Test_Results

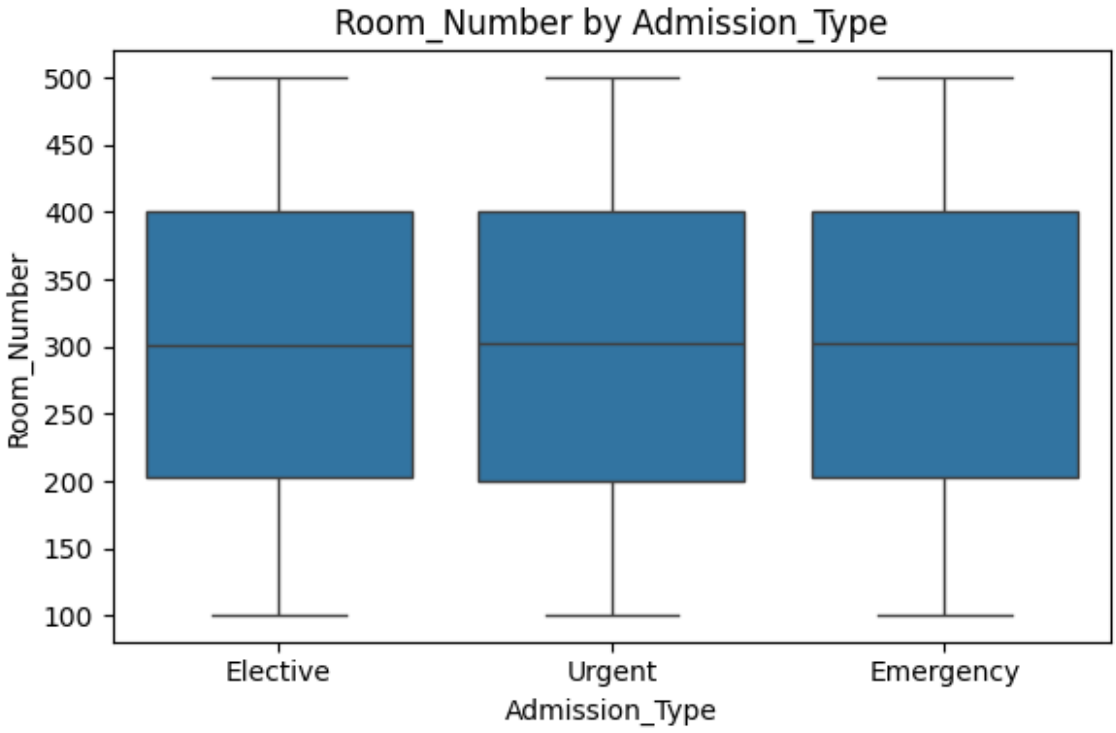

Room_Number by Gender
Room_Number
500
450
400
350
300
250
200
150
100
Male
Female
Gender

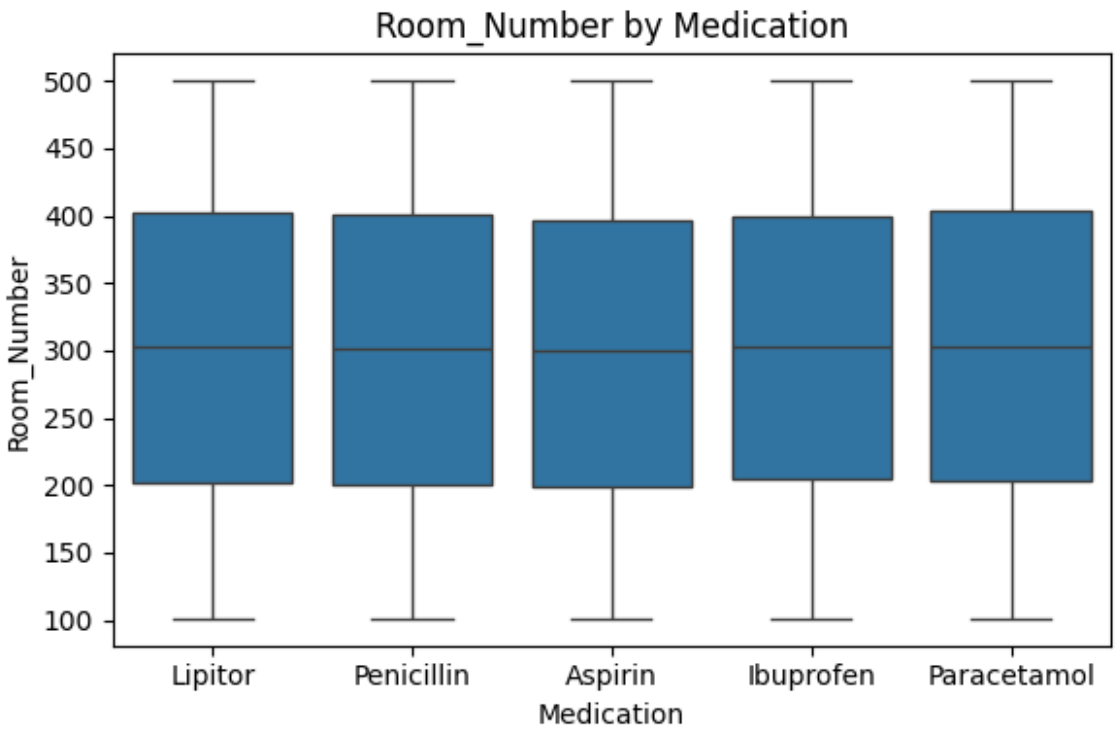

Room_Number by Ethnicity
Room_Number
500
450
400
350
300
250
200
150
100
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

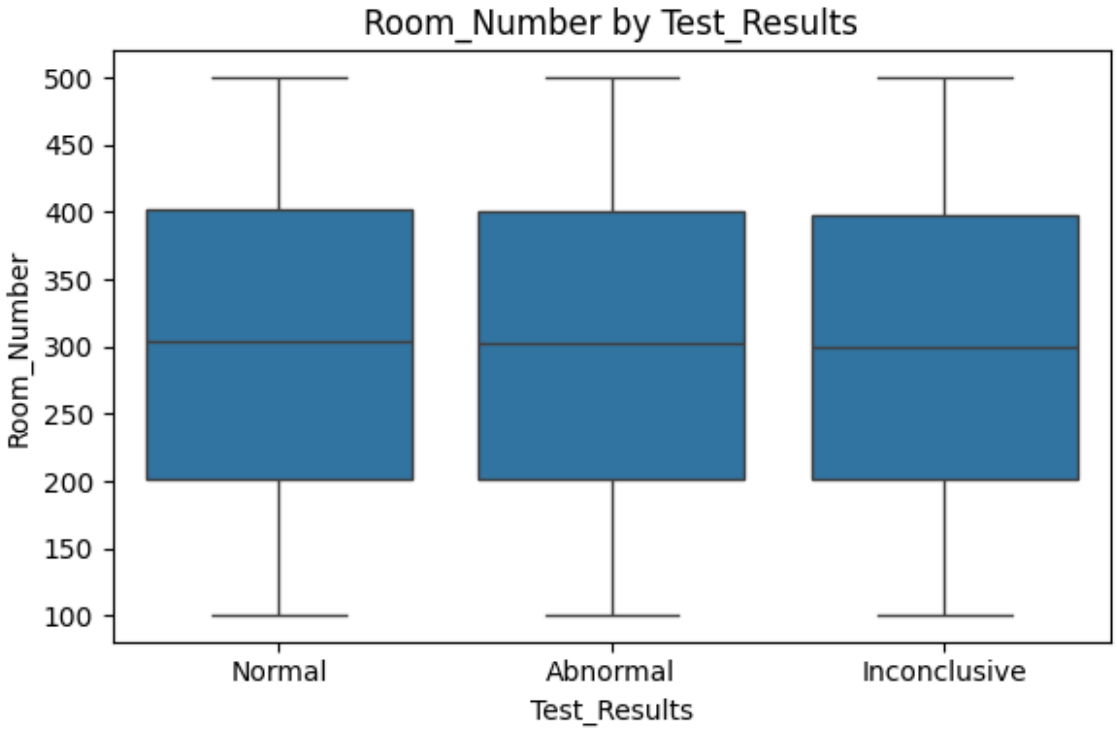

Room_Number by Blood_Type
Room_Number
500
450
400
350
300
250
200
150
100
B+
A-
AB-
AB+
O-
B-
O+
A+
Blood_Type

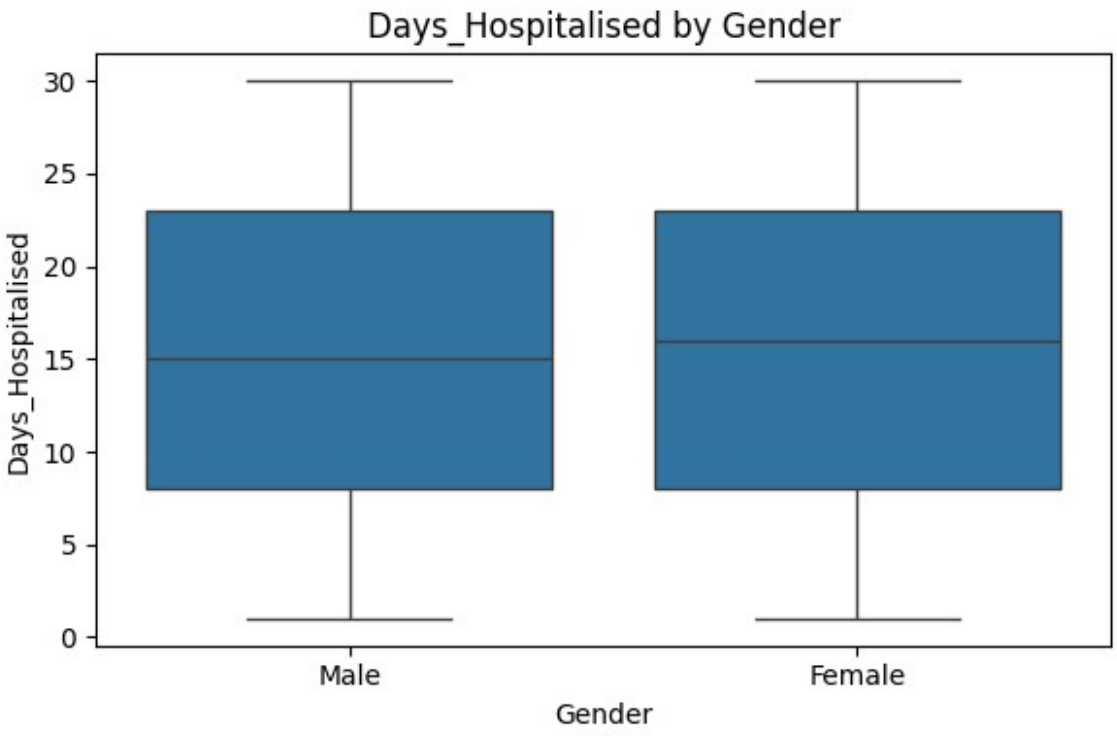

Room_Number by Medical_Condition
Room_Number
500
450
400
350
300
250
200
150
100
Cancer
Hypertension
Arthritis
Asthma
Obesity
Diabetes
Medical_Condition

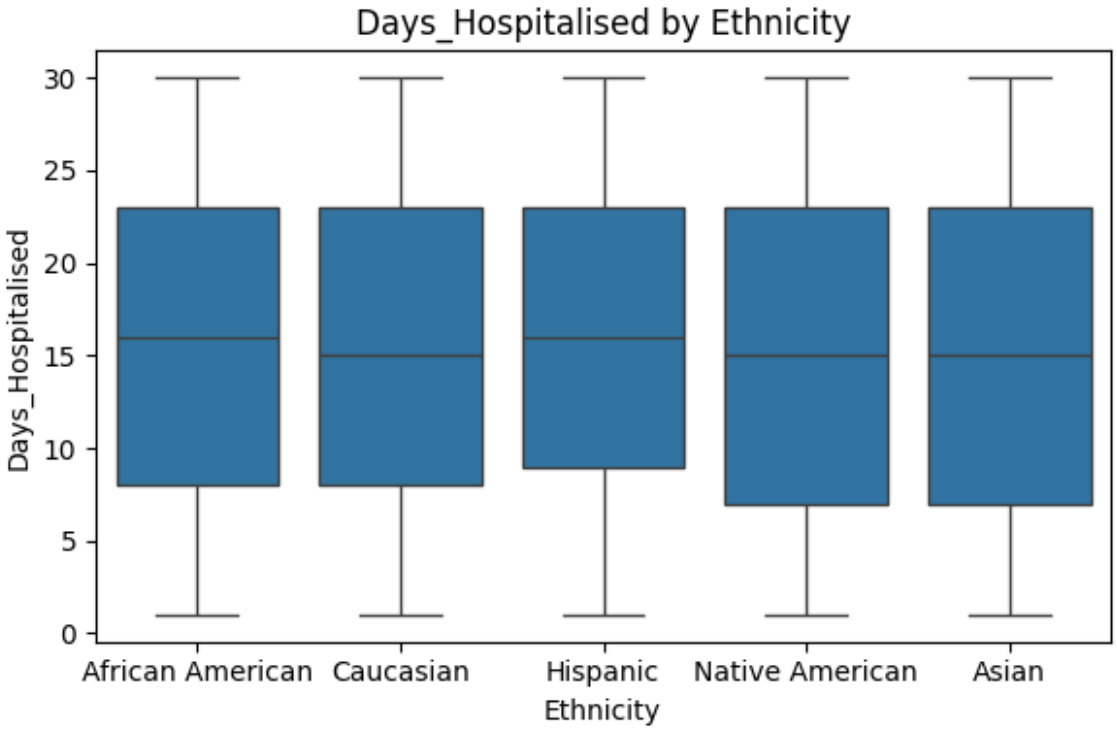

Room_Number by Insurance_Provider
Room_Number
500
450
400
350
300
250
200
150
100
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider

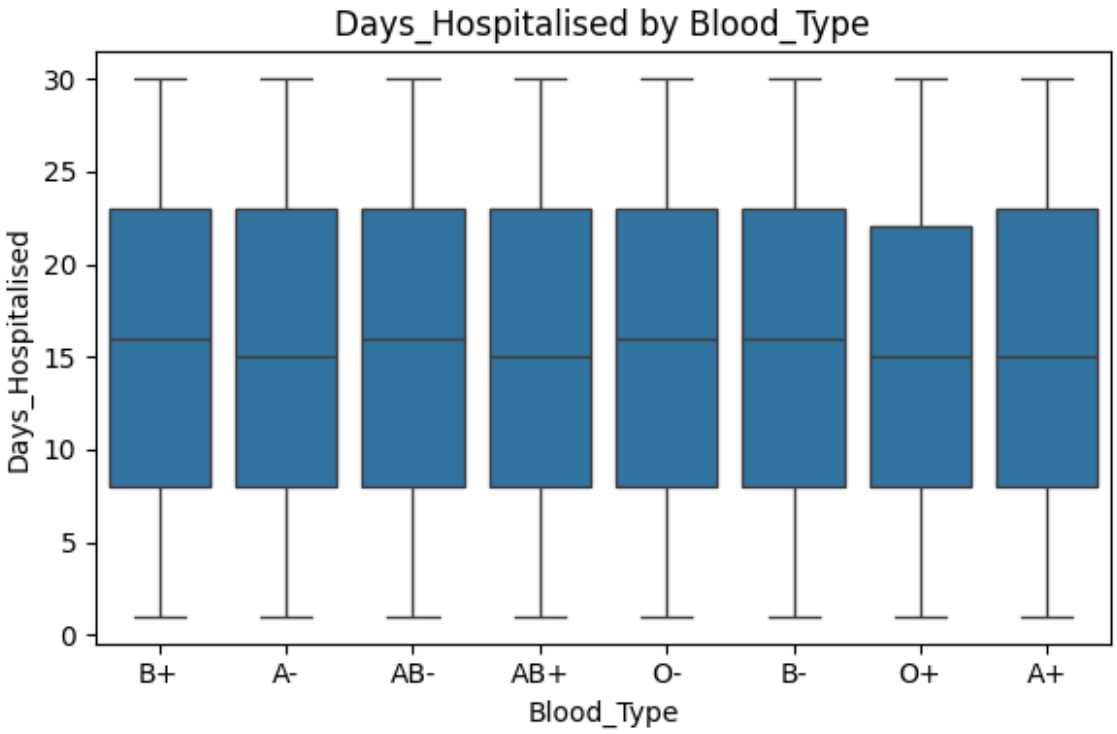

Room_Number by Admission_Type
Room_Number
500
450
400
350
300
250
200
150
100
Elective
Urgent
Emergency
Admission_Type

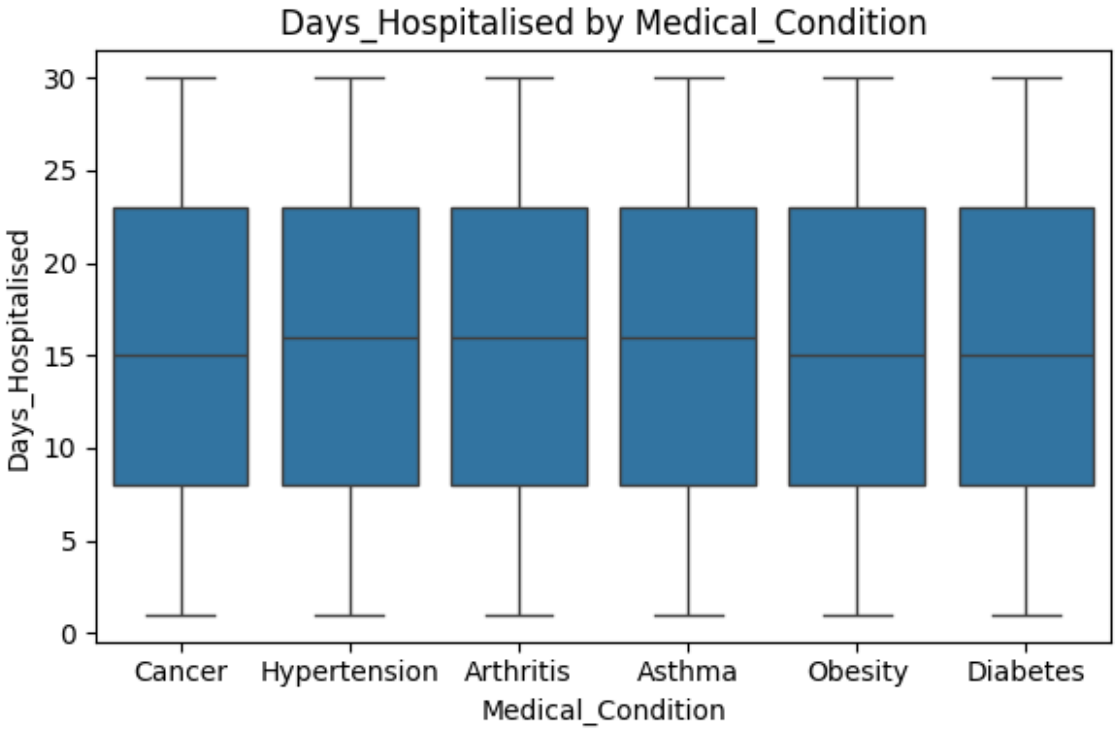

Room_Number by Medication
Room_Number
500
450
400
350
300
250
200
150
100
Lipitor
Penicillin
Aspirin
Ibuprofen
Paracetamol
Medication

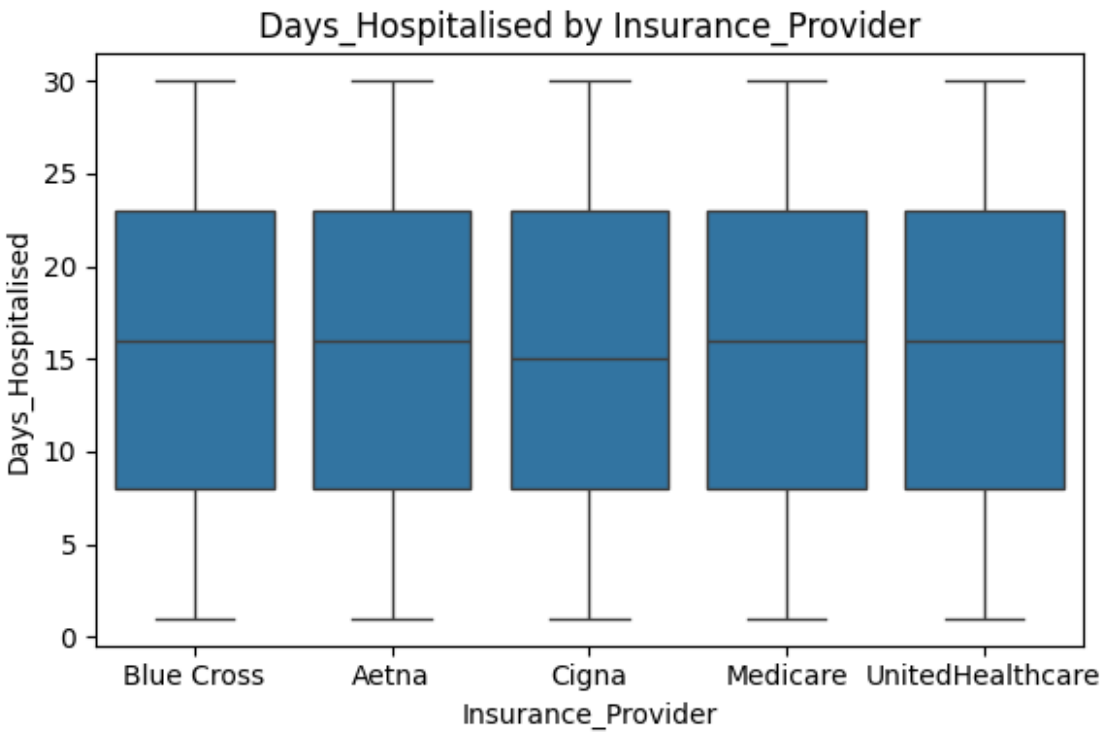

Room_Number by Test_Results
Room_Number
500
450
400
350
300
250
200
150
100
Normal
Abnormal
Inconclusive
Test_Results

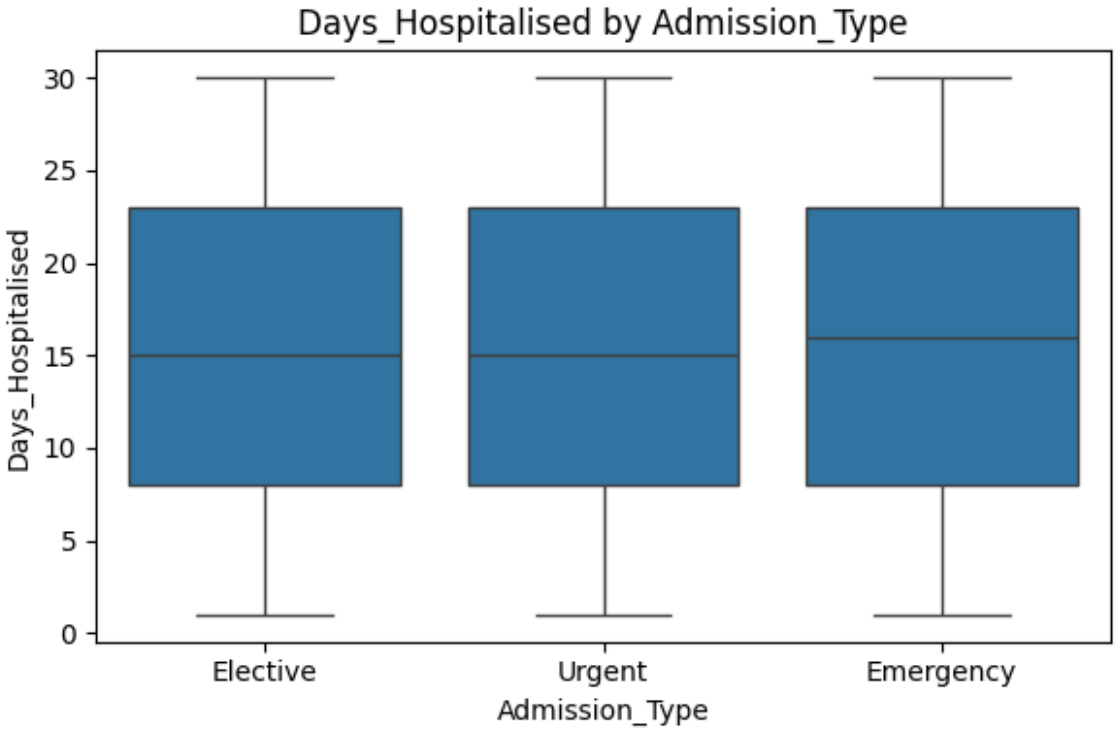

Days_Hospitalised by Gender
Days_Hospitalised
30
25
20
15
10
5
0
Male
Female
Gender

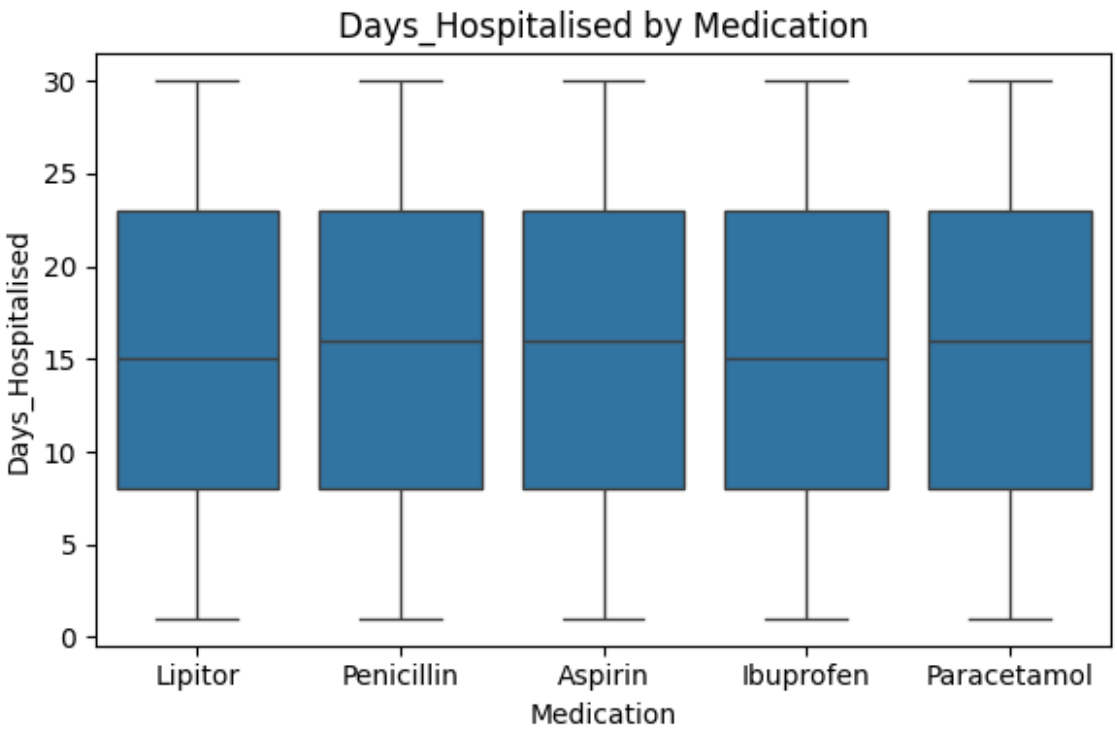

Days_Hospitalised by Ethnicity
Days_Hospitalised
30
25
20
15
10
5
0
African American
Caucasian
Hispanic
Native American
Asian
Ethnicity

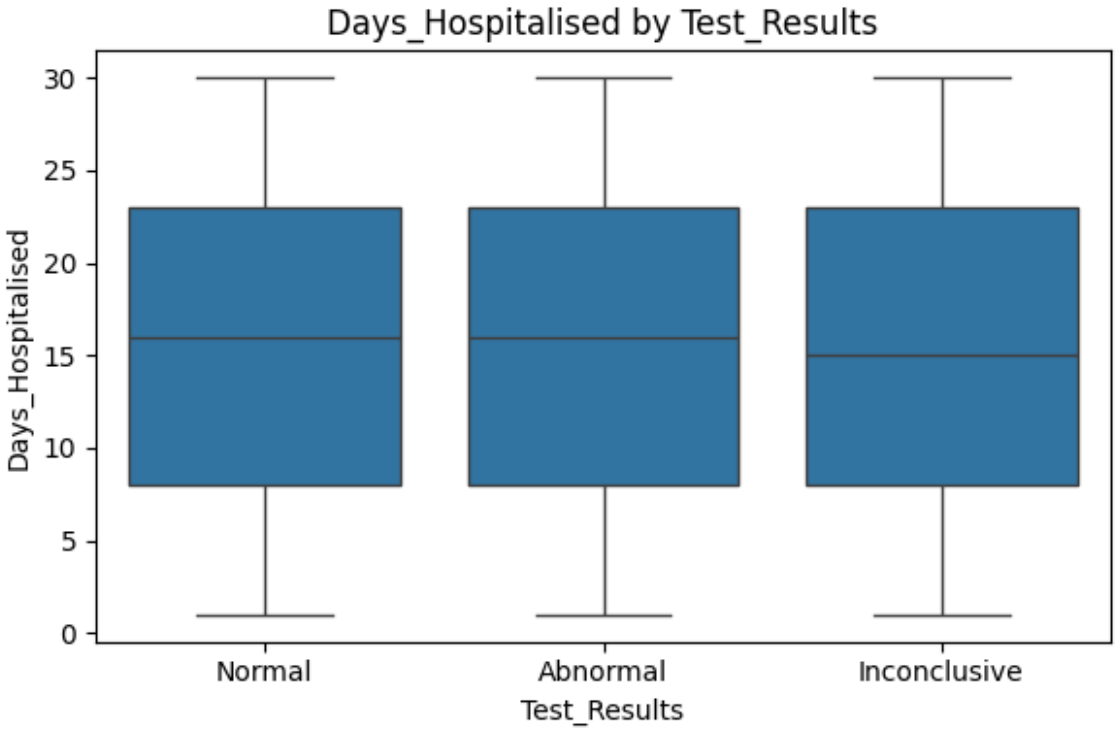

Days_Hospitalised by Blood_Type
Days_Hospitalised
0
5
10
15
20
25
30
B+
A-
AB-
AB+
O-
B-
O+
A+
Blood_Type


Days_Hospitalised by Medical_Condition
Days_Hospitalised
0
5
10
15
20
25
30
Cancer
Hypertension
Arthritis
Asthma
Obesity
Diabetes
Medical_Condition

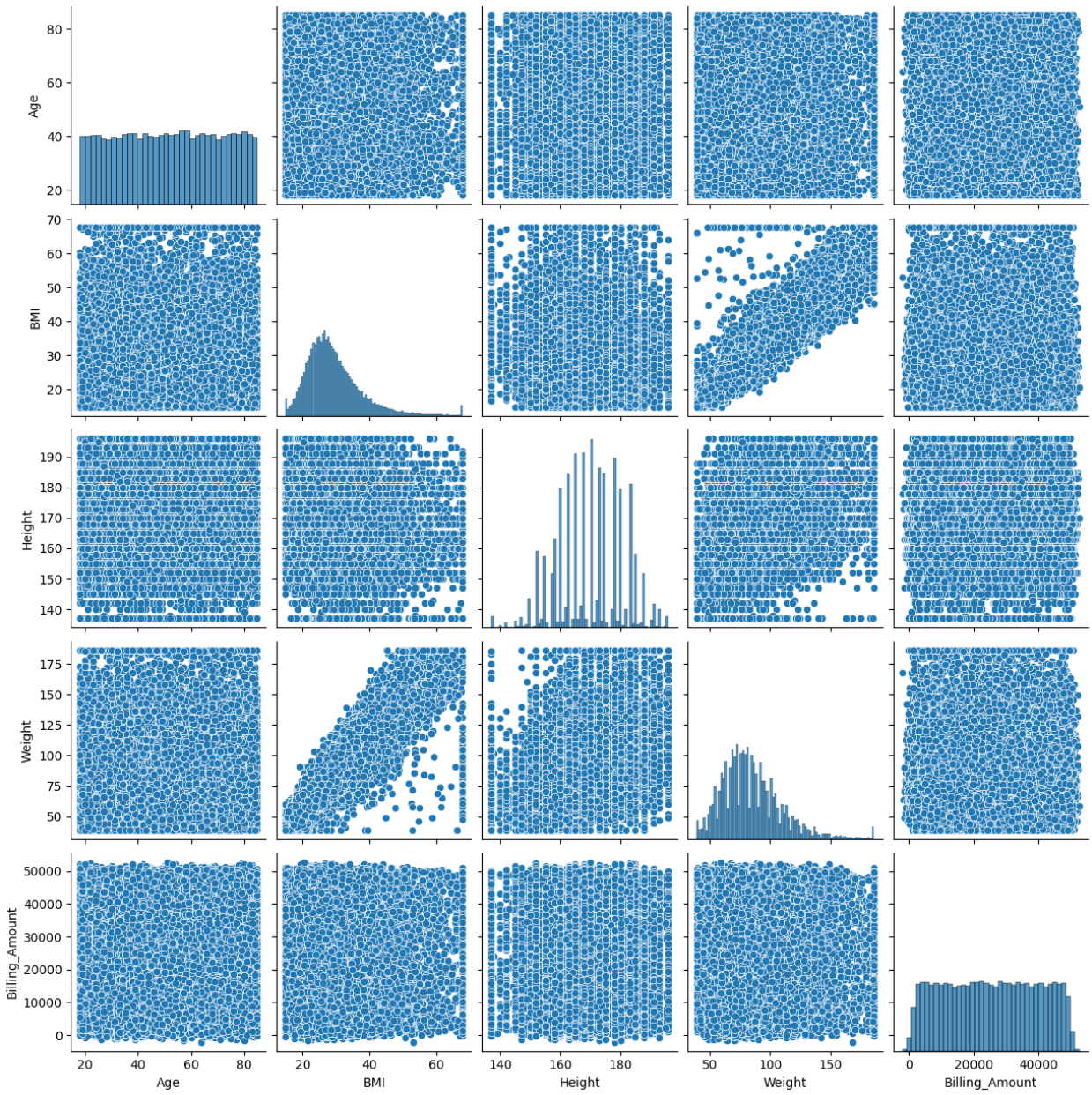

Days_Hospitalised by Insurance_Provider
Days_Hospitalised
0
5
10
15
20
25
30
Blue Cross
Aetna
Cigna
Medicare
UnitedHealthcare
Insurance_Provider


Days_Hospitalised by Admission_Type
Days_Hospitalised
0
5
10
15
20
25
30
Elective
Urgent
Emergency
Admission_Type

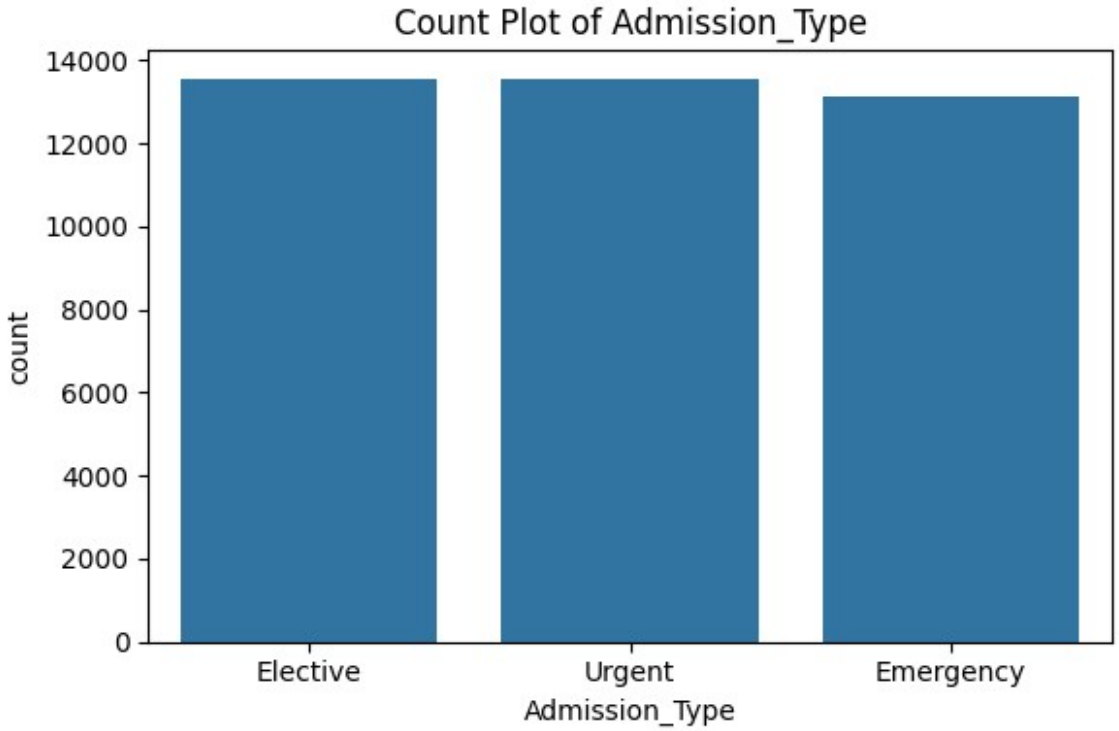




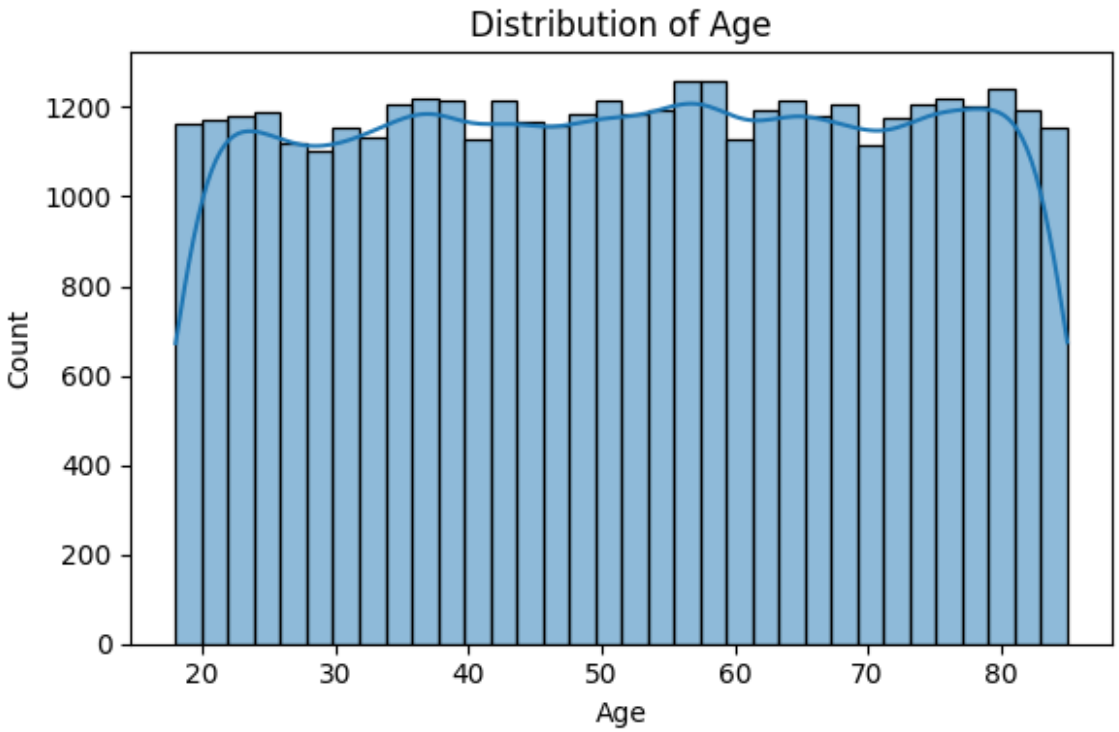

```
Creating pairplot...
```



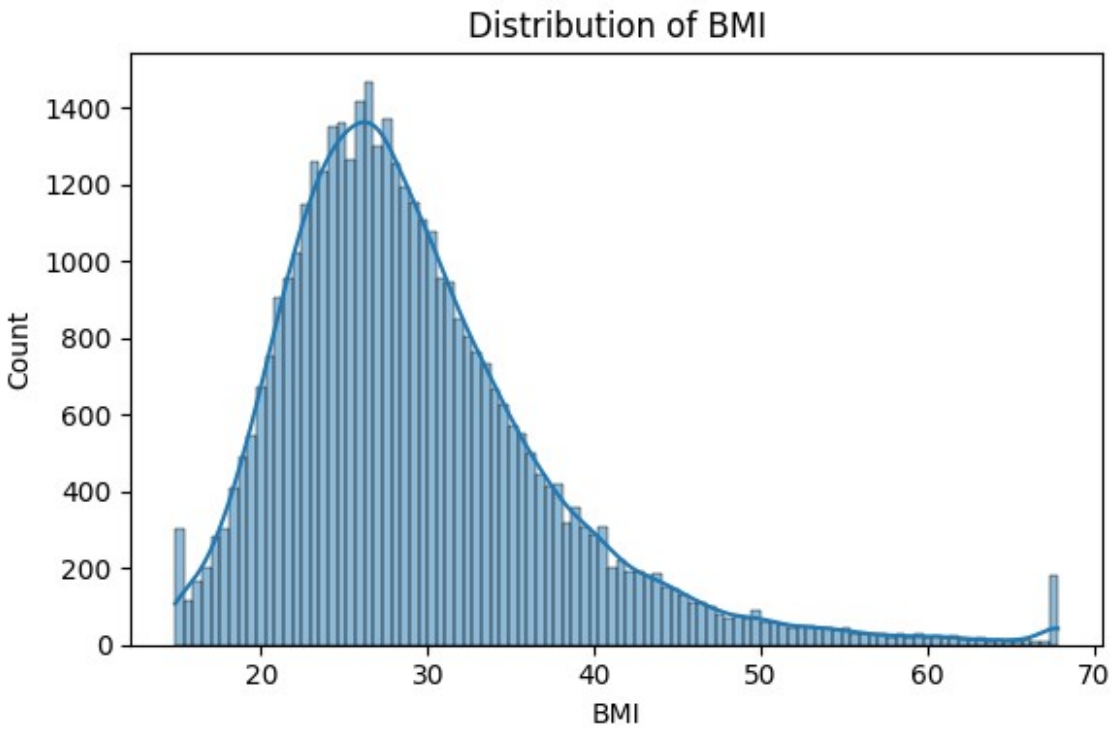

```
EDA report saved in 'eda_report' folder.
Admission_Type_plot.png
```


Count Plot of Admission_Type
14000
12000
10000
8000
6000
4000
2000
0
count
Elective
Urgent
Emergency
Admission_Type

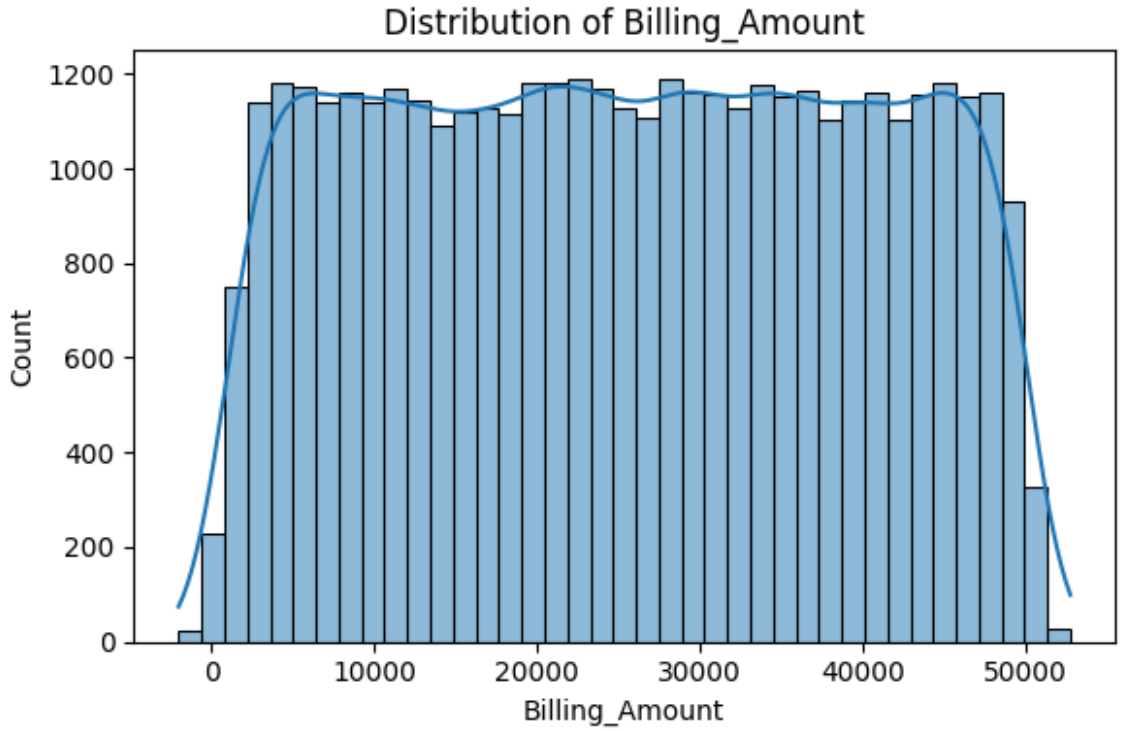
```
Age_plot.png
```


Distribution of Age
1200
1000
800
600
400
200
0
Count
20
30
40
50
60
70
80
Age

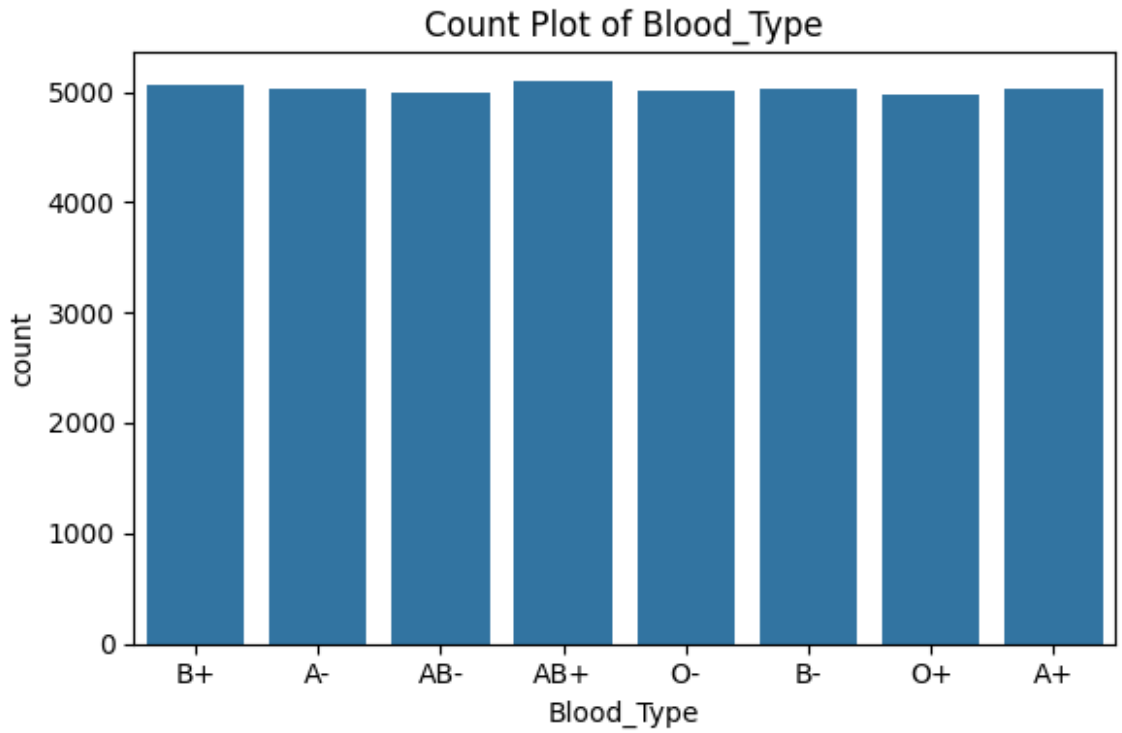
```
BMI_plot.png
```



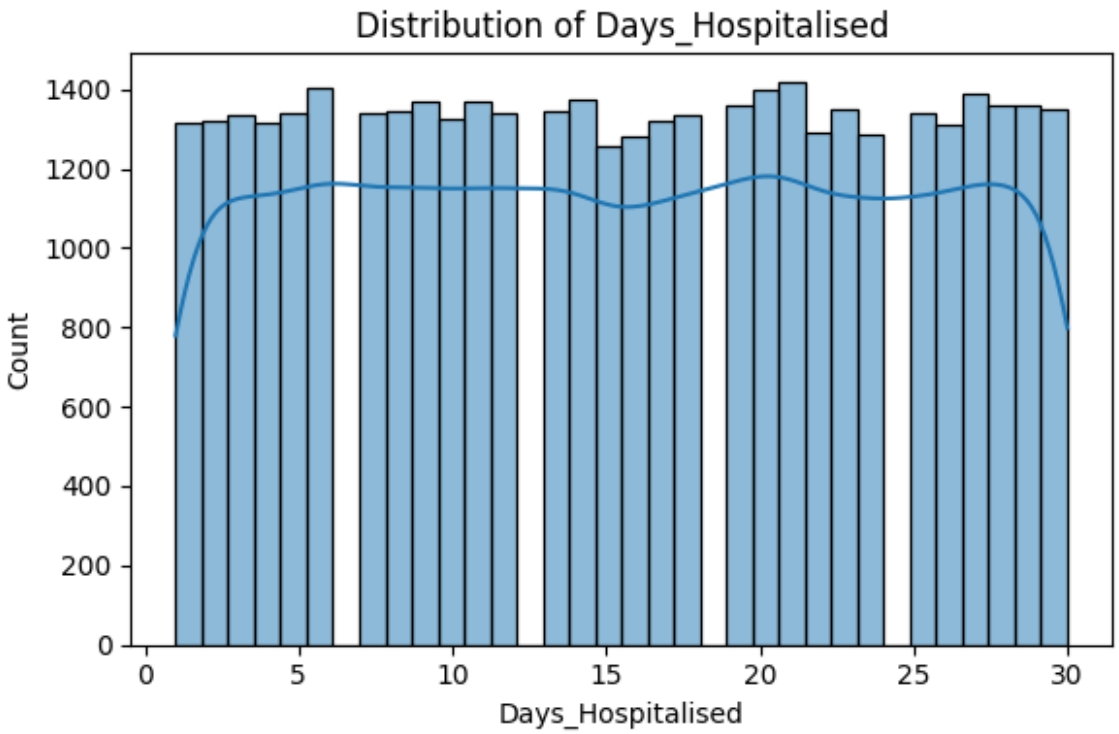

```
Billing_Amount_plot.png
```



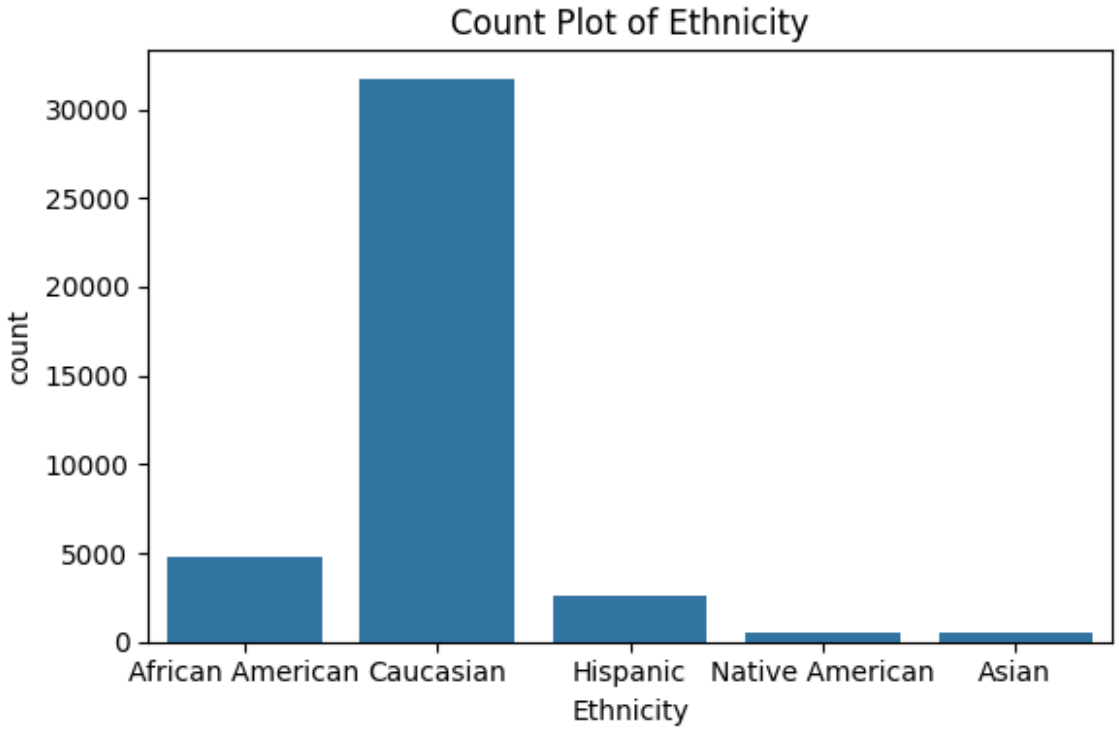

```
Blood_Type_plot.png
```



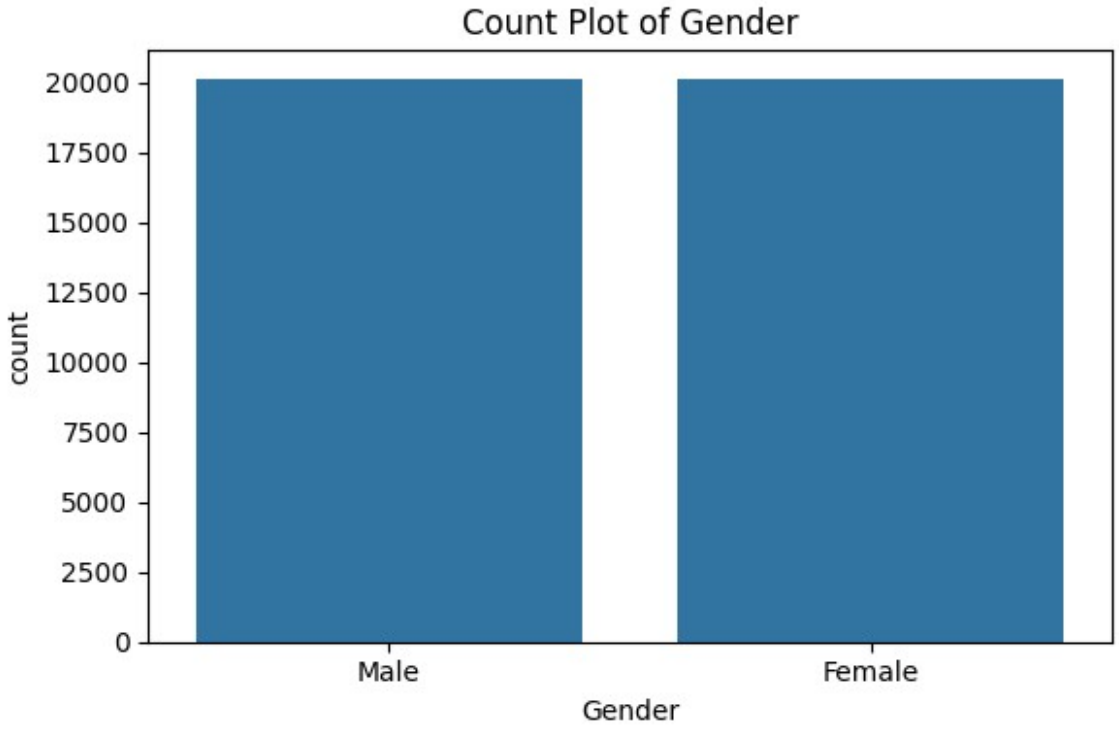
```
Days_Hospitalised_plot.png
```



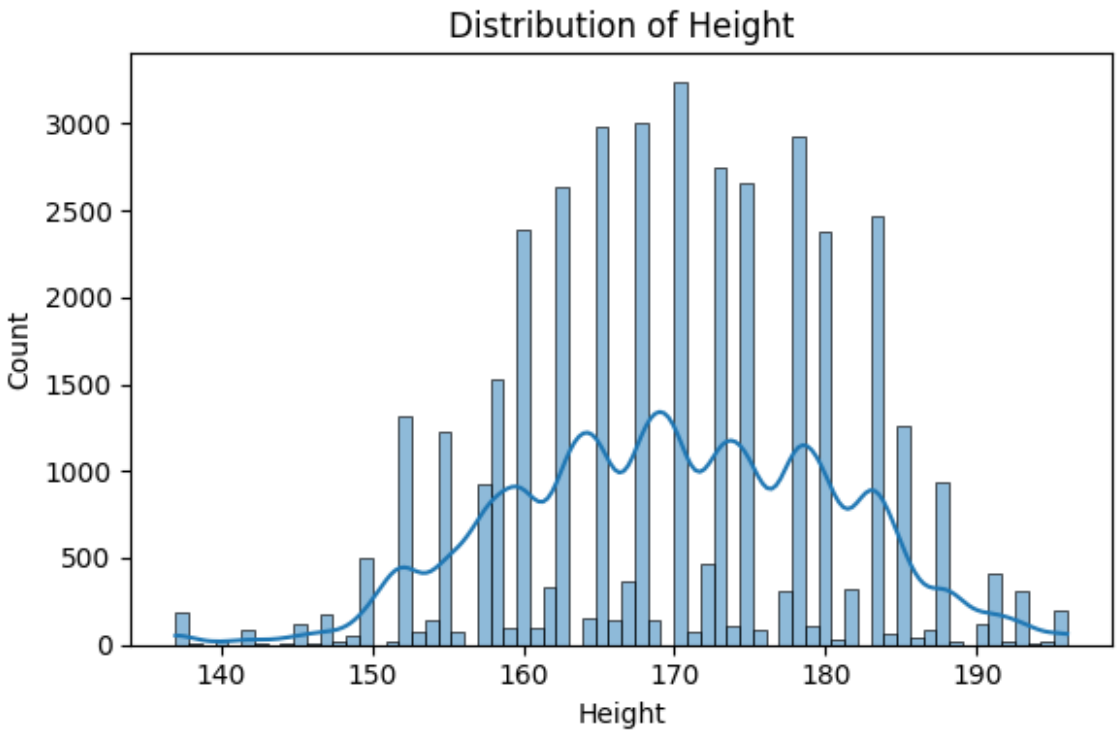

```
Ethnicity_plot.png
```



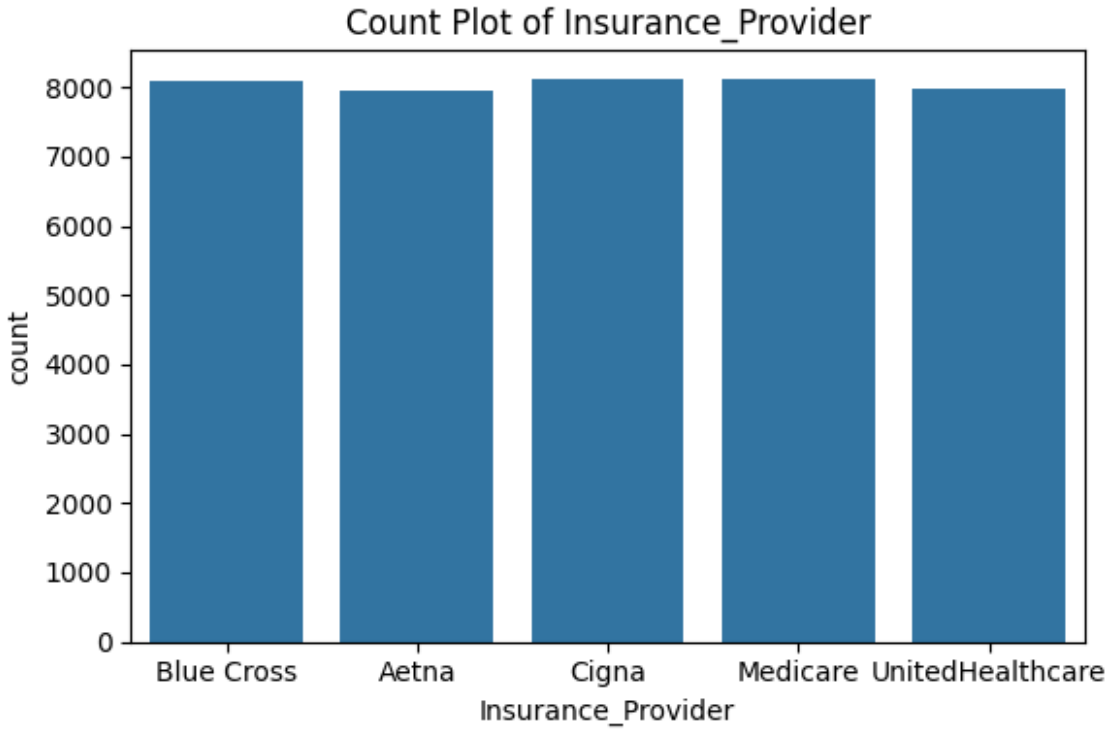

```
Gender_plot.png
```



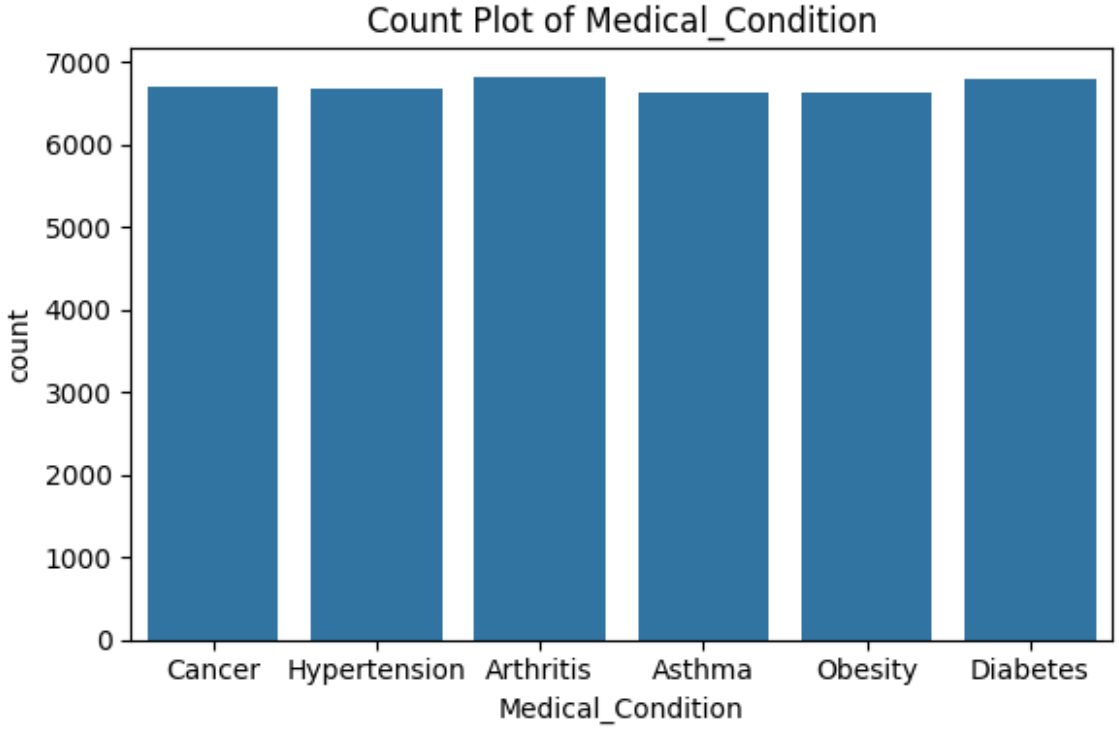
```
Height_plot.png
```



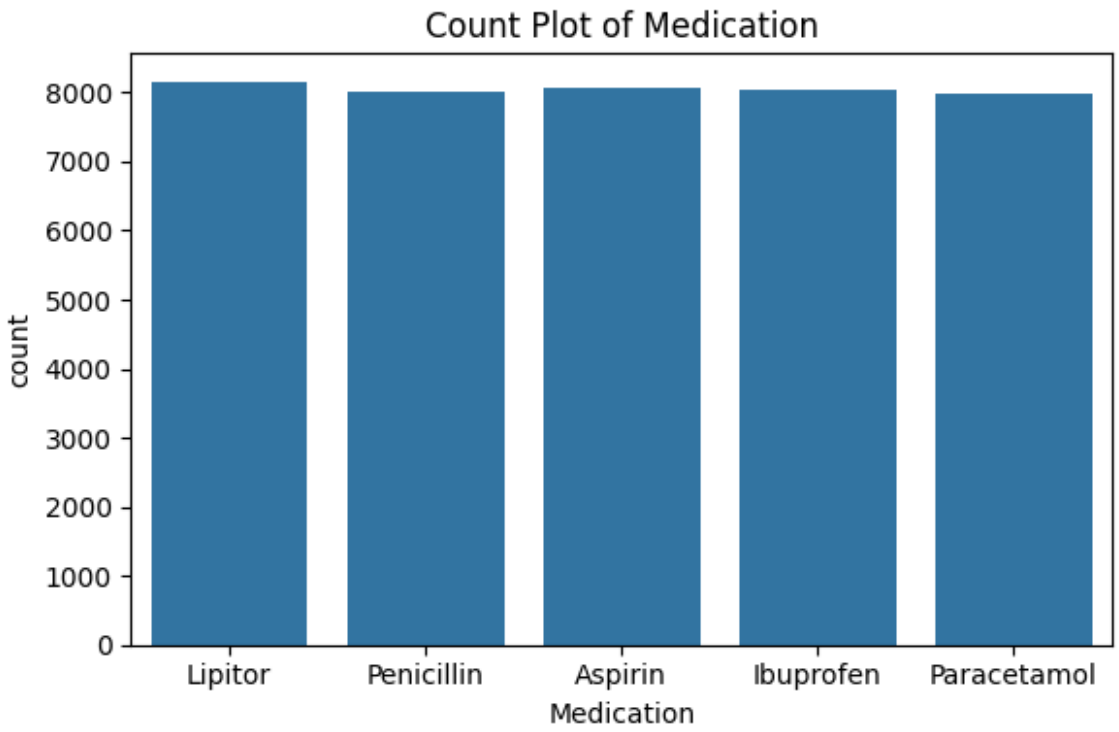

```
Insurance_Provider_plot.png
```



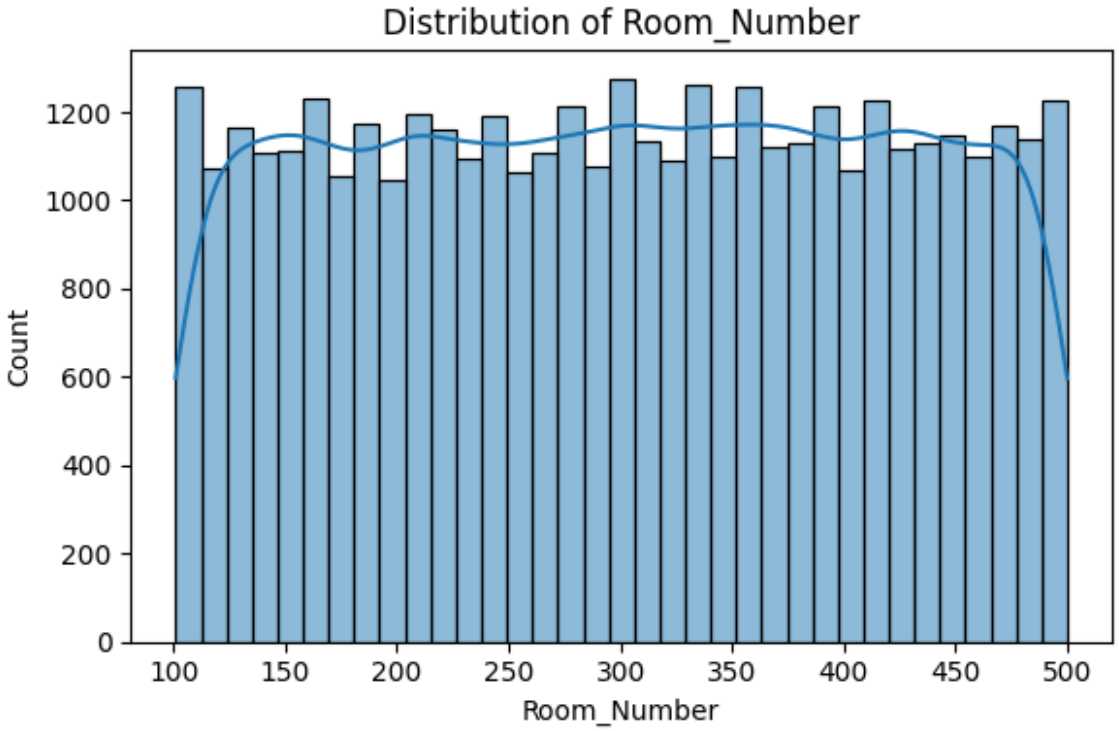

```
Medical_Condition_plot.png
```



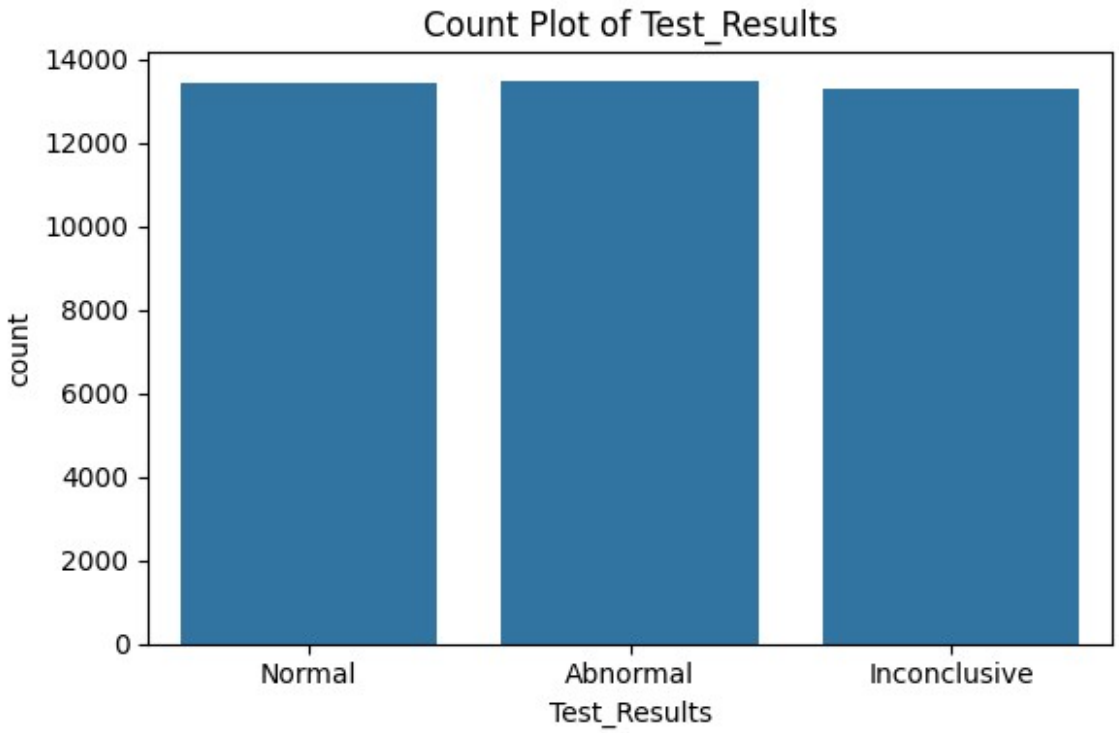
```
Medication_plot.png
```



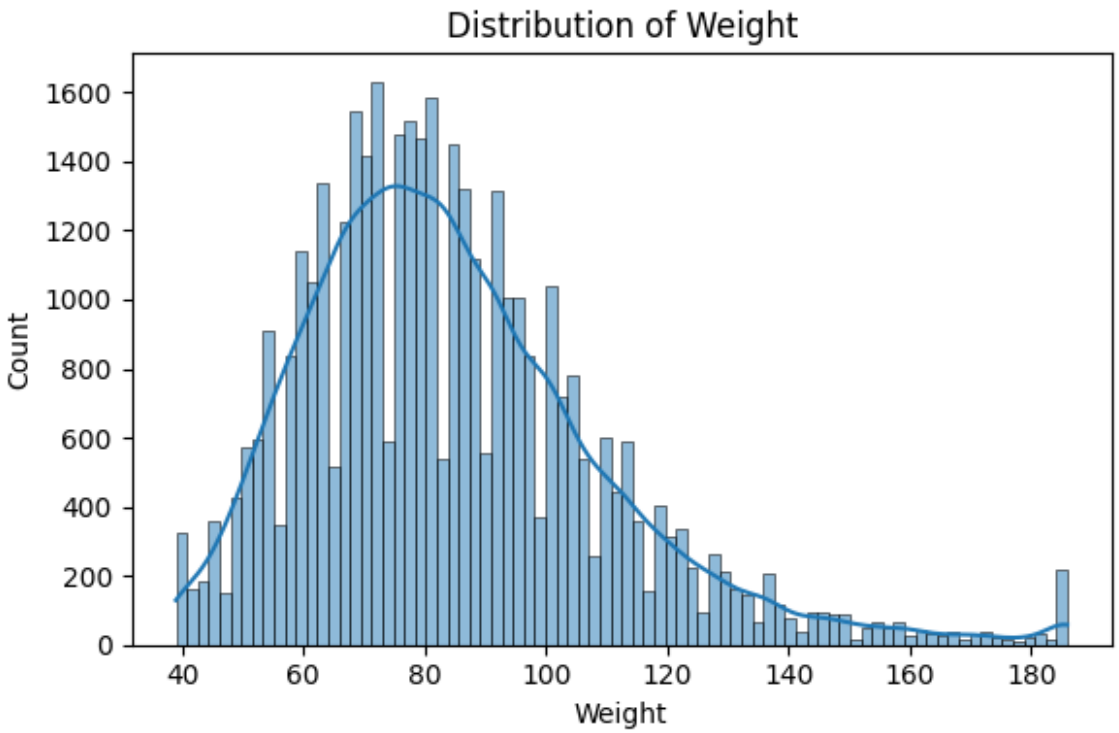

```
Room_Number_plot.png
```



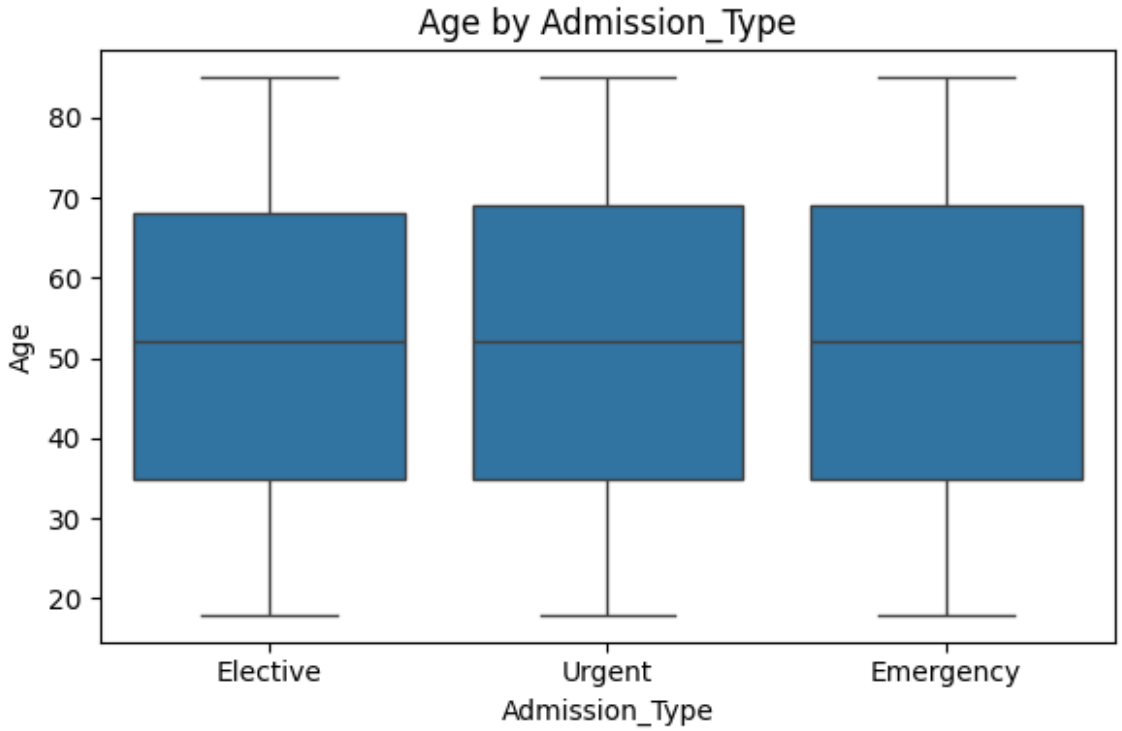

```
Test_Results_plot.png
```



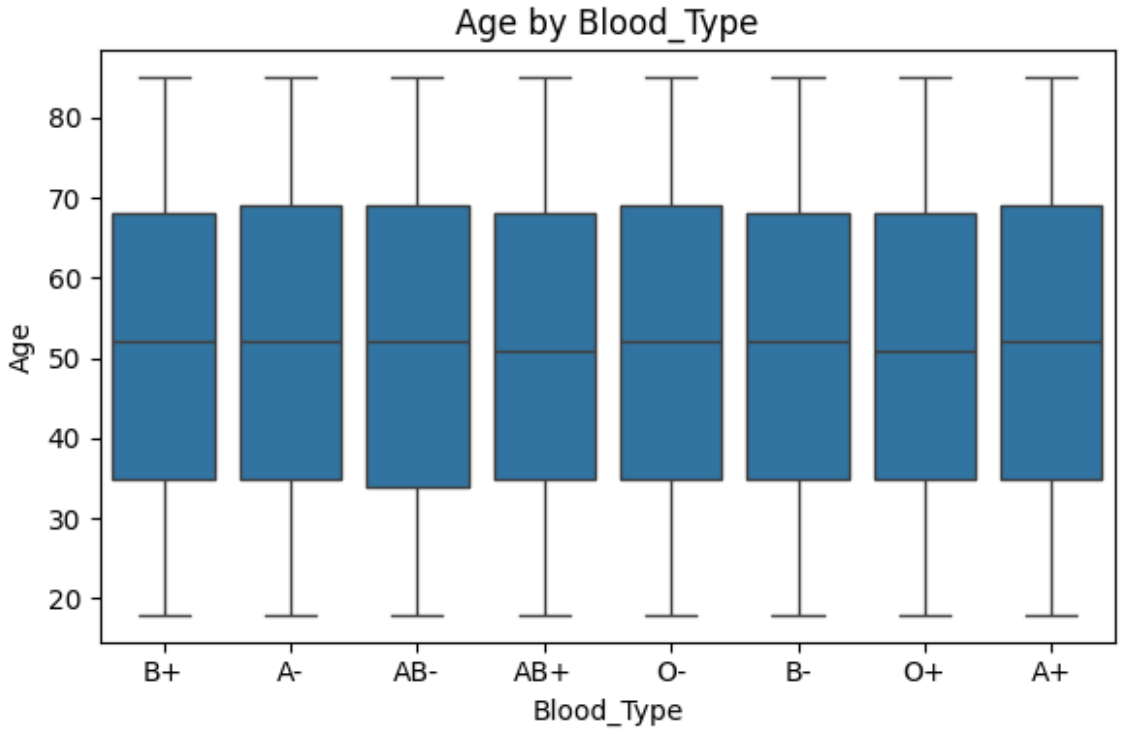
```
Weight_plot.png
```



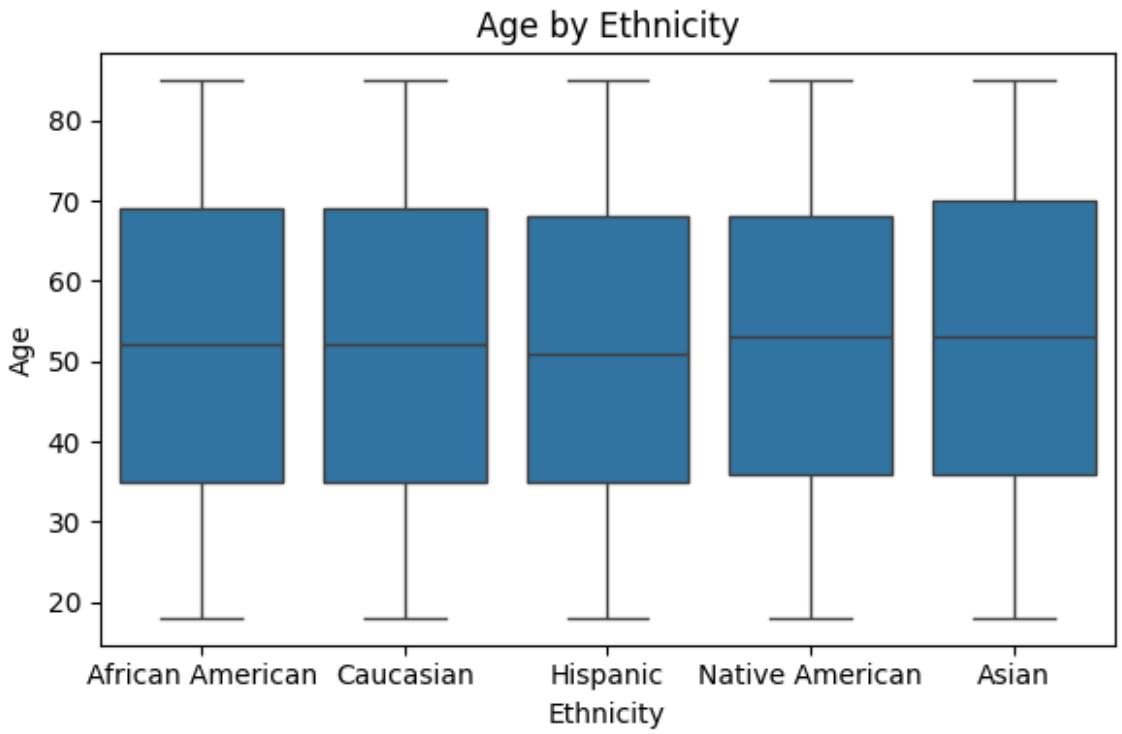

```
boxplot_Age_by_Admission_Type.png
```



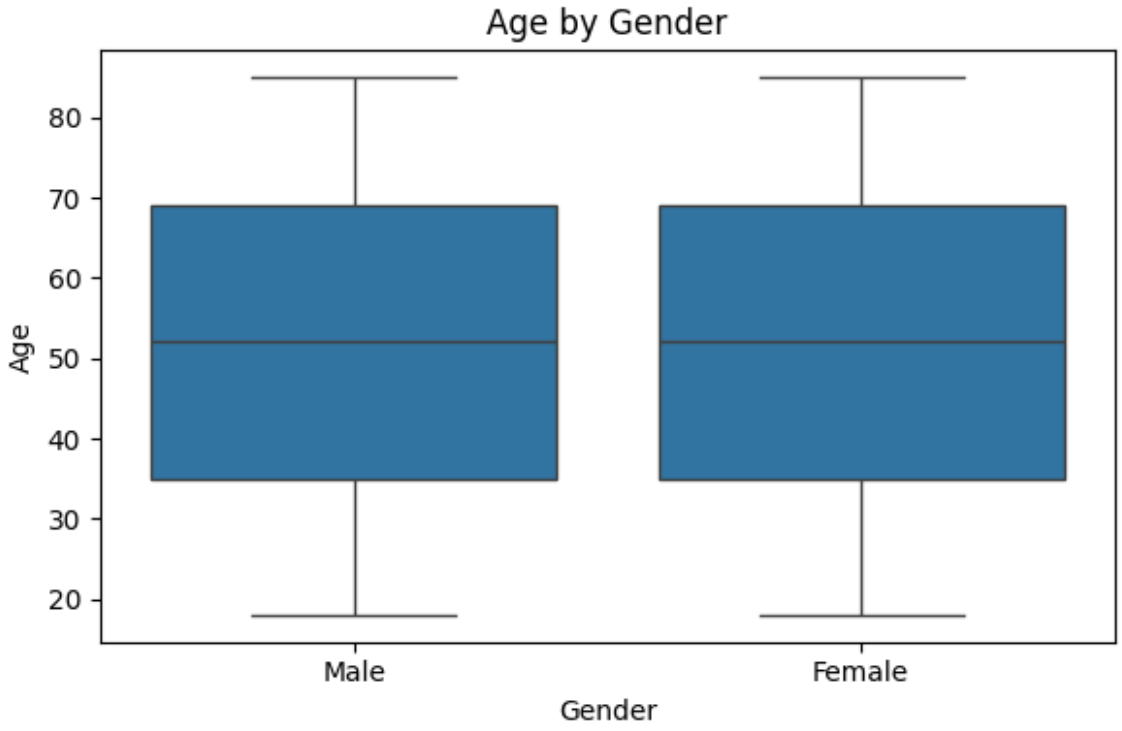

```
boxplot_Age_by_Blood_Type.png
```



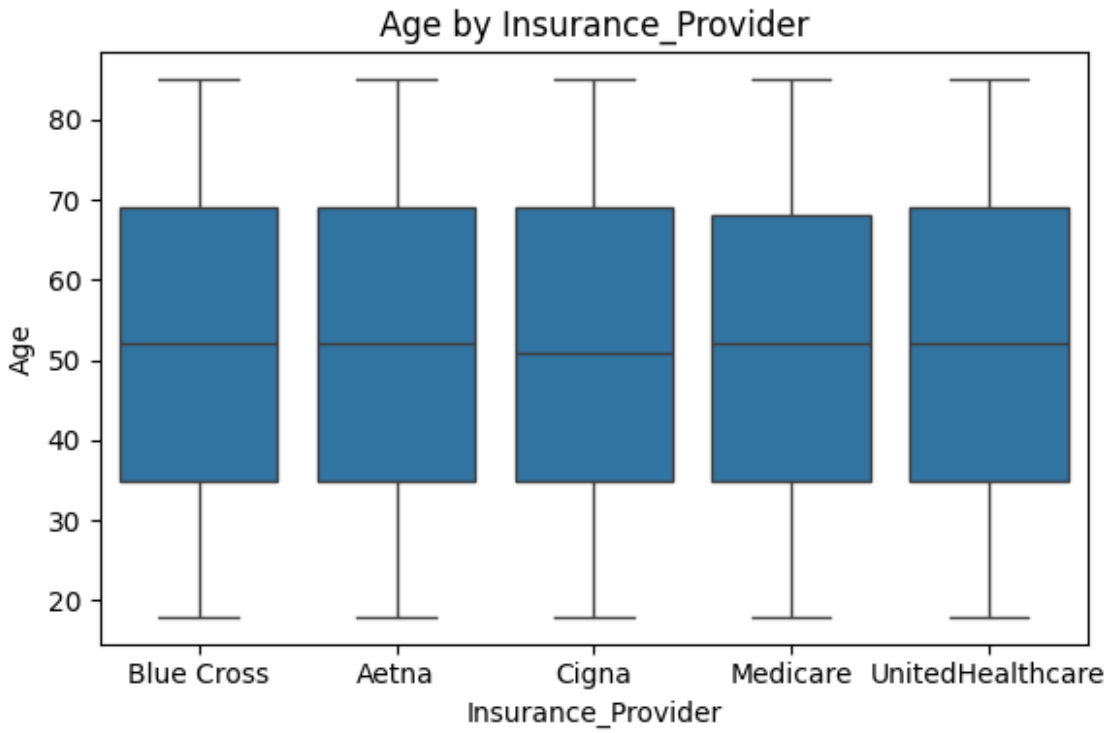
```
boxplot_Age_by_Ethnicity.png
```



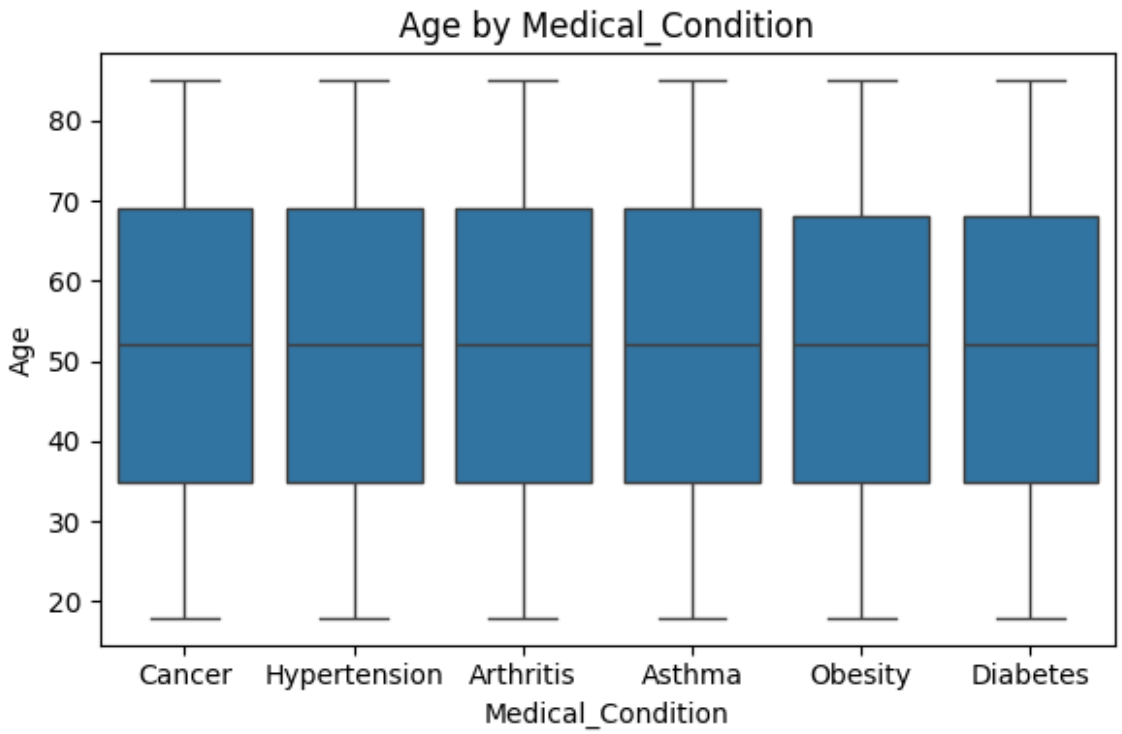

```
boxplot_Age_by_Gender.png
```



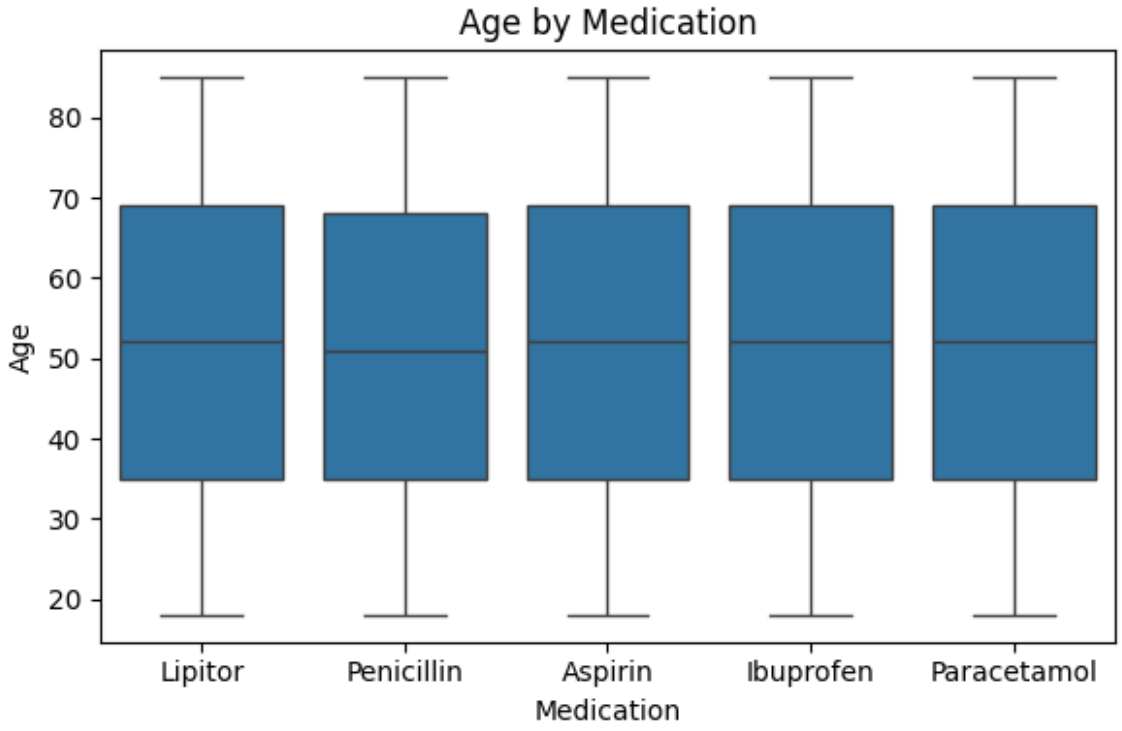

```
boxplot_Age_by_Insurance_Provider.png
```



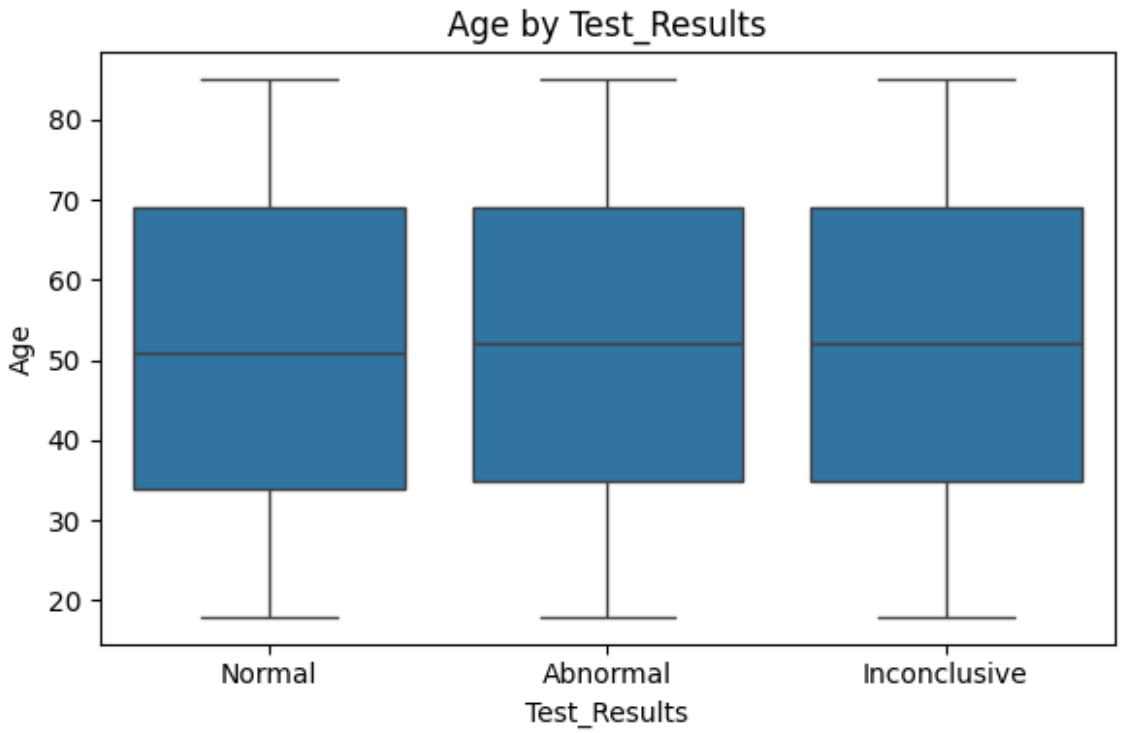
```
boxplot_Age_by_Medical_Condition.png
```



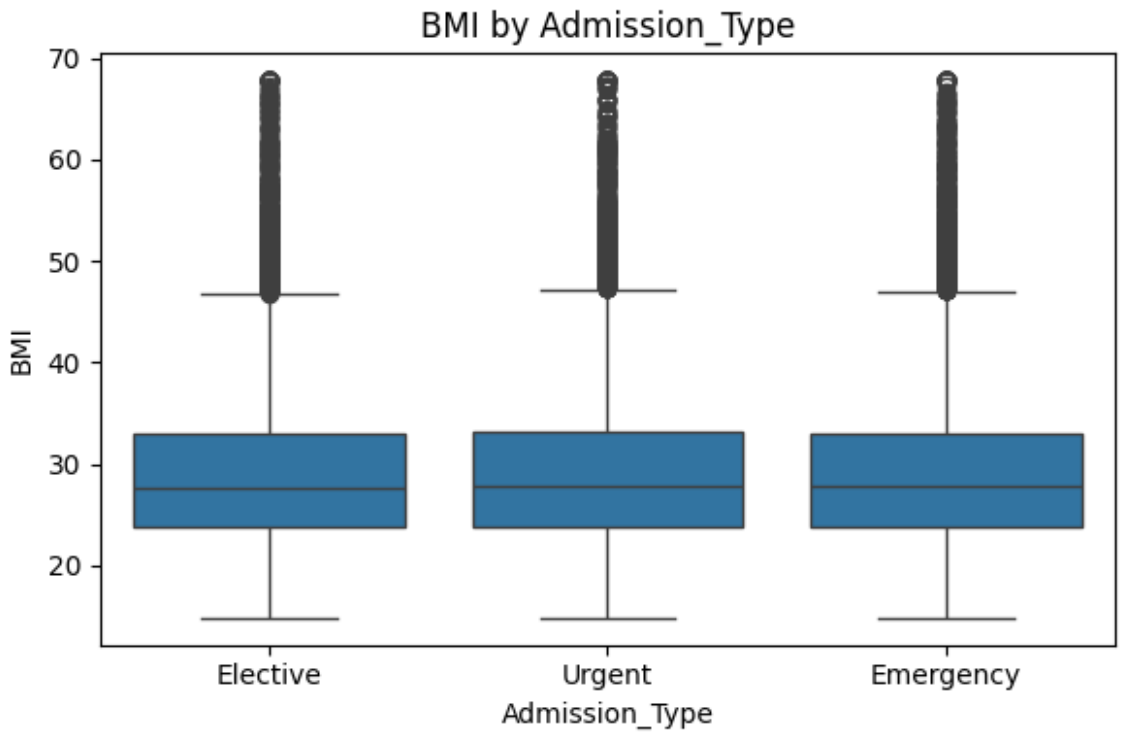

```
boxplot_Age_by_Medication.png
```



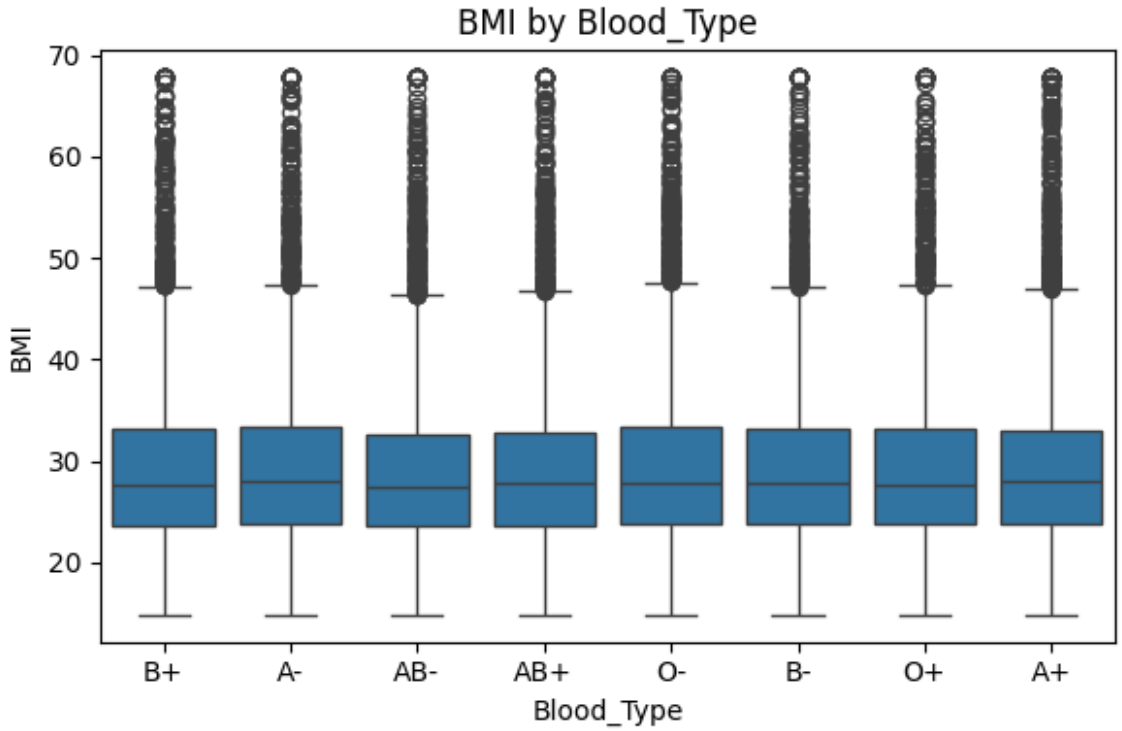

```
boxplot_Age_by_Test_Results.png
```



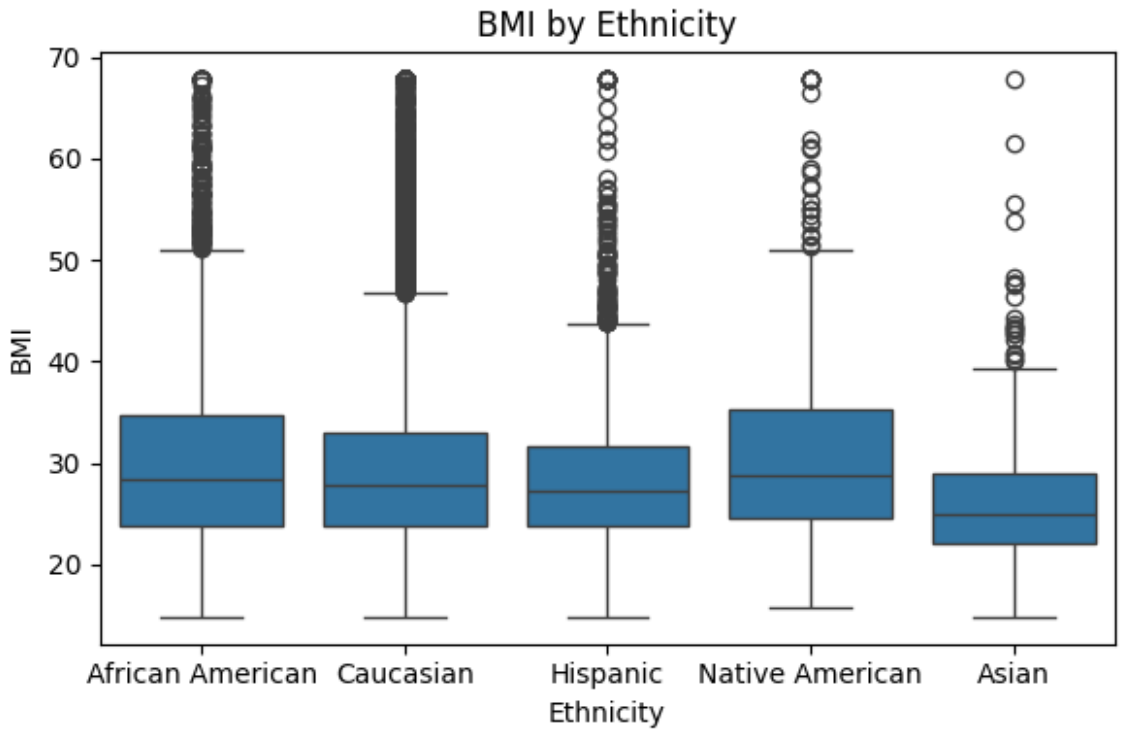
```
boxplot_BMI_by_Admission_Type.png
```



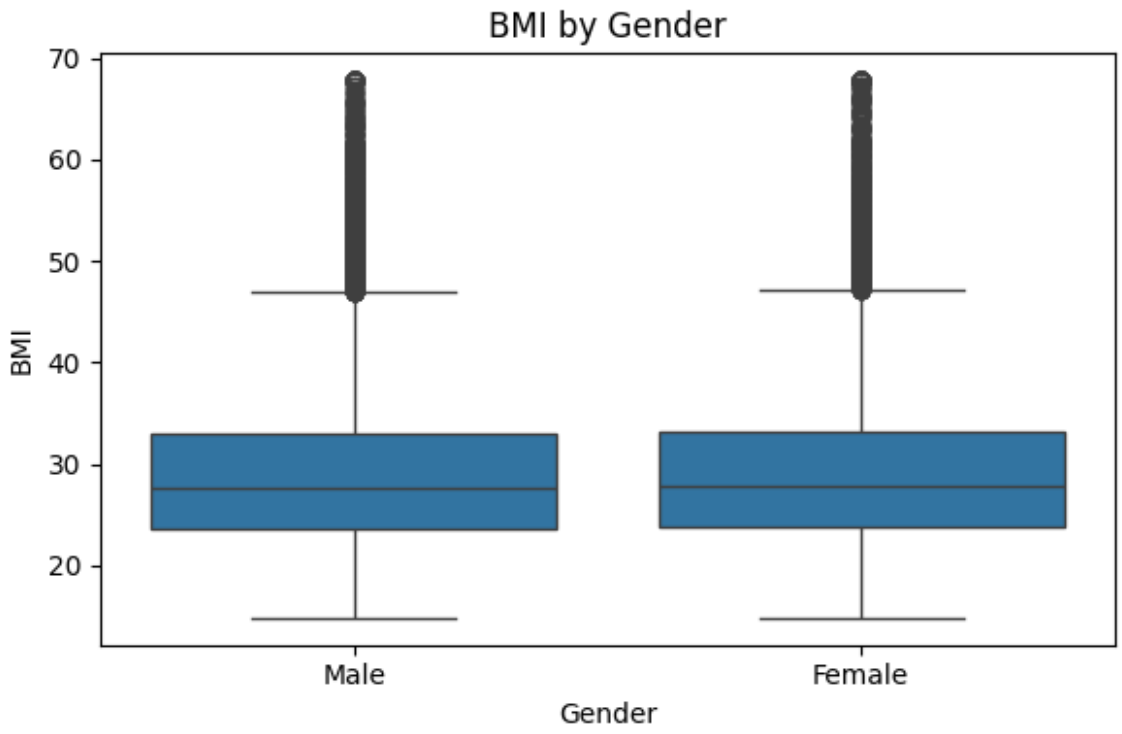

```
boxplot_BMI_by_Blood_Type.png
```



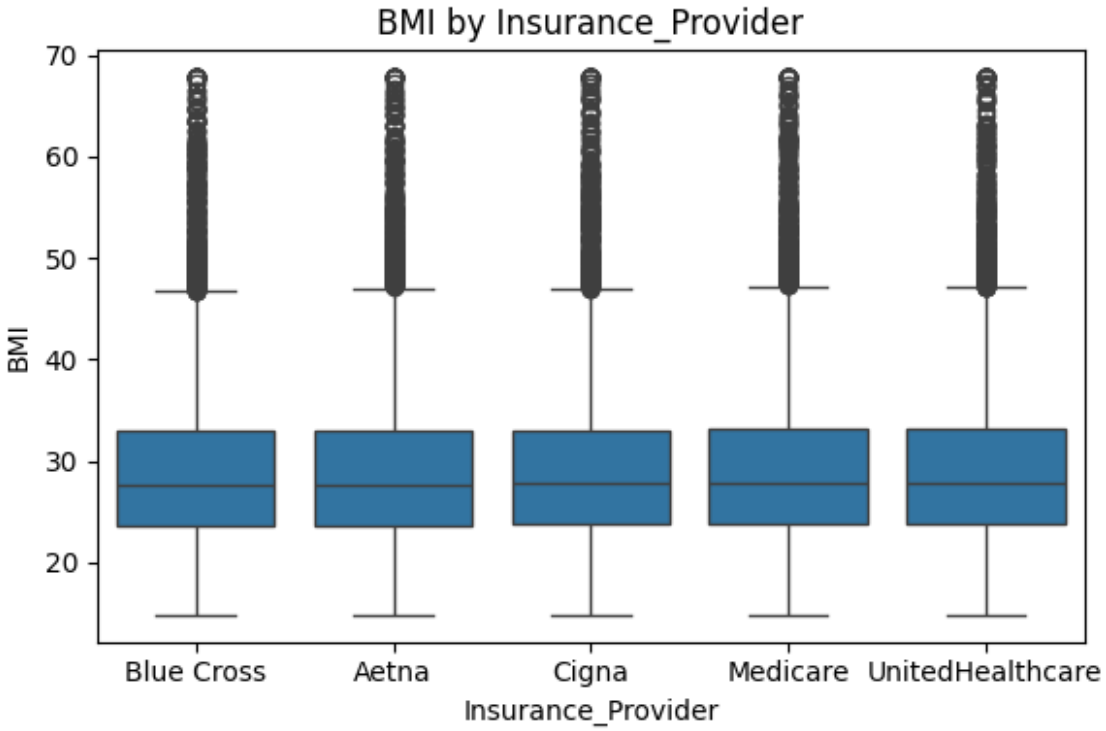

```
boxplot_BMI_by_Ethnicity.png
```



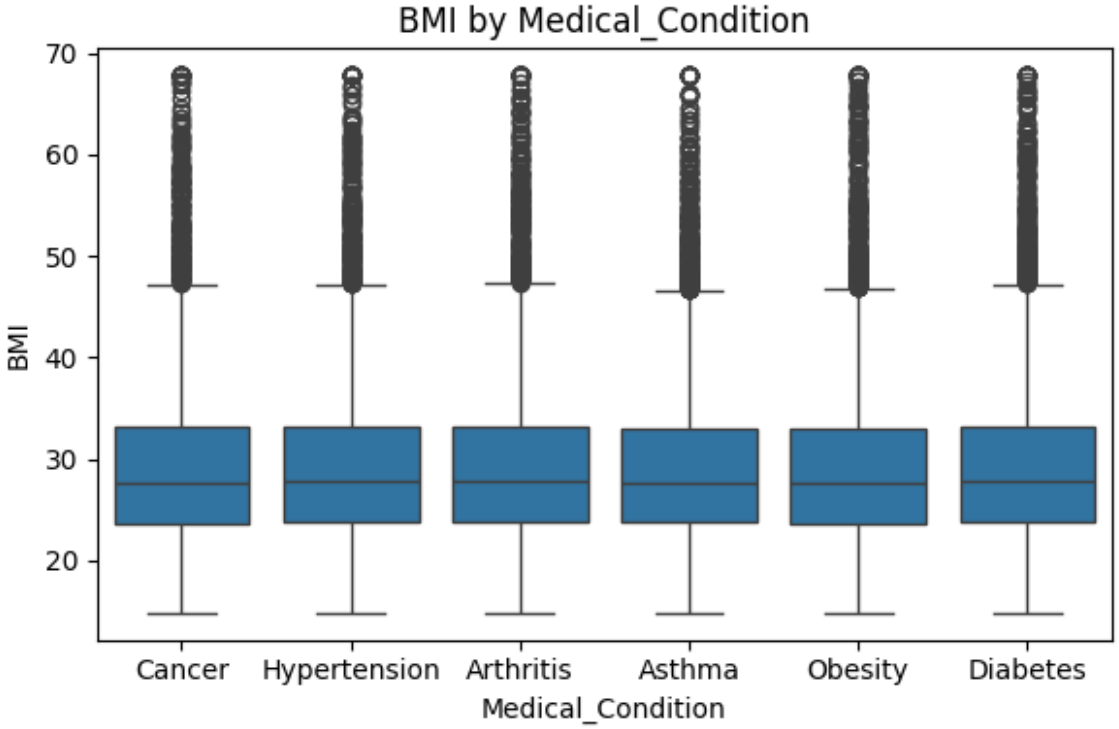
```
boxplot_BMI_by_Gender.png
```



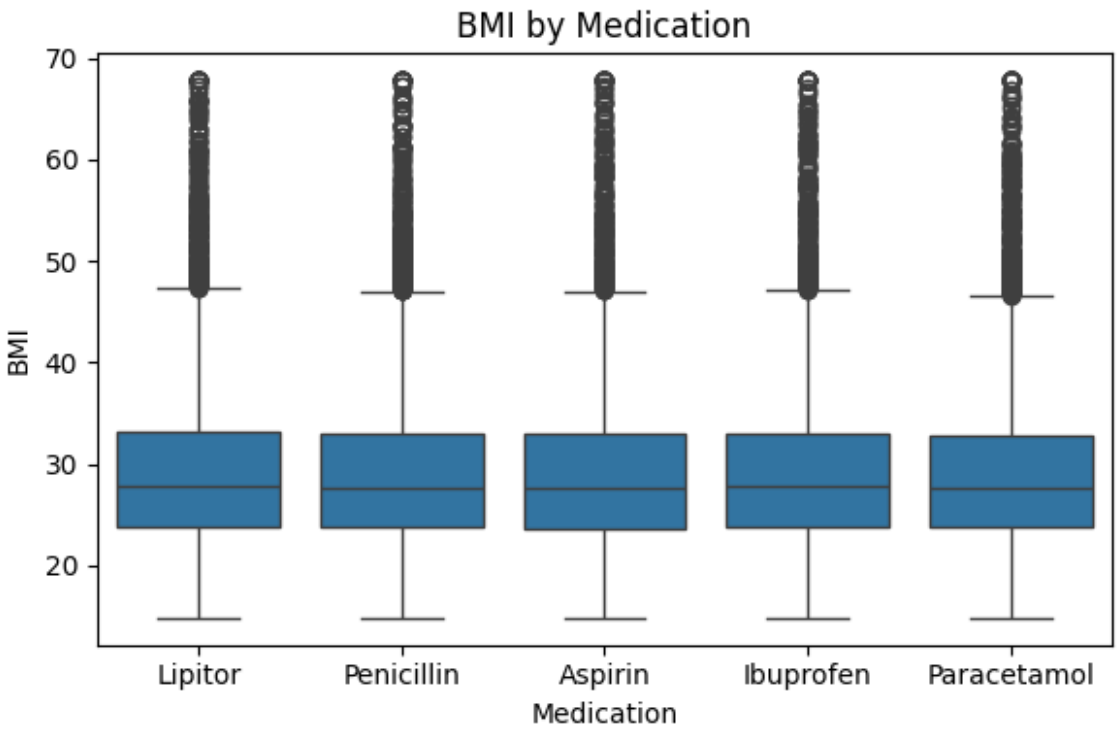

```
boxplot_BMI_by_Insurance_Provider.png
```



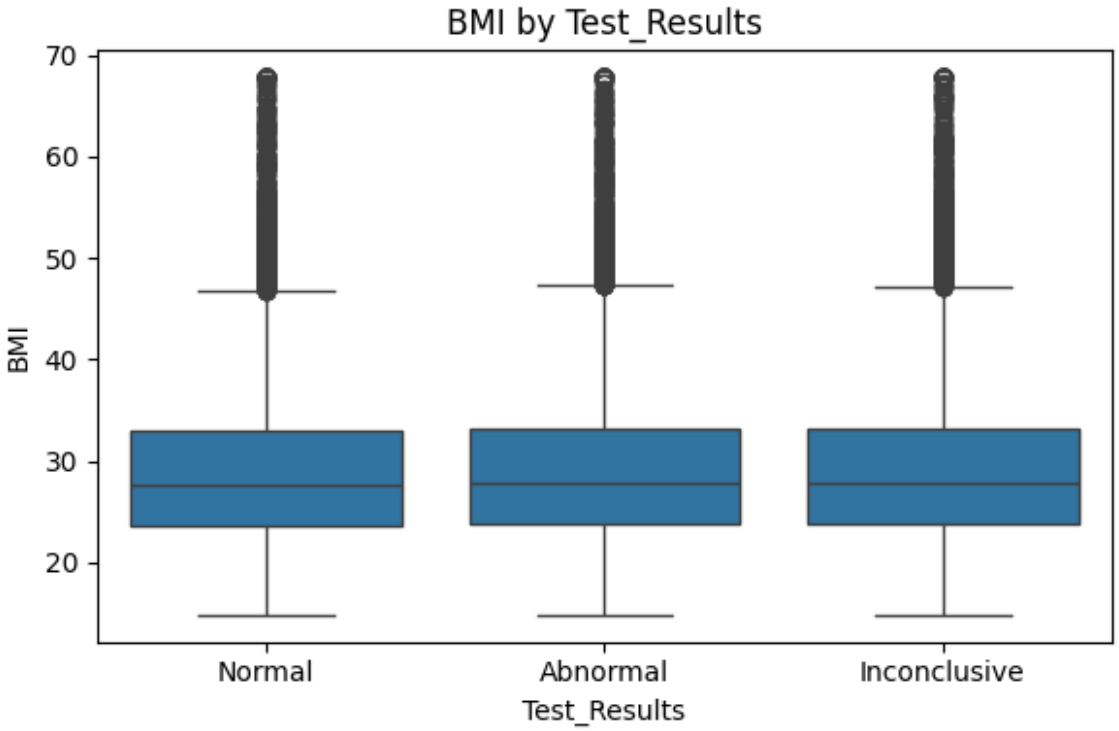

```
boxplot_BMI_by_Medical_Condition.png
```



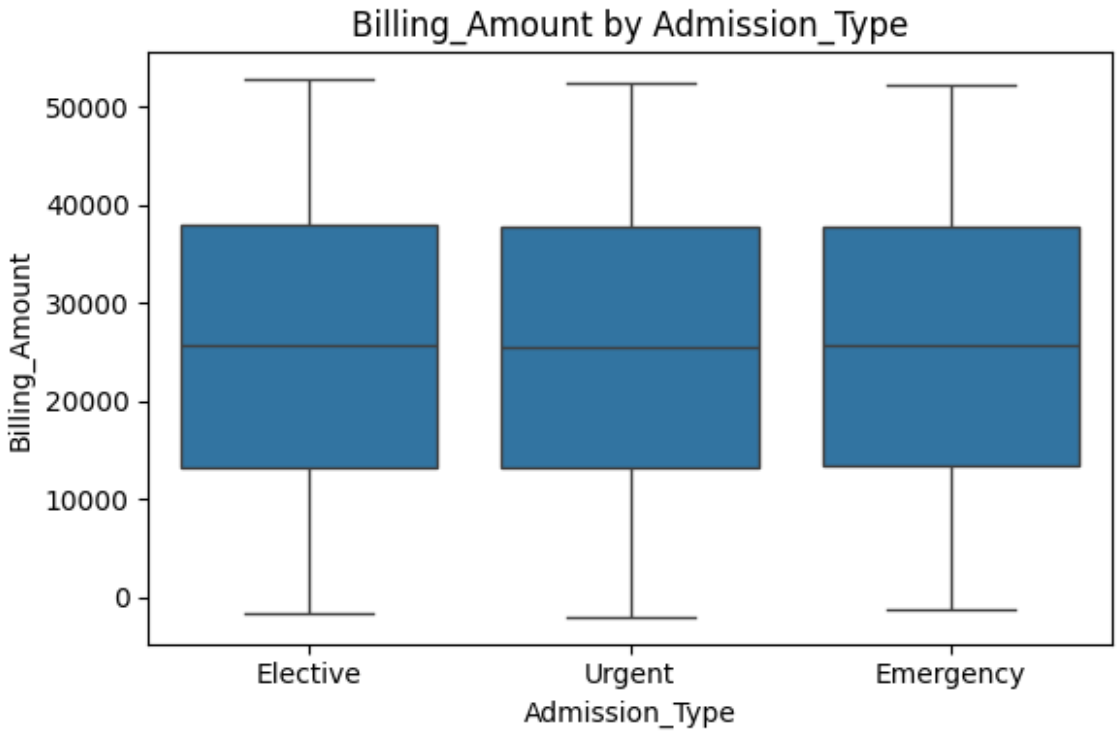
```
boxplot_BMI_by_Medication.png
```



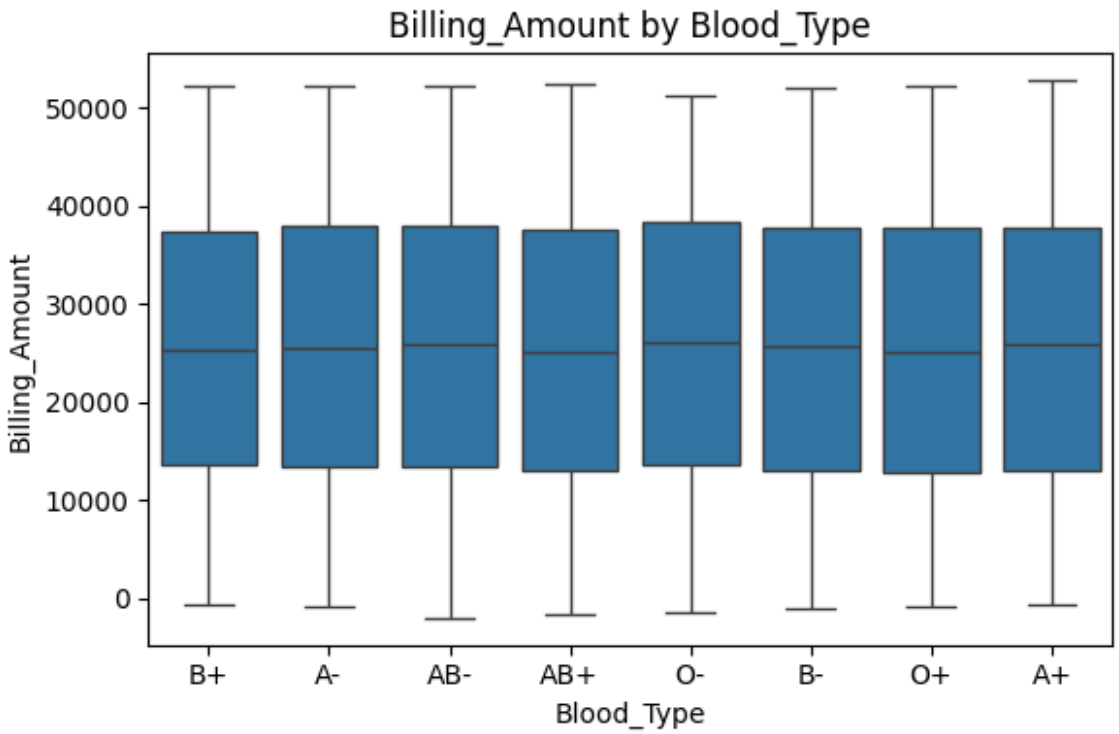

```
boxplot_BMI_by_Test_Results.png
```



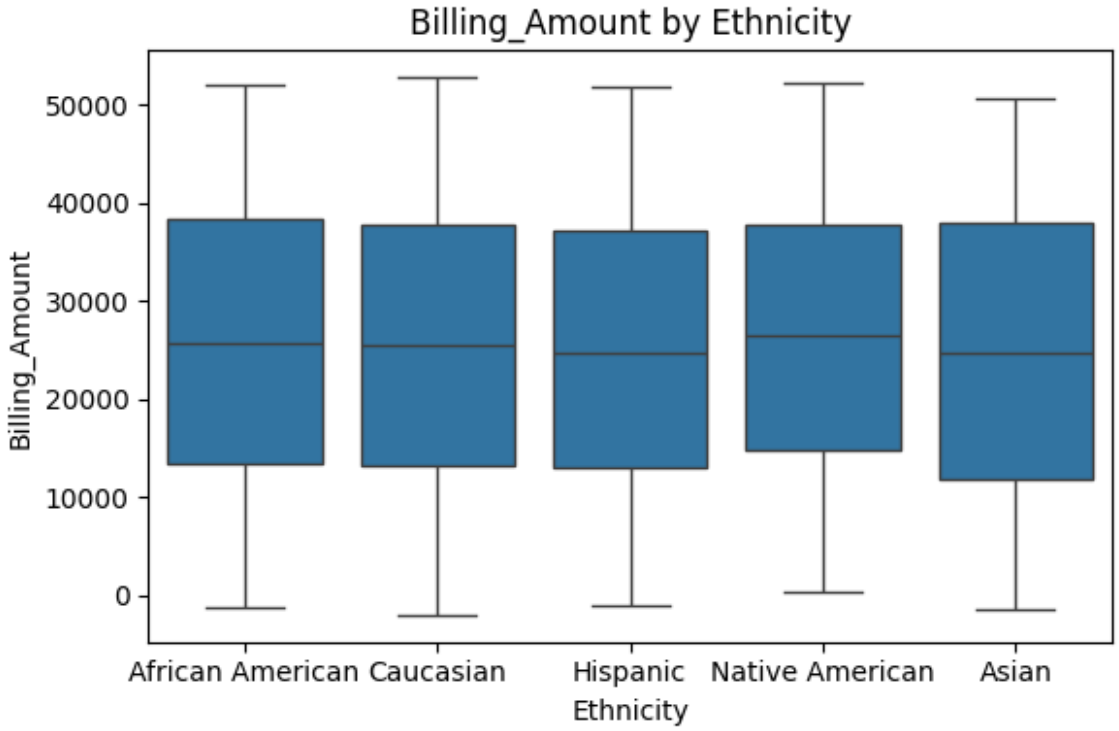

```
boxplot_Billing_Amount_by_Admission_Type.png
```



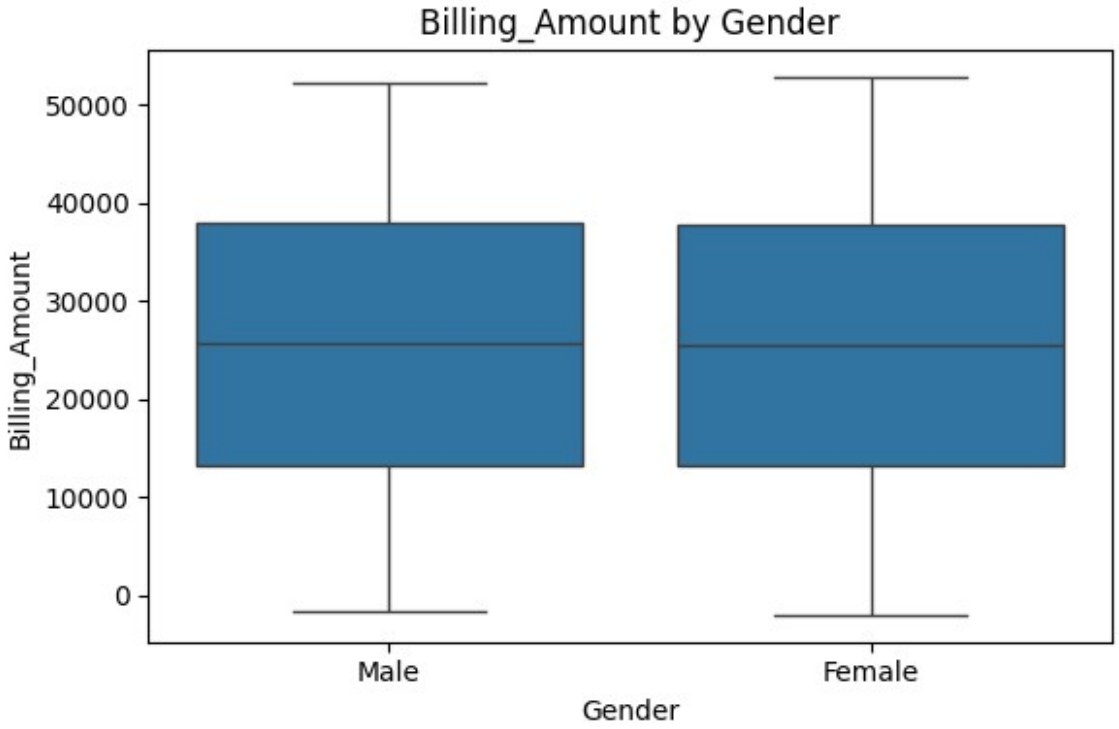
```
boxplot_Billing_Amount_by_Blood_Type.png
```



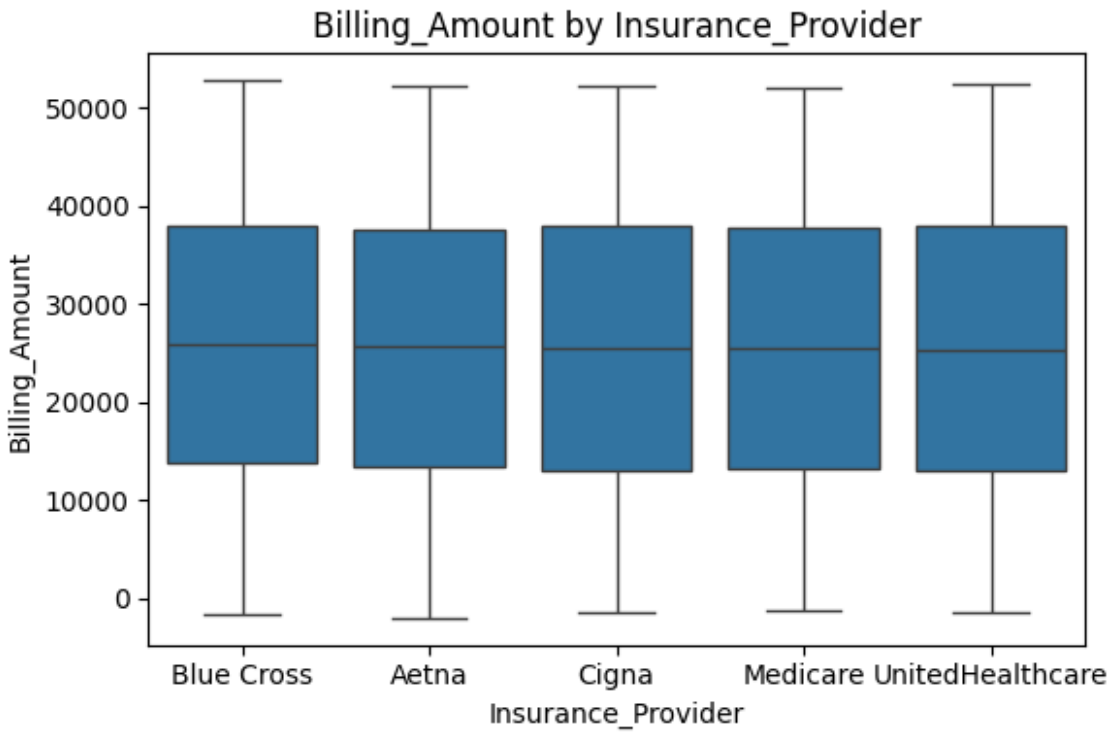

```
boxplot_Billing_Amount_by_Ethnicity.png
```



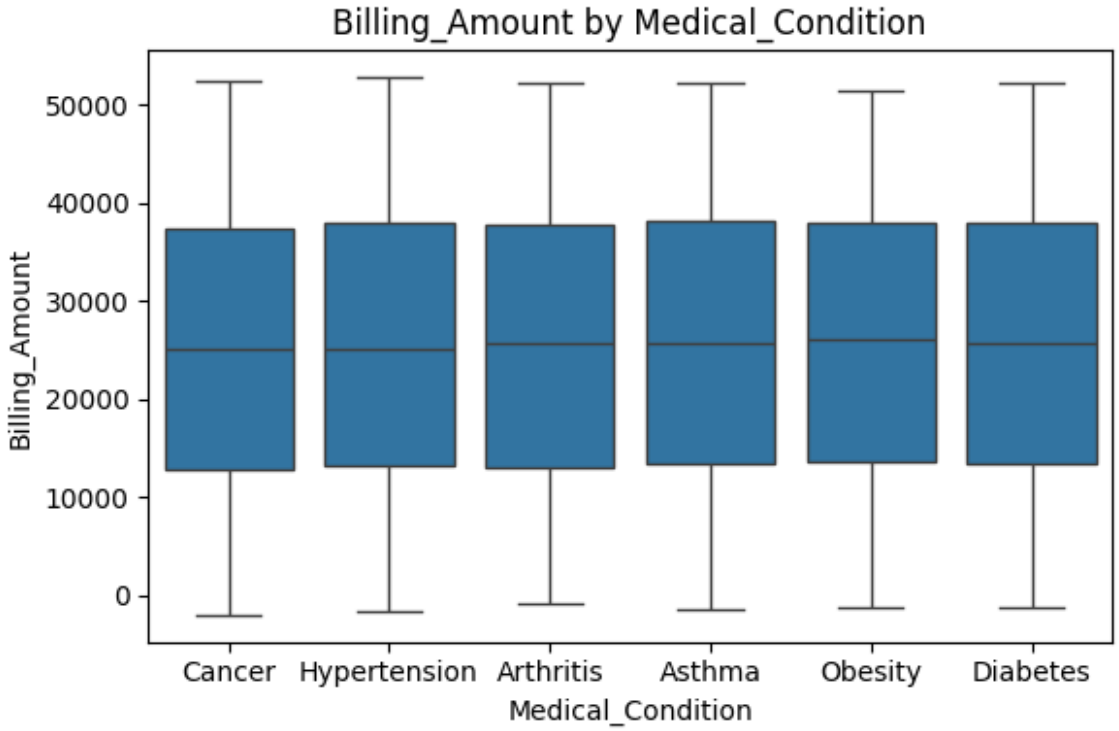

```
boxplot_Billing_Amount_by_Gender.png
```



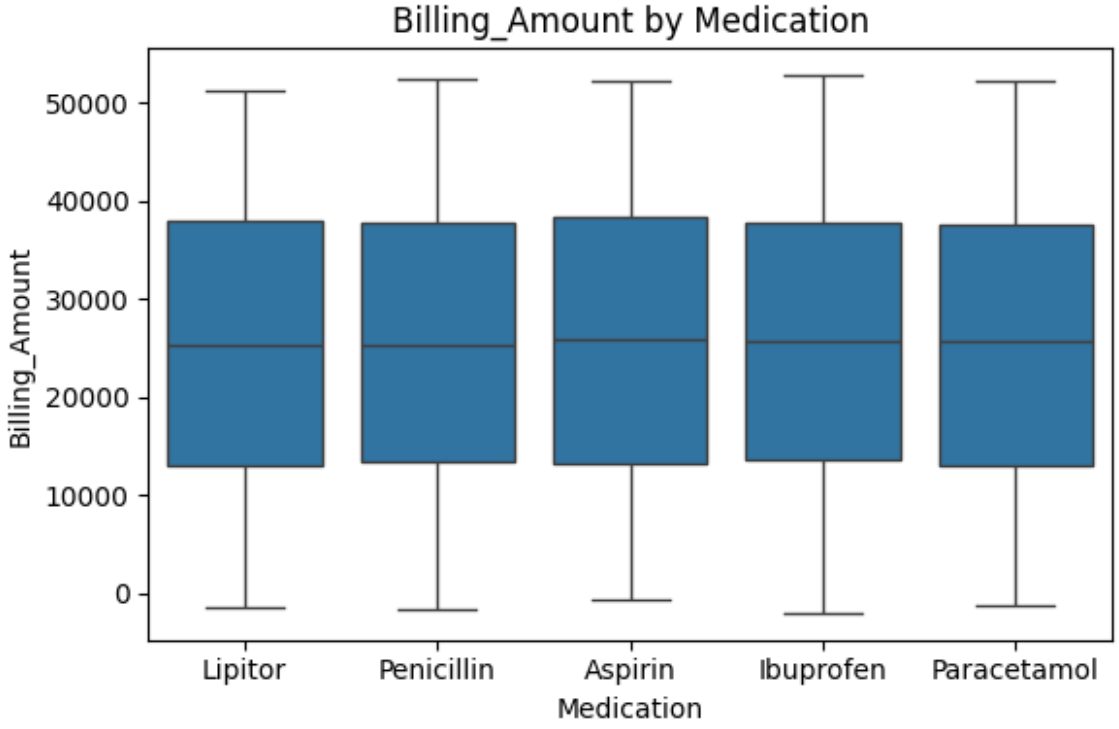
```
boxplot_Billing_Amount_by_Insurance_Provider.png
```



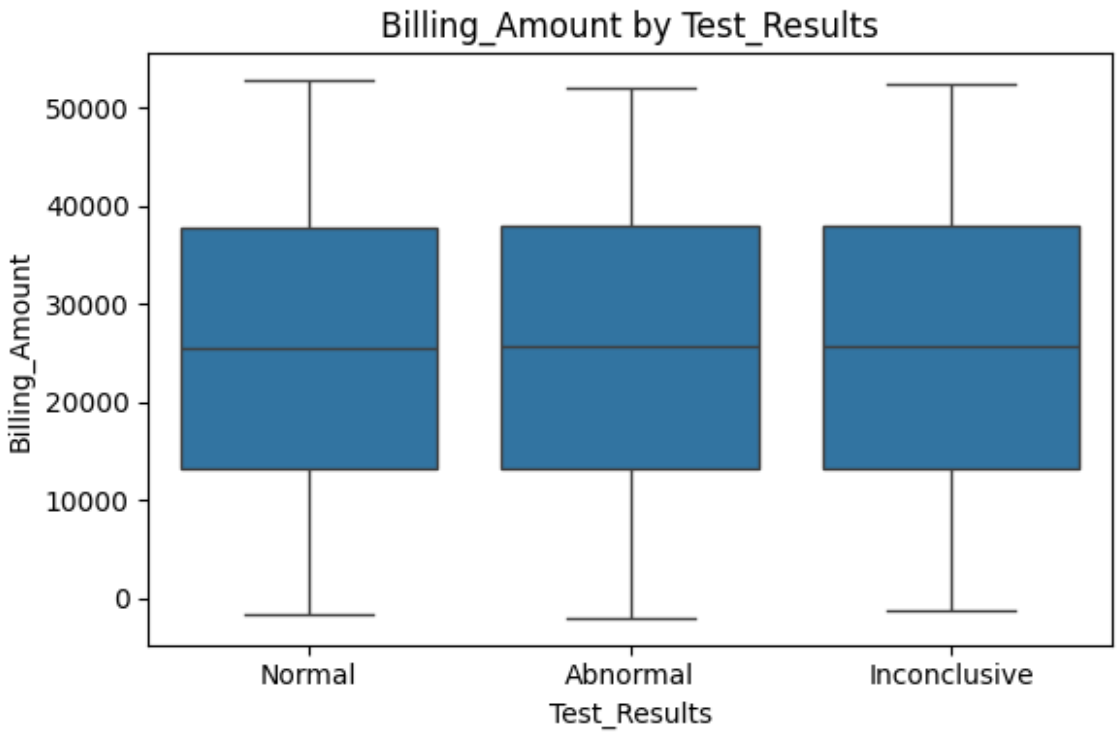

```
boxplot_Billing_Amount_by_Medical_Condition.png
```



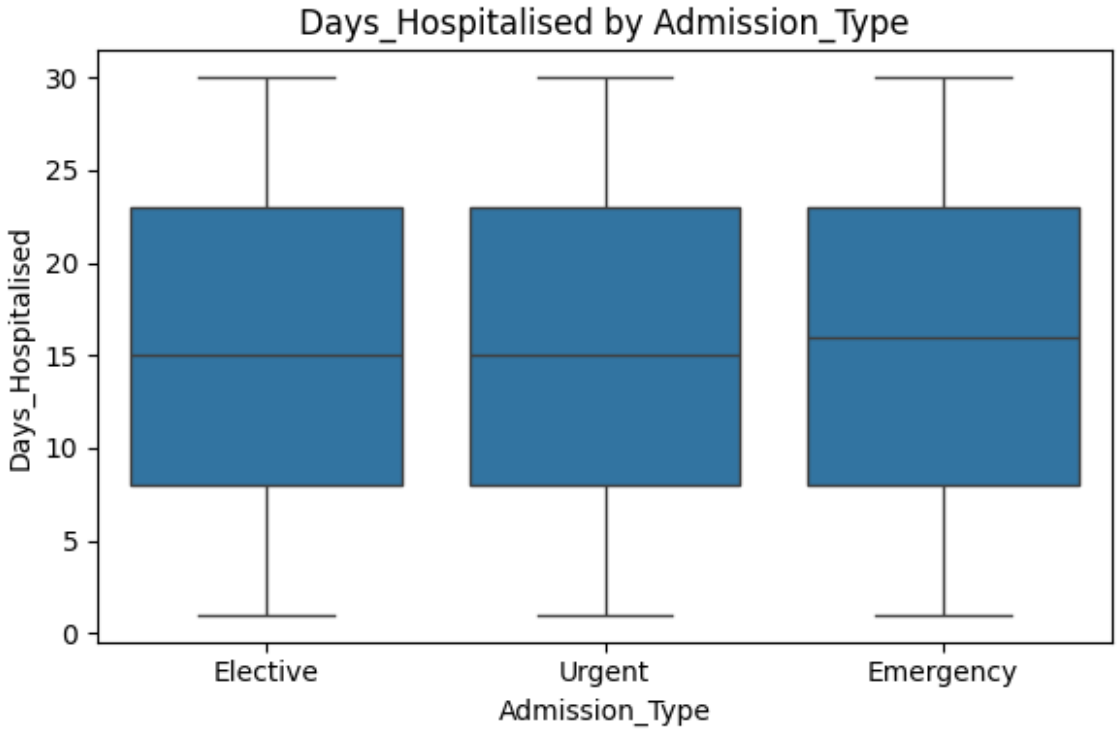

```
boxplot_Billing_Amount_by_Medication.png
```



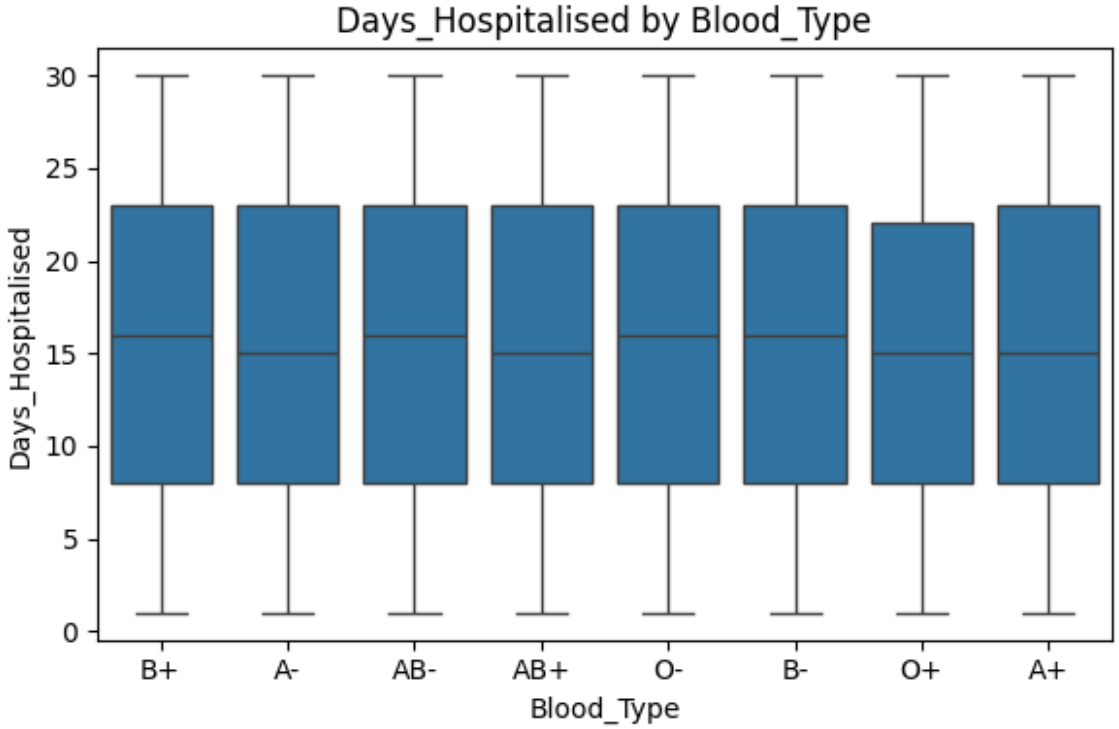
```
boxplot_Billing_Amount_by_Test_Results.png
```



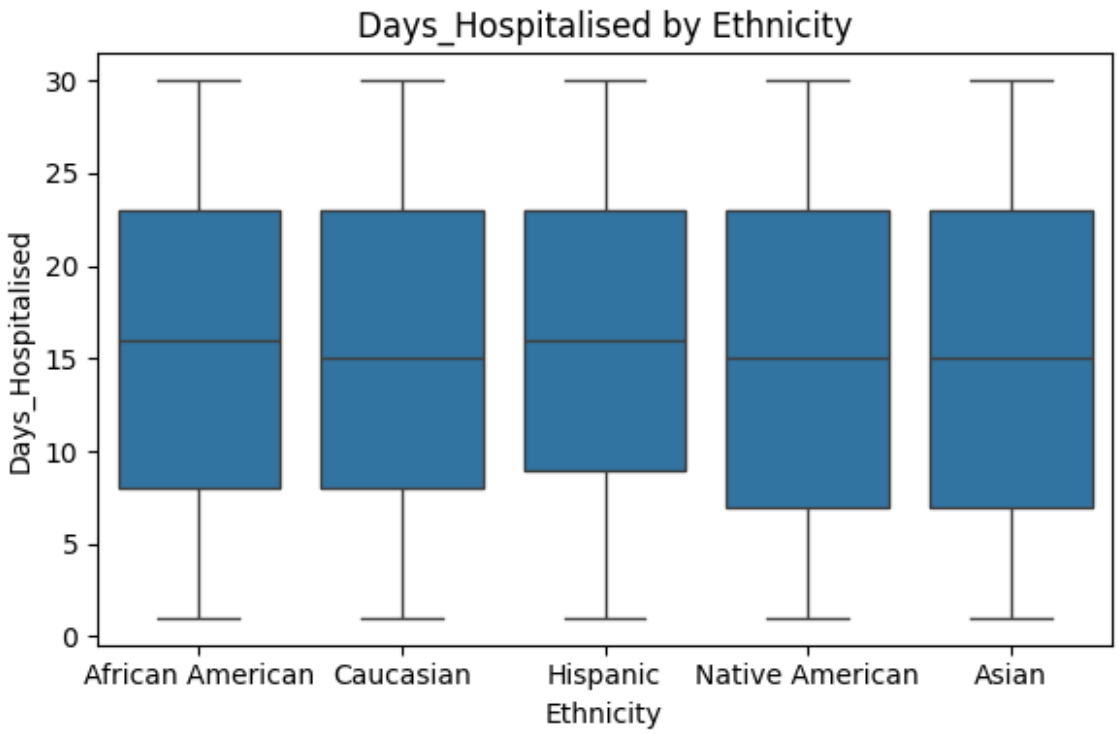

```
boxplot_Days_Hospitalised_by_Admission_Type.png
```



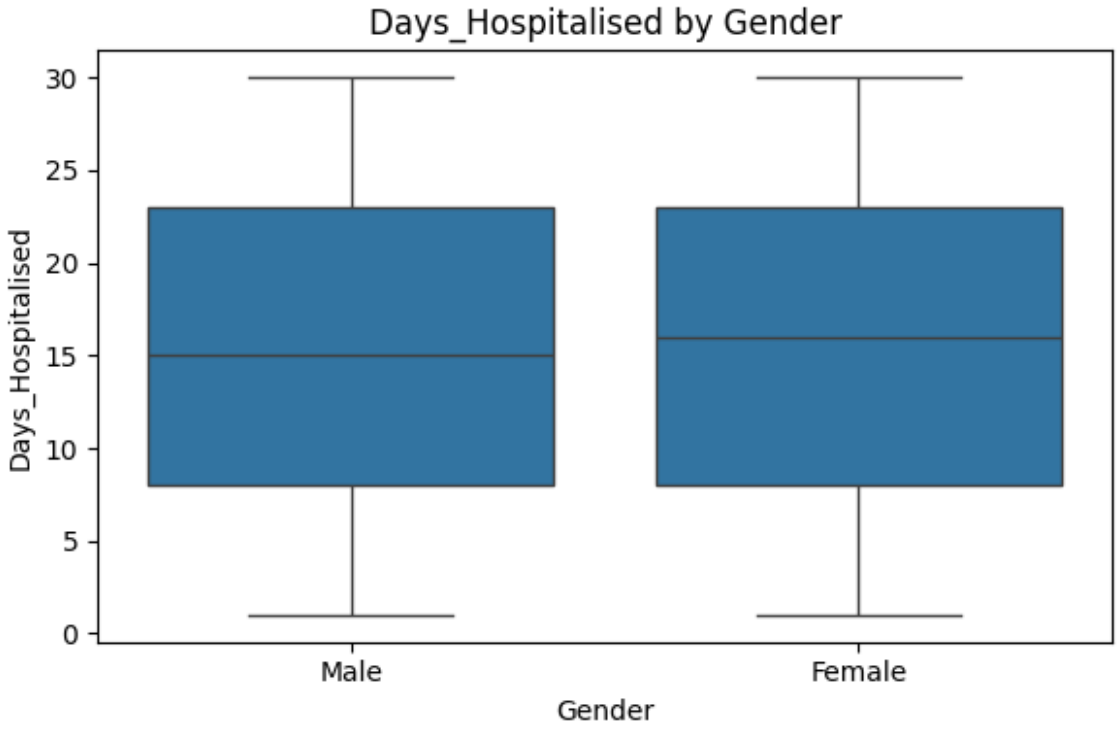

```
boxplot_Days_Hospitalised_by_Blood_Type.png
```



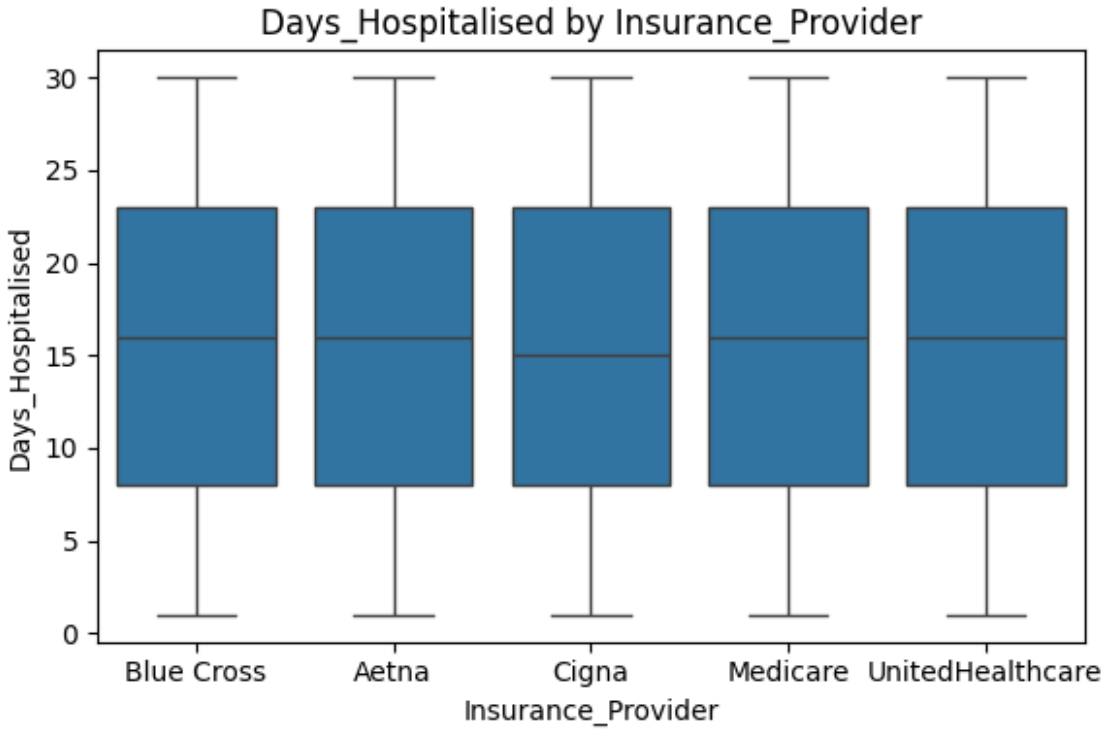
```
boxplot_Days_Hospitalised_by_Ethnicity.png
```



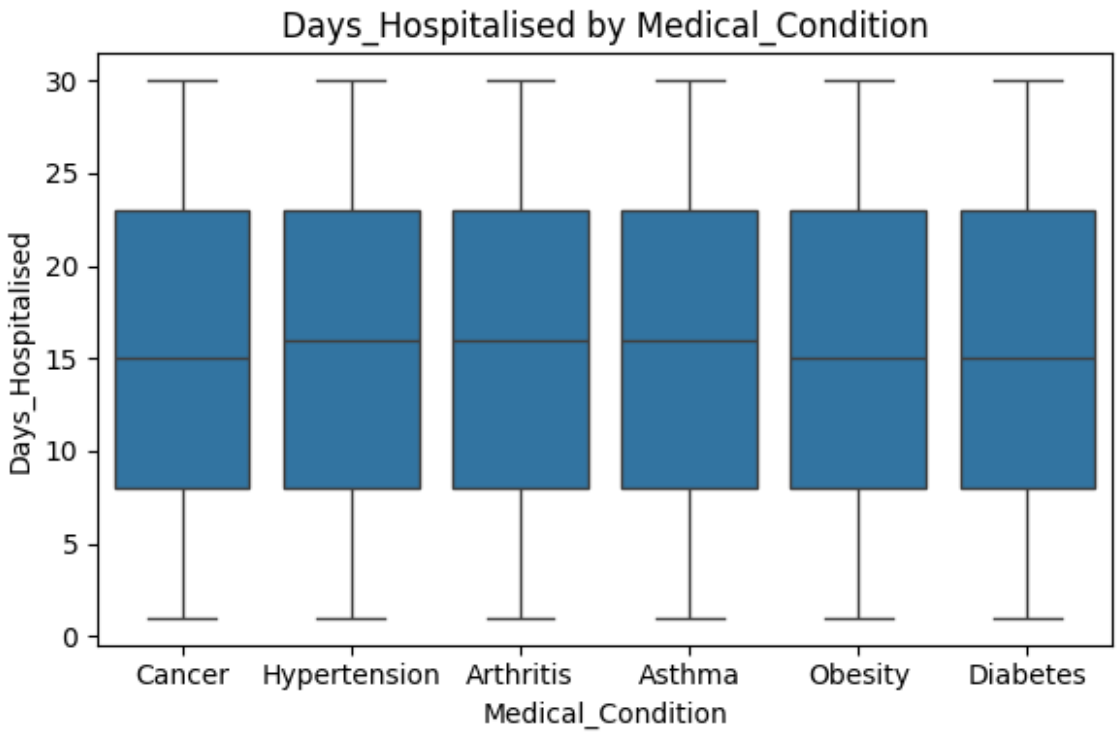

```
boxplot_Days_Hospitalised_by_Gender.png
```



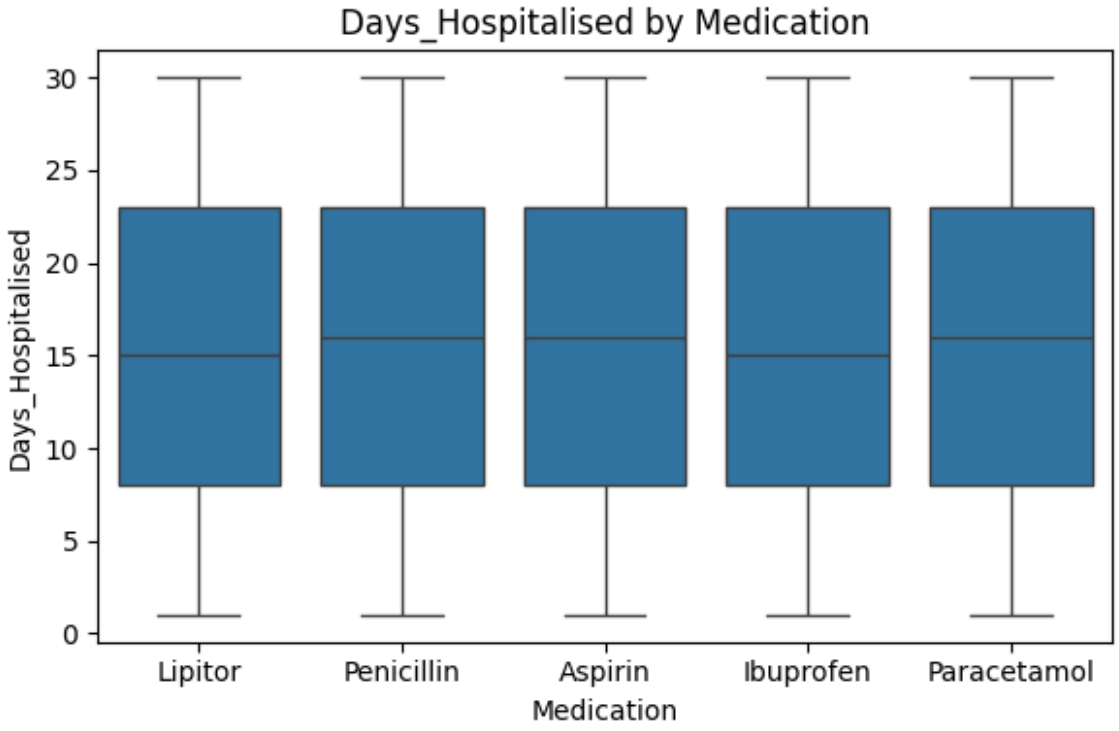

```
boxplot_Days_Hospitalised_by_Insurance_Provider.png
```



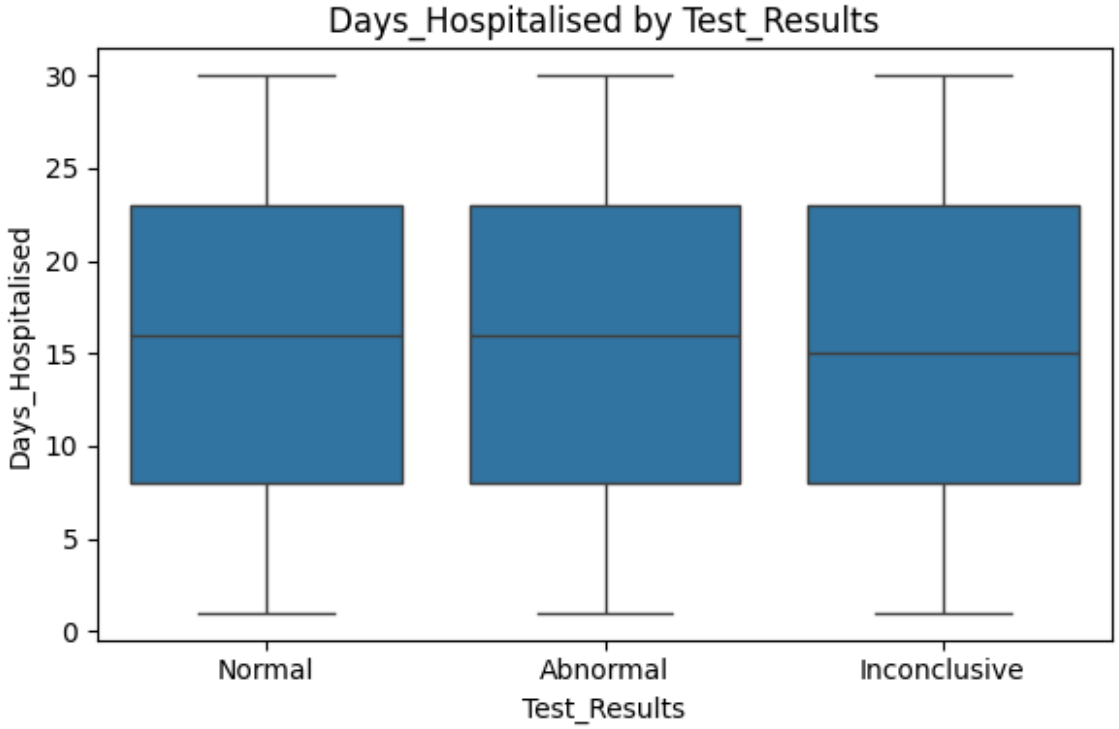
```
boxplot_Days_Hospitalised_by_Medical_Condition.png
```



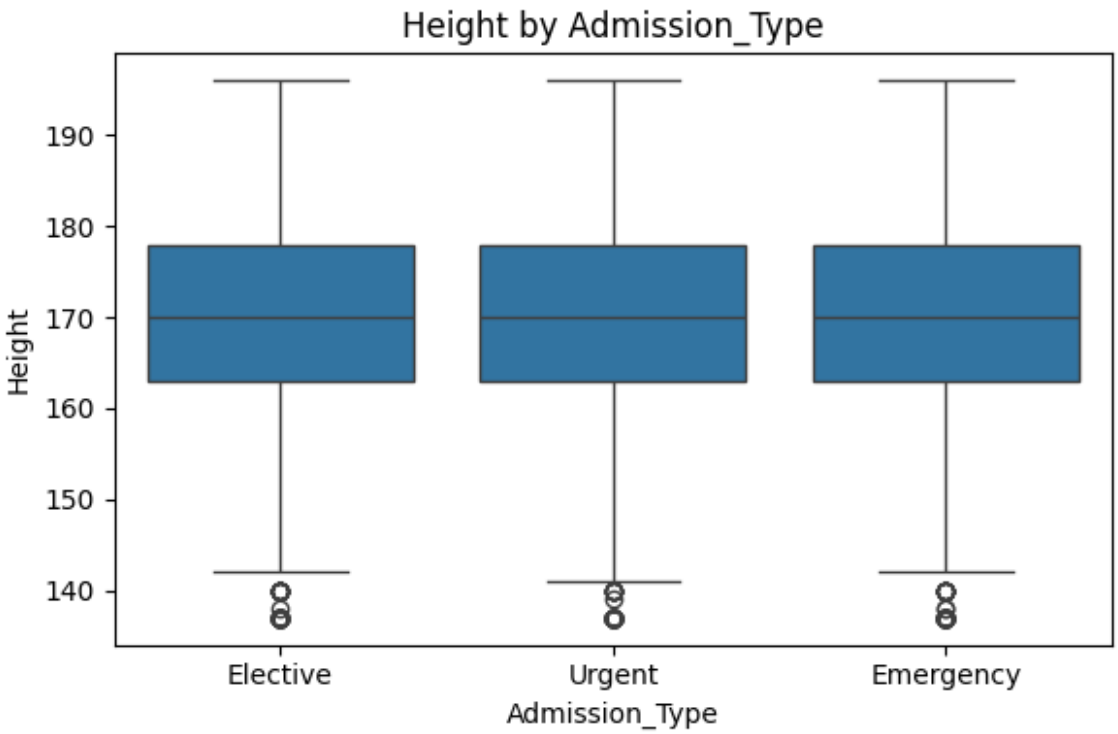

```
boxplot_Days_Hospitalised_by_Medication.png
```



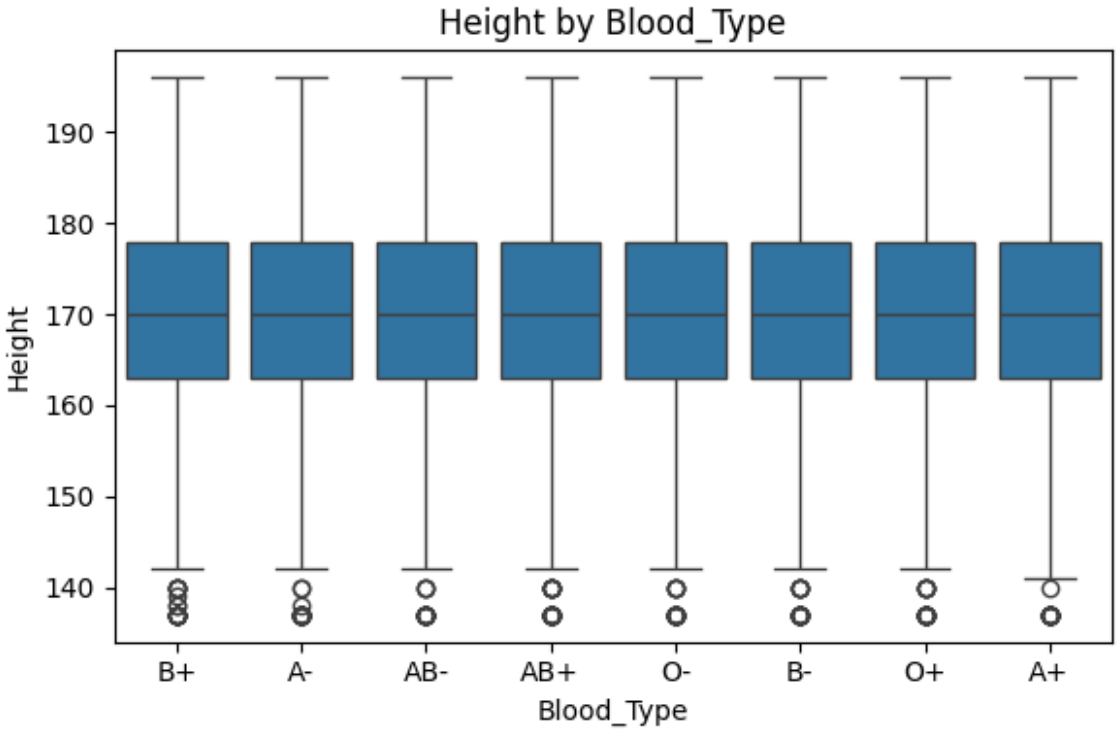

```
boxplot_Days_Hospitalised_by_Test_Results.png
```



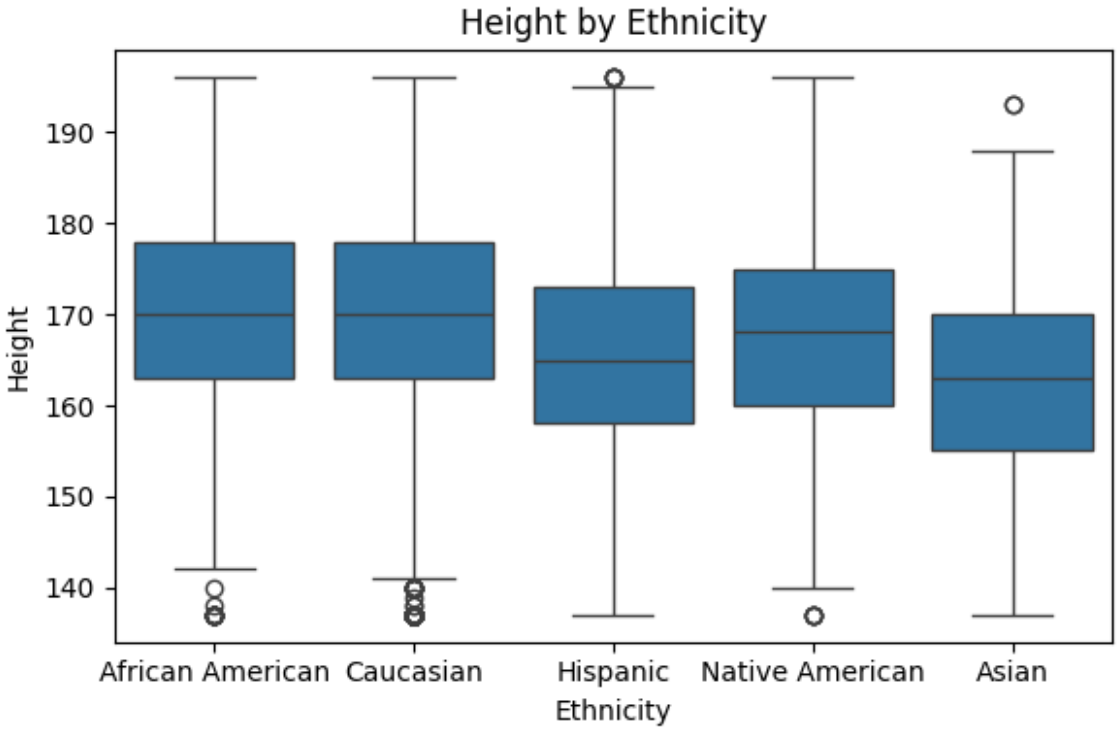
```
boxplot_Height_by_Admission_Type.png
```



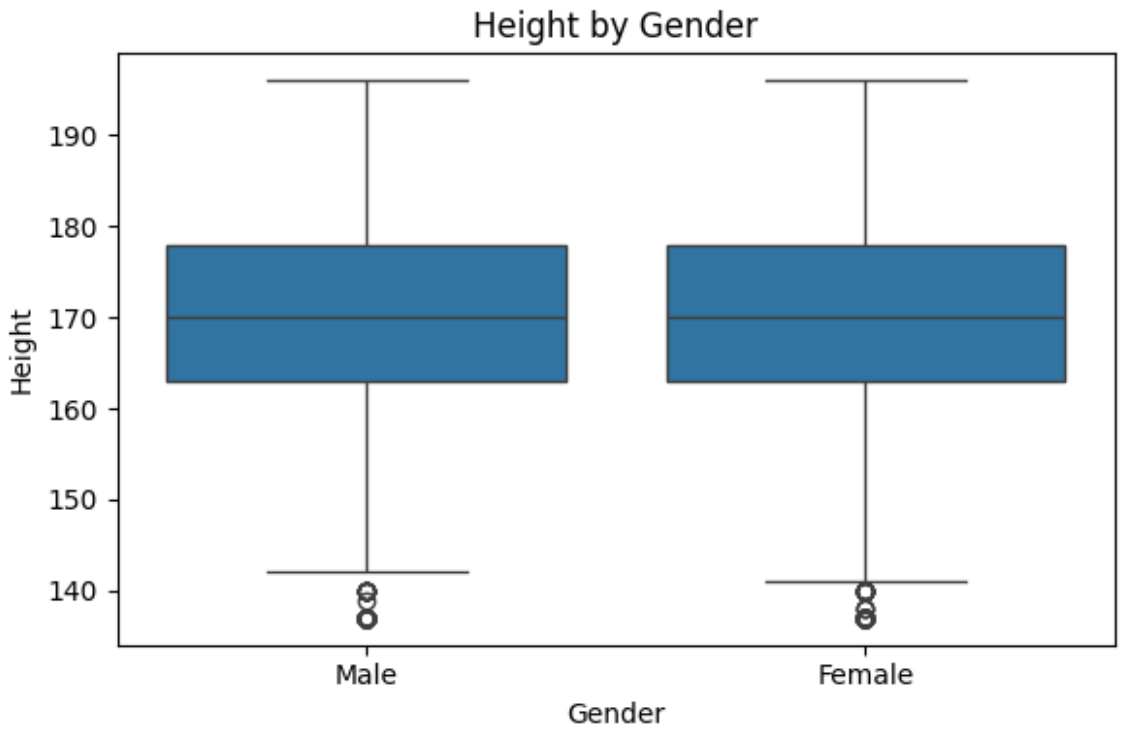

```
boxplot_Height_by_Blood_Type.png
```



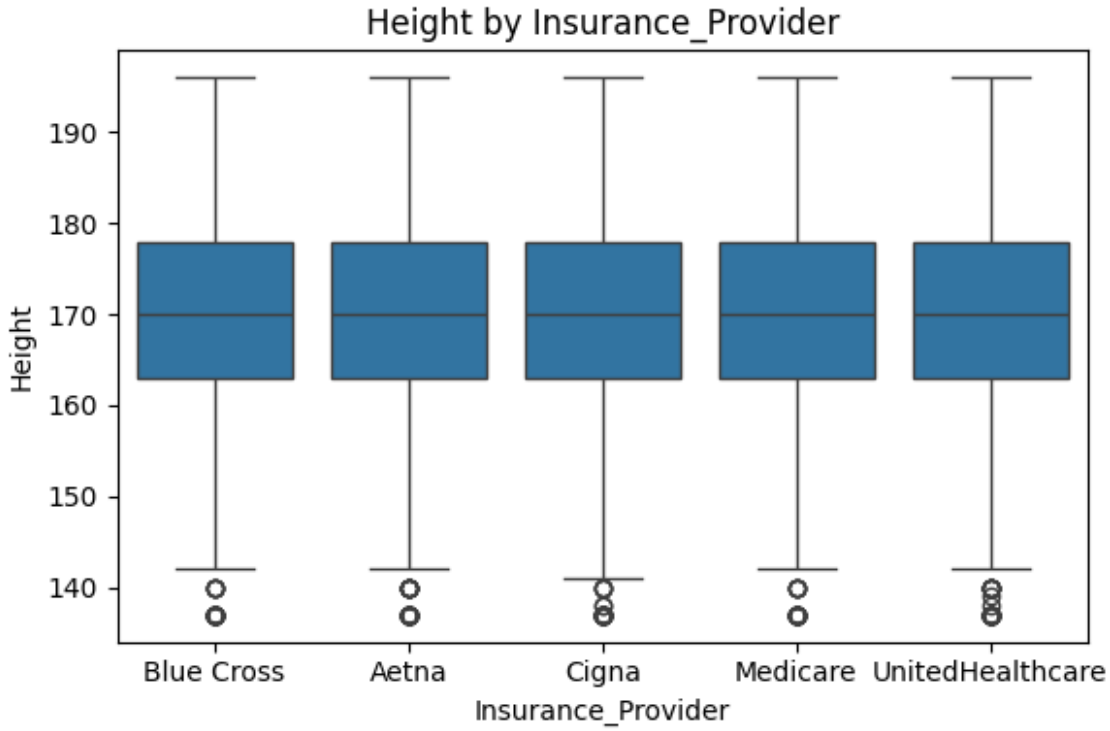

```
boxplot_Height_by_Ethnicity.png
```



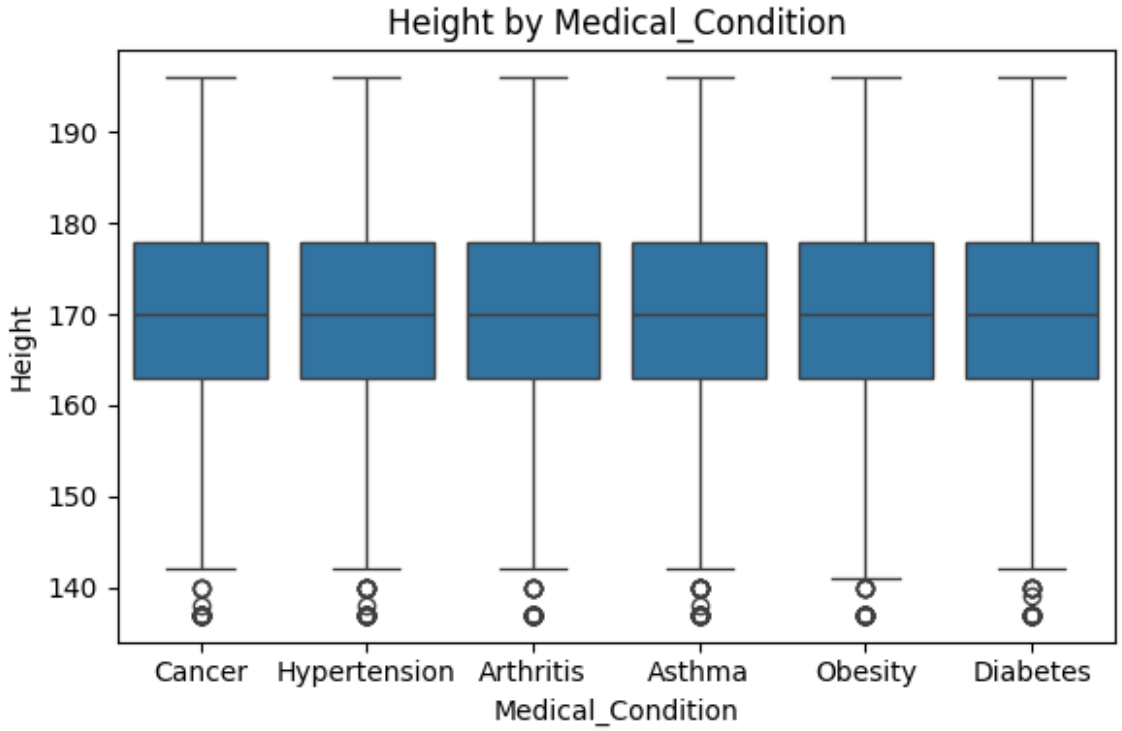
```
boxplot_Height_by_Gender.png
```



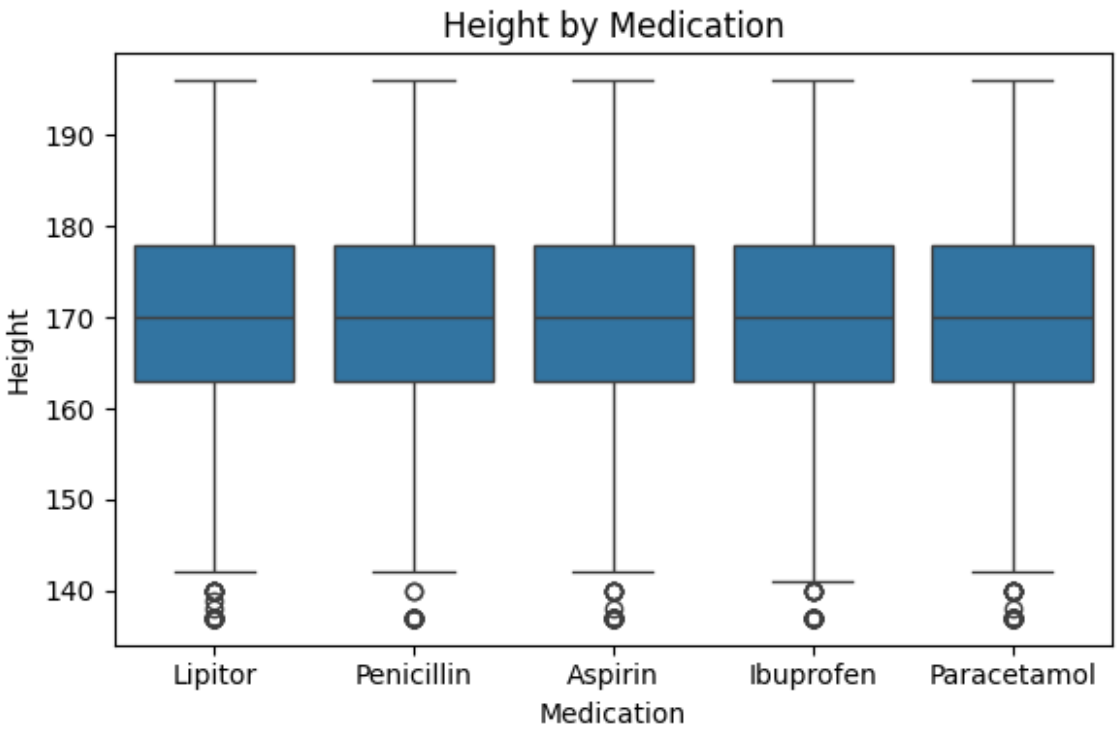

```
boxplot_Height_by_Insurance_Provider.png
```



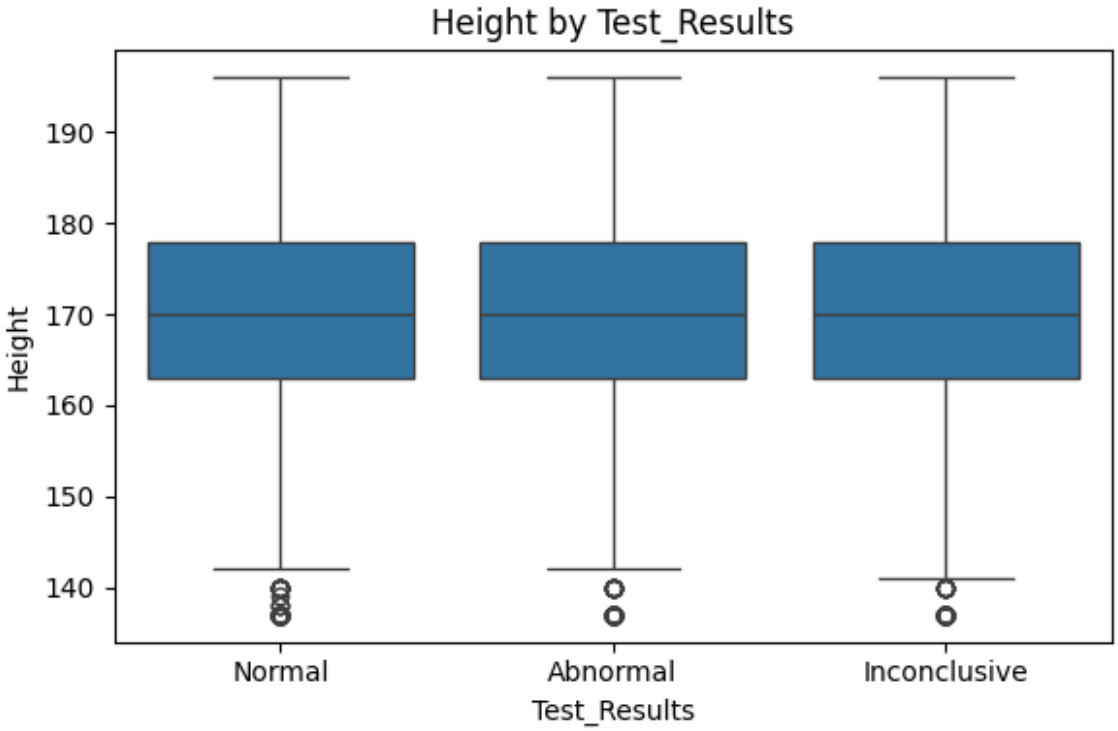

```
boxplot_Height_by_Medical_Condition.png
```



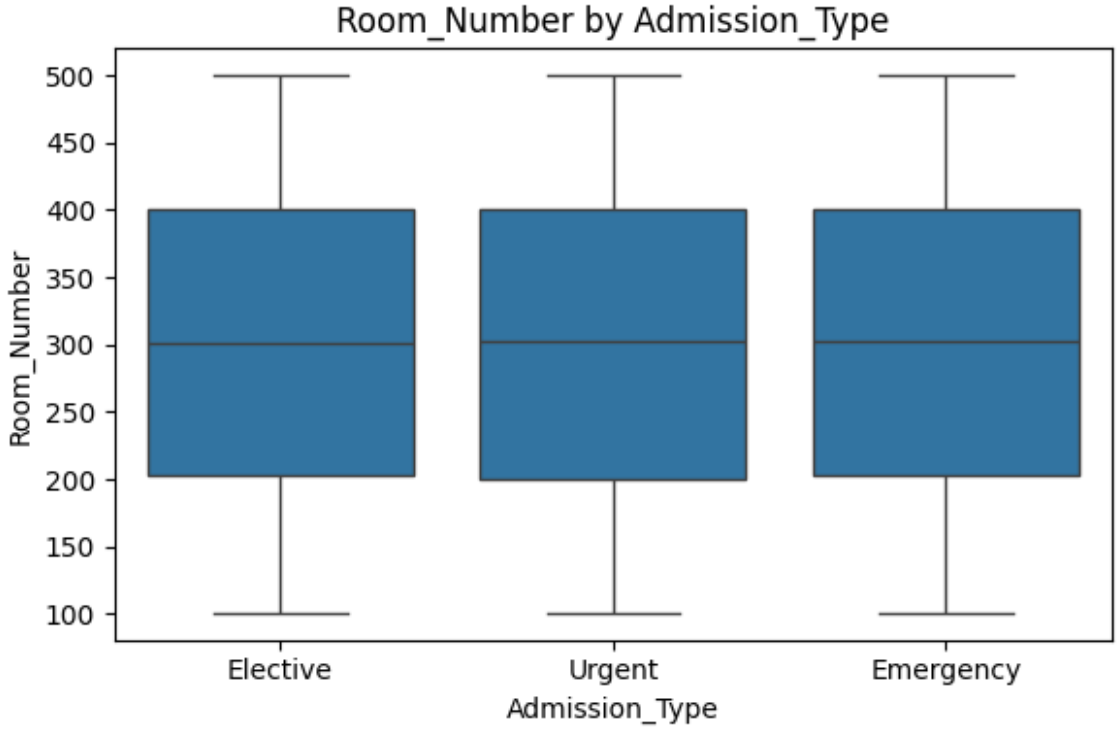
```
boxplot_Height_by_Medication.png
```



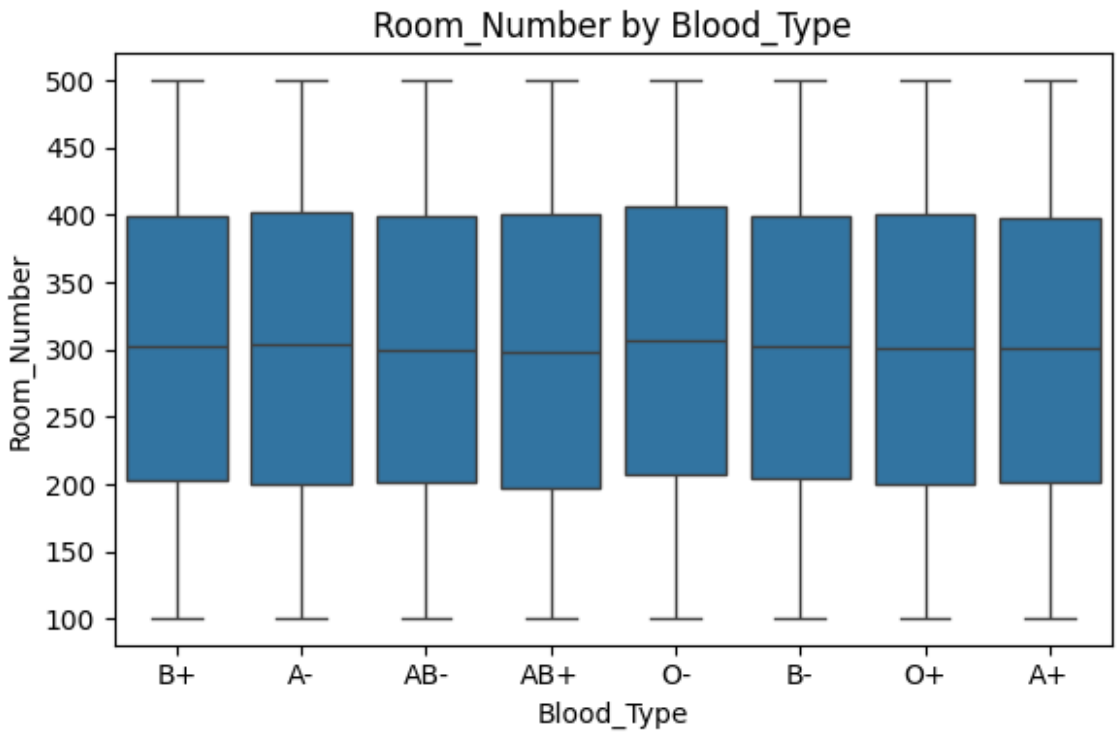

```
boxplot_Height_by_Test_Results.png
```



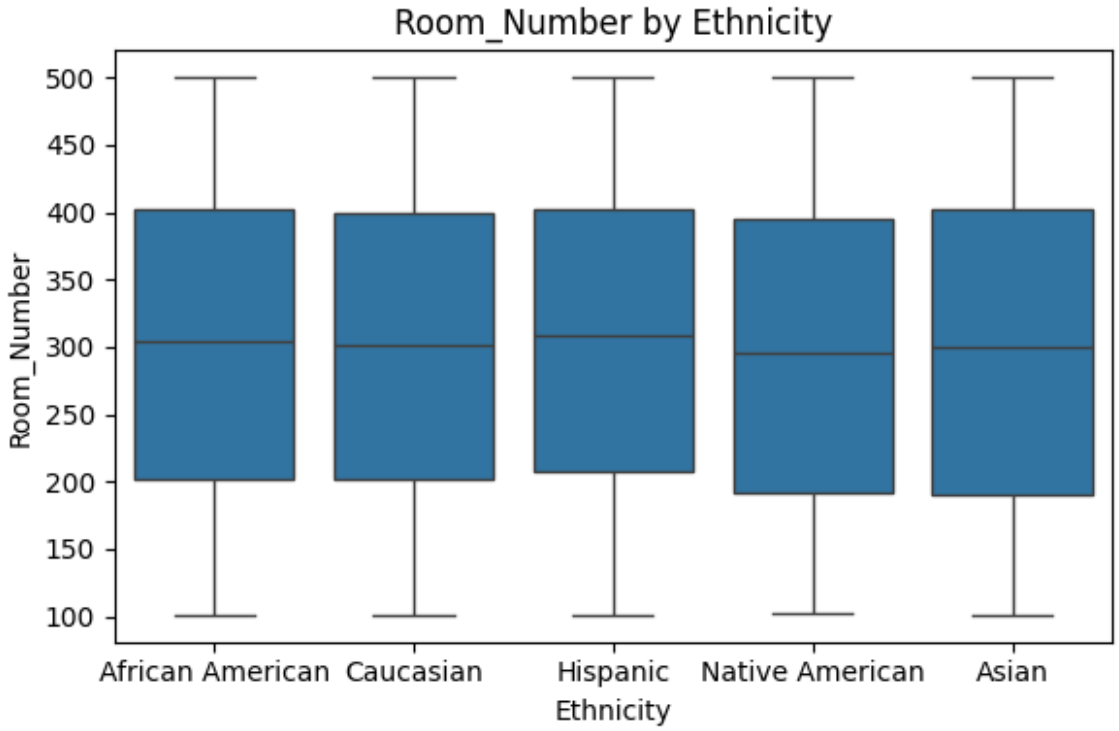

```
boxplot_Room_Number_by_Admission_Type.png
```



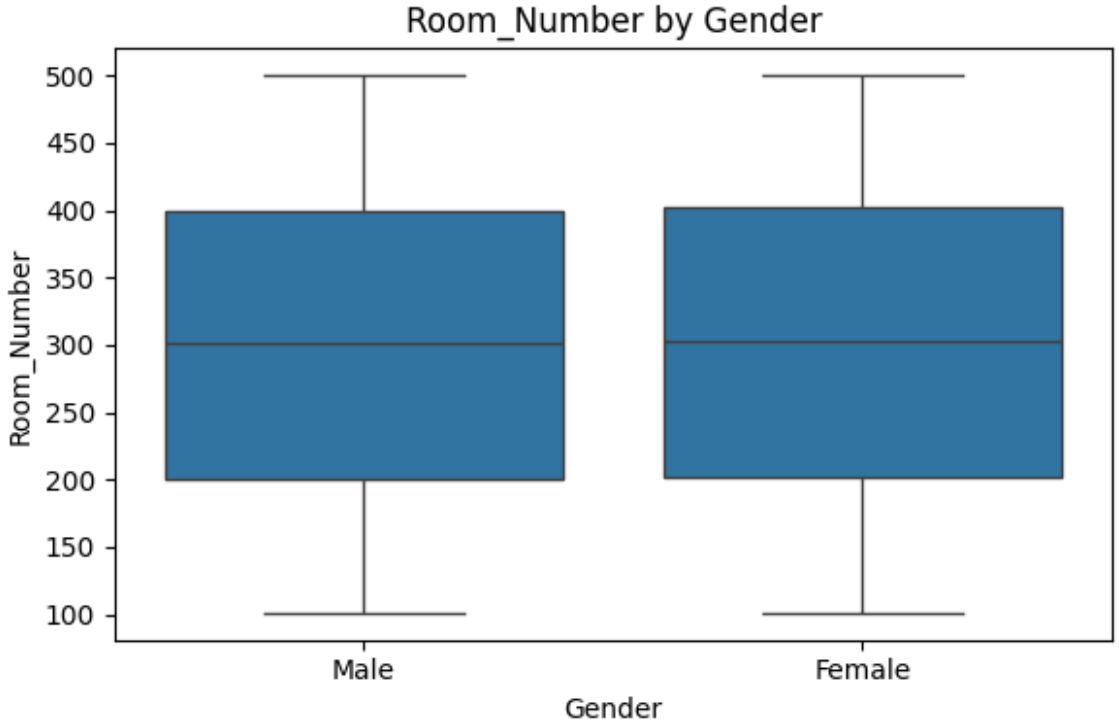
```
boxplot_Room_Number_by_Blood_Type.png
```



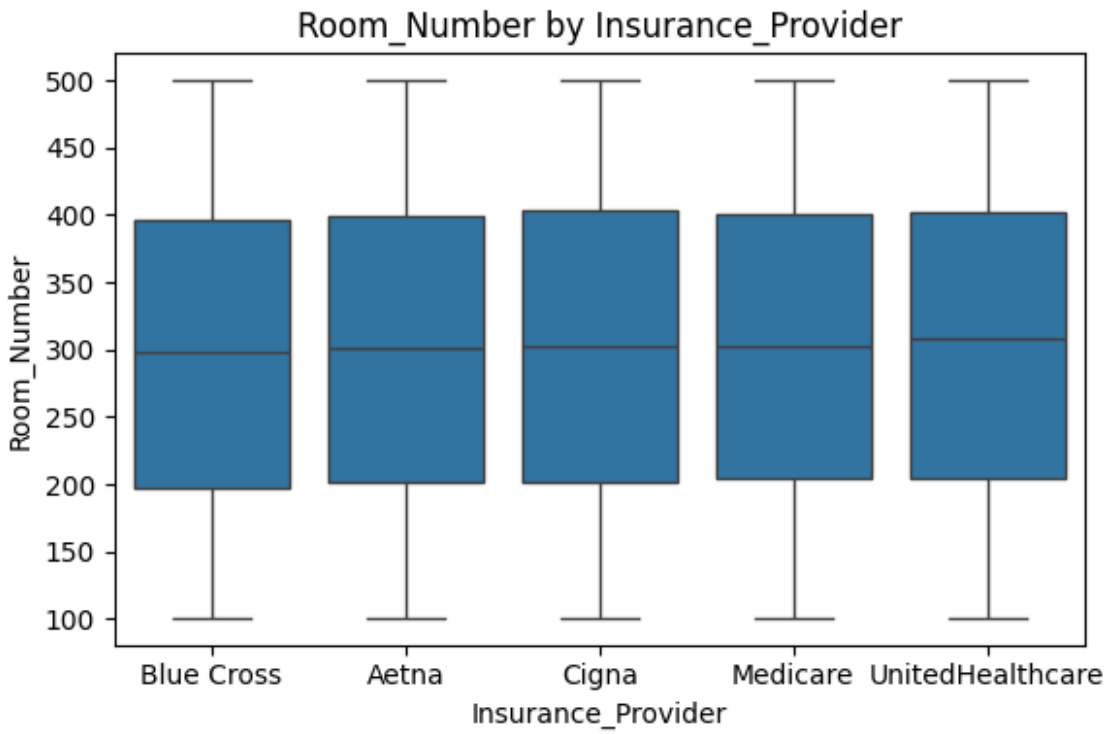

```
boxplot_Room_Number_by_Ethnicity.png
```



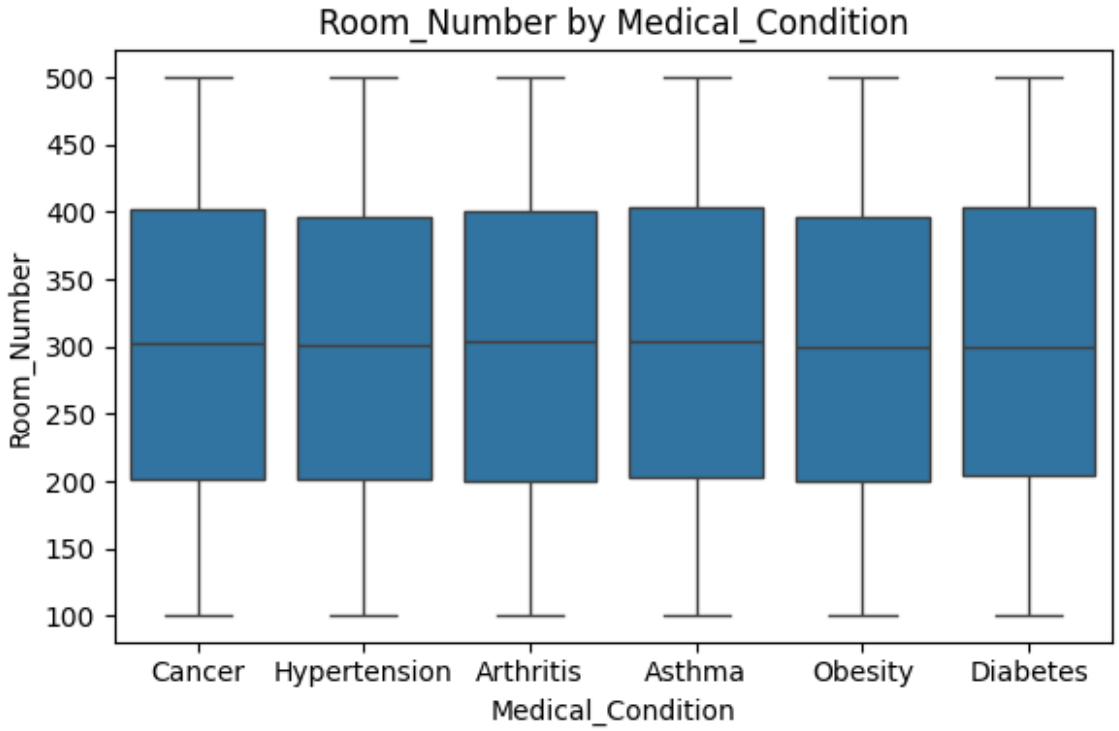

```
boxplot_Room_Number_by_Gender.png
```



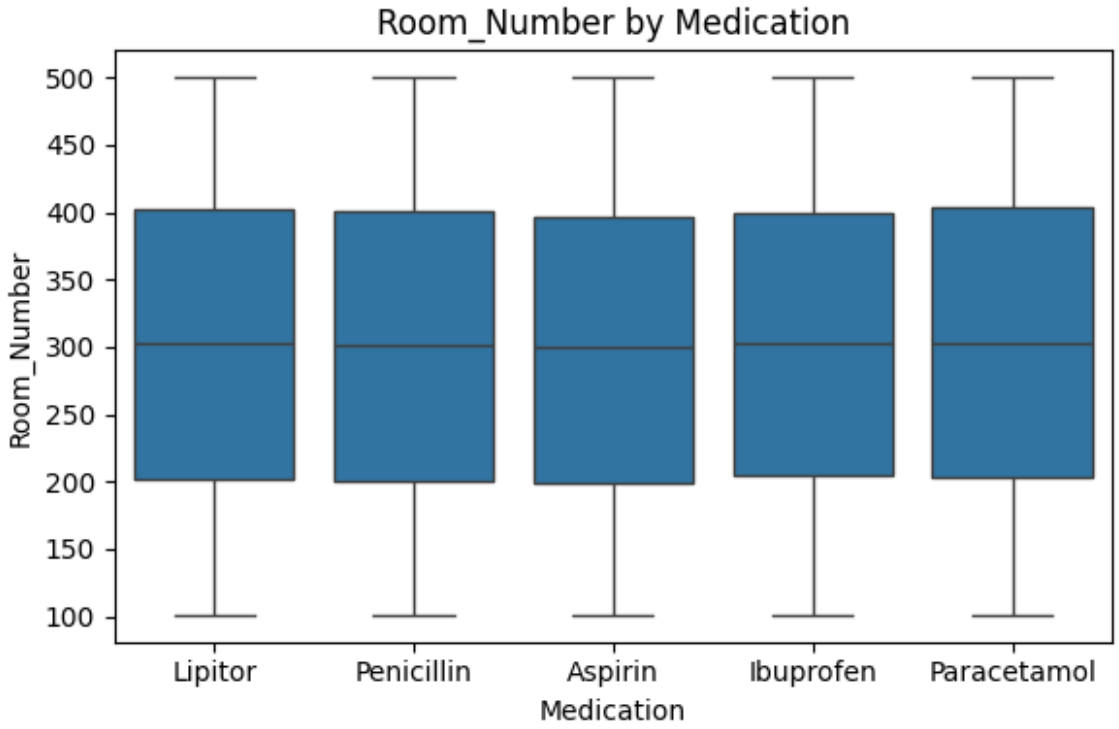
```
boxplot_Room_Number_by_Insurance_Provider.png
```



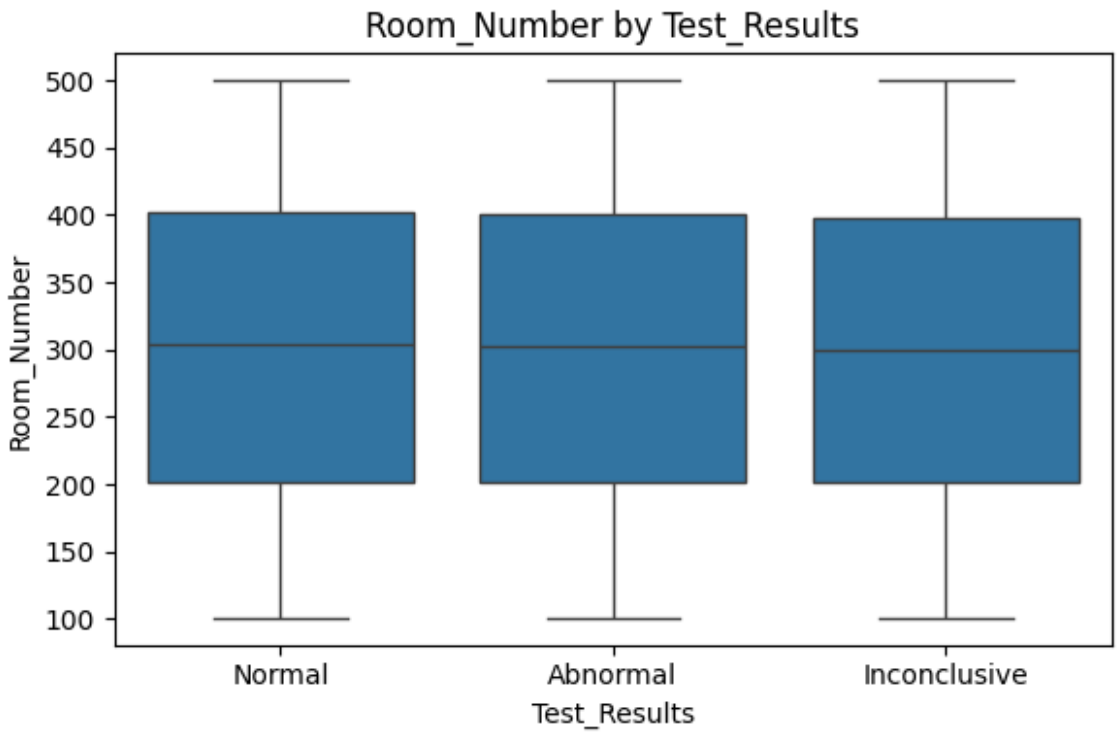

```
boxplot_Room_Number_by_Medical_Condition.png
```



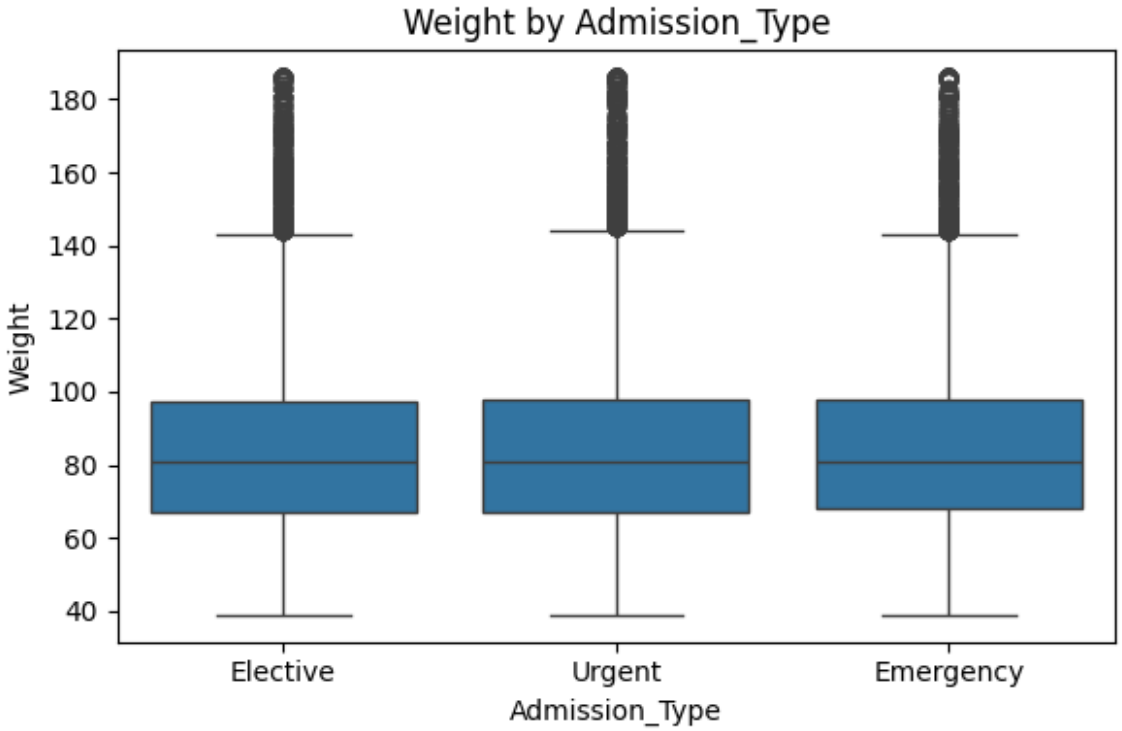

```
boxplot_Room_Number_by_Medication.png
```



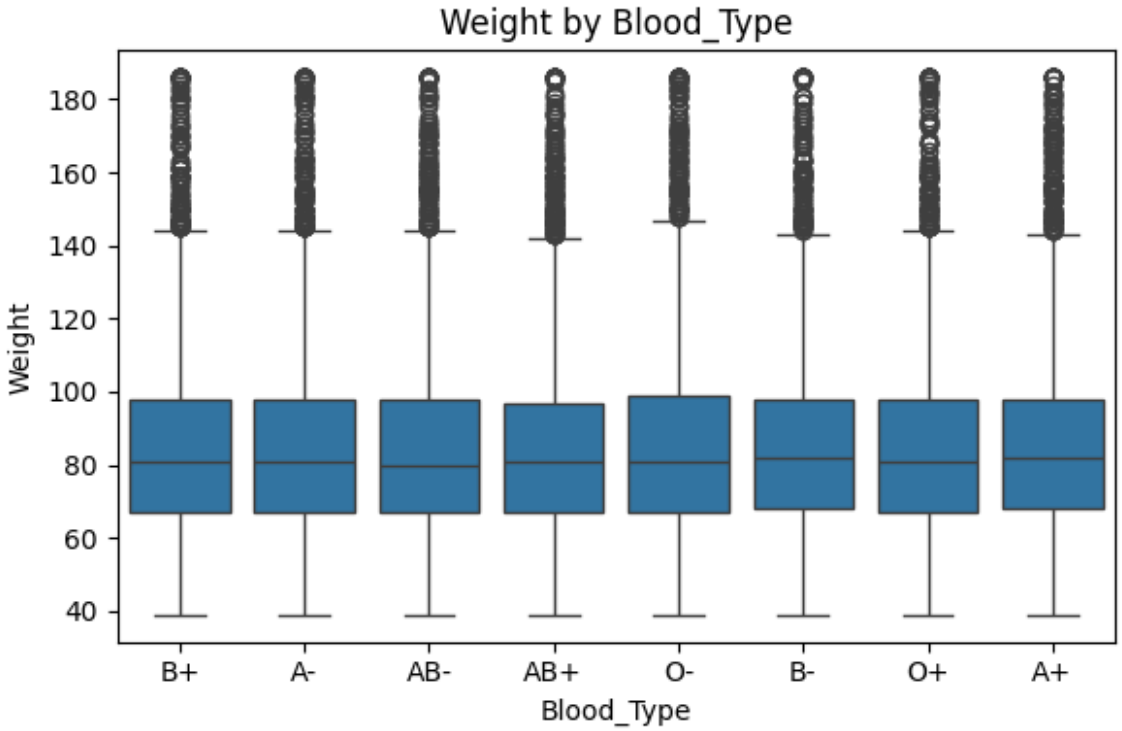
```
boxplot_Room_Number_by_Test_Results.png
```



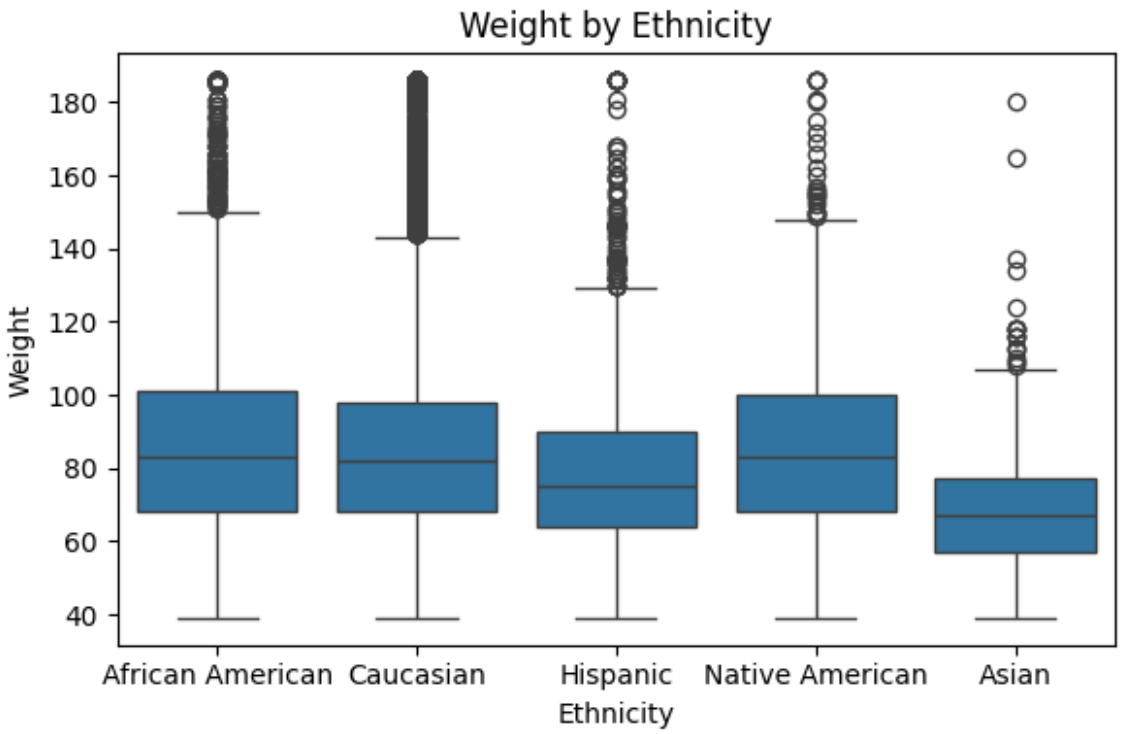

```
boxplot_Weight_by_Admission_Type.png
```



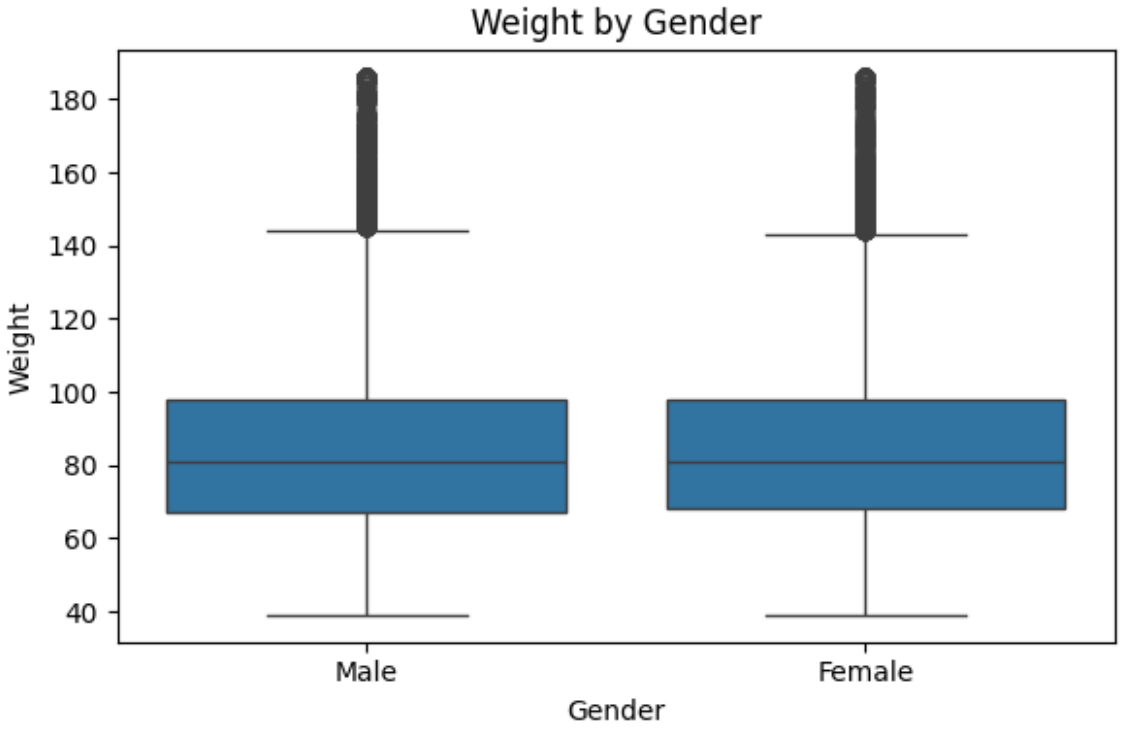

```
boxplot_Weight_by_Blood_Type.png
```



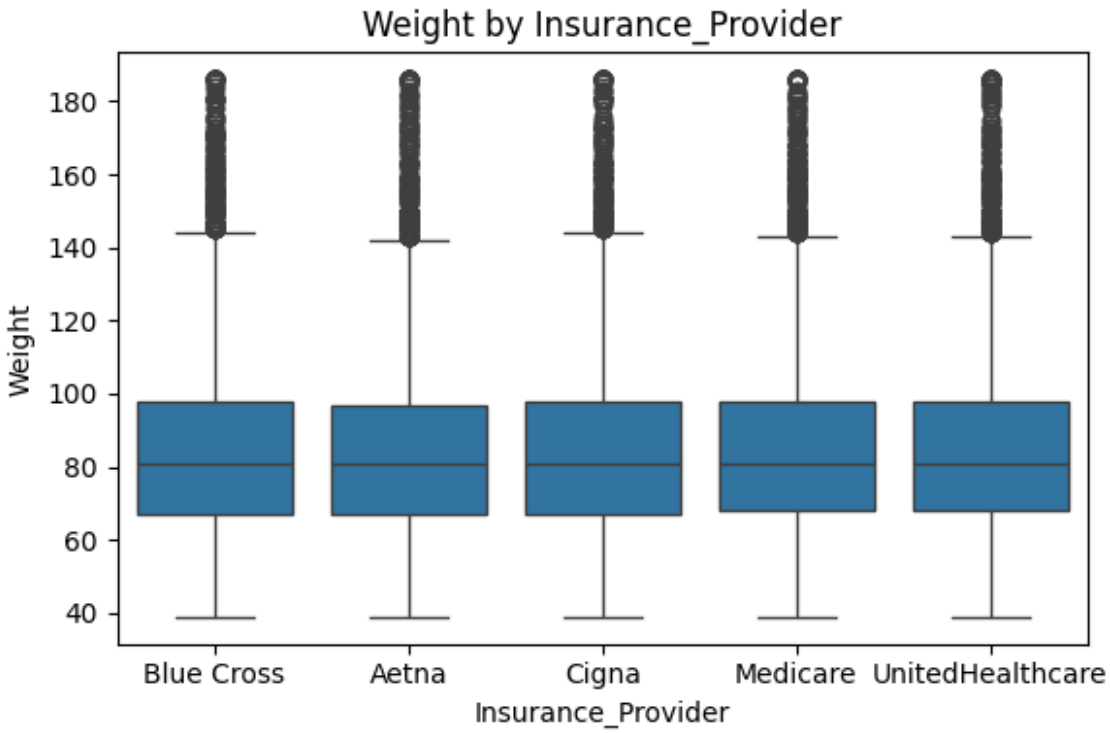
```
boxplot_Weight_by_Ethnicity.png
```



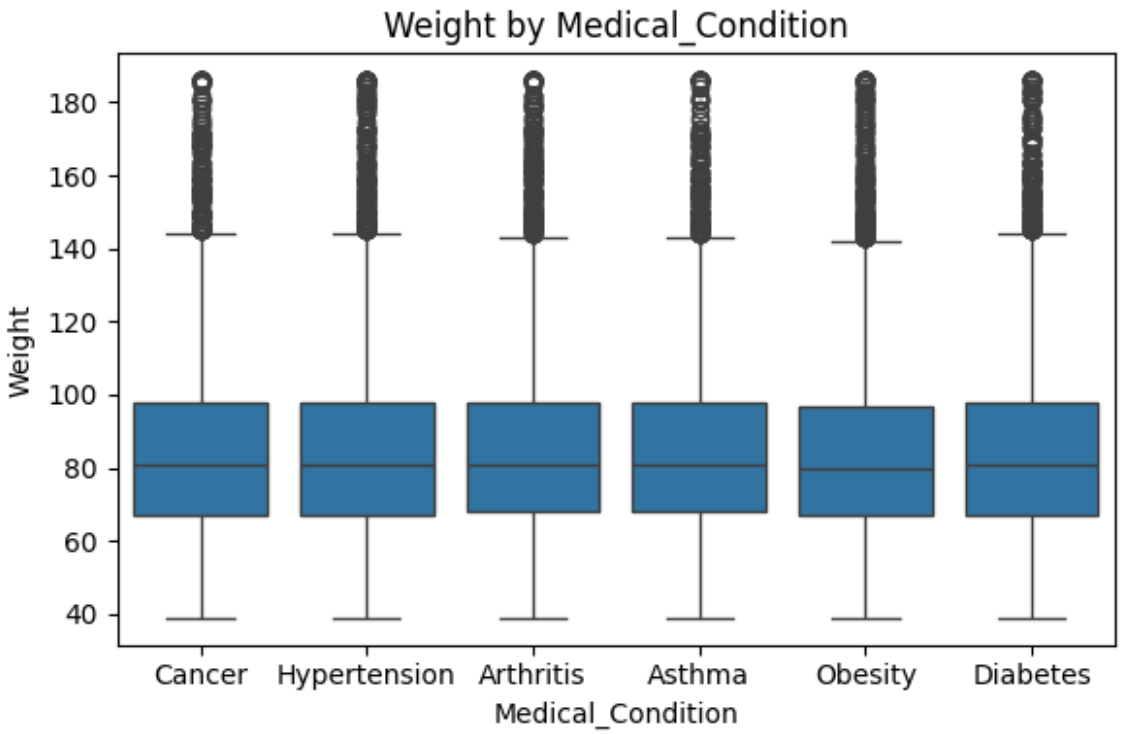

```
boxplot_Weight_by_Gender.png
```



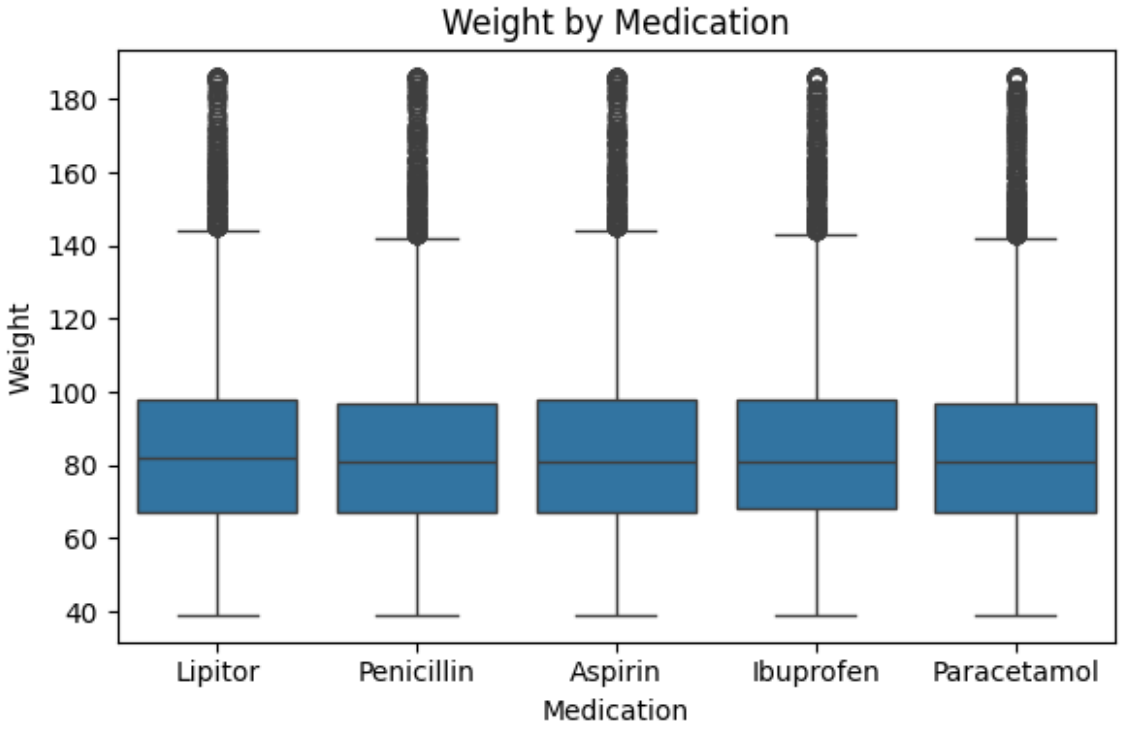

```
boxplot_Weight_by_Insurance_Provider.png
```



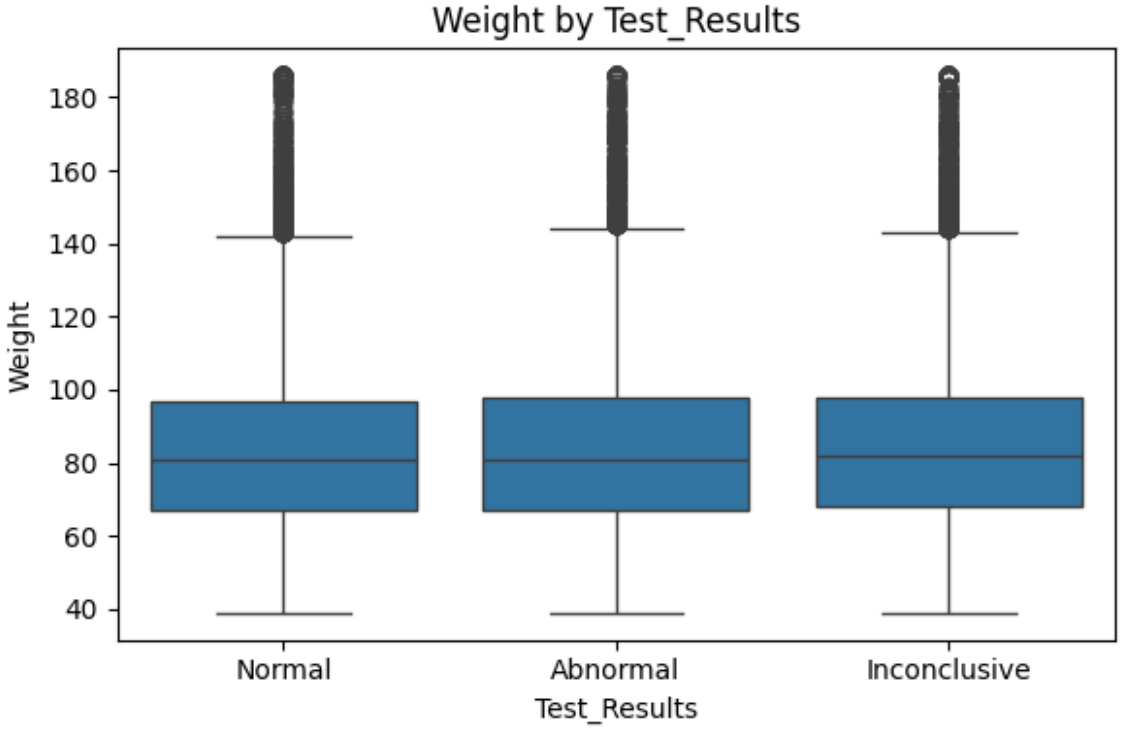
```
boxplot_Weight_by_Medical_Condition.png
```



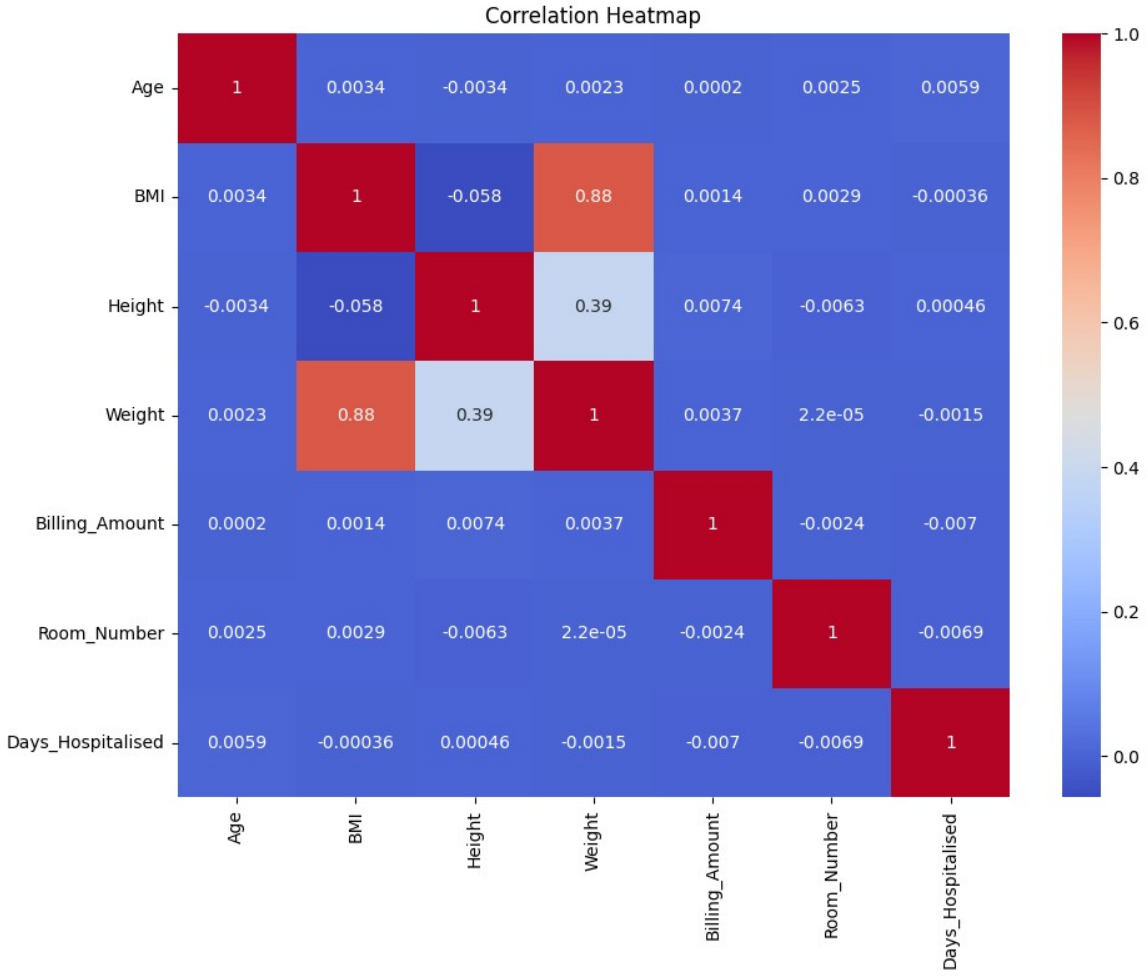

```
boxplot_Weight_by_Medication.png
```



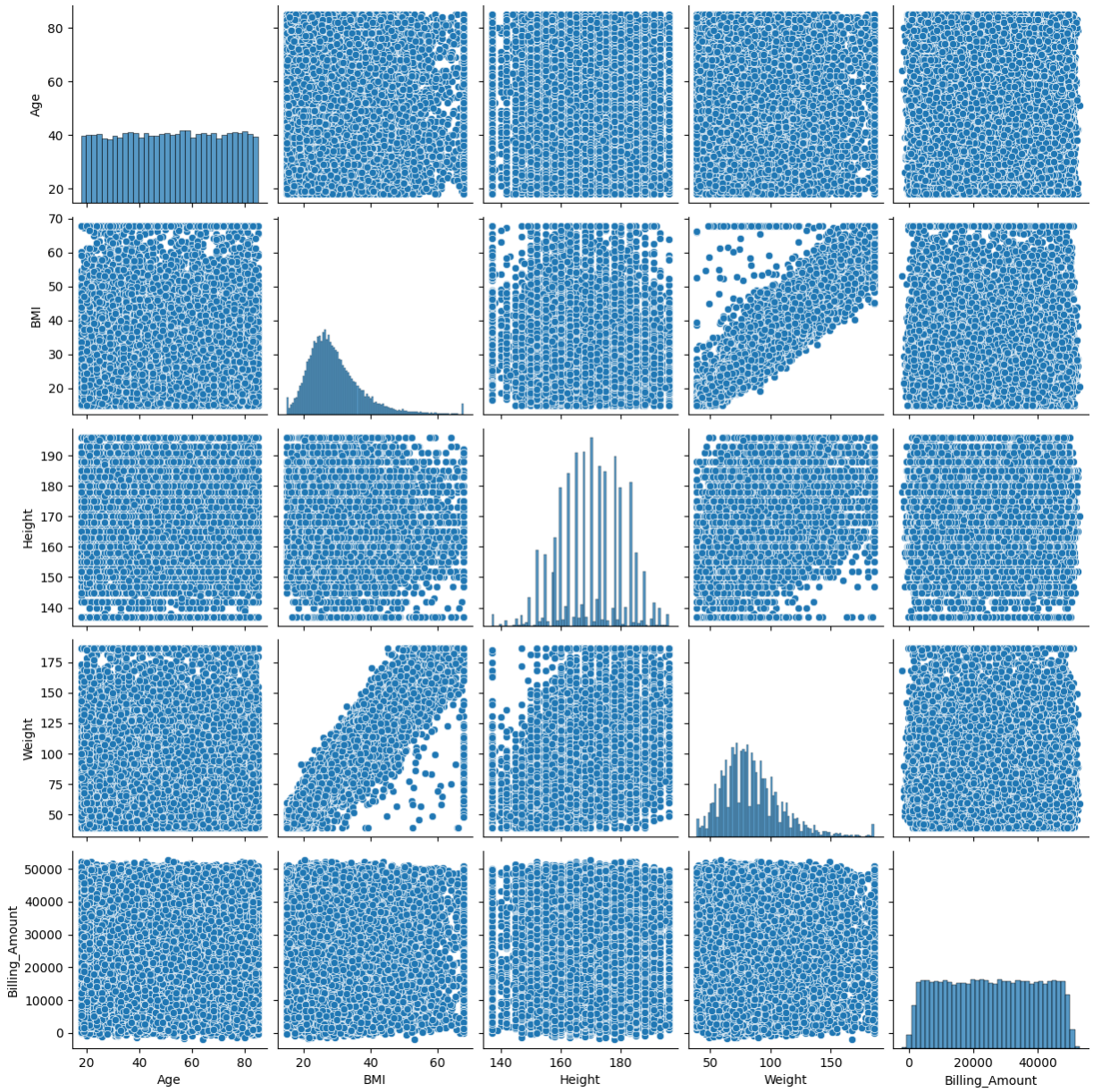

```
boxplot_Weight_by_Test_Results.png
```



`correlation_heatmap.png`



`pairplot.png`



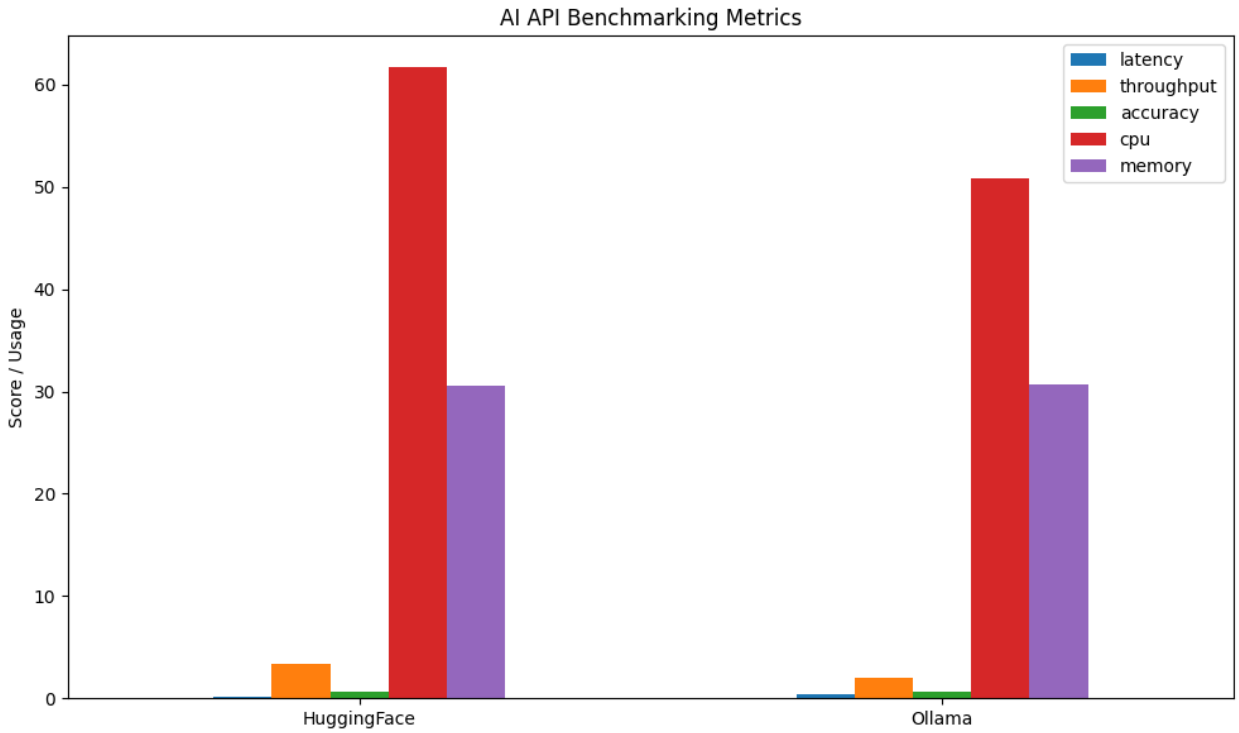
1. AI API Performance Benchmarking Design a benchmarking suite that: - Compares at least two AI APIs (e.g., Hugging Face, Ollama) - Measures latency, throughput, and accuracy - Reports hardware usage statistics during execution

```
import time
import random
import psutil
import pandas as pd
import matplotlib.pyplot as plt

#  Simulated API 1: Hugging Face-style sentiment classifier
def huggingface_api(text):
    time.sleep(0.2)  # Simulate latency
```

```
    return "positive" if "love" in text or "great" in text else
"negative"

#  Simulated API 2: Ollama-style sentiment classifier
def ollama_api(text):
    time.sleep(0.4)  # Simulate longer latency
    return "positive" if "excellent" in text or "fast" in text else
"negative"

#  Benchmarking function
def benchmark_api(api_func, test_data):
    latencies = []
    correct = 0
    cpu_usages = []
    mem_usages = []

    start_time = time.time()

    for item in test_data:
        text = item["text"]
        expected = item["label"]

        cpu_usages.append(psutil.cpu_percent(interval=0.1))
        mem_usages.append(psutil.virtual_memory().percent)

        t0 = time.time()
        prediction = api_func(text)
        t1 = time.time()

        latencies.append(t1 - t0)
        if prediction == expected:
            correct += 1

    end_time = time.time()
    total_time = end_time - start_time
    throughput = len(test_data) / total_time
    avg_latency = sum(latencies) / len(latencies)
    accuracy = correct / len(test_data)

    return {
        "latency": round(avg_latency, 3),
        "throughput": round(throughput, 2),
        "accuracy": round(accuracy, 2),
        "cpu": round(sum(cpu_usages) / len(cpu_usages), 2),
        "memory": round(sum(mem_usages) / len(mem_usages), 2)
    }

# Sample test data
test_data = [
    {"text": "I love this product", "label": "positive"},
```

```
    {"text": "This was a terrible experience", "label": "negative"},
    {"text": "Excellent service and fast delivery", "label":
"positive"},
    {"text": "Not worth the price", "label": "negative"},
    {"text": "Great quality and packaging", "label": "positive"},
    {"text": "Slow response and bad support", "label": "negative"}
]

# Run benchmarks
hf_results = benchmark_api(huggingface_api, test_data)
ollama_results = benchmark_api(ollama_api, test_data)

# Compile results
df = pd.DataFrame([hf_results, ollama_results], index=["HuggingFace",
"Ollama"])
df.to_csv("benchmark_results.csv")
print(df)

# Plot comparison
df.plot(kind="bar", figsize=(10, 6), title="AI API Benchmarking
Metrics")
plt.ylabel("Score / Usage")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("benchmark_metrics_plot.png")
plt.show()
```

```
             latency  throughput  accuracy    cpu  memory
HuggingFace      0.2        3.32      0.67  61.67   30.58
Ollama           0.4        2.00      0.67  50.83   30.73
```




```
from google.colab import files
files.upload()

<IPython.core.display.HTML object>

---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipython-input-1850841700.py in <cell line: 0>()
      1 from google.colab import files
----> 2 files.upload()

/usr/local/lib/python3.11/dist-packages/google/colab/files.py in upload(target_dir)
     70   """
     71
---> 72   uploaded_files = _upload_files(multiple=True)
     73   # Mapping from original filename to filename as saved locally.
     74   local_filenames = dict()

/usr/local/lib/python3.11/dist-packages/google/colab/files.py in _upload_files(multiple)
    162
    163   # First result is always an indication that the file picker has completed.
```

```
--> 164   result = _output.eval_js(
    165       'google.colab._files._uploadFiles("{input_id}",
"{output_id}")'.format(
    166           input_id=input_id, output_id=output_id

/usr/local/lib/python3.11/dist-packages/google/colab/output/_js.py in
eval_js(script, ignore_result, timeout_sec)
     38   if ignore_result:
     39     return
---> 40   return _message.read_reply_from_input(request_id,
timeout_sec)
     41
     42

/usr/local/lib/python3.11/dist-packages/google/colab/_message.py in
read_reply_from_input(message_id, timeout_sec)
     94     reply = _read_next_input_message()
     95     if reply == _NOT_READY or not isinstance(reply, dict):
---> 96       time.sleep(0.025)
     97       continue
     98     if (

KeyboardInterrupt:
```

```
#for git
!pip install nbstripout
!nbstripout week2_assg.ipynb
```

```
Requirement already satisfied: nbstripout in
/usr/local/lib/python3.11/dist-packages (0.8.1)
Requirement already satisfied: nbformat in
/usr/local/lib/python3.11/dist-packages (from nbstripout) (5.10.4)
Requirement already satisfied: fastjsonschema>=2.15 in
/usr/local/lib/python3.11/dist-packages (from nbformat->nbstripout)
(2.21.1)
Requirement already satisfied: jsonschema>=2.6 in
/usr/local/lib/python3.11/dist-packages (from nbformat->nbstripout)
(4.25.0)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
/usr/local/lib/python3.11/dist-packages (from nbformat->nbstripout)
(5.8.1)
Requirement already satisfied: traitlets>=5.1 in
/usr/local/lib/python3.11/dist-packages (from nbformat->nbstripout)
(5.7.1)
Requirement already satisfied: attrs>=22.2.0 in
/usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6-
>nbformat->nbstripout) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
/usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6-
>nbformat->nbstripout) (2025.4.1)
```

```
Requirement already satisfied: referencing>=0.28.4 in
/usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6-
>nbformat->nbstripout) (0.36.2)
Requirement already satisfied: rpds-py>=0.7.1 in
/usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6-
>nbformat->nbstripout) (0.27.0)
Requirement already satisfied: platformdirs>=2.5 in
/usr/local/lib/python3.11/dist-packages (from jupyter-core!
=5.0.*,>=4.12->nbformat->nbstripout) (4.3.8)
Requirement already satisfied: typing-extensions>=4.4.0 in
/usr/local/lib/python3.11/dist-packages (from referencing>=0.28.4-
>jsonschema>=2.6->nbformat->nbstripout) (4.14.1)
Could not strip 'week2_assg.ipynb': file not found
```